



CS 329Z: Engineering AI Agents

LLMs for Builders

Diyi Yang, Michael Ryan, John Yang

The LLM Stack

LLM
Applications

Agents

CS329Z

Chatbots RAG

Agent Frameworks
Multi Agent
Agent Memory
Proactive Agents
Agent Evals
Agent Design
...

Finetuning

Distillation

CS224N

LLM
Adaptation

In-Context Learning

LoRA

Learning from Feedback (RLHF/DPO)

LLM
Engineering

Distributed Compute

Architectures

GPU Kernels

CS336

Scaling Laws

Post-training

Pretraining

RL

Tokenizer



We are going to operate at the **highest level of the LLM stack**, but it is still important to understand a bit about **how they work...**



Learning Objectives

- **Understand and explain** the core design decisions that go into training and inferencing modern Large Language Models (LLMs).
- **Synthesize** how these design decisions lead to limitations that impact the way in which applications leverage LLMs.



Lecture Overview

- **What is a Language Model?**
- **Language Model Architecture**
- **Training**
- **Inference**
- **LLMs for Agents**



What is a Language Model?

Language Models?

Architecture

Training

Inference

LLMs for Agents



$$\underline{P(x_{1:T})} = \prod_{t=1}^T \underline{P(x_t \mid x_{<t})}$$

**Probability of a
sequence of tokens** **Equals**

**Product from the
first to last token** **of the probability of the current
token given the previous tokens**



$$P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

The class was about agents .

P(The) = 0.0377 → 0.0377

P(class | The) = 0.0001 → 0.0001

P(was | The , class) = 0.0344 → 0.0344

...

...

P(. | The , class , was , about , agents) ≠ 0.0252

*in practice we typically take the sum of the log of the probabilities instead for numerical stability. 1.2e-17



$$P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

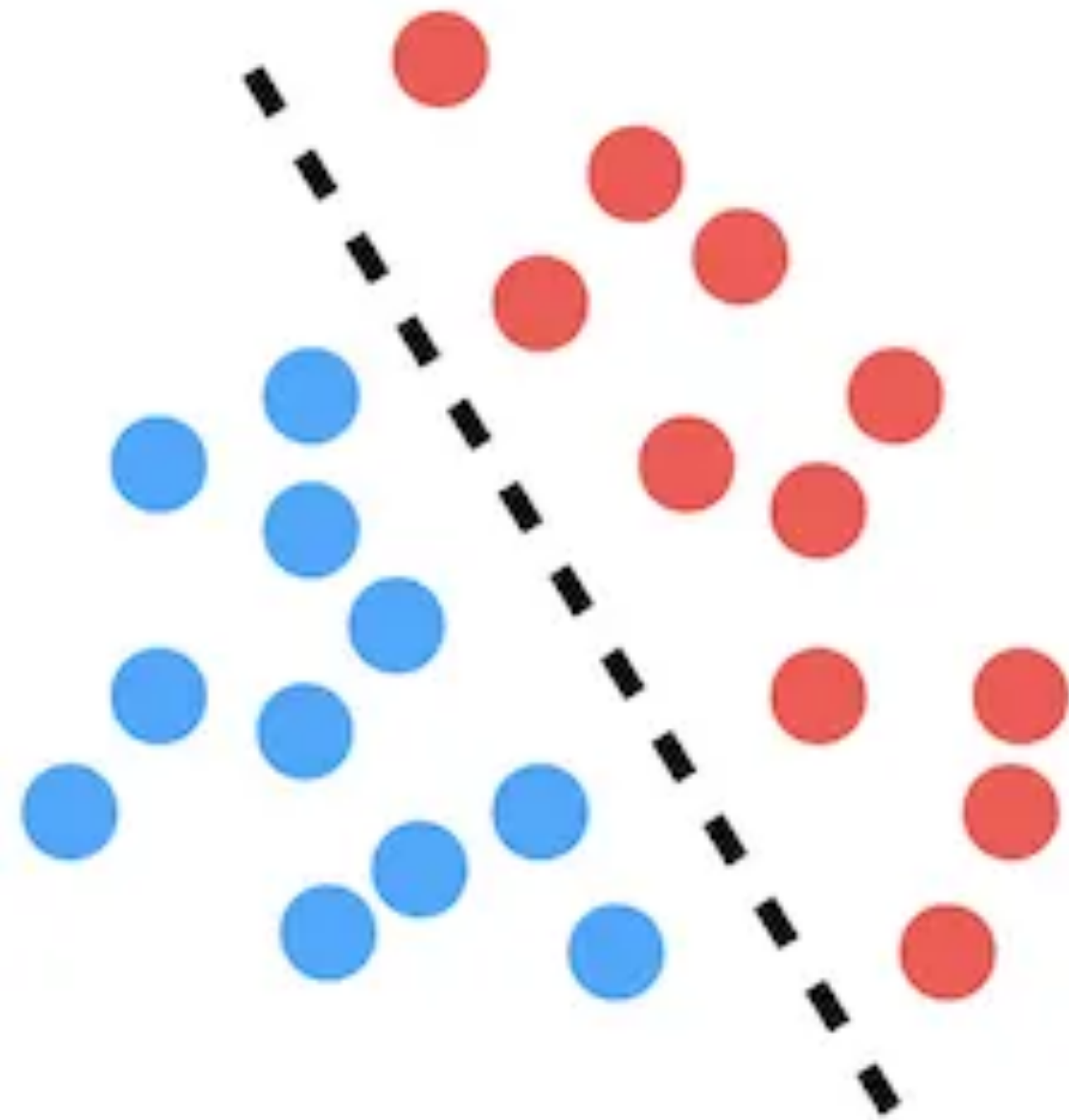
**This is a Language
Model**



Generative vs. Discriminative Models



Discriminative

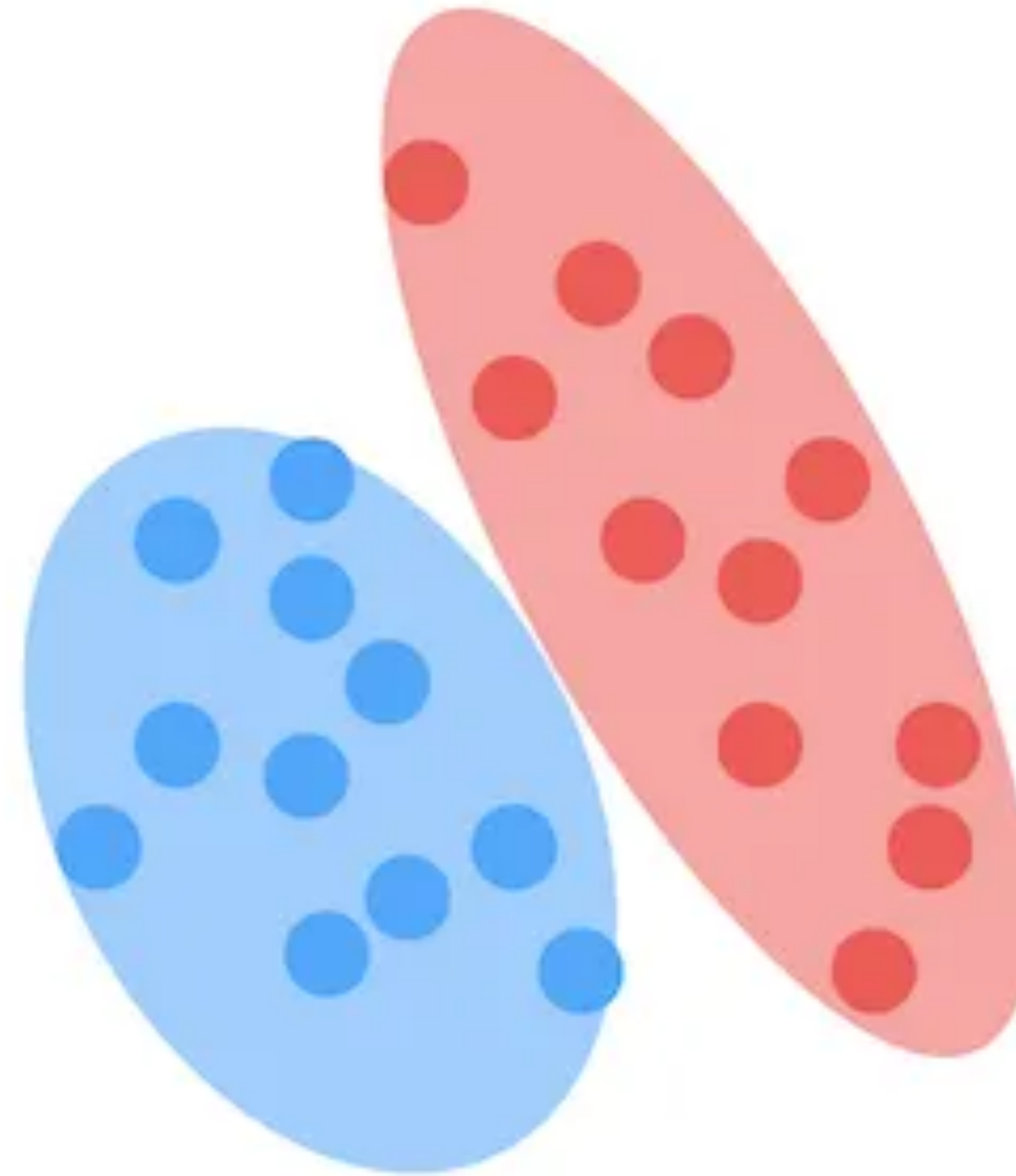


Learn to Distinguish Classes

E.g.

- Logistic Regression
- SVM

Generative



Learn Distribution of Data

E.g.

- Naive Bayes
- Diffusion models



Knowledge Check

Are Language Models that model the probability of a token sequence as shown below **generative** or **discriminative**?

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

This isn't meant to be a trick question, but the notation here can make it tricky.



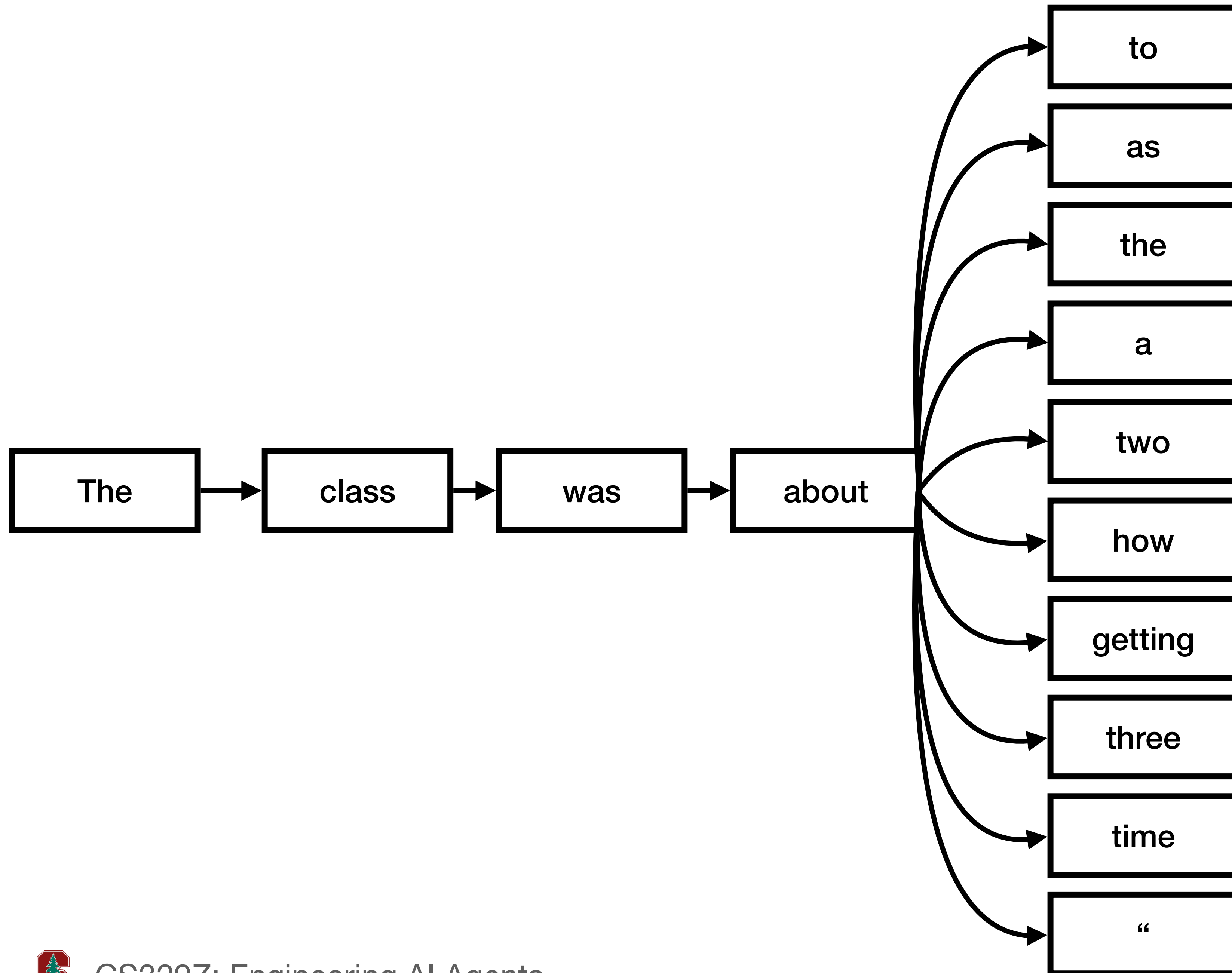
$$P(x_{1:T}) = P(x_1, x_2, x_3, \dots, x_T)$$

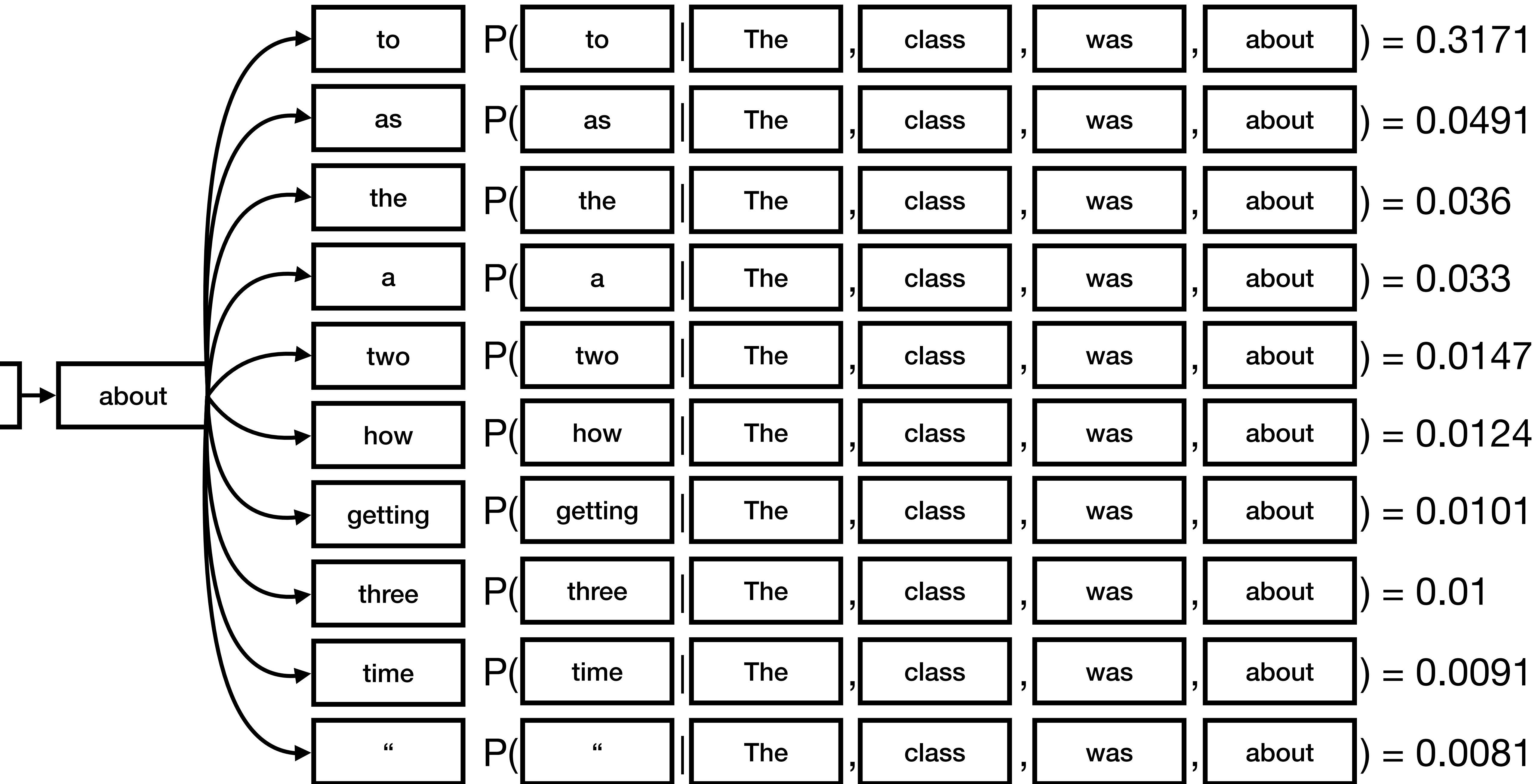
LMs are generative, which means we can sample from them.

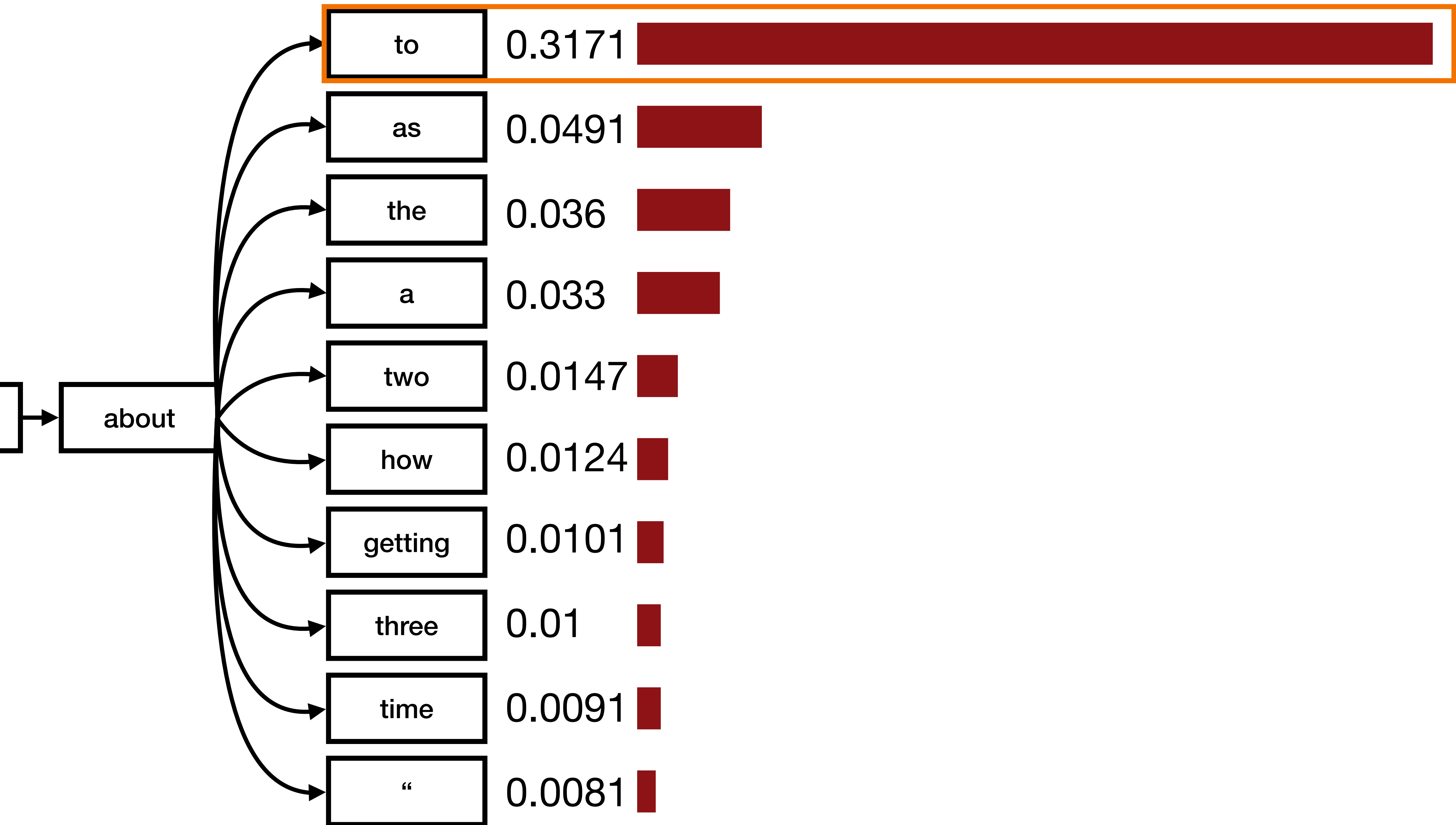


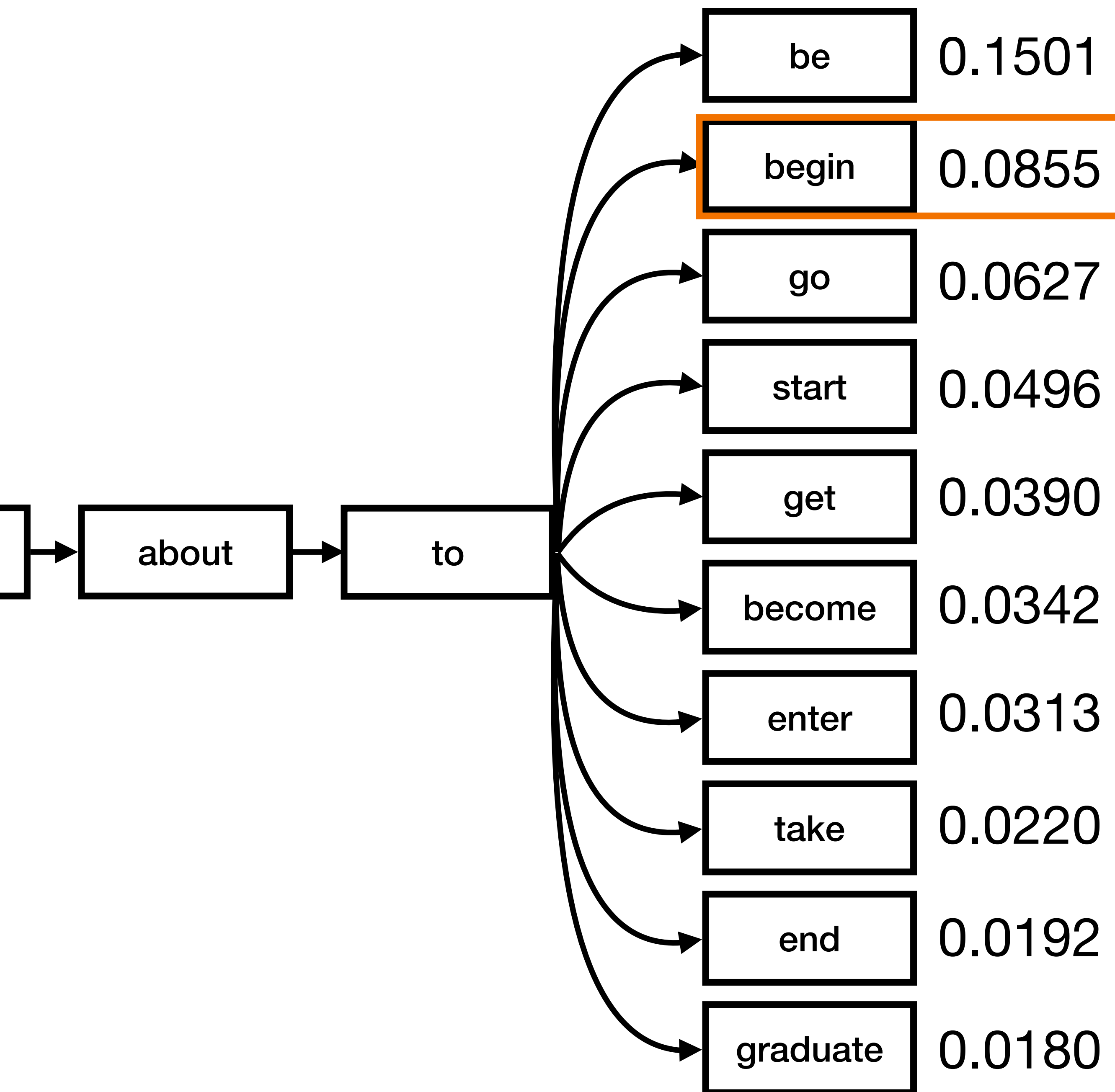
Language Model Sampling











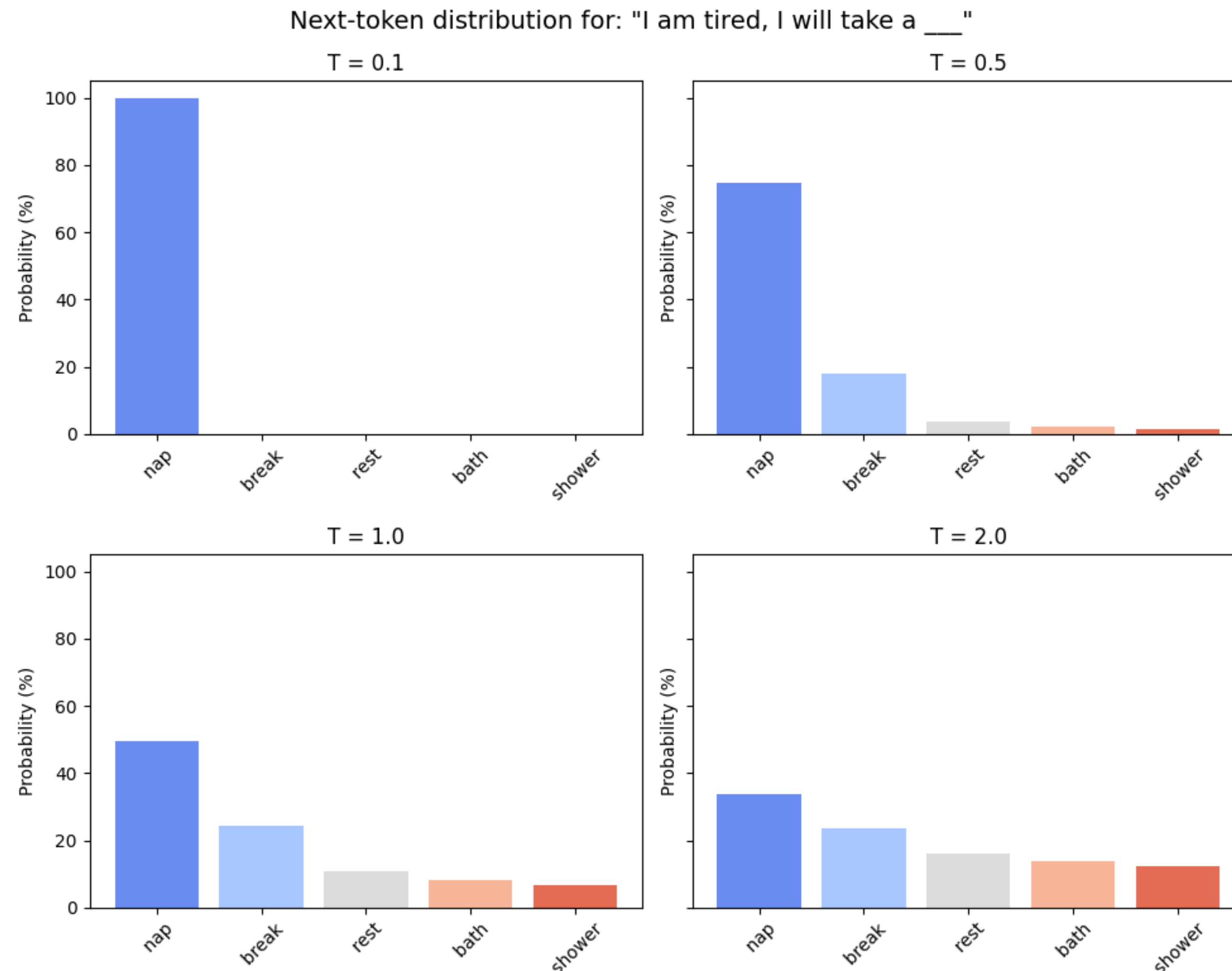
We don't always have to pick the most likely next token. There are different sampling (decoding) strategies.

- Greedy
- Nucleus (top p)
- Top k
- ...





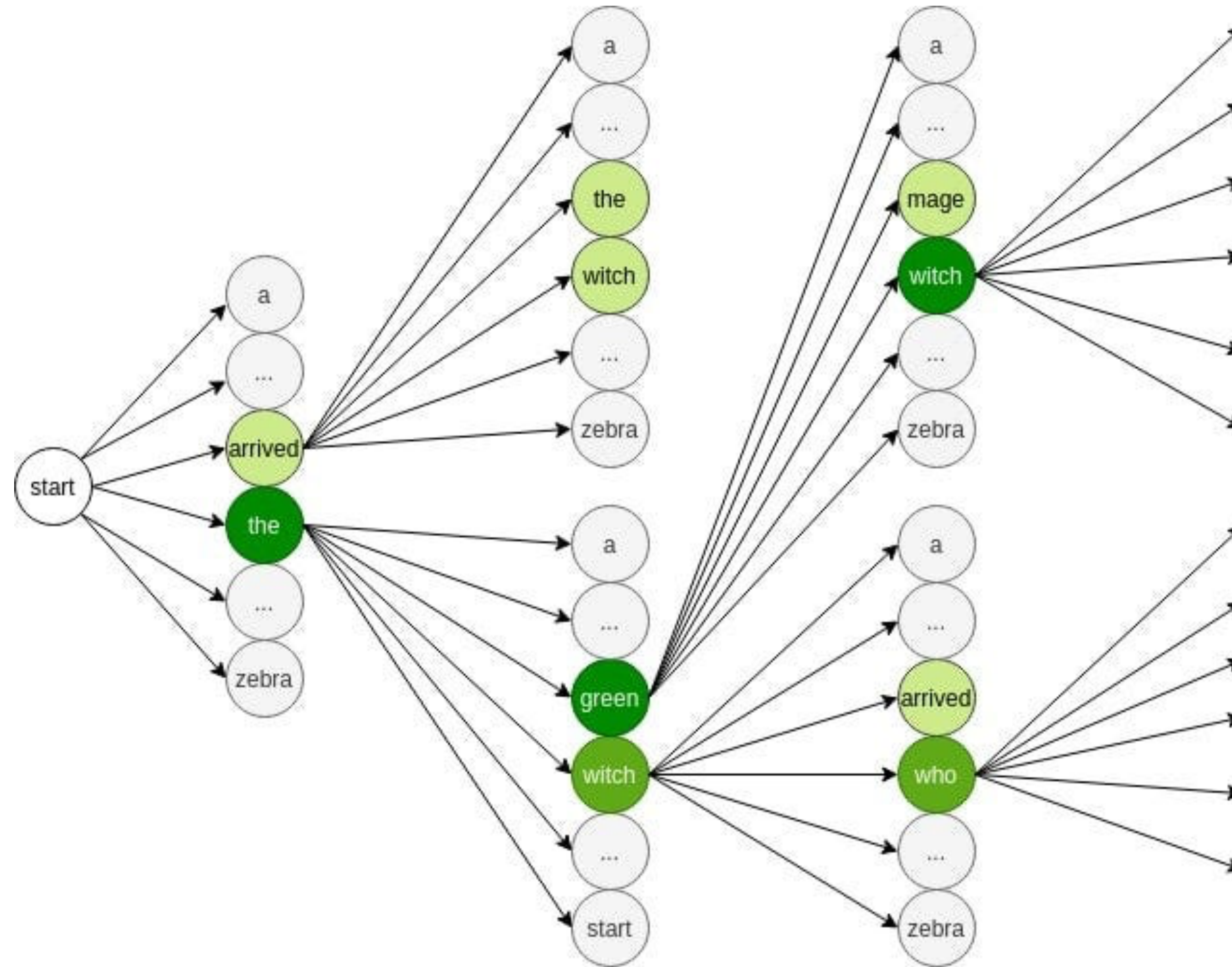
Temperature



Divide all the probabilities by some temperature T and then renormalize to sum to 1.



Beam Search



Considerations for Sampling

- Each Sampling Strategy has different tradeoffs
- Temperature and Top-p/Top-k help with diversity
- The strategy you use may depend on your task
 - Math/Code are more predictable (lower temperature)
 - Creative Writing may benefit from higher diversity
- Your strategy also depends on your budget (beam search has higher time and memory complexity)



Further Reading

- **Decoding Strategies in Large Language Models:** <https://huggingface.co/blog/mlabonne/decoding-strategies>
- **Tokenizers:** <https://huggingface.co/learn/llm-course/en/chapter2/4>
- **Generative Models:** <https://deepgenerativemodels.github.io/notes/introduction/>



Language Model Architecture

Language Models?

Architecture

Training

Inference

LLMs for Agents



A brief history of Language Model Architectures

- N-gram Models (Pre 2010s)
- First Neural Language Model (2003)
- Recurrent Neural Network Language Models (2010)
- Long Short Term Memory (Invented 1997 & popularized for LMs 2014)

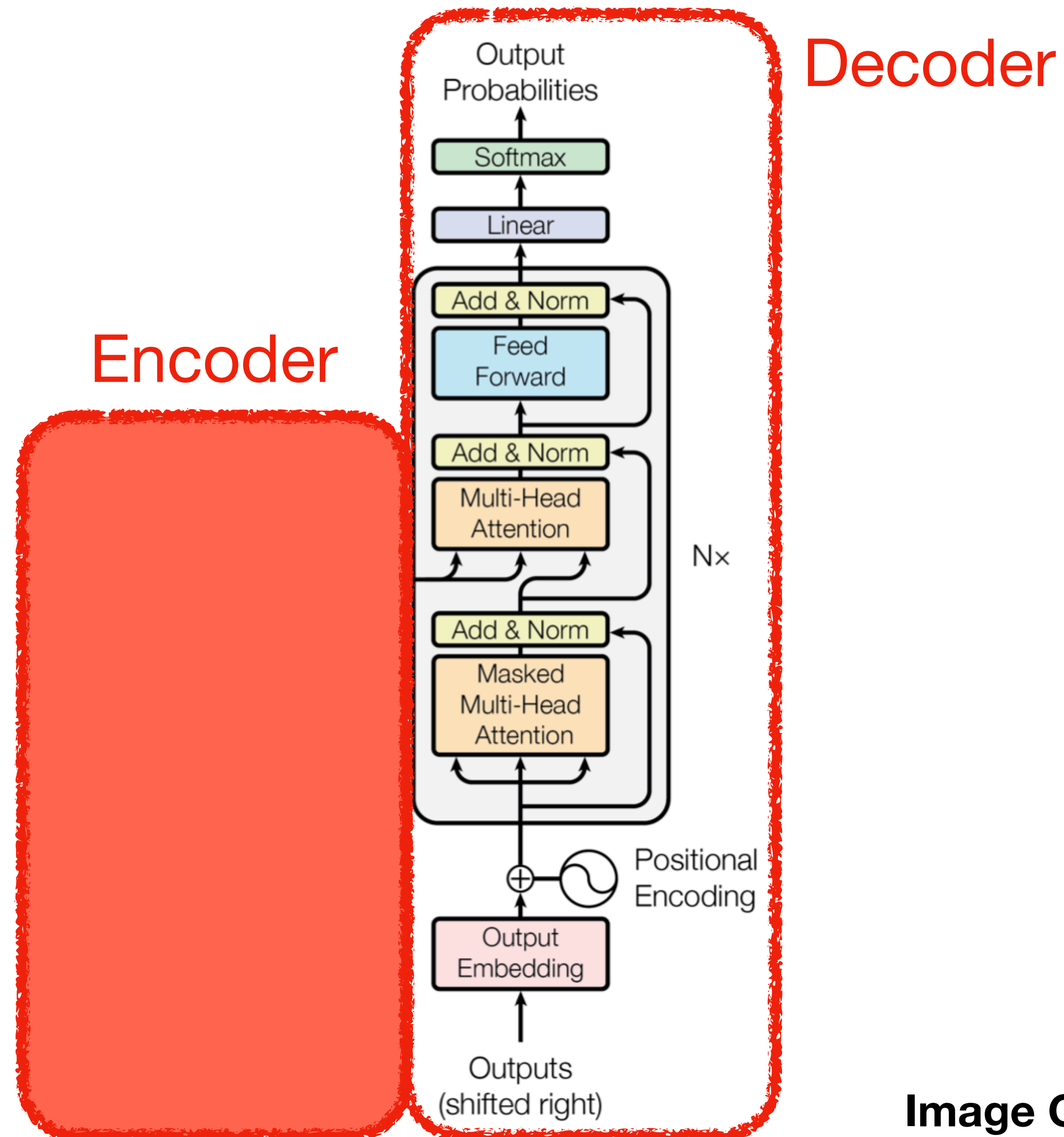


Transformers



Transformers: Attention Is All You Need (2017)

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin



Decoder-only Transformer

The

class

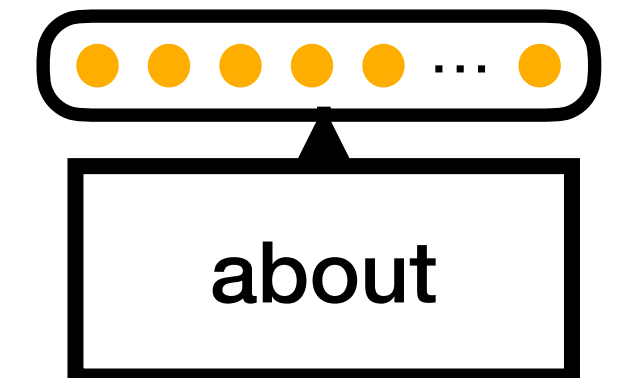
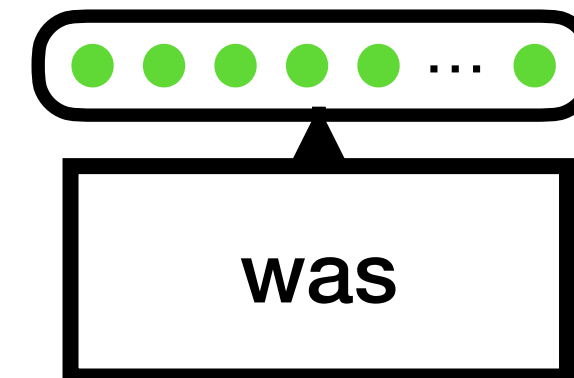
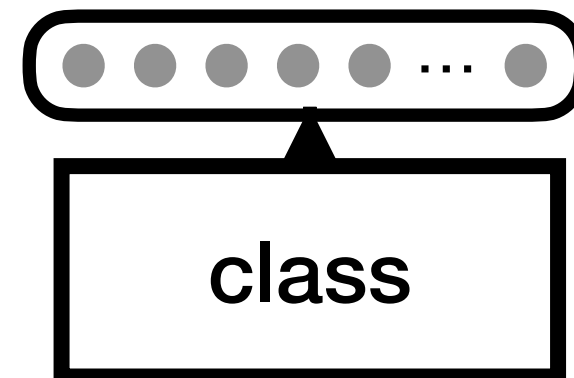
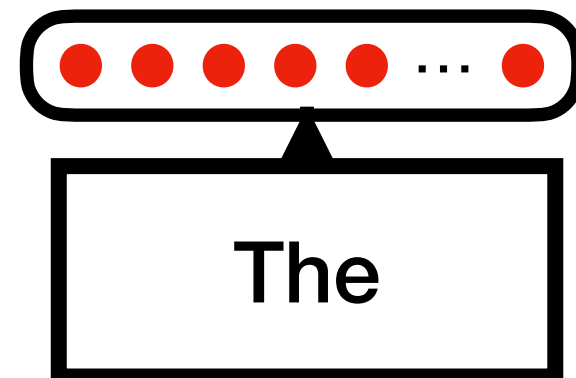
was

about

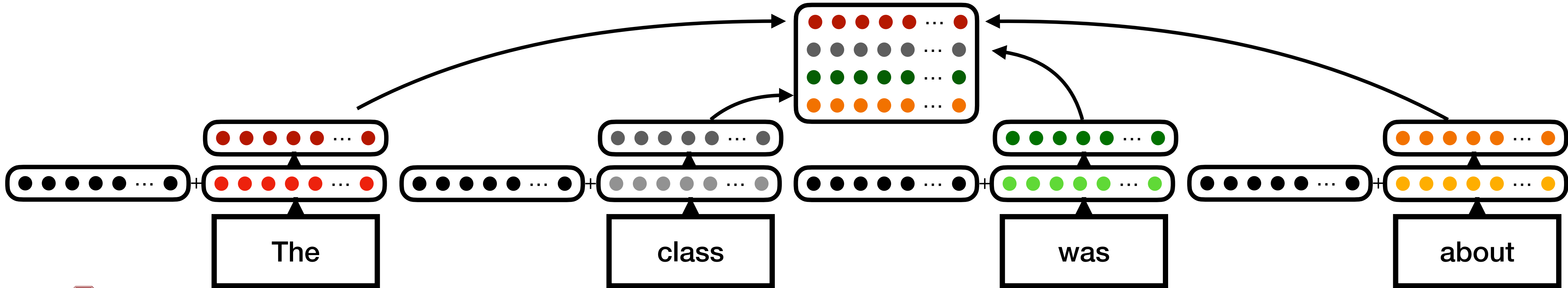


Decoder-only Transformer

Embedding



Decoder-only Transformer

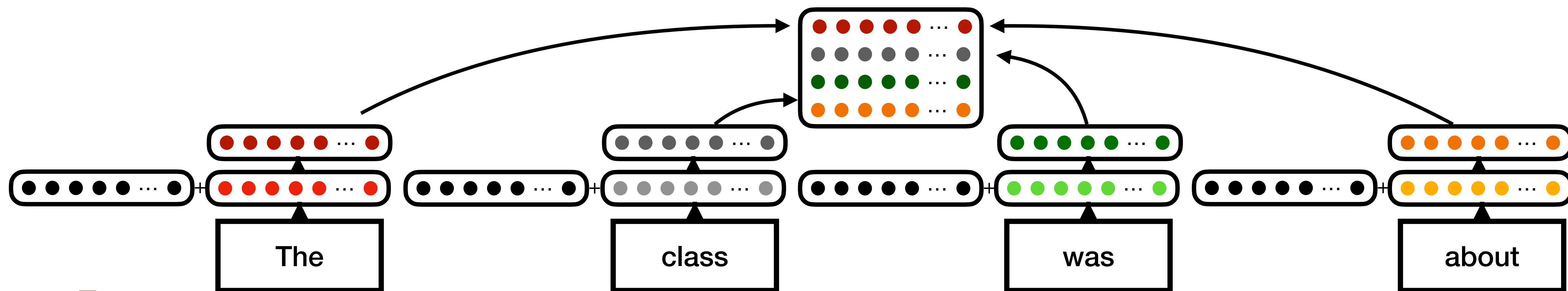


Important! Pay Close *Attention* Here!

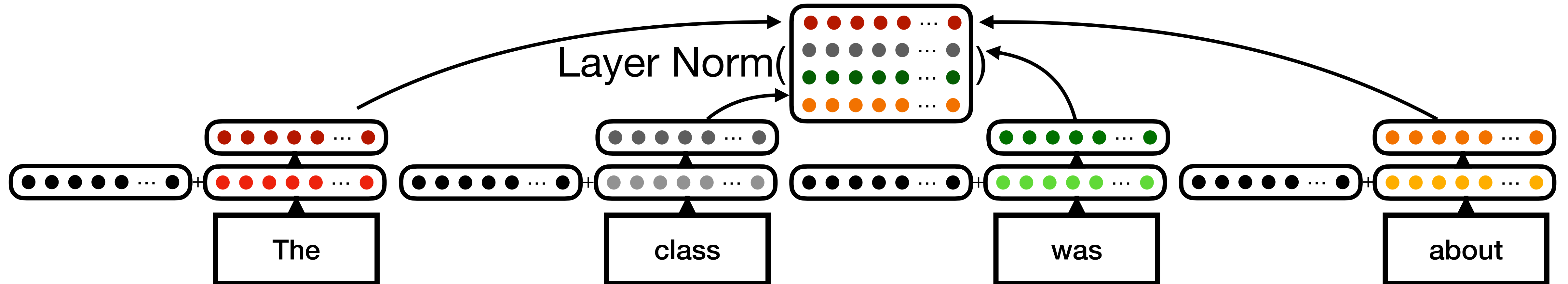
**If you get lost here much of the rest of the lecture will be
very confusing!**



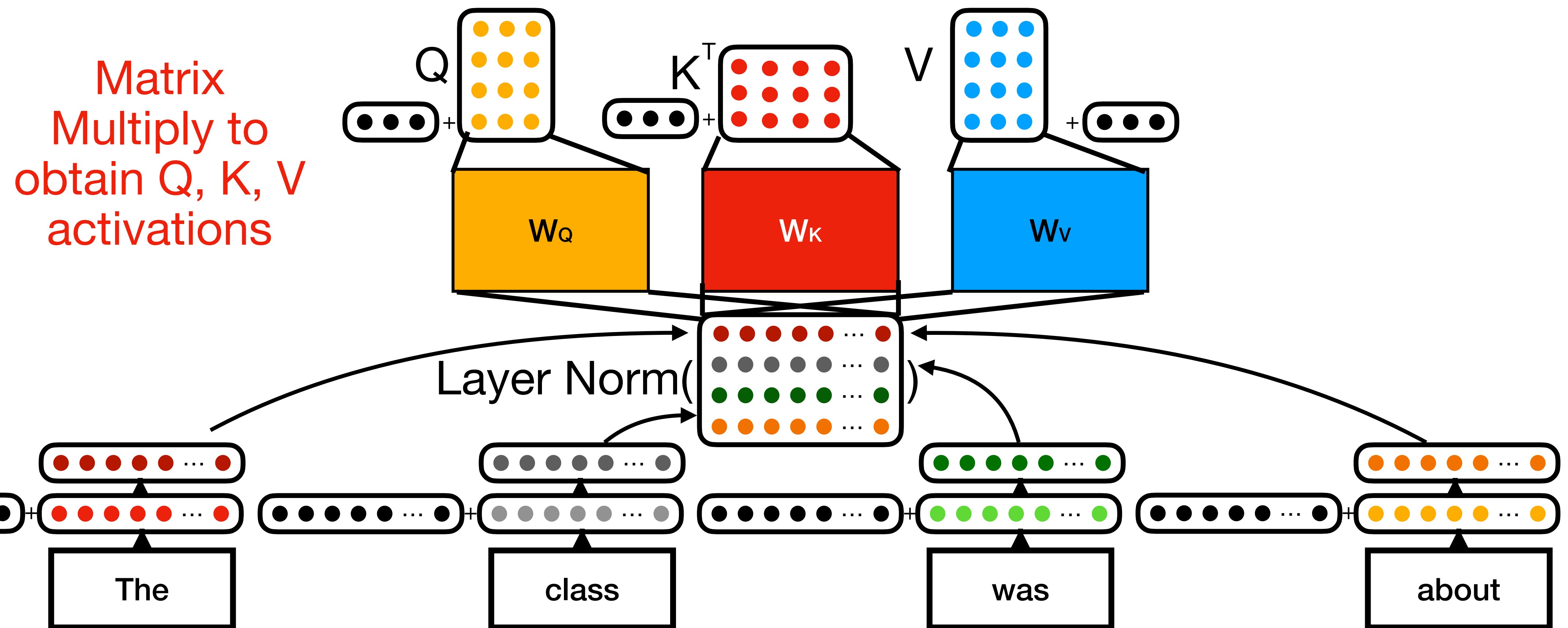
Attention Layer



Attention Layer

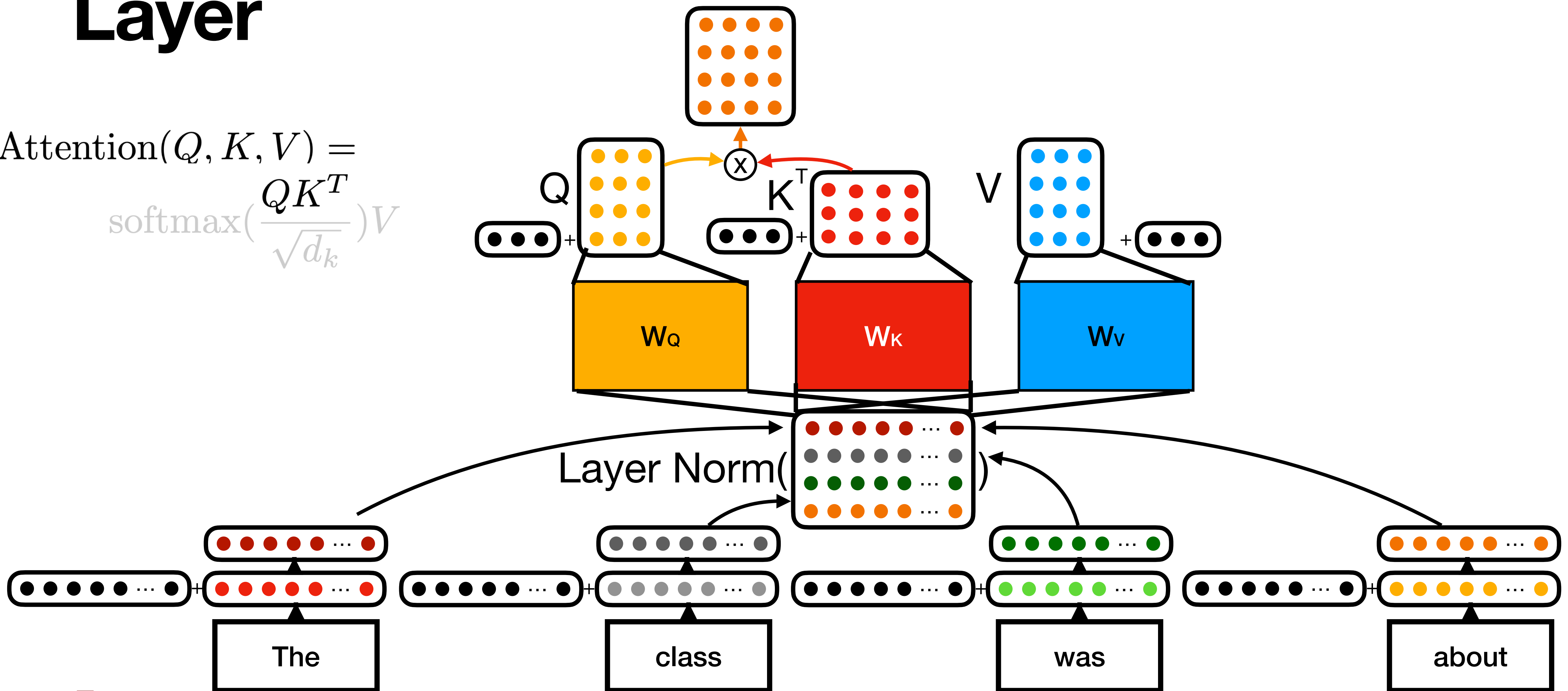


Attention Layer



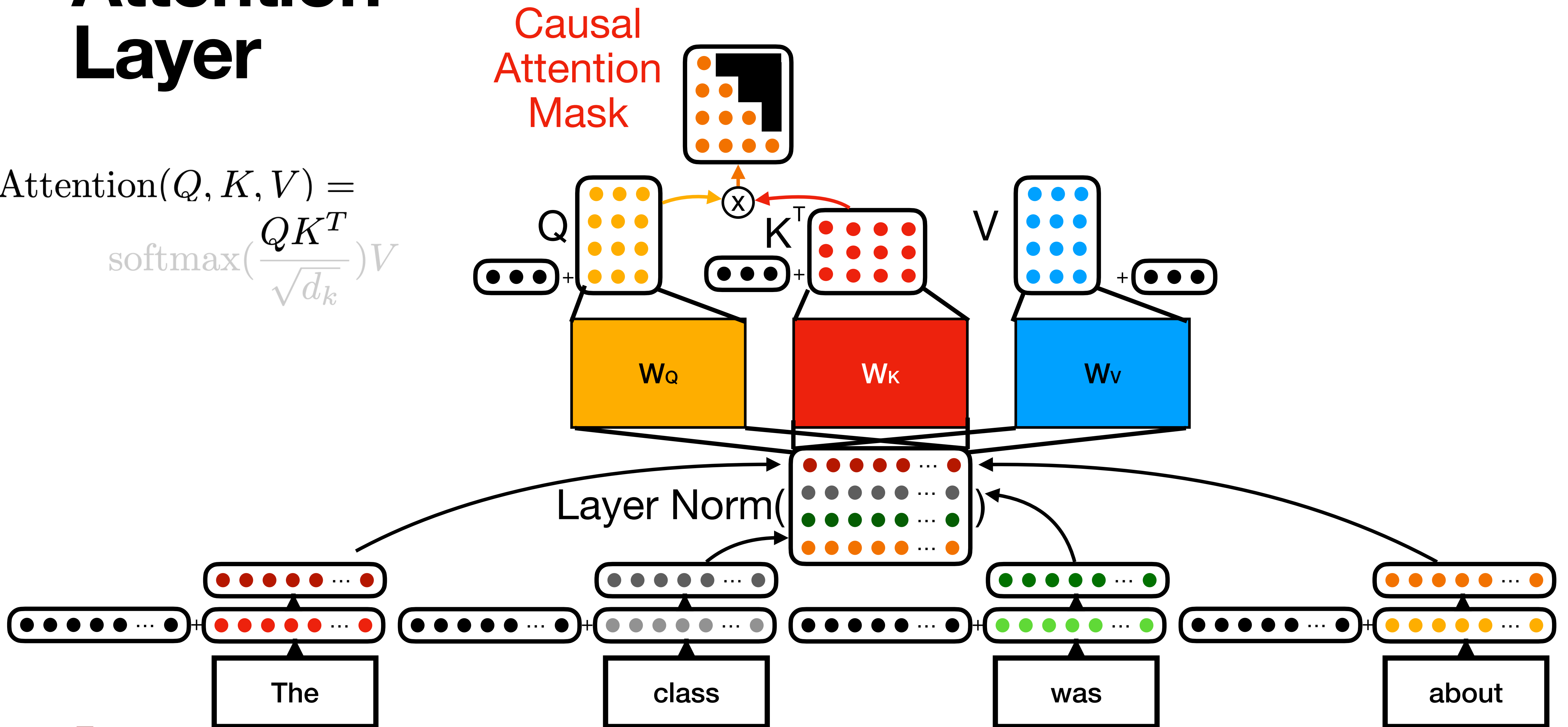
Attention Layer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



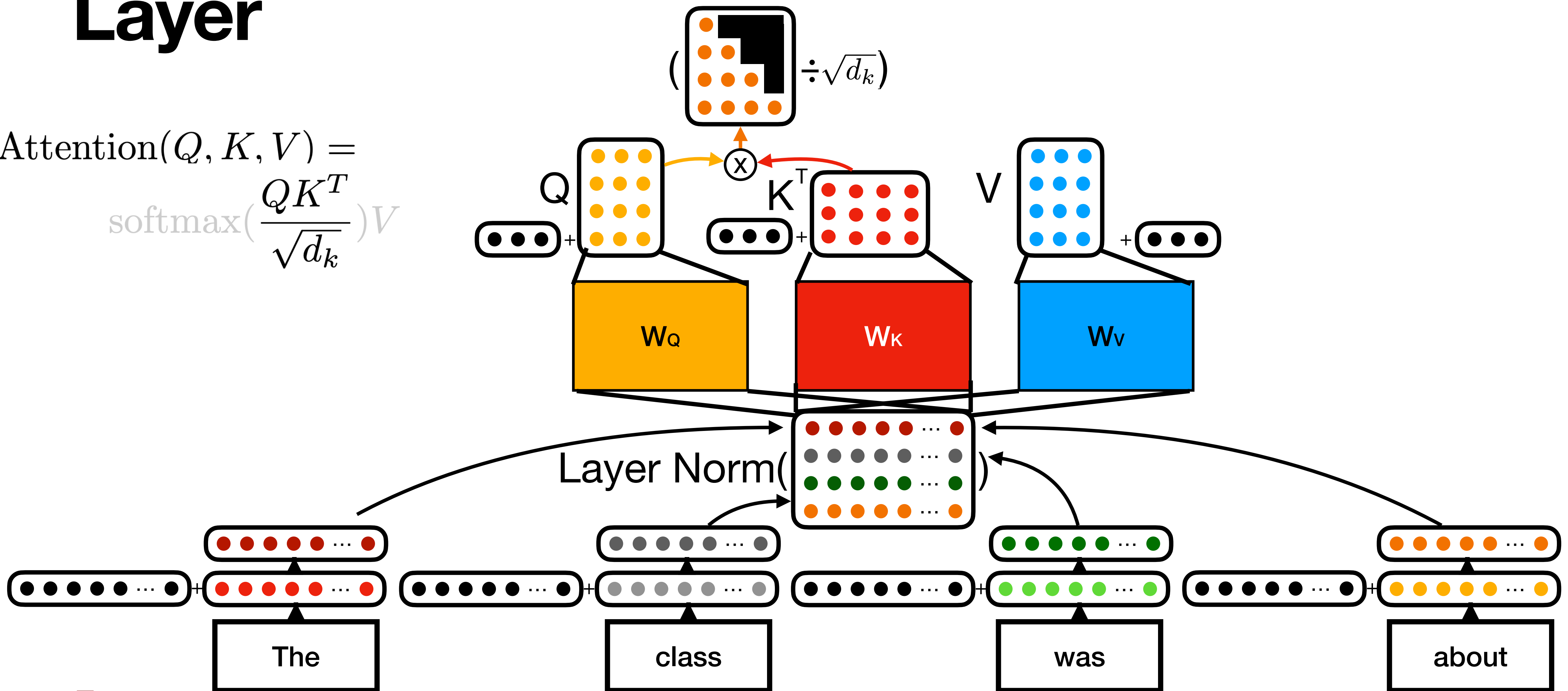
Attention Layer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



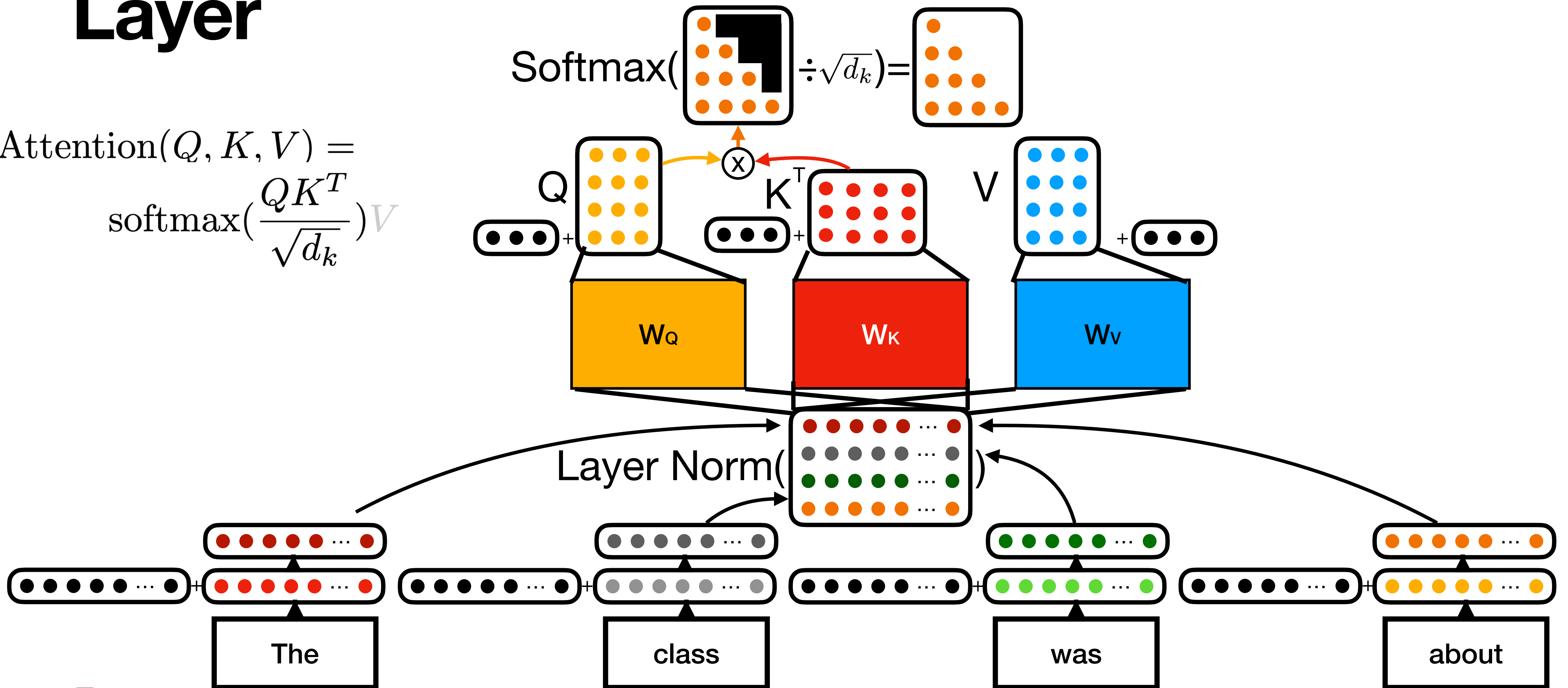
Attention Layer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



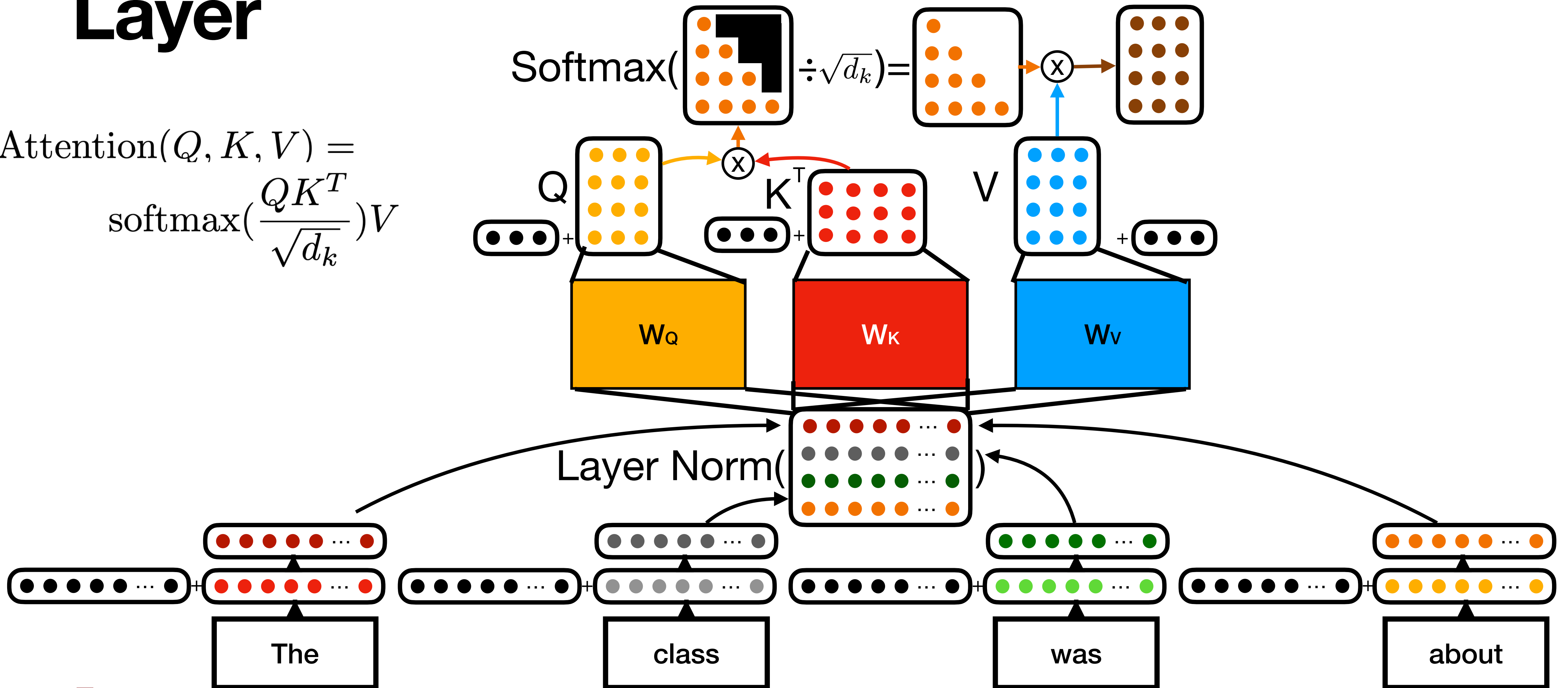
Attention Layer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



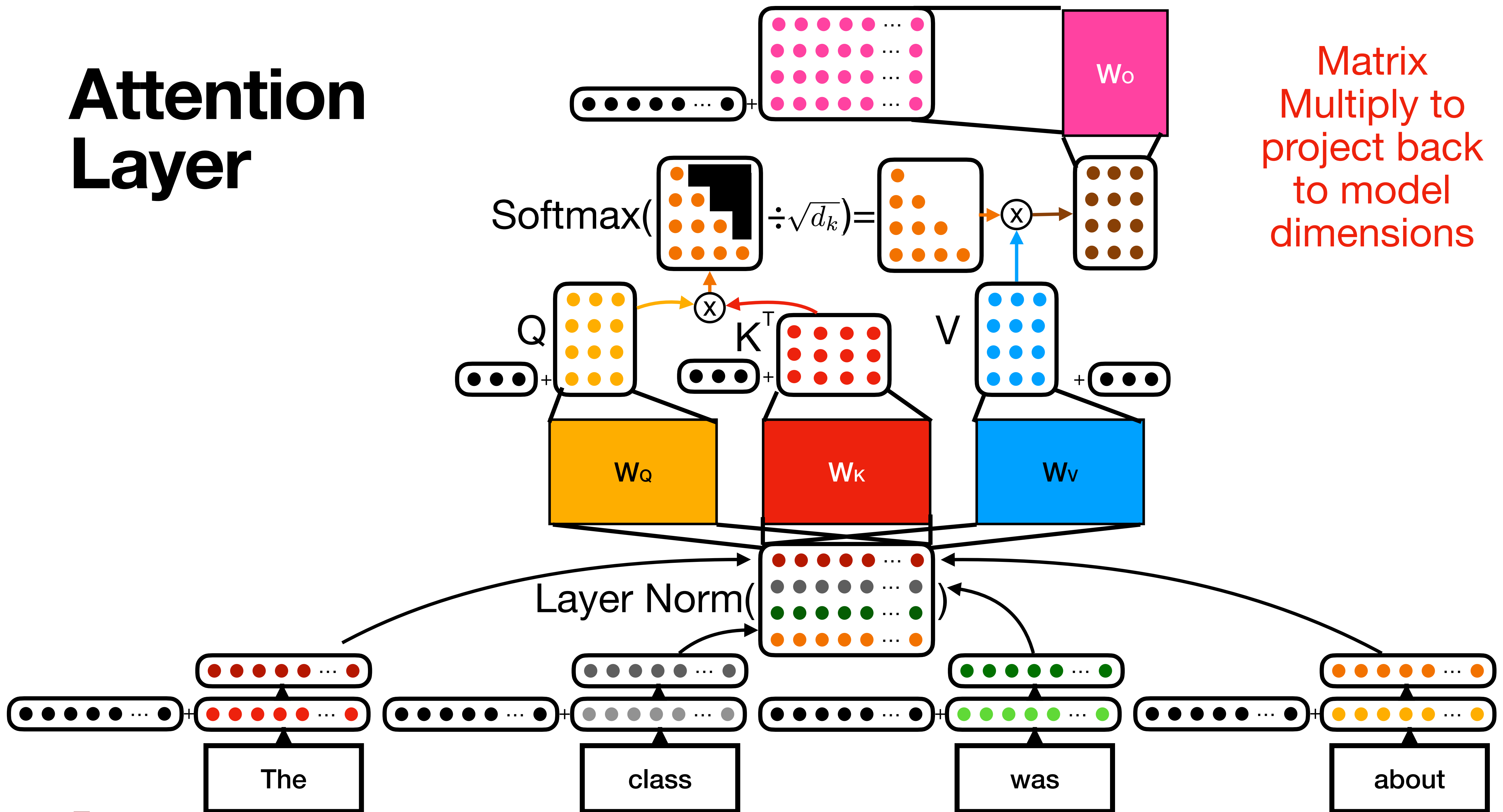
Attention Layer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



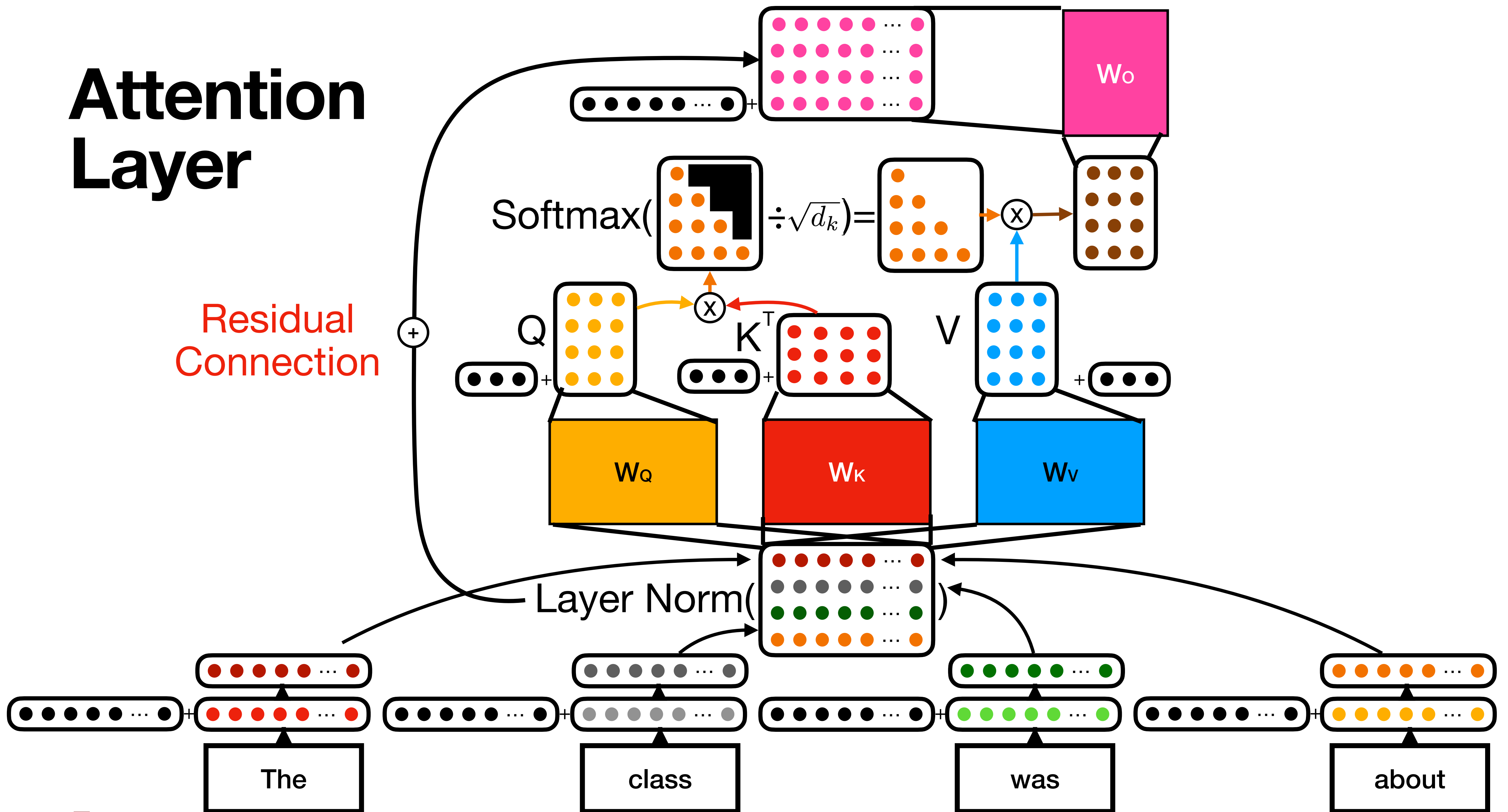
Attention Layer

Matrix
Multiply to
project back
to model
dimensions



Attention Layer

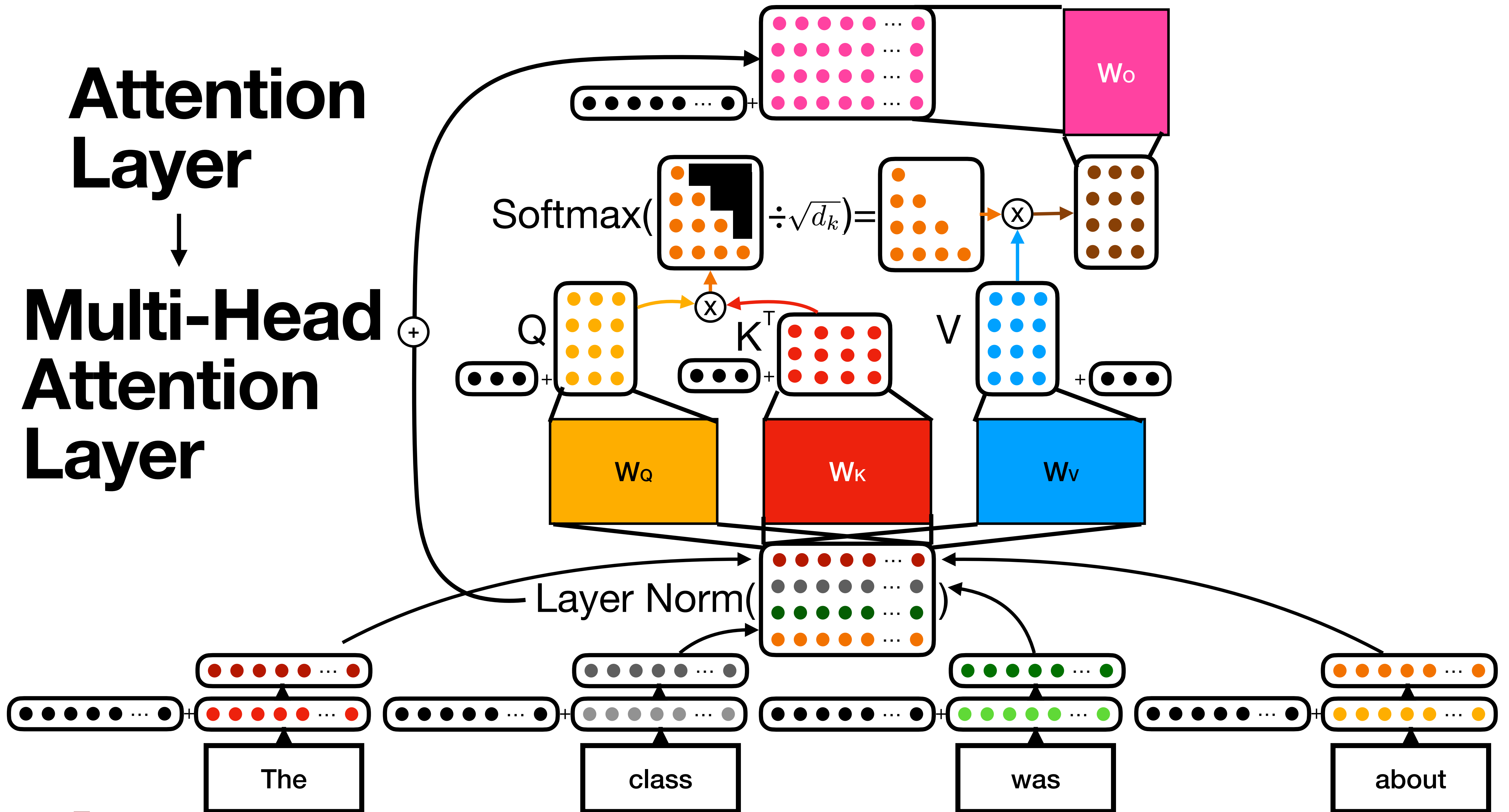
Residual Connection



Attention Layer

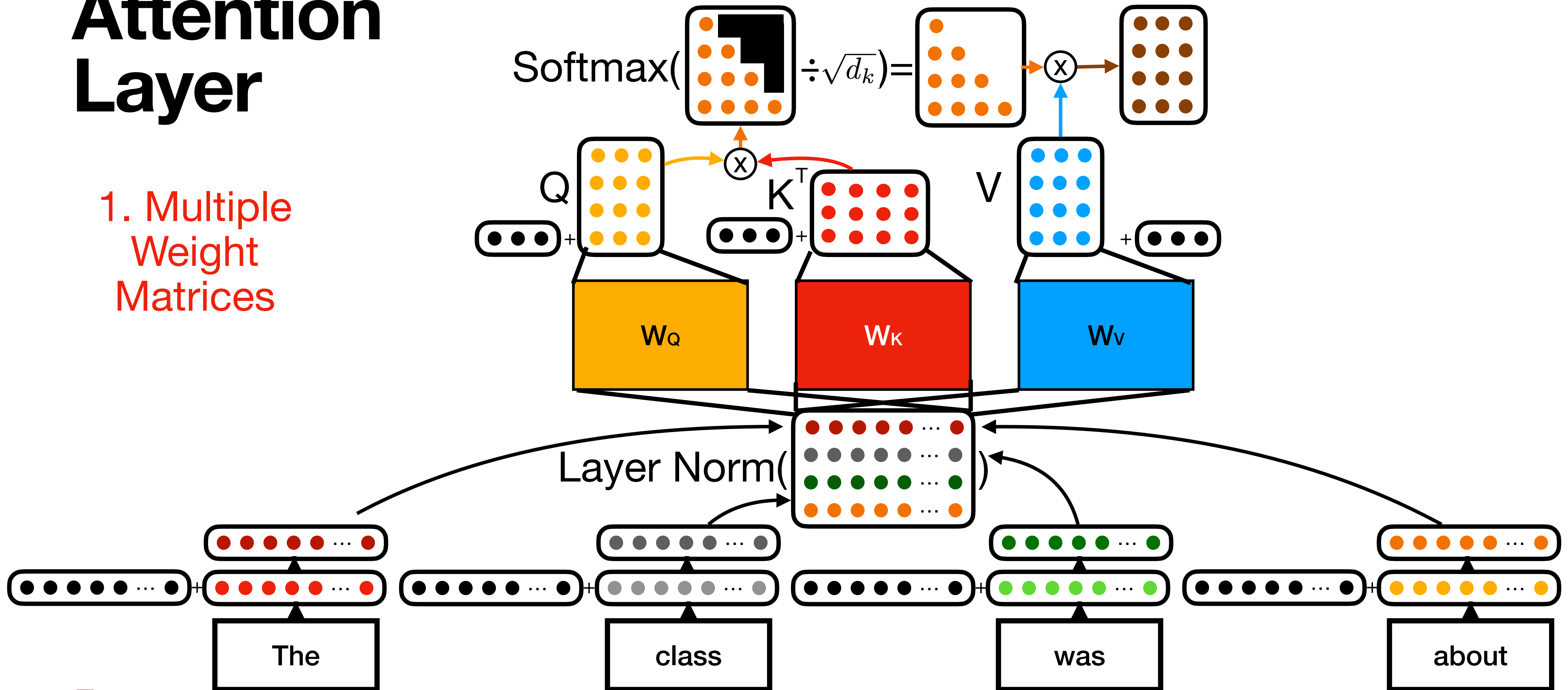


Multi-Head Attention Layer



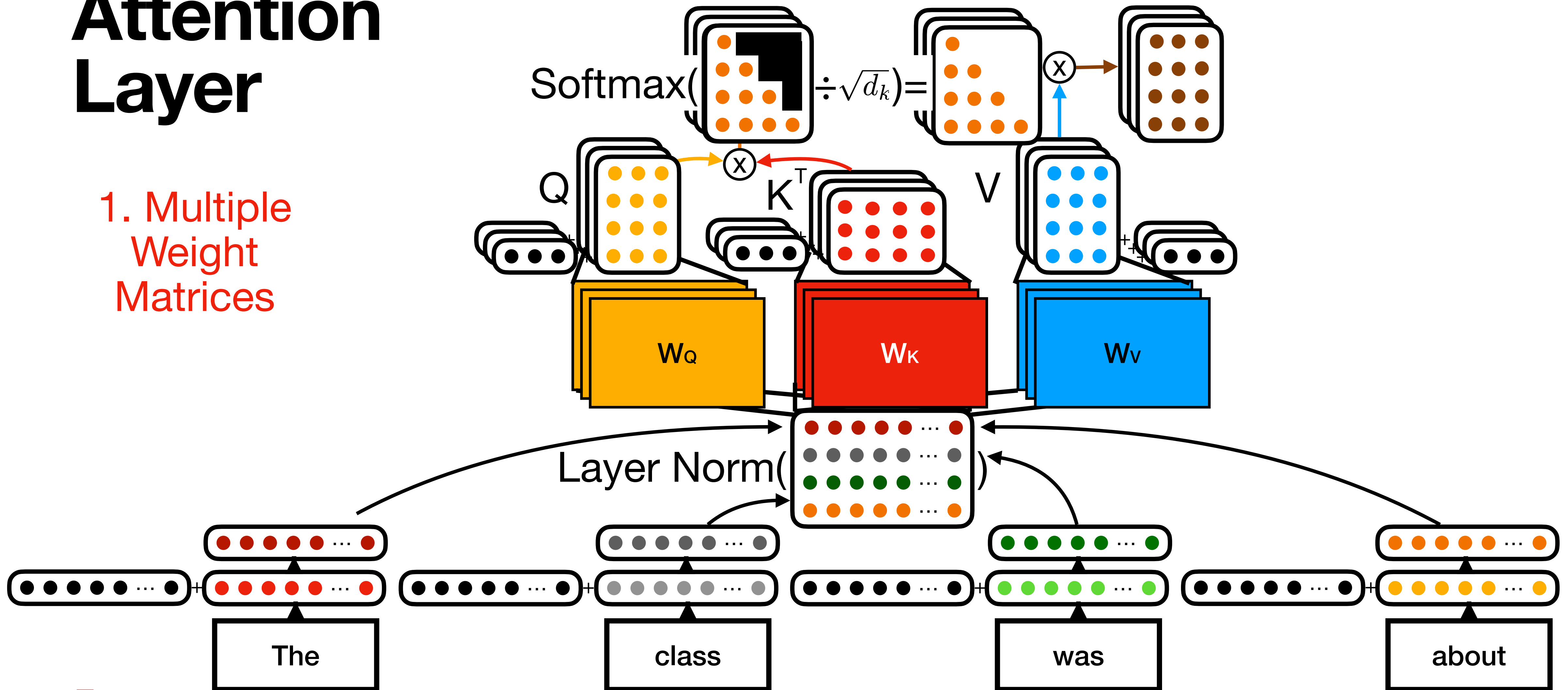
Multi-Head Attention Layer

1. Multiple Weight Matrices



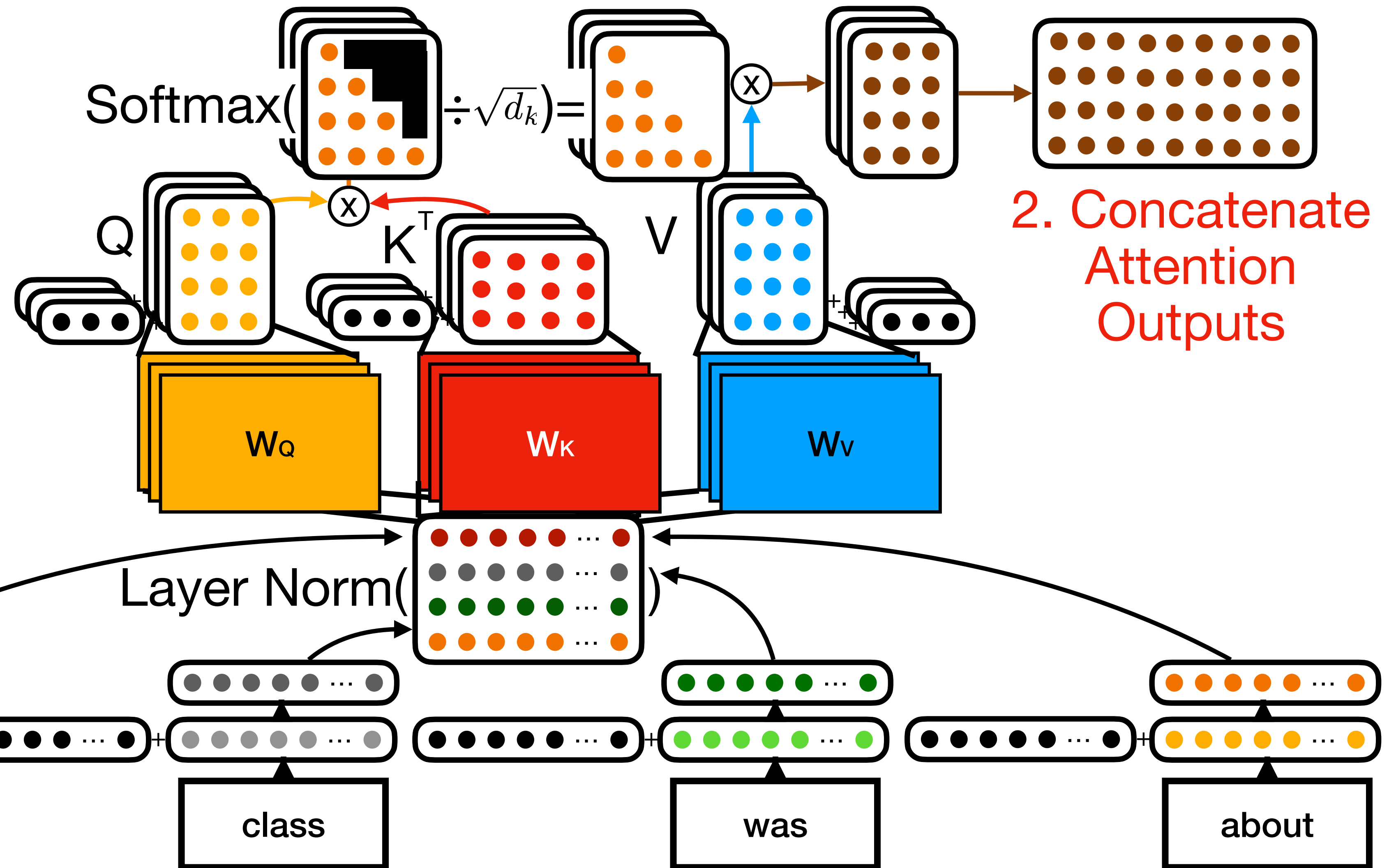
Multi-Head Attention Layer

1. Multiple Weight Matrices



Multi-Head Attention Layer

1. Multiple Weight Matrices

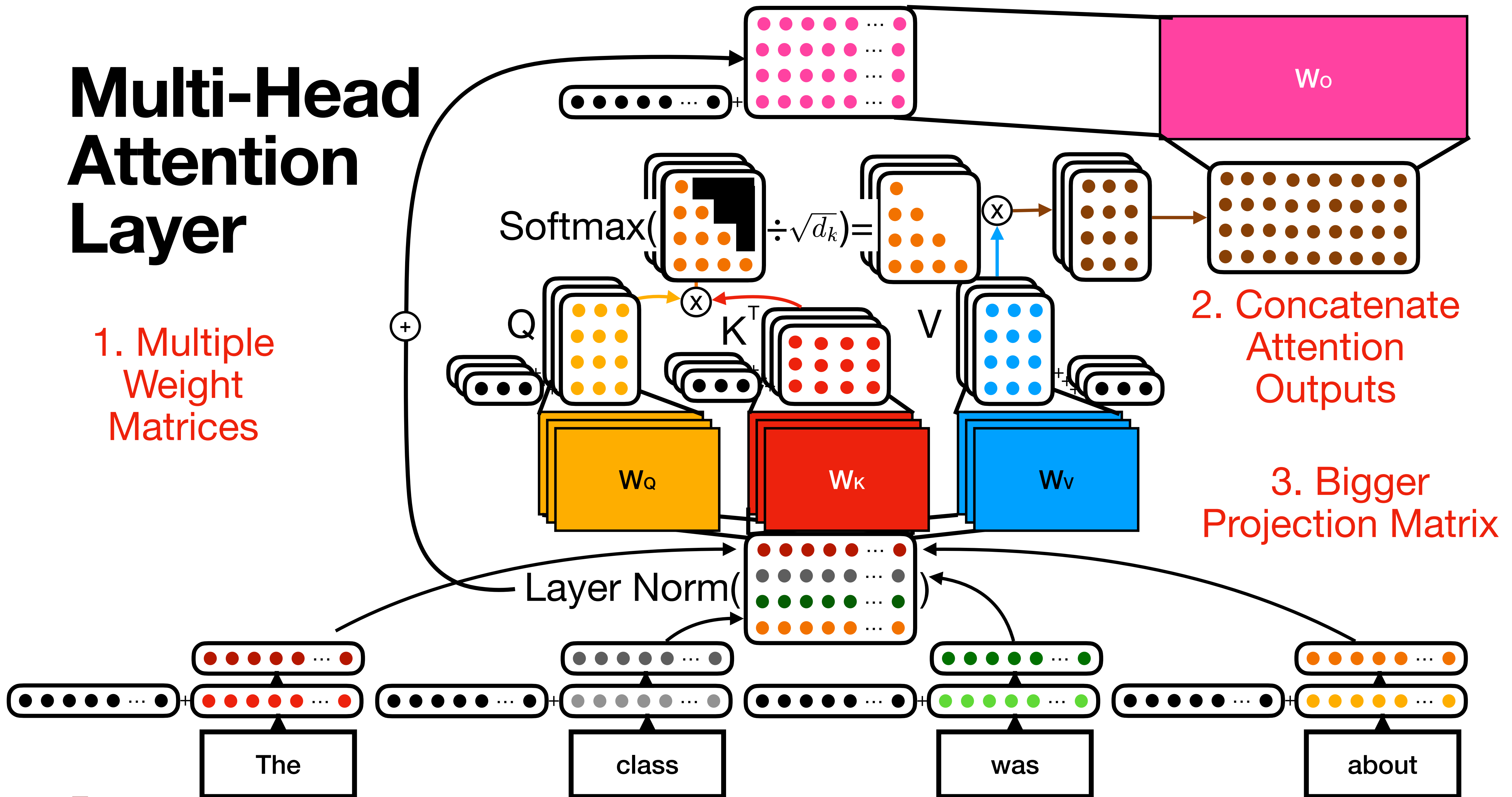


Multi-Head Attention Layer

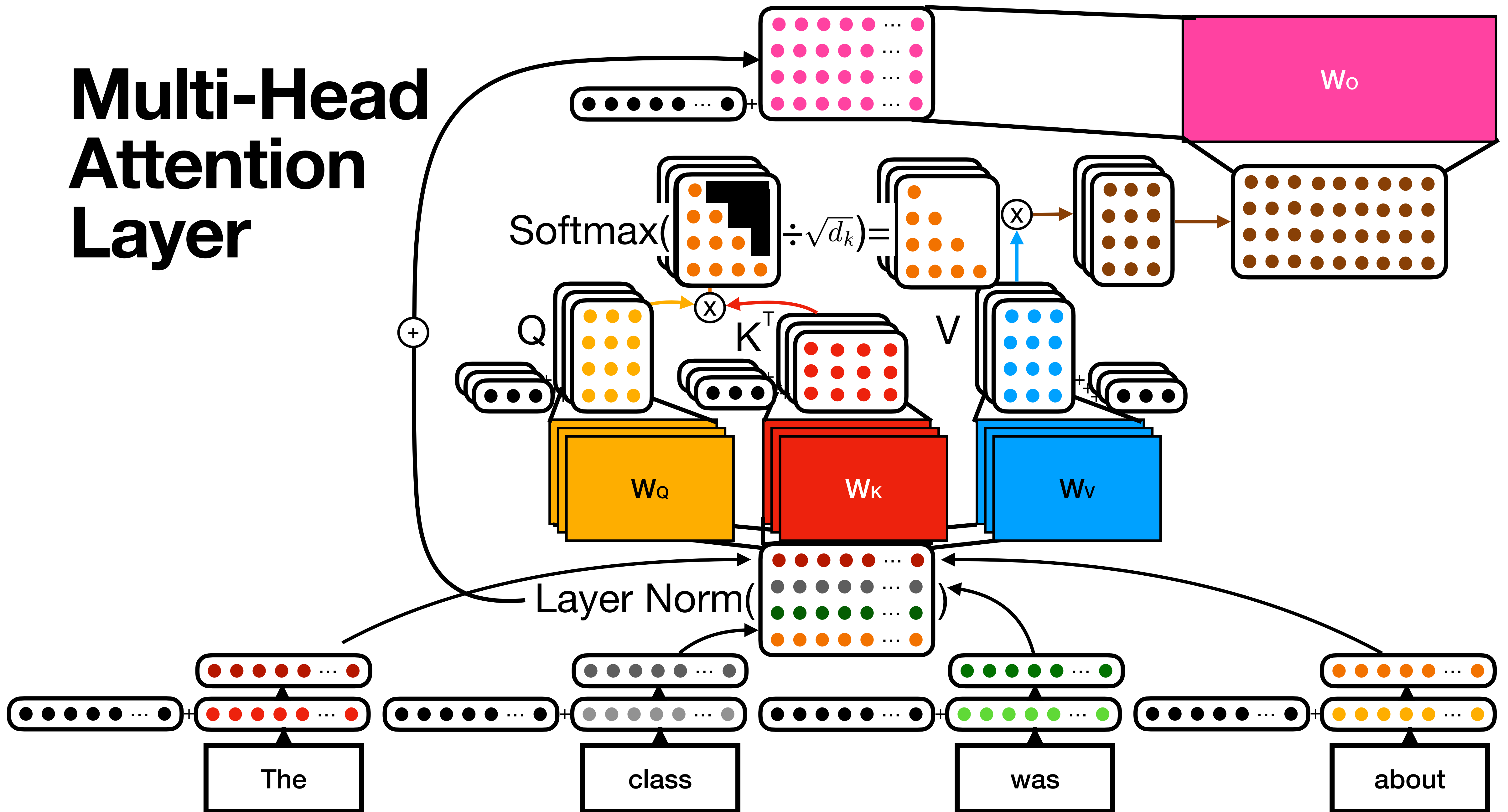
1. Multiple Weight Matrices

2. Concatenate Attention Outputs

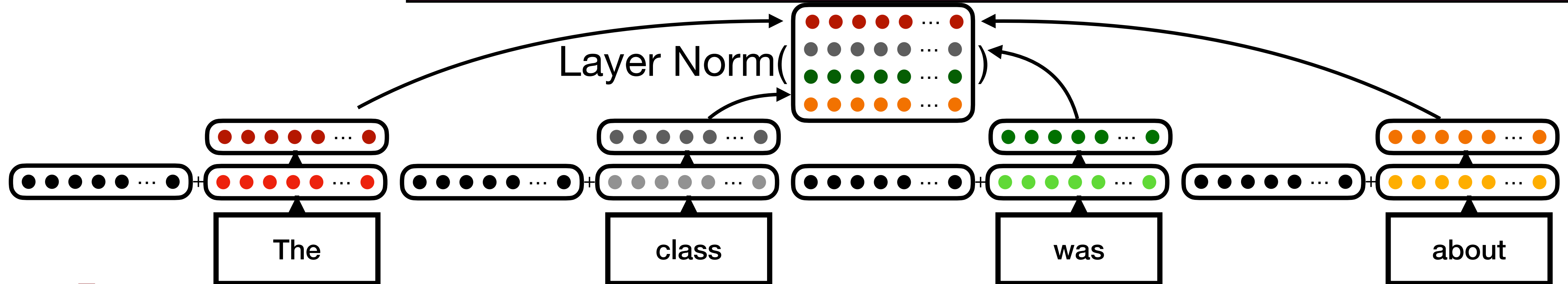
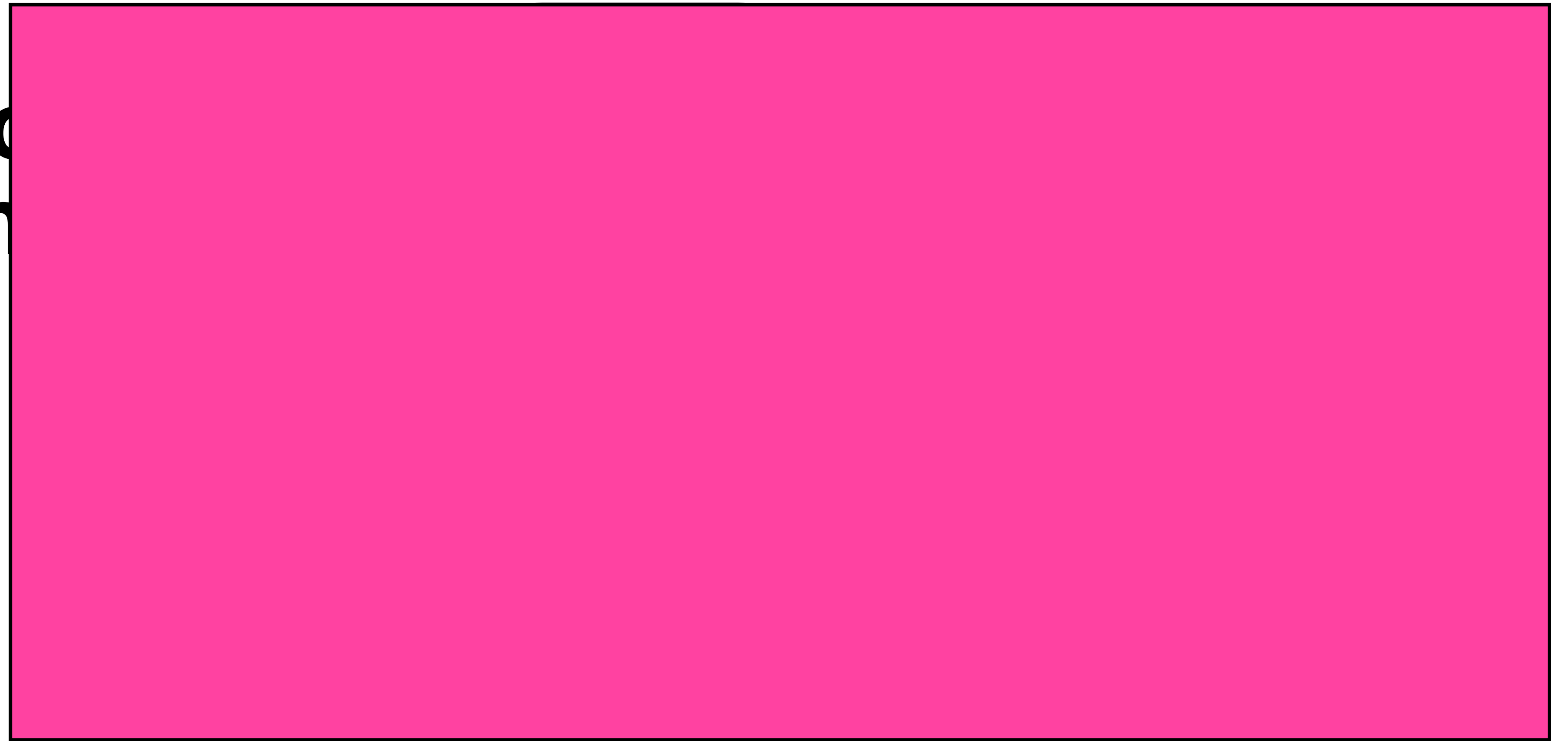
3. Bigger Projection Matrix



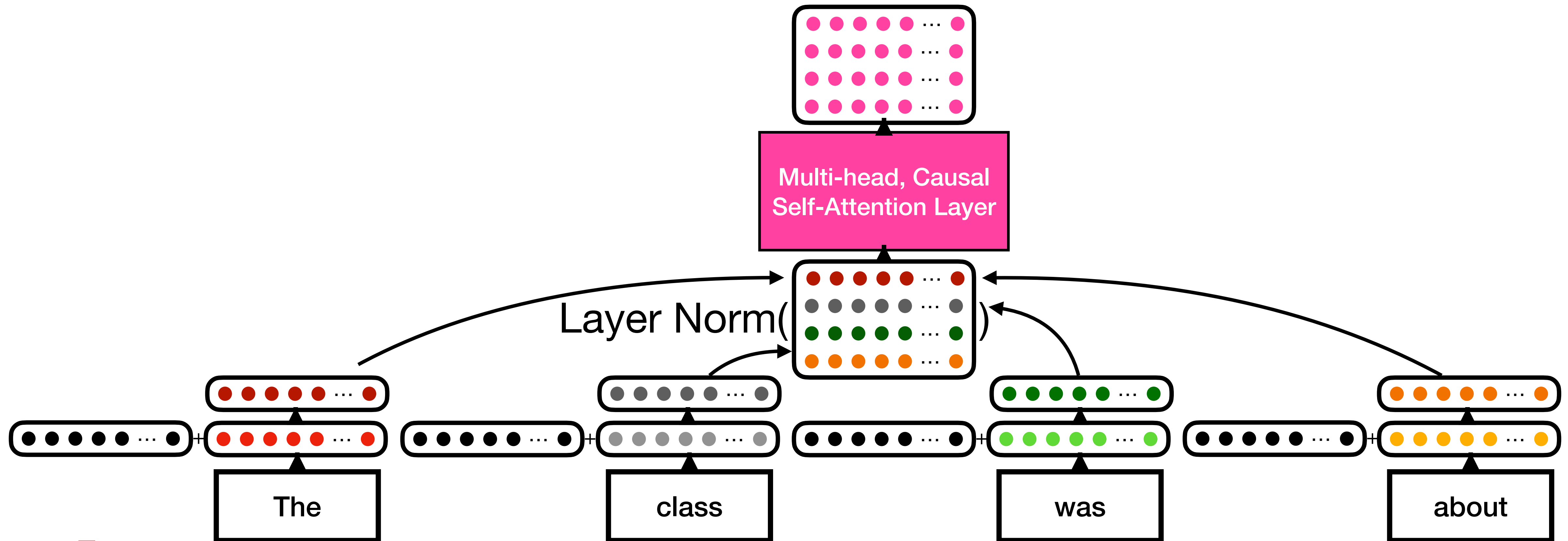
Multi-Head Attention Layer



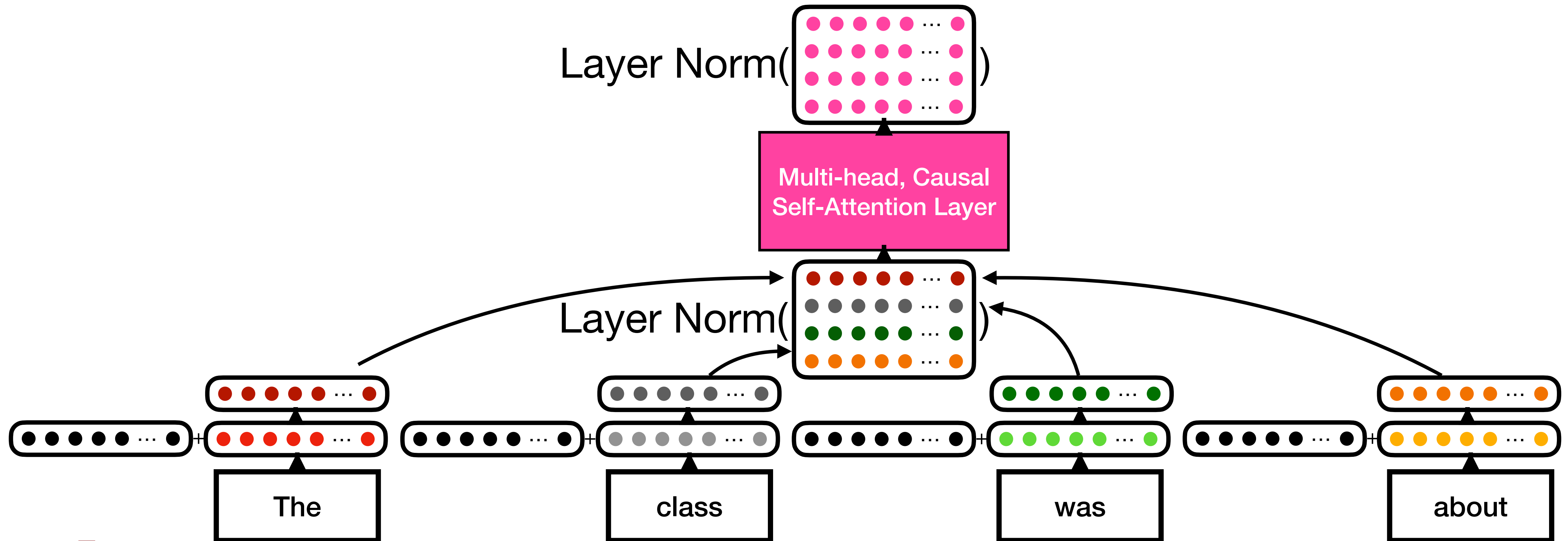
Decoder- Transform



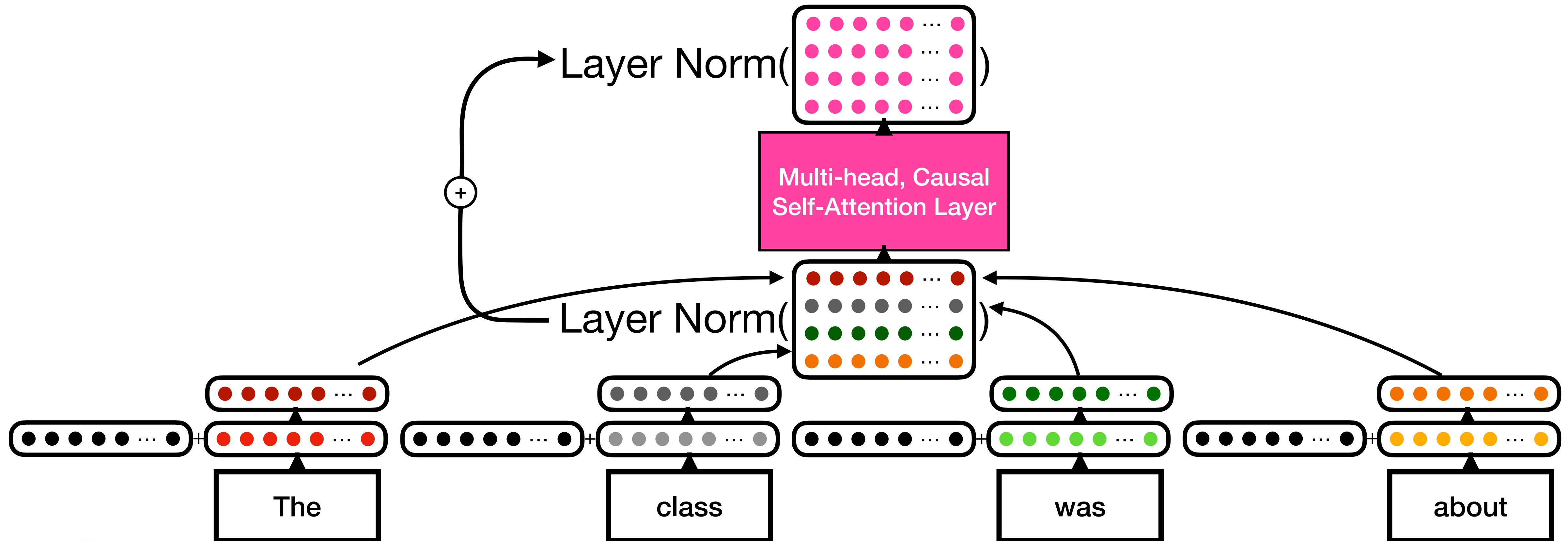
Decoder-only Transformer



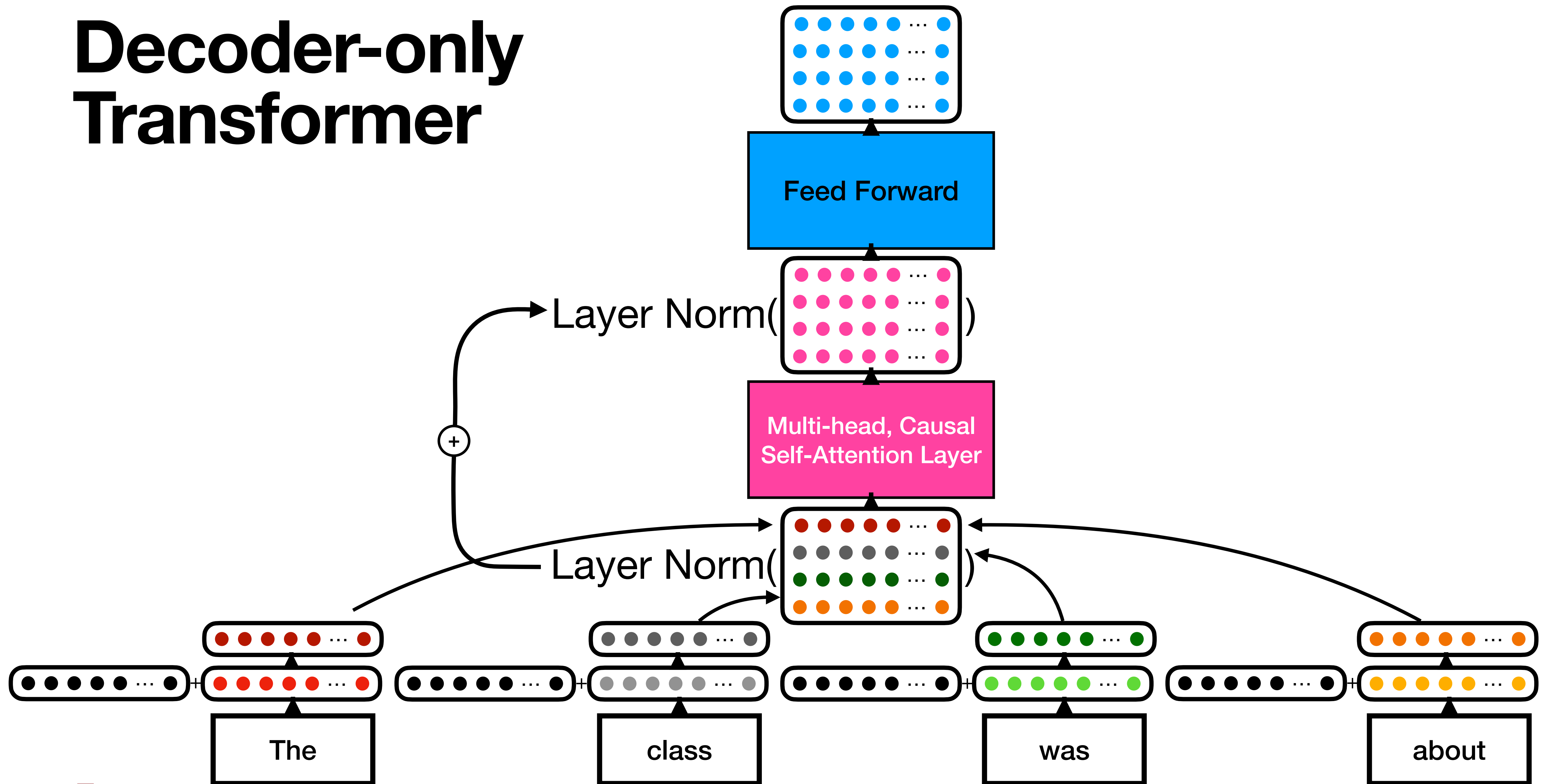
Decoder-only Transformer



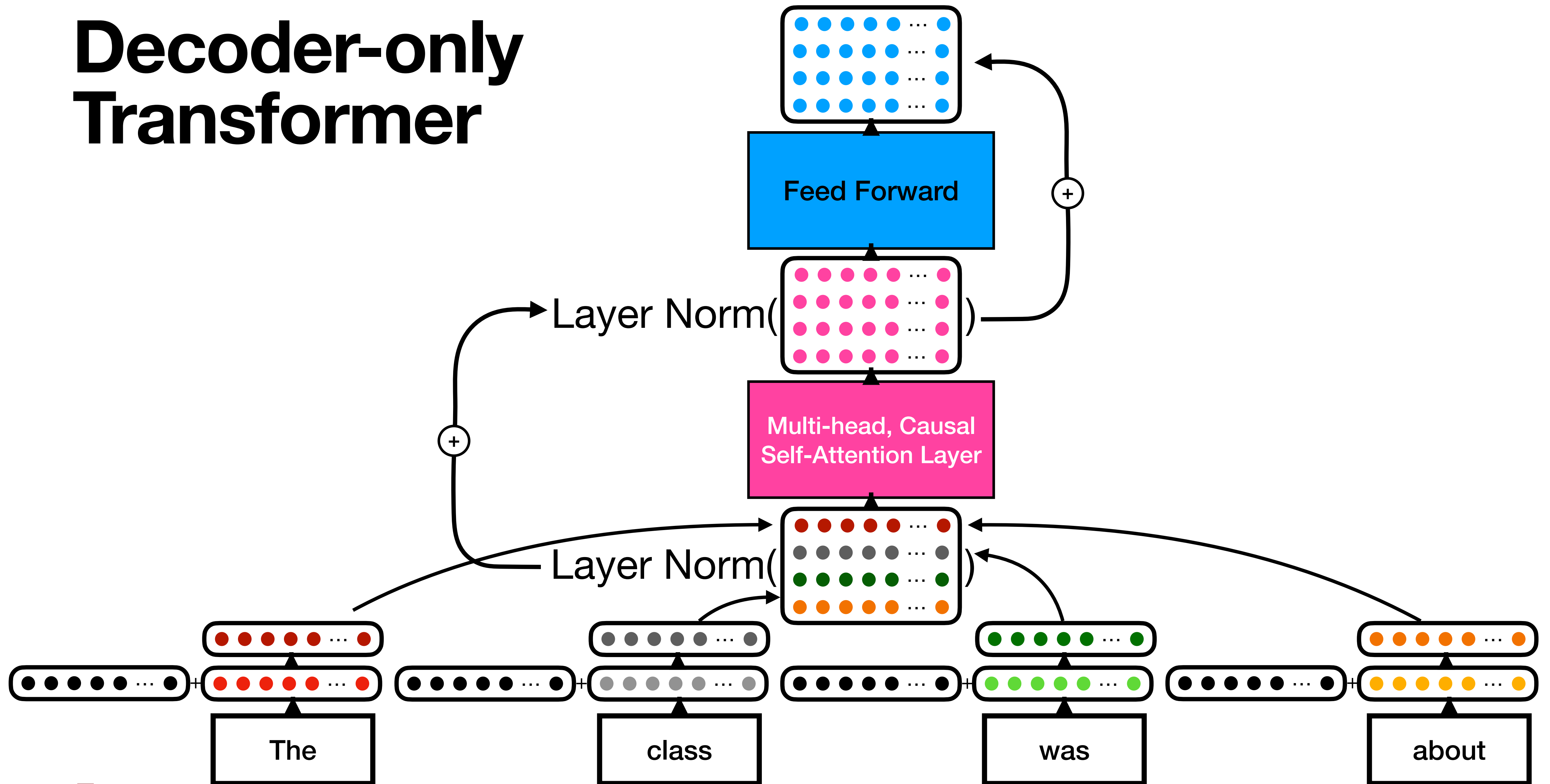
Decoder-only Transformer



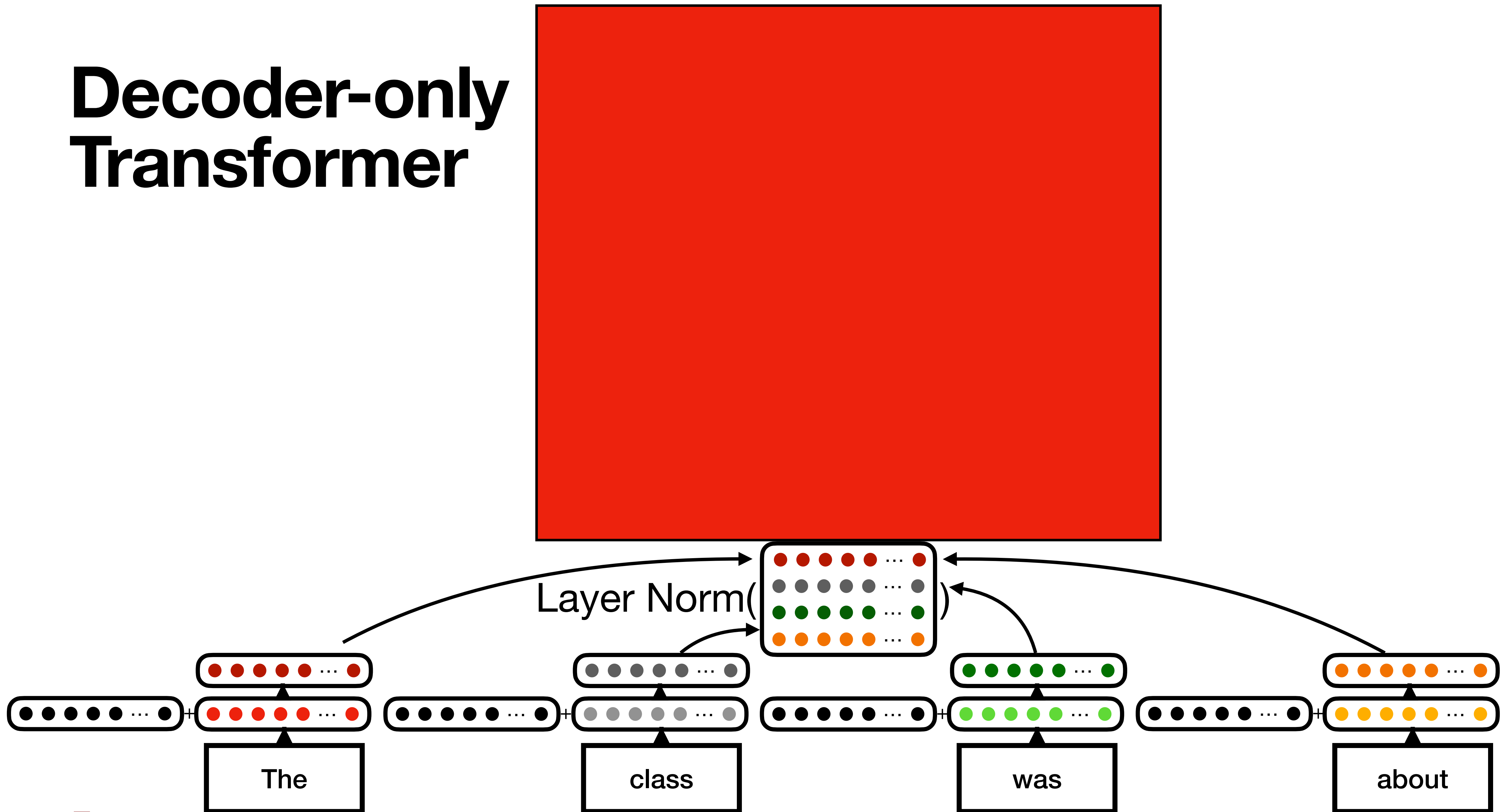
Decoder-only Transformer



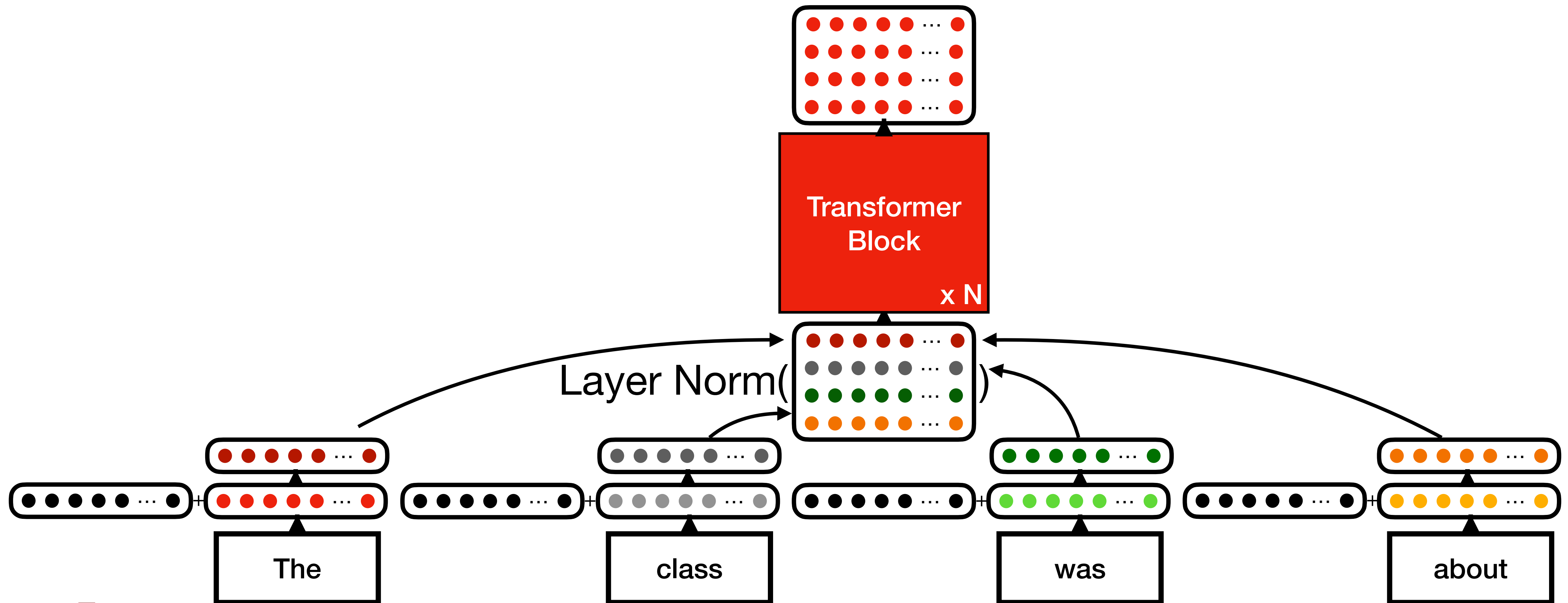
Decoder-only Transformer



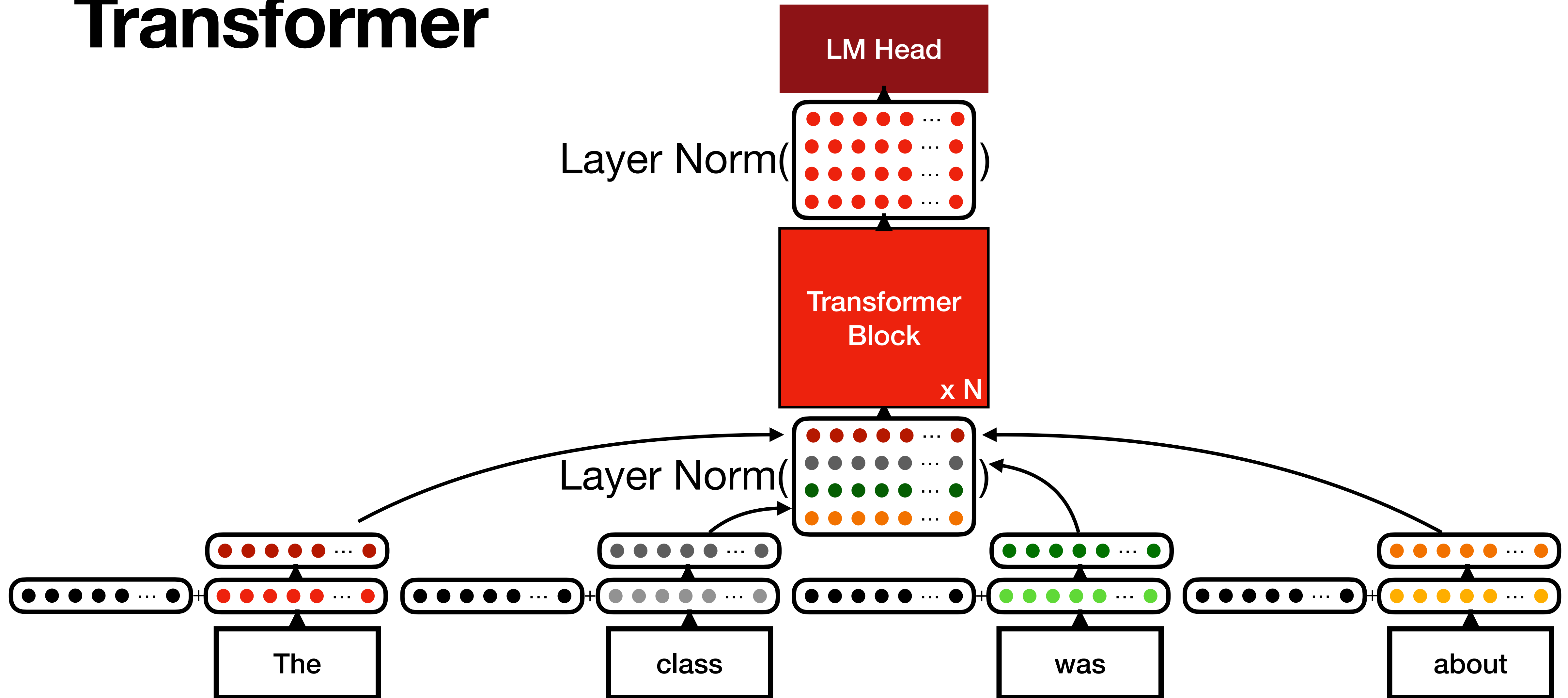
Decoder-only Transformer



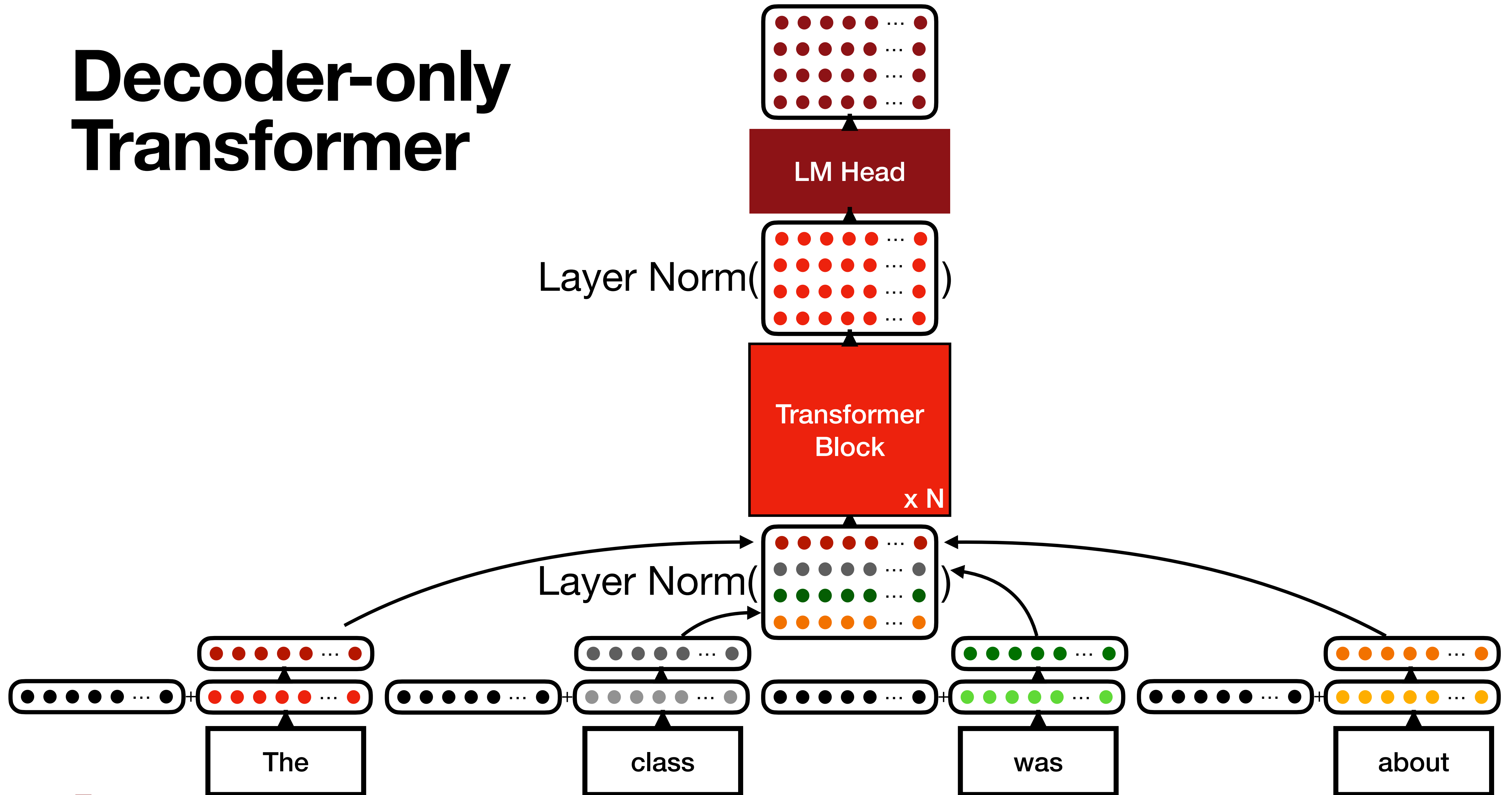
Decoder-only Transformer



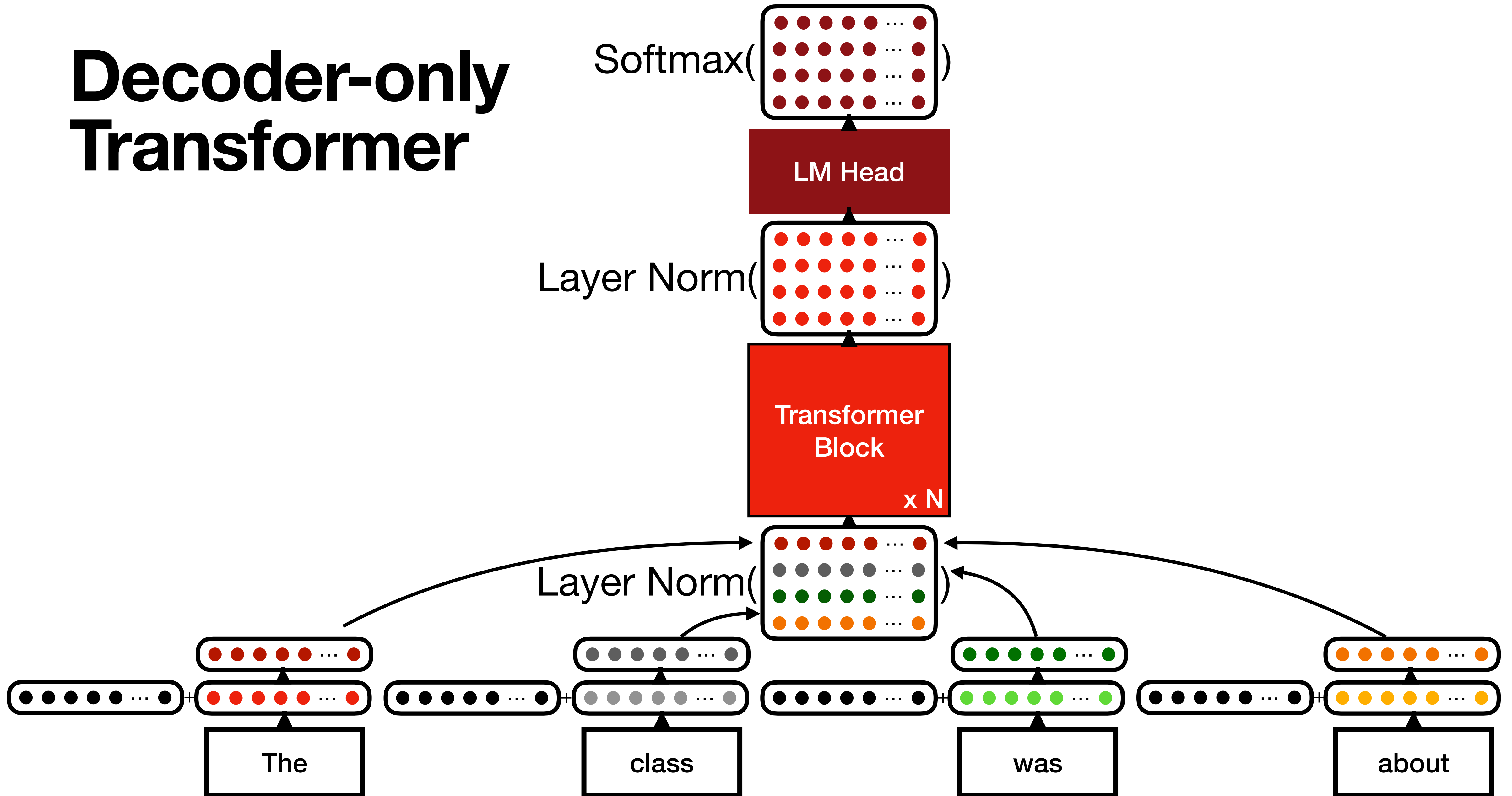
Decoder-only Transformer

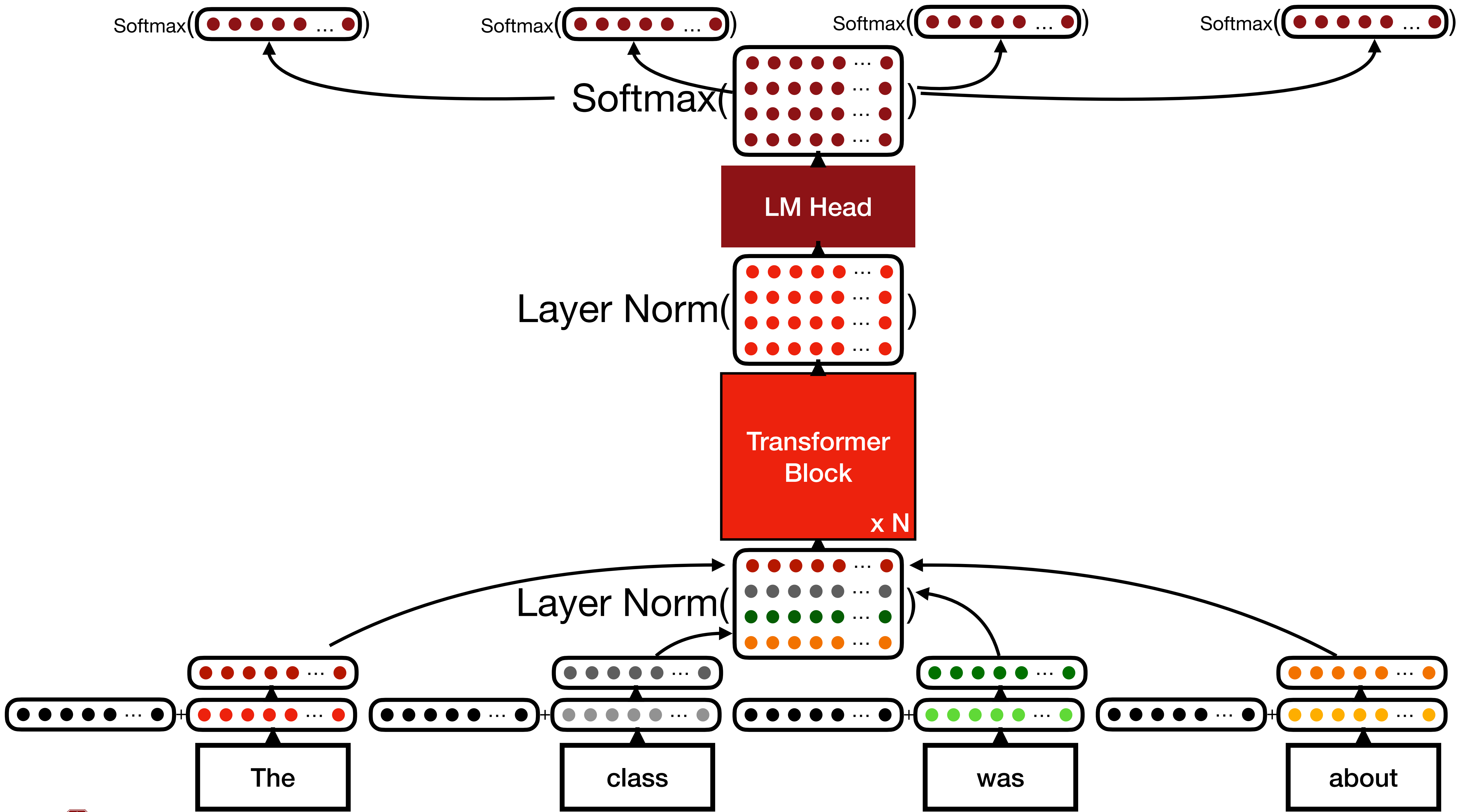


Decoder-only Transformer



Decoder-only Transformer





Attention vs Recurrence

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

$O(1)$ Sequential Operations is a huge deal for parallel training on GPUs!

$O(n^2)$ Sequence length means long context becomes expensive.



Transformers were better and cheaper

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

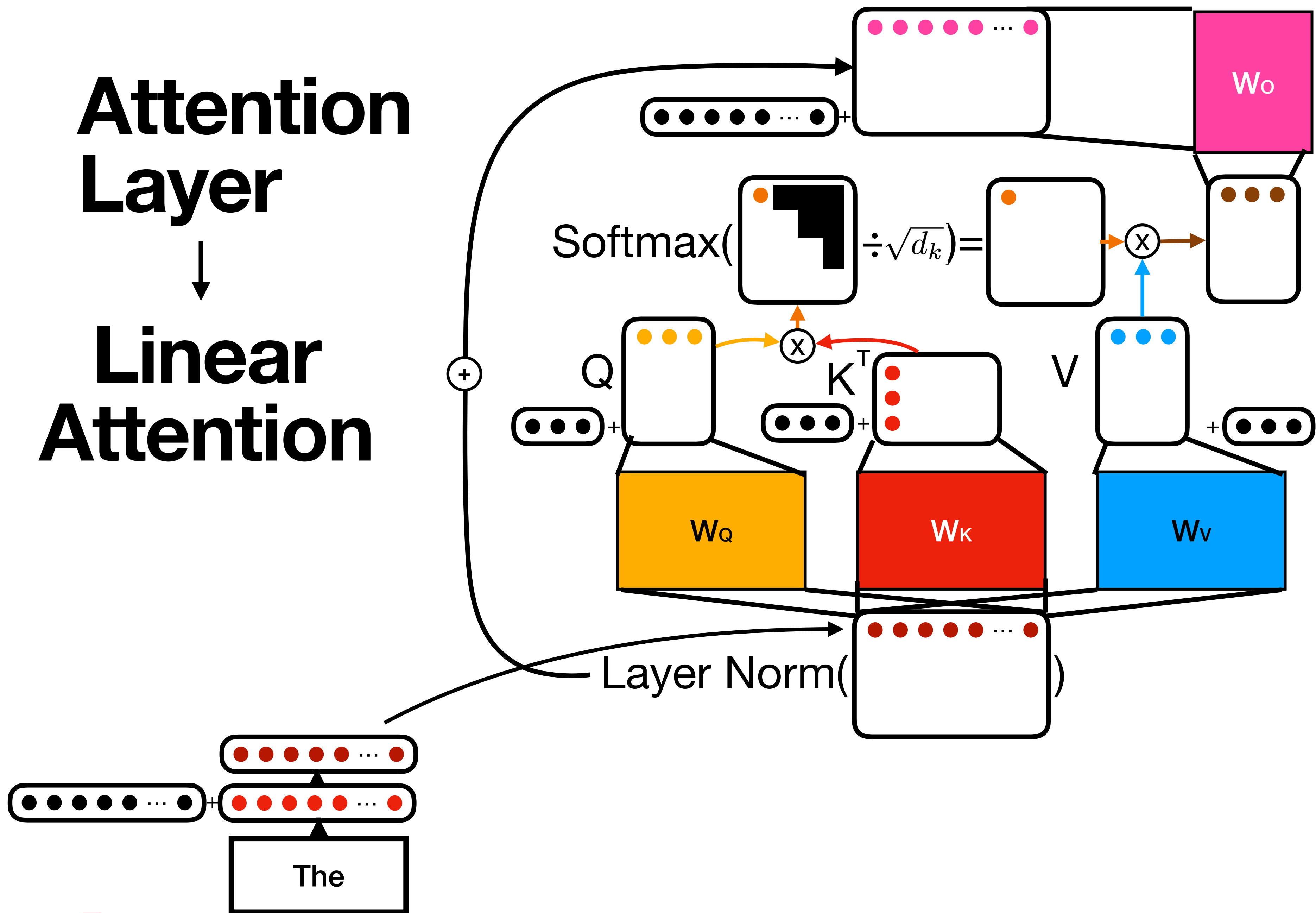


Linear Attention

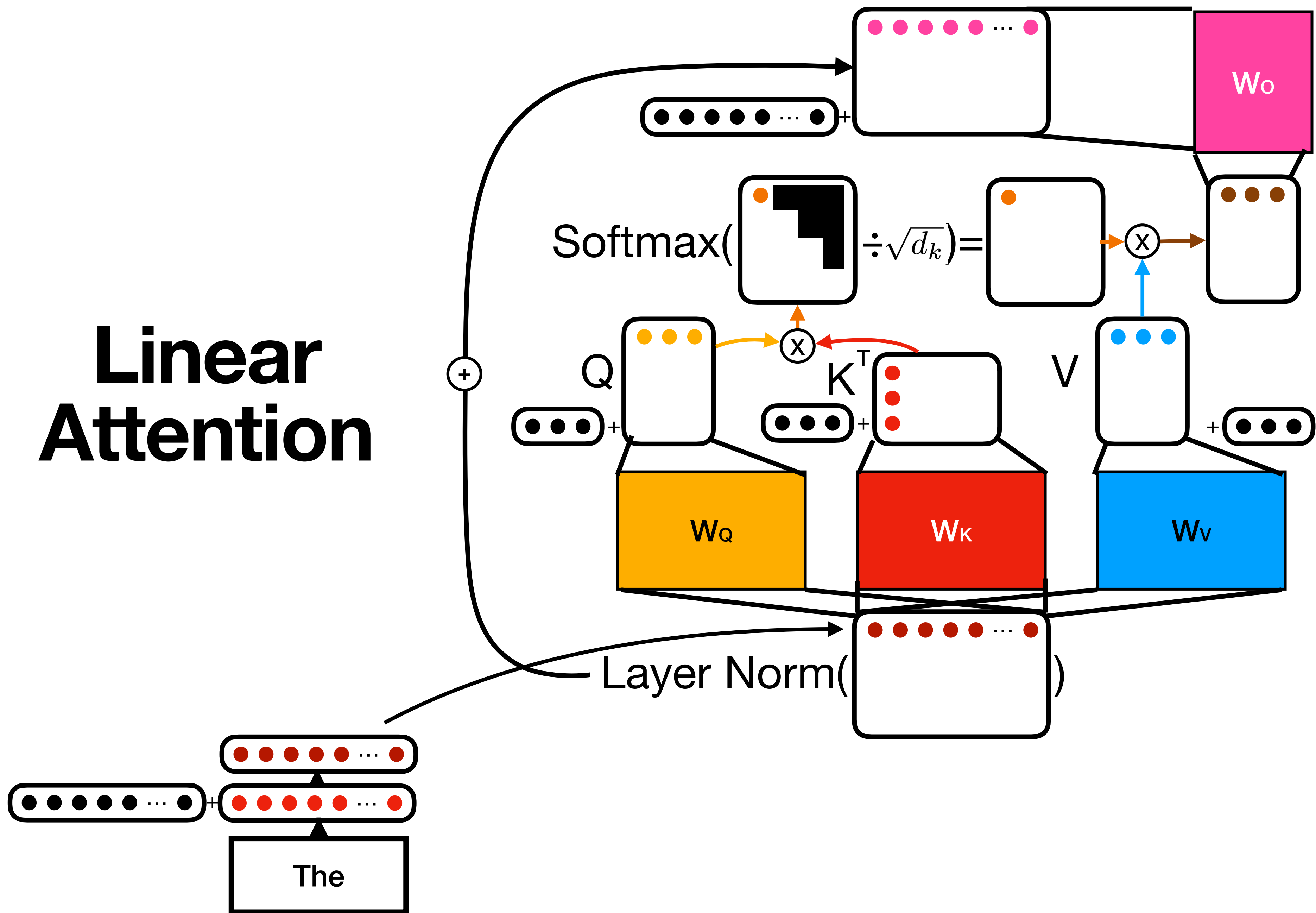


Attention Layer

Linear Attention

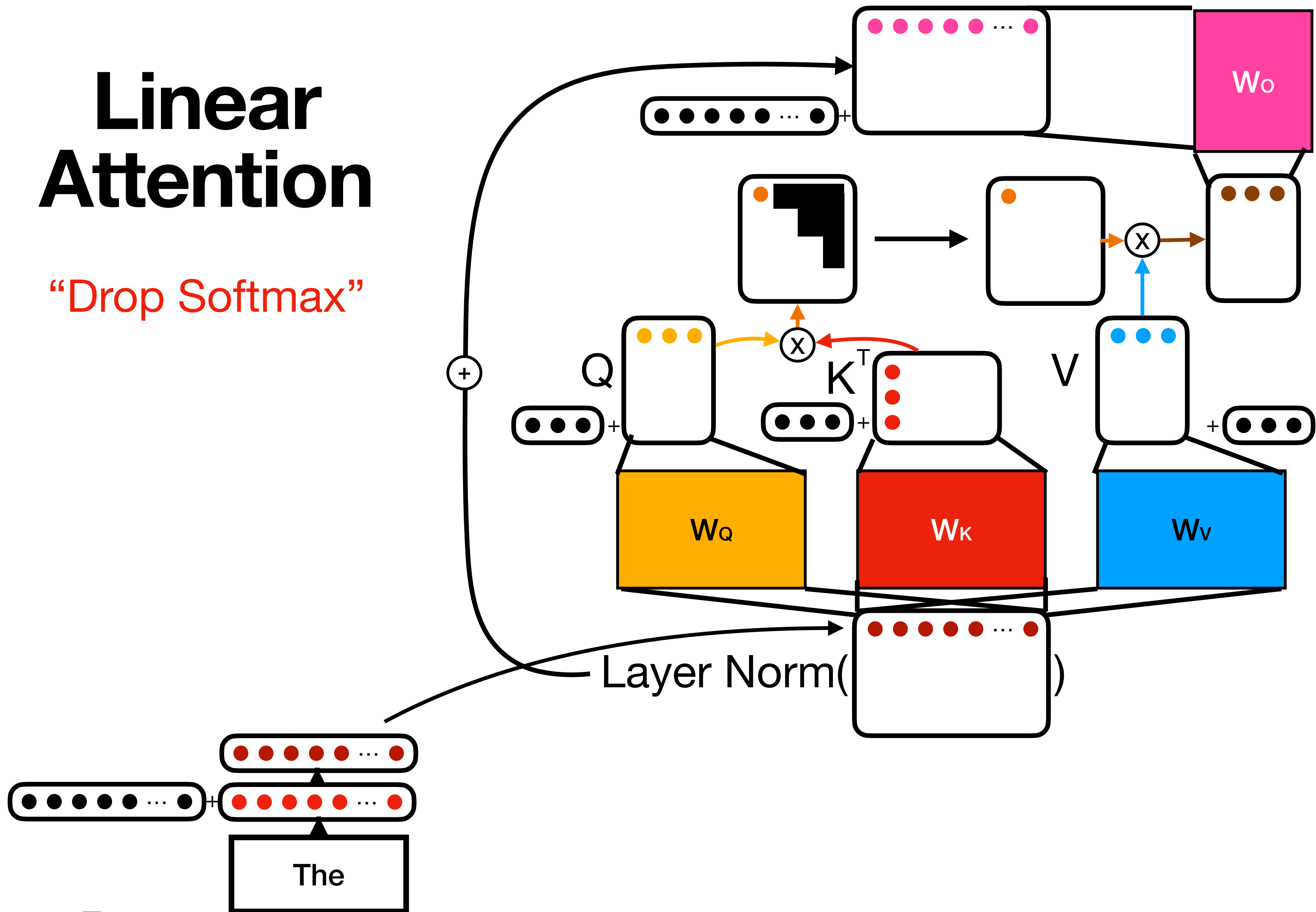


Linear Attention



Linear Attention

“Drop Softmax”



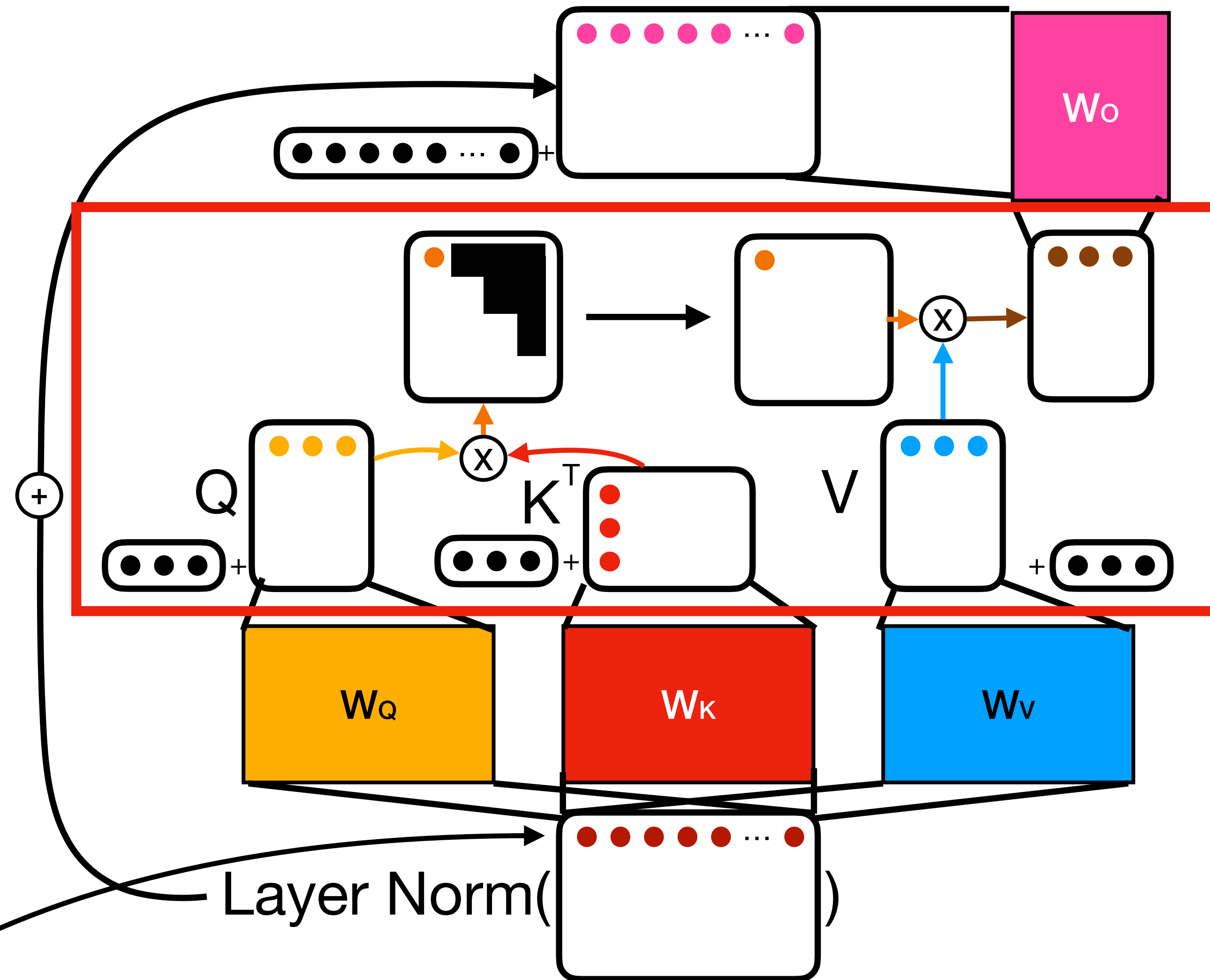
Linear Attention

“Drop Softmax”

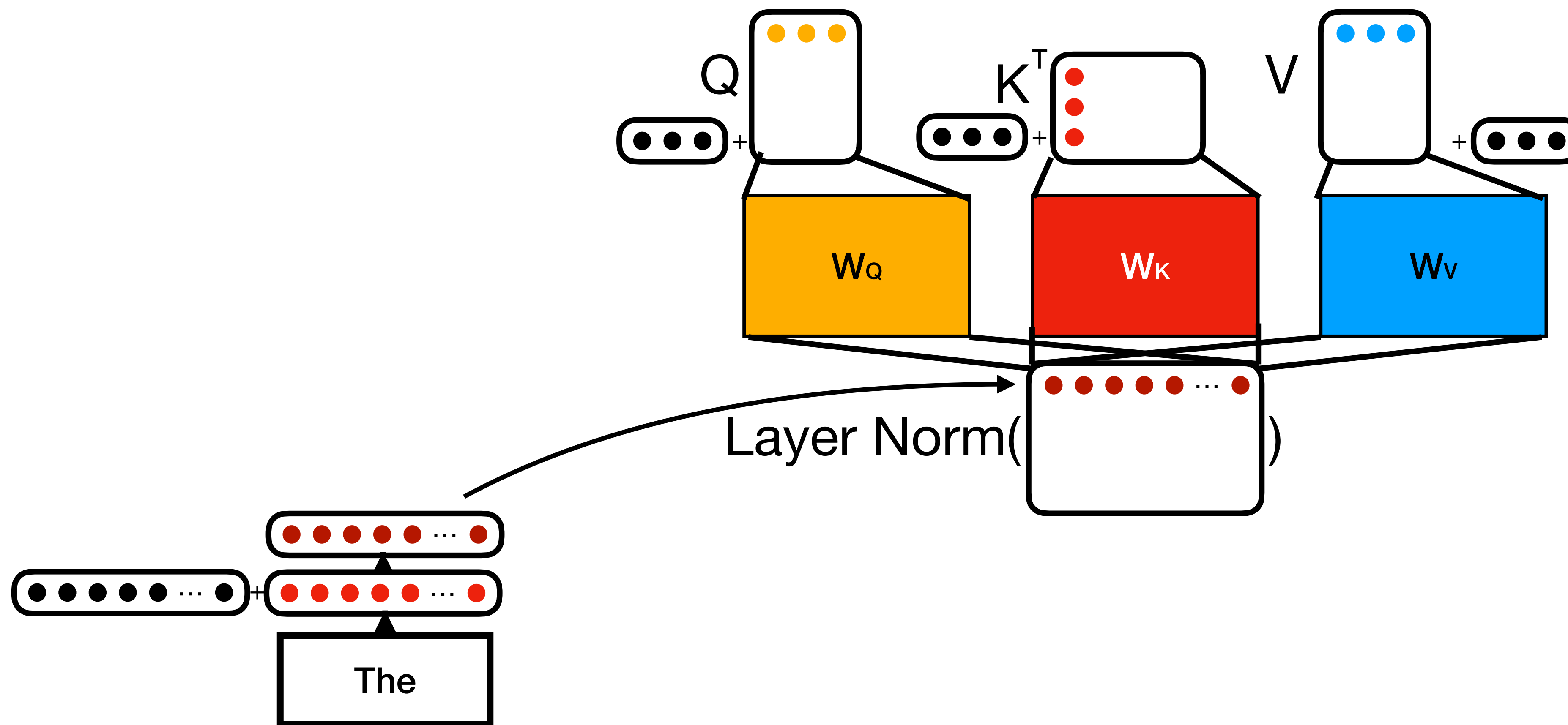
Everything here is now linear!

We can change the associativity.

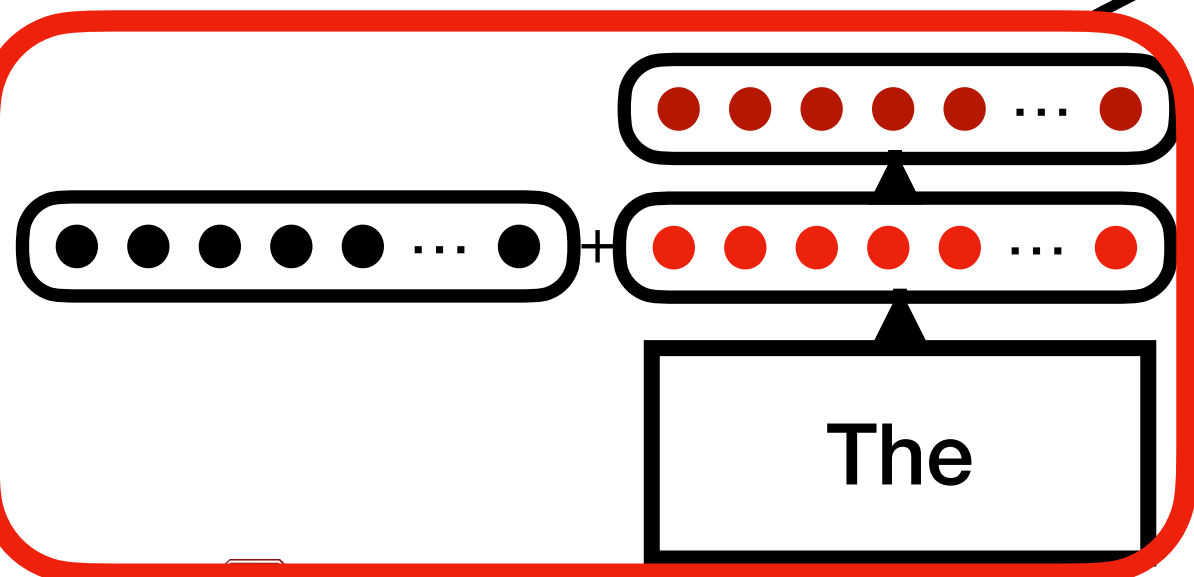
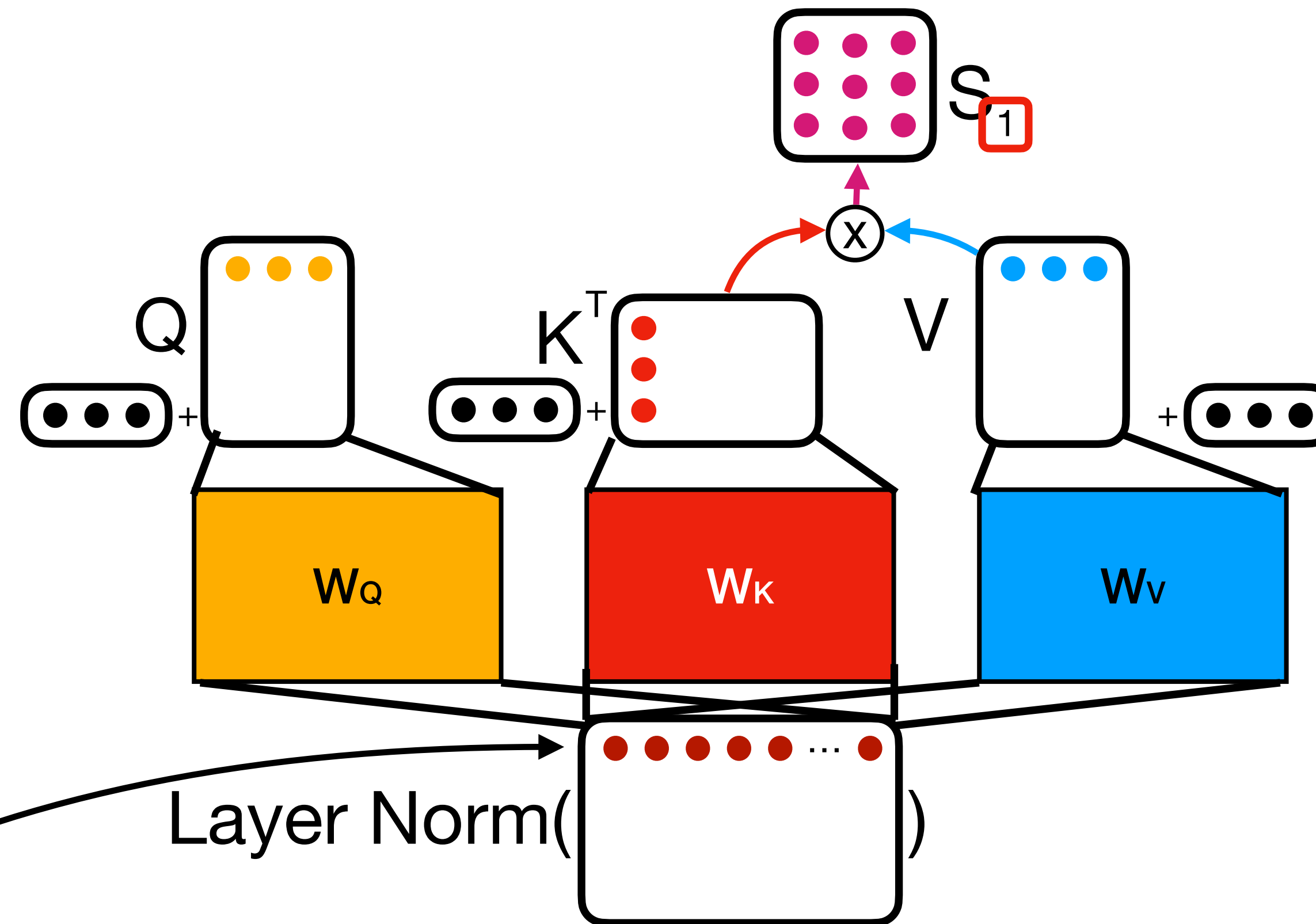
Let's Multiply K and V instead.



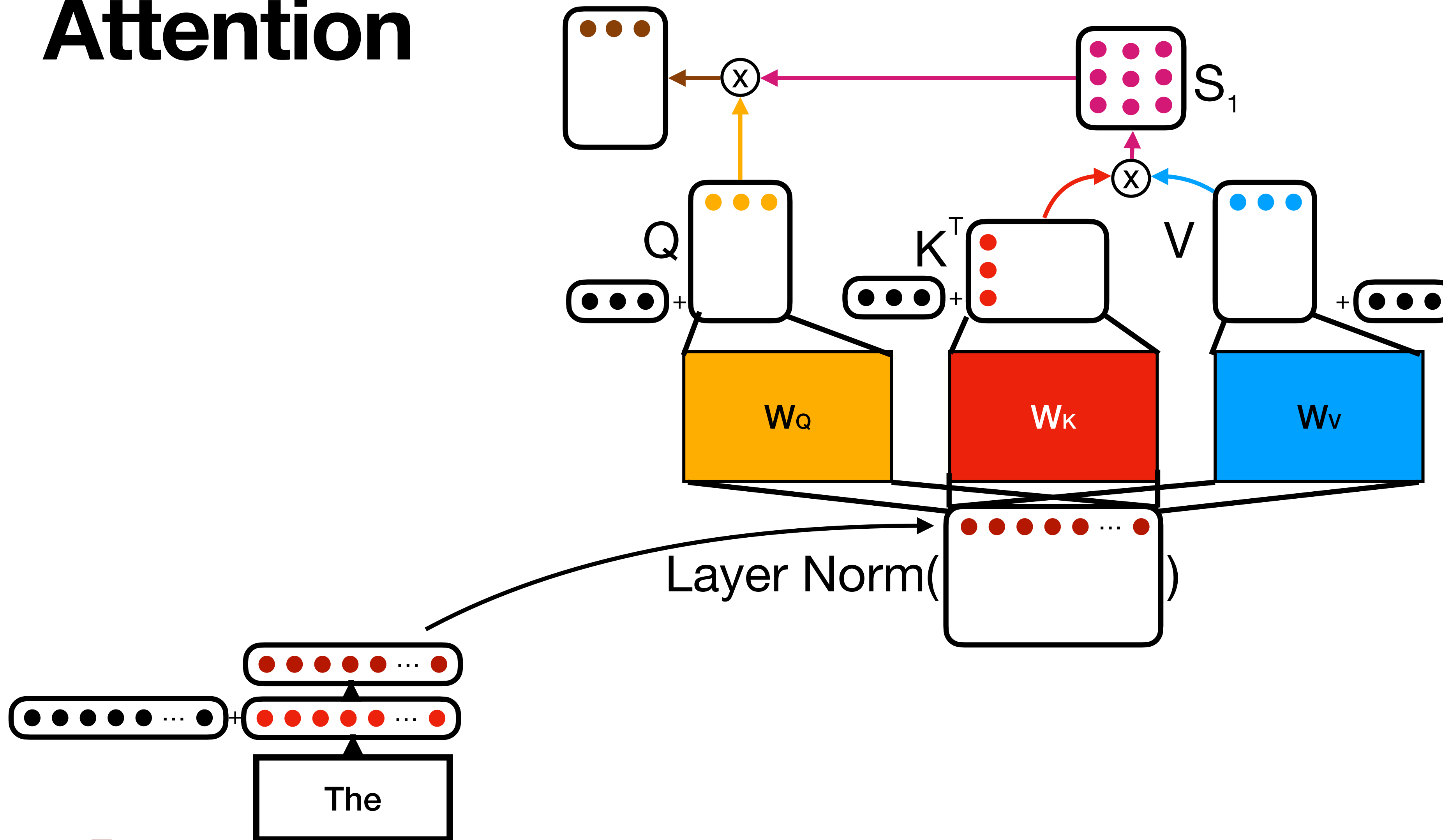
Linear Attention



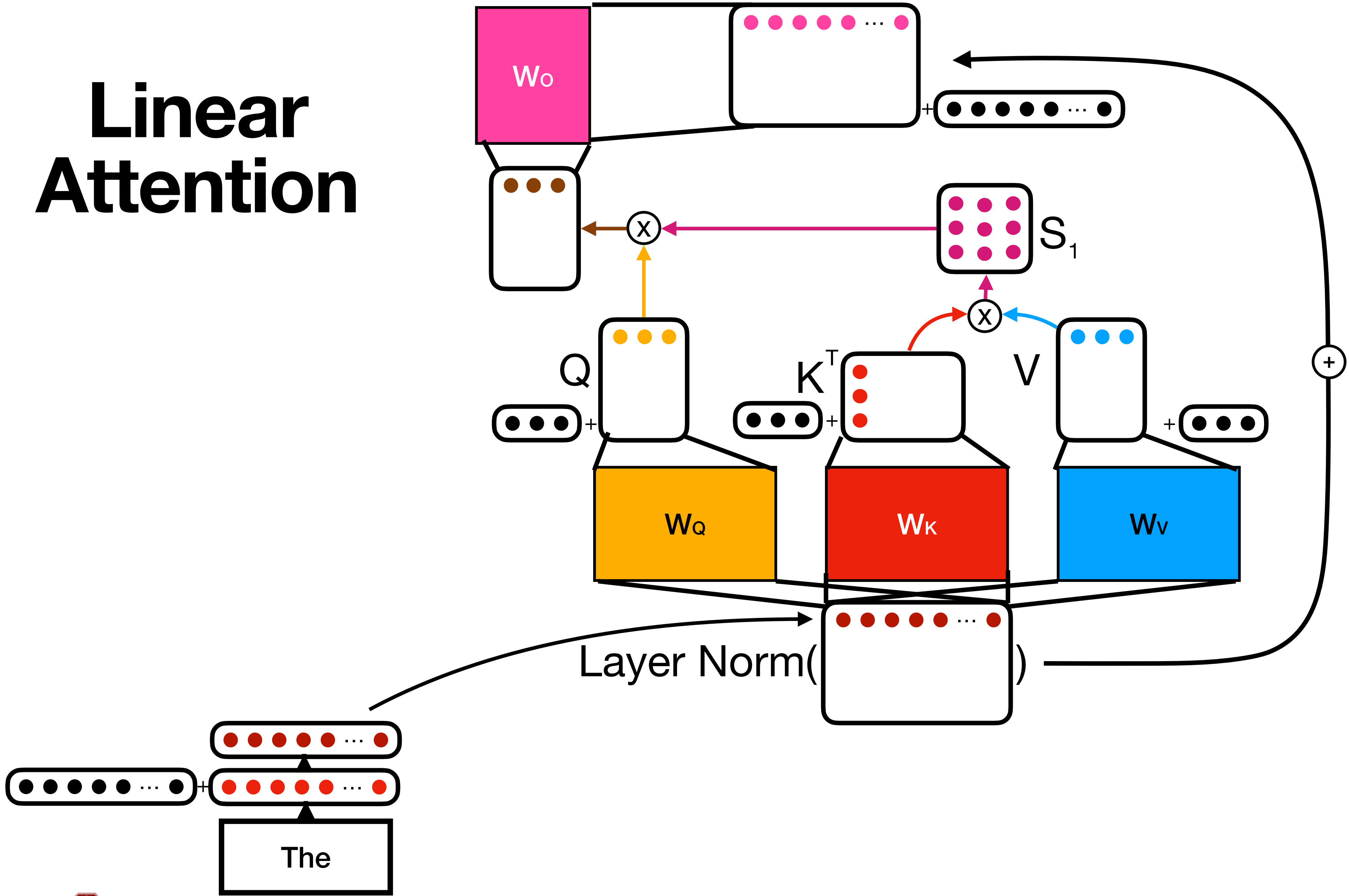
Linear Attention



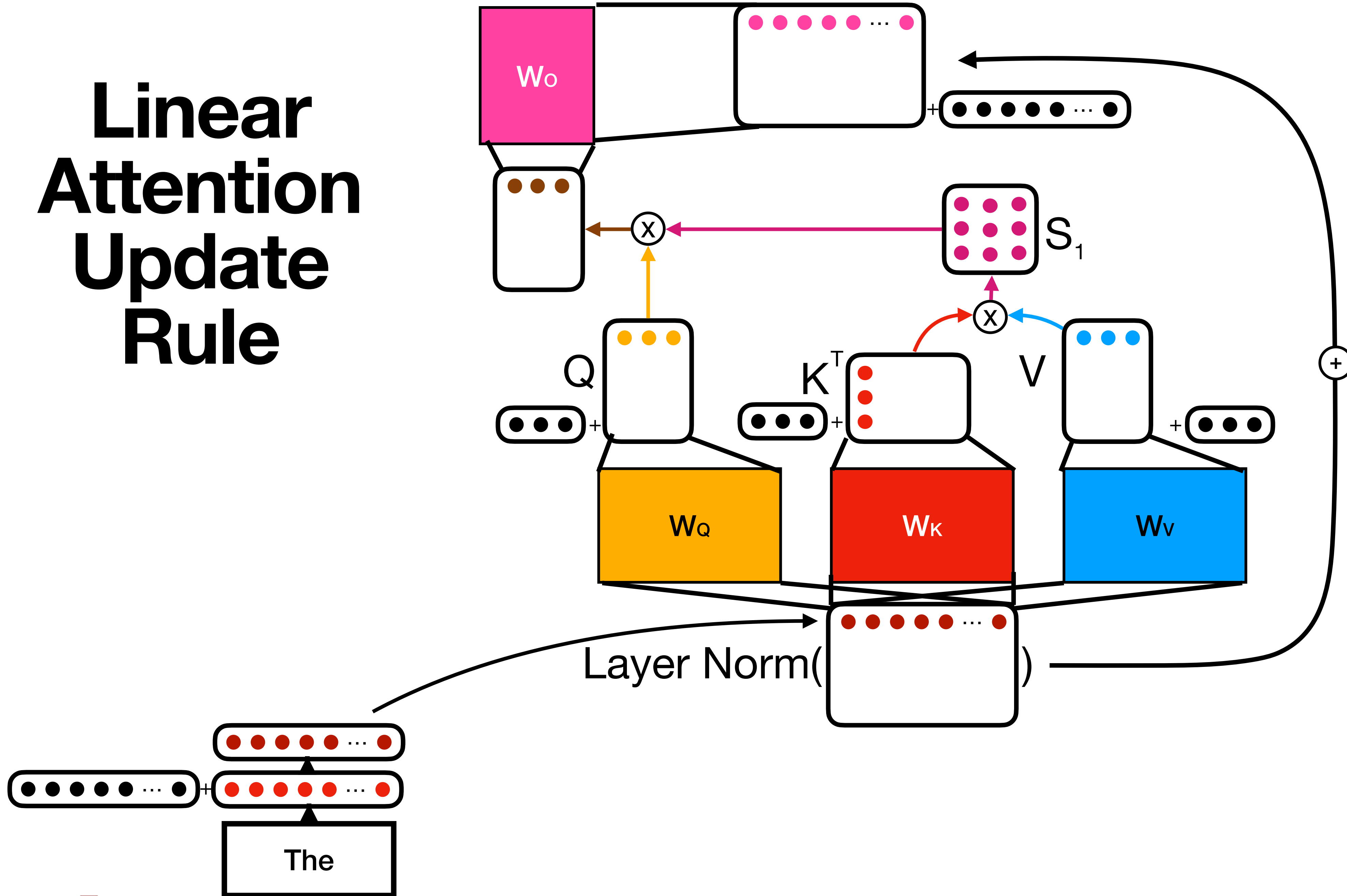
Linear Attention



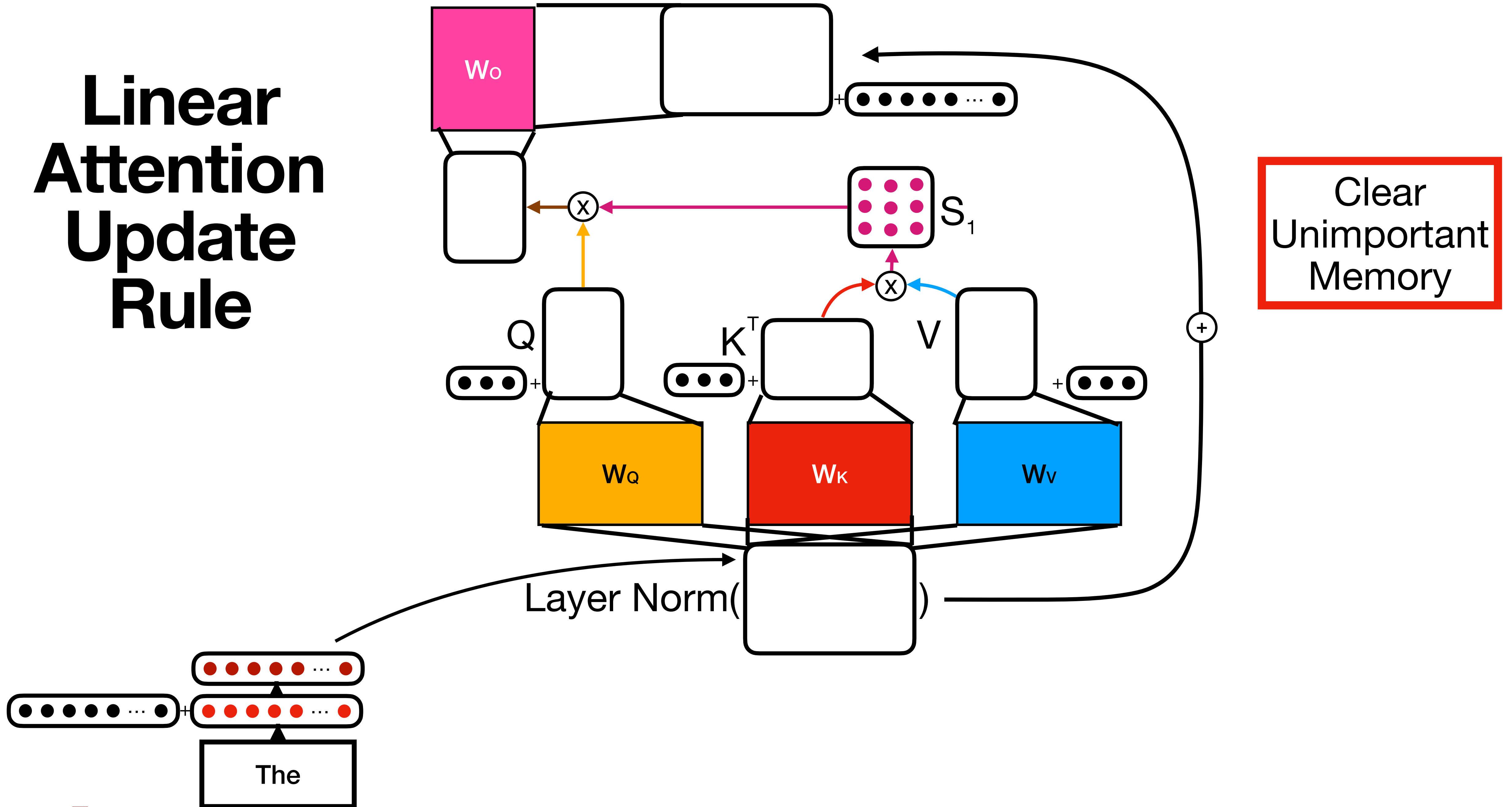
Linear Attention



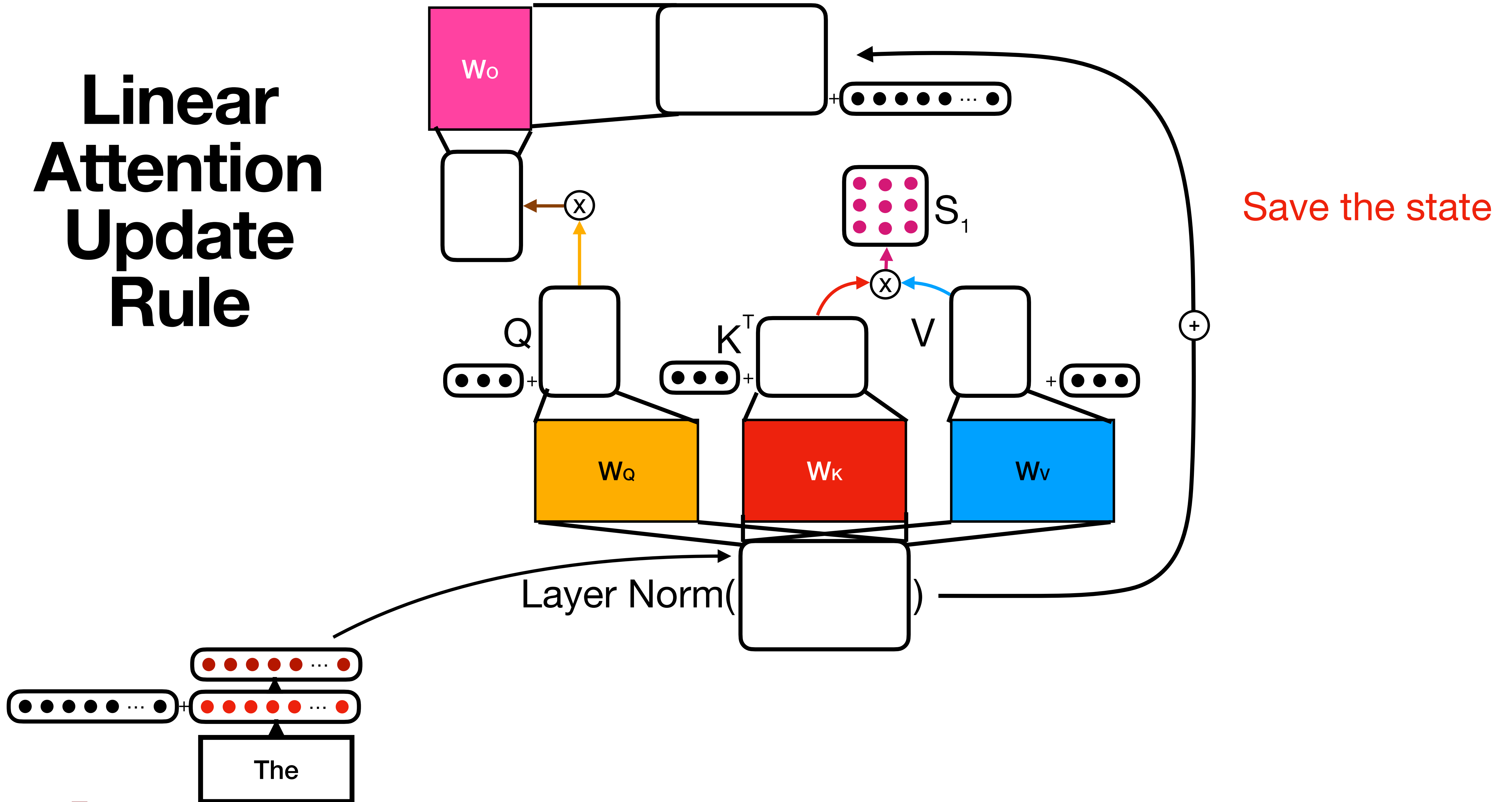
Linear Attention Update Rule



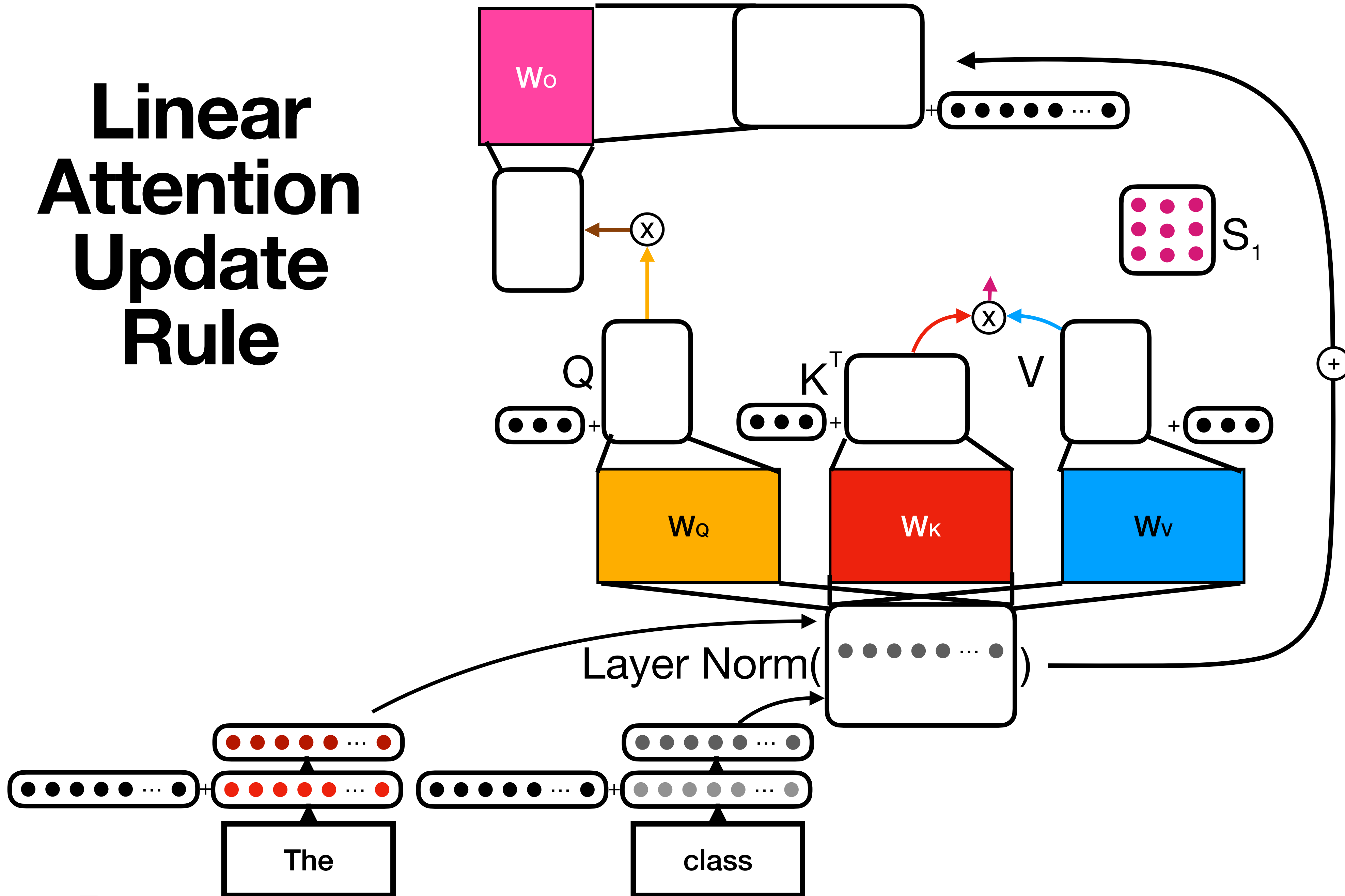
Linear Attention Update Rule



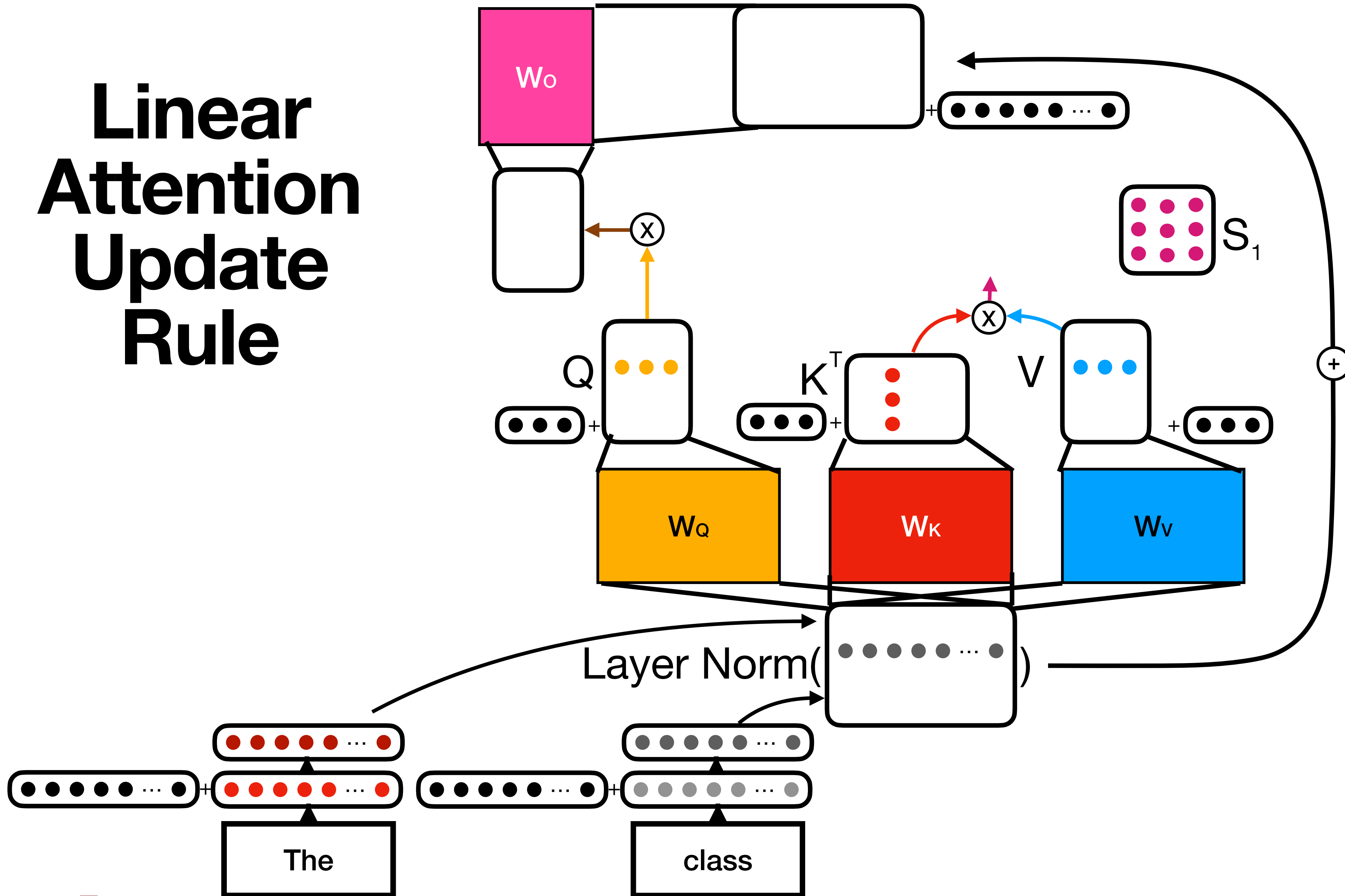
Linear Attention Update Rule



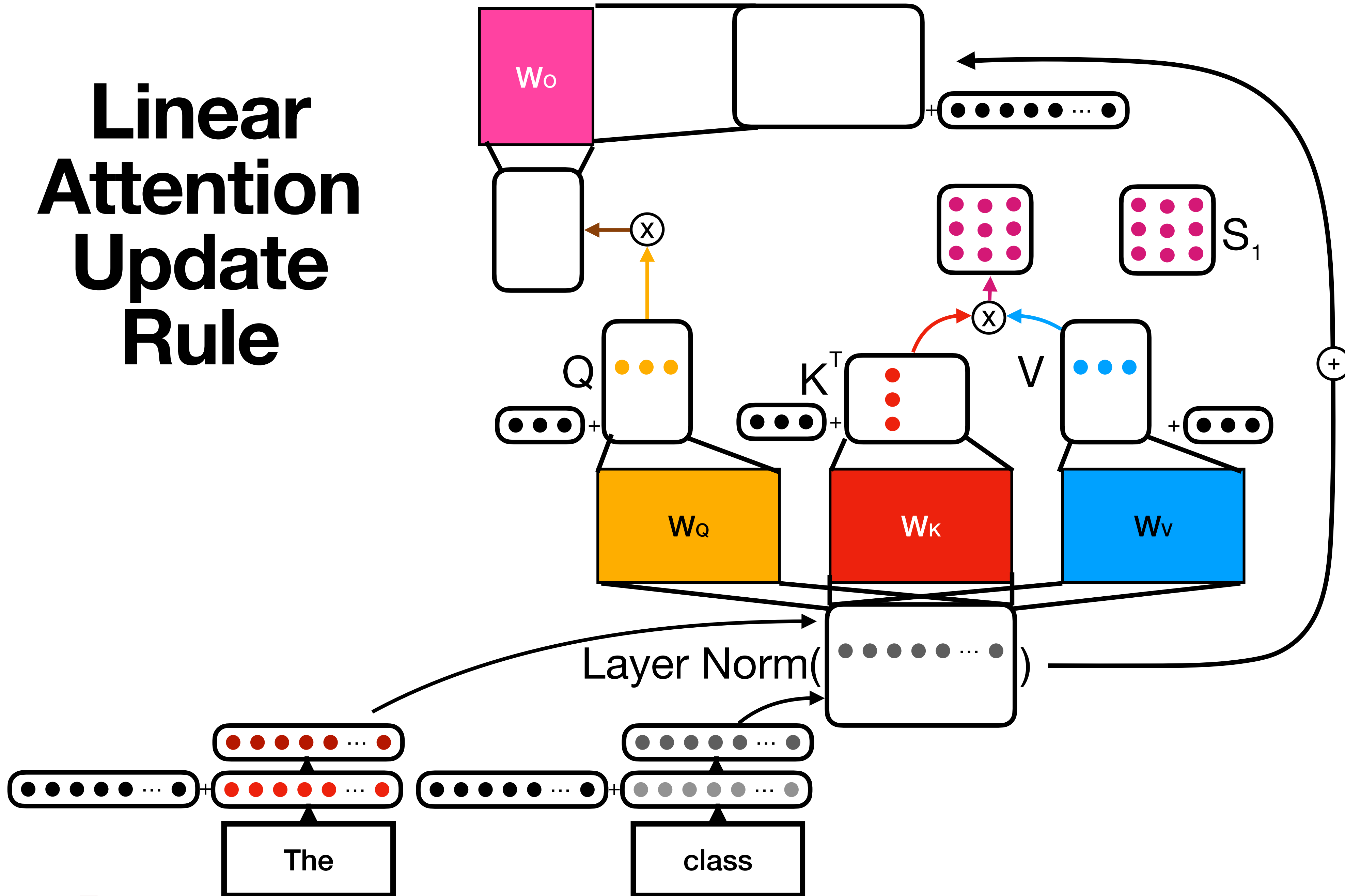
Linear Attention Update Rule



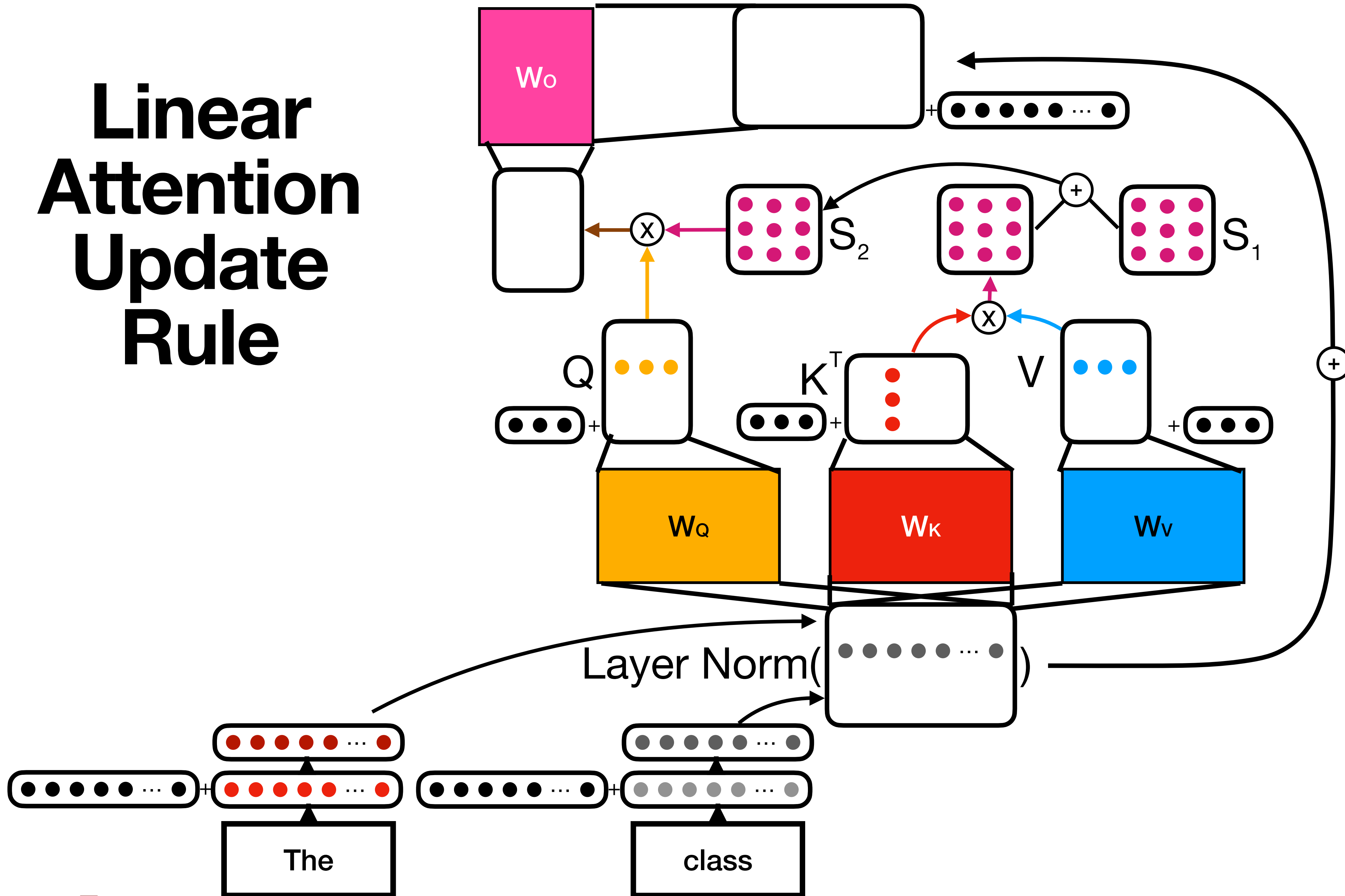
Linear Attention Update Rule



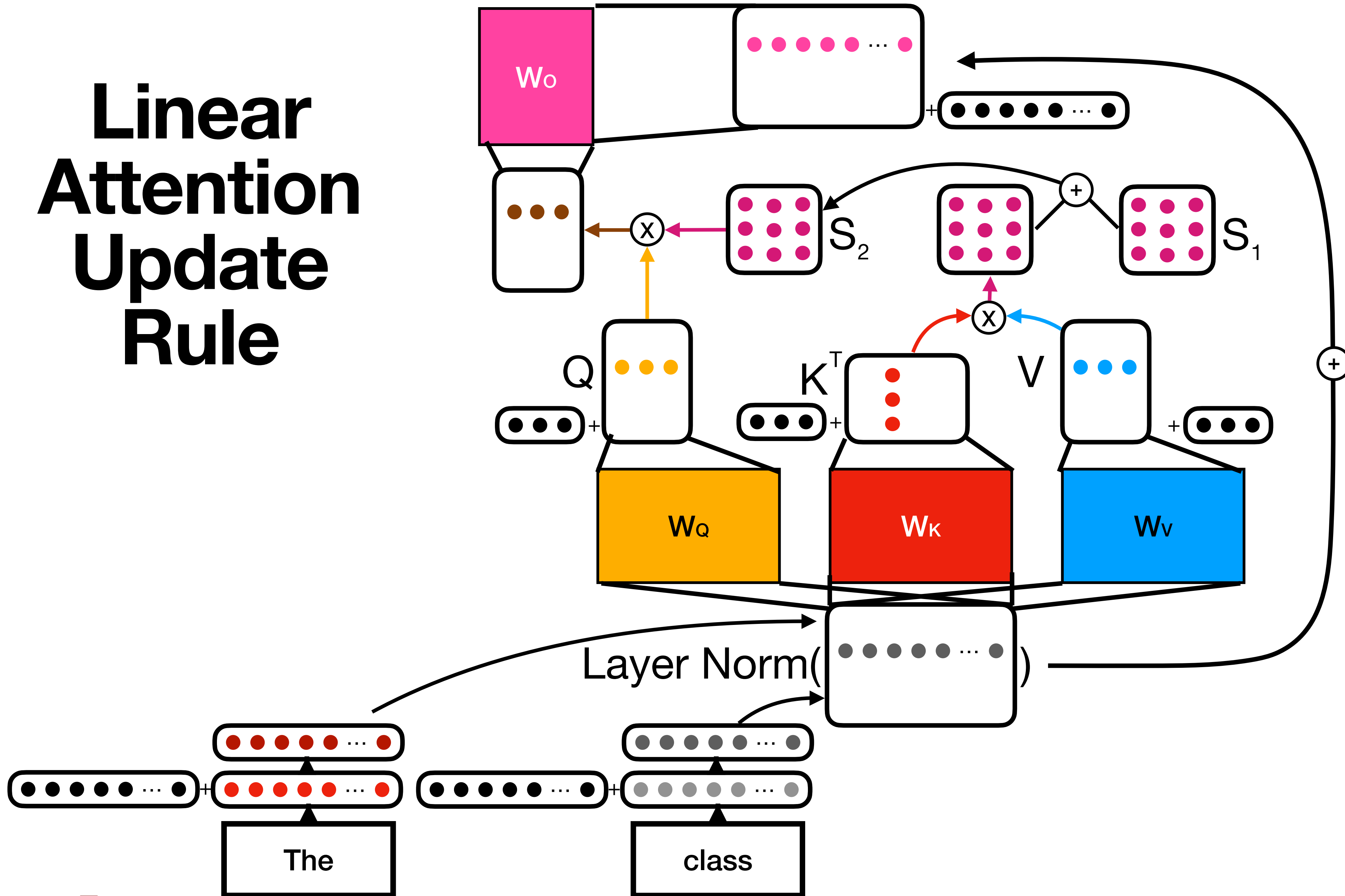
Linear Attention Update Rule



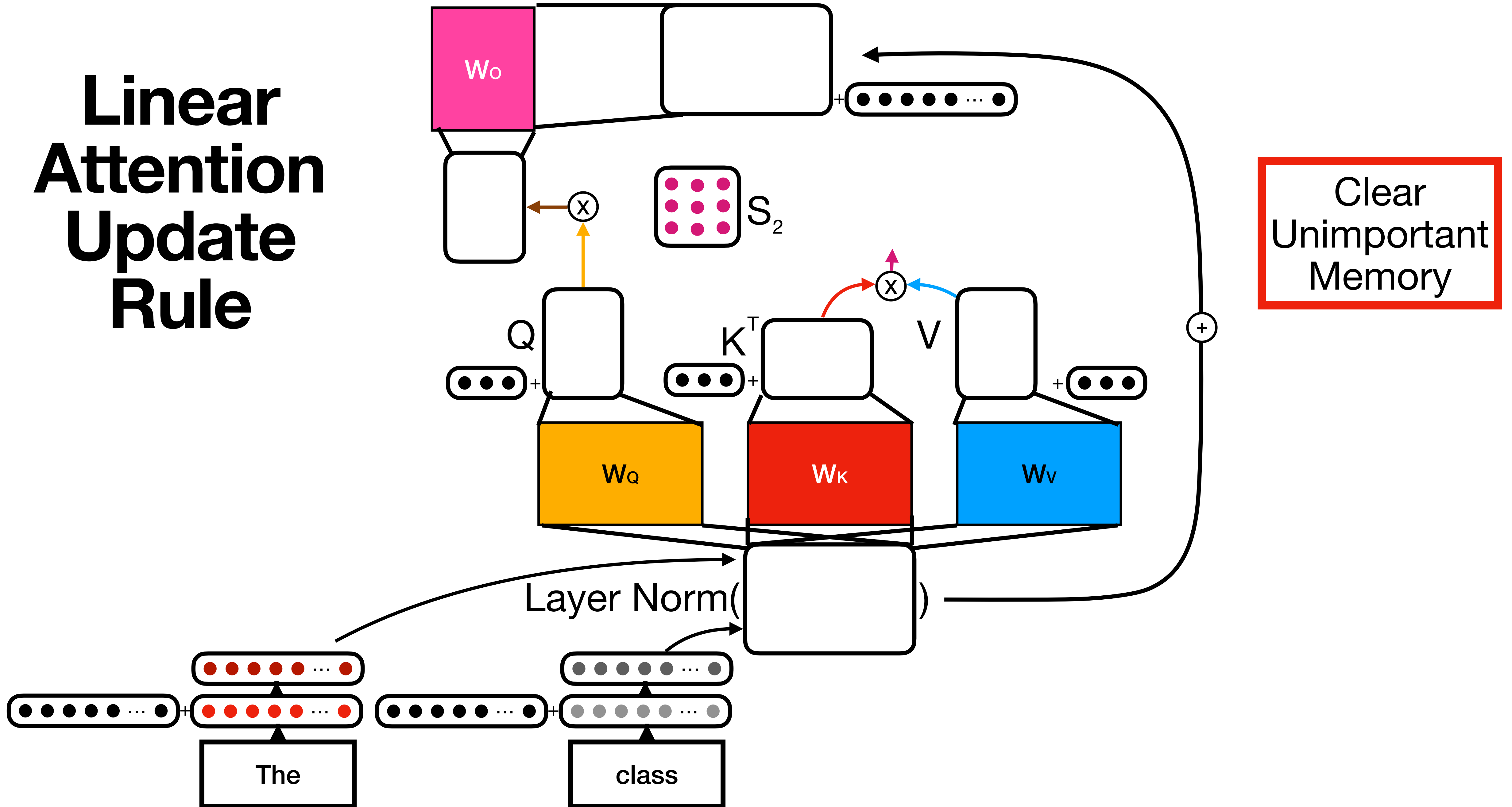
Linear Attention Update Rule



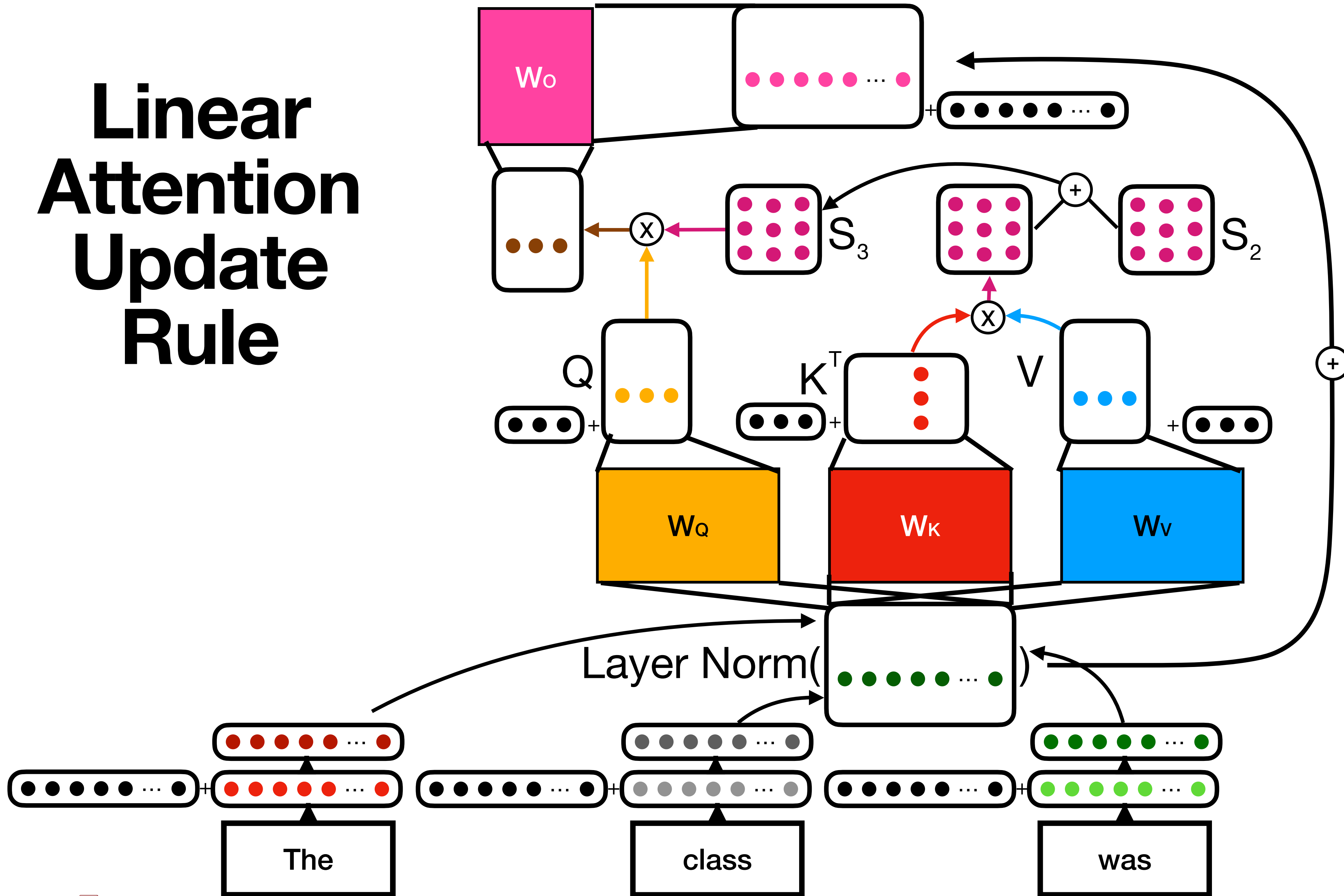
Linear Attention Update Rule



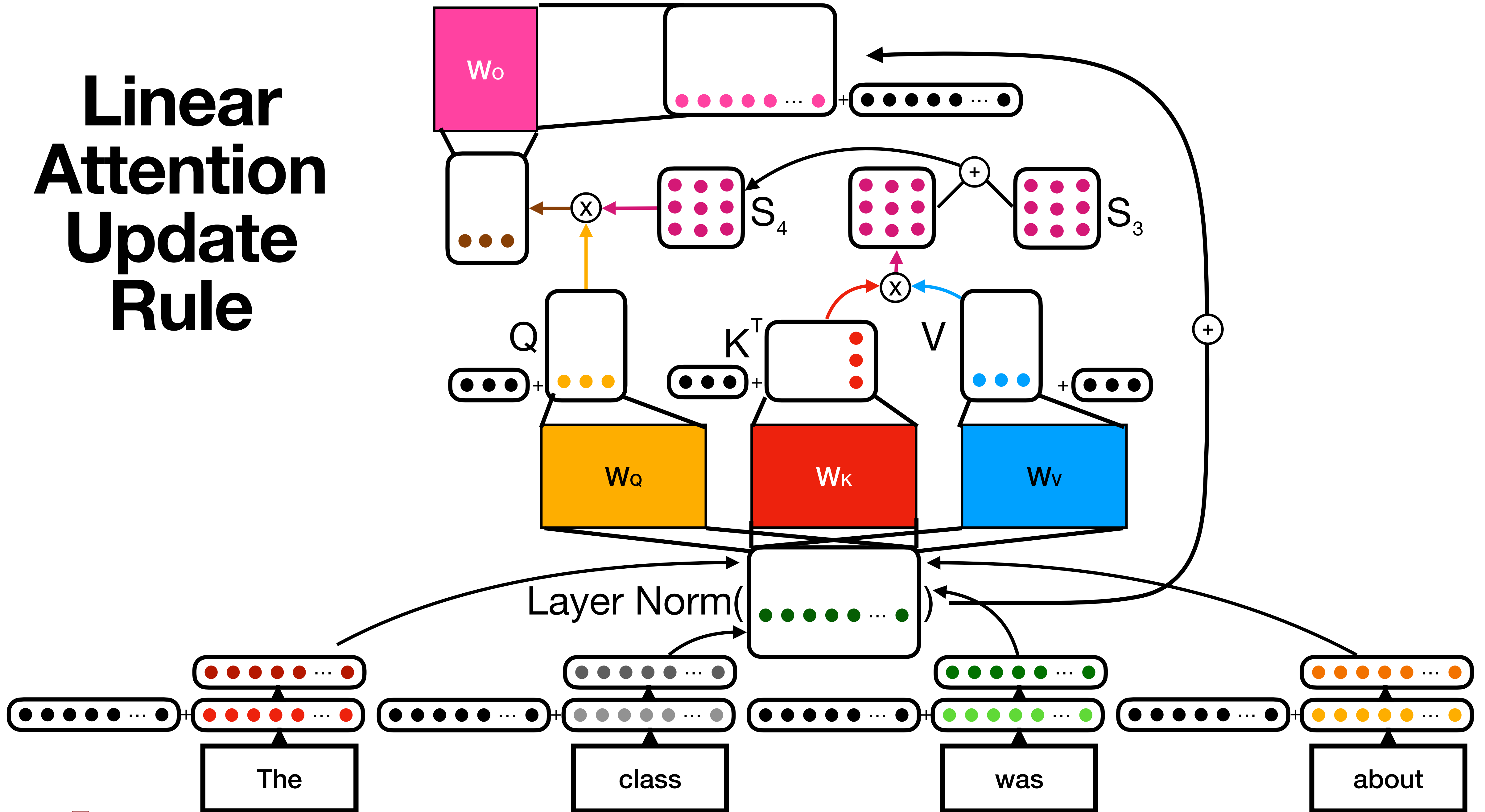
Linear Attention Update Rule



Linear Attention Update Rule



Linear Attention Update Rule



Linear Attention is way faster and has a better memory footprint

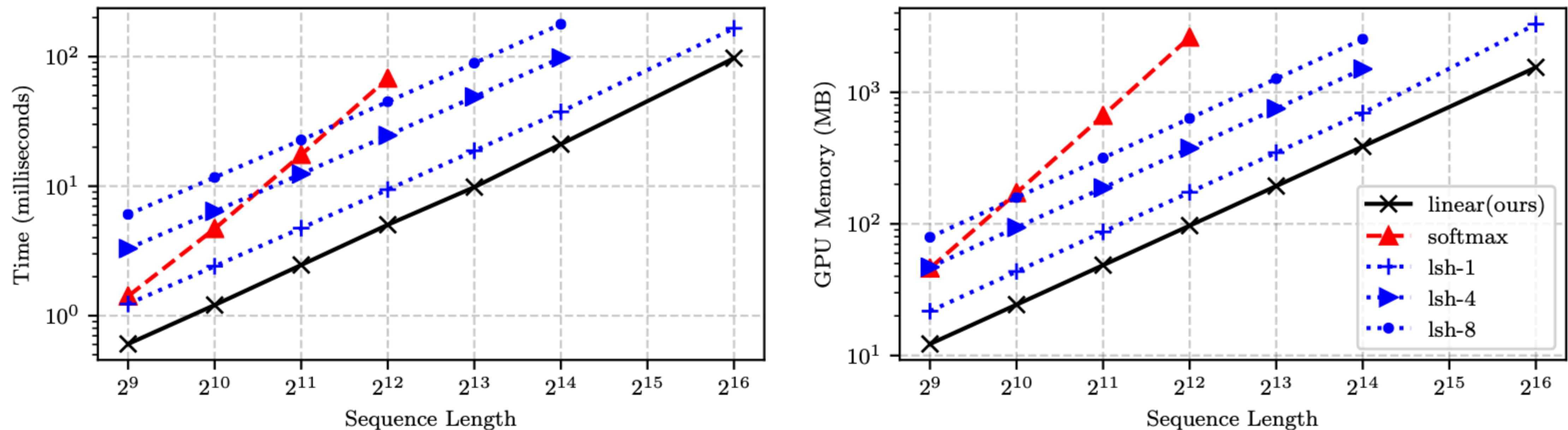
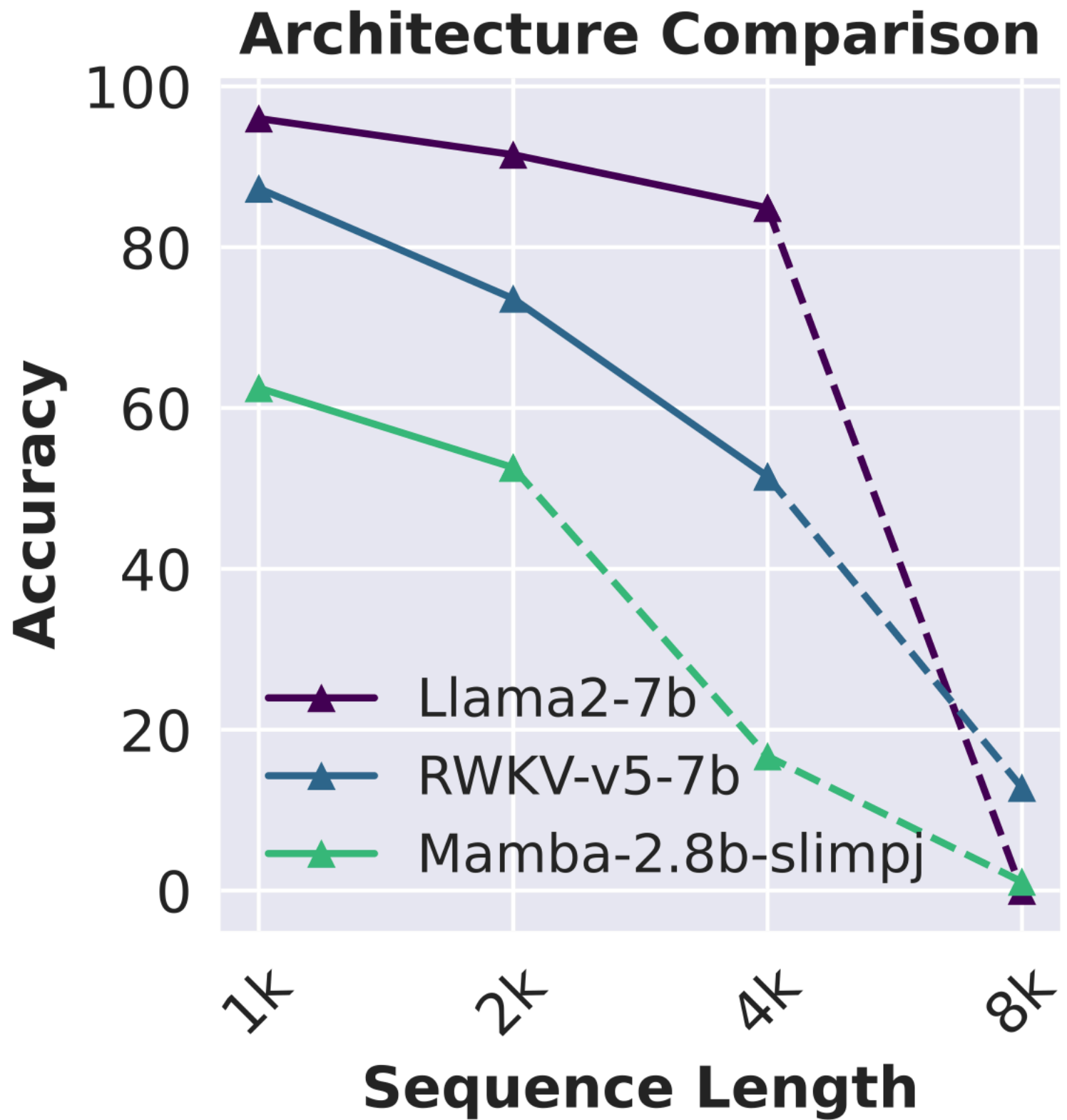


Figure 1: Comparison of the computational requirements for a forward/backward pass for Reformer (lsh-X), softmax attention and linear attention. Linear and Reformer models scale linearly with the sequence length unlike softmax which scales with the square of the sequence length both in memory and time. Full details of the experiment can be found in § 4.1.



However... it struggles more with longer context

Task	Configuration	Example
Single NIAH (S-NIAH)	type_key = word type_value = number type_haystack = essay size_haystack $\propto$ context length	(essays) One of the special magic numbers for long-context is: 12345. What is the special magic number for long-context mentioned in the provided text? Answer: 12345
Multi-keys NIAH (MK-NIAH)	num_keys = 2 type_key = word type_value = number type_haystack = essay size_haystack $\propto$ context length	(essays) One of the special magic numbers for long-context is: 12345. One of the special magic numbers for large-model is: 54321. What is the special magic number for long-context mentioned in the provided text? Answer: 12345
Multi-values NIAH (MV-NIAH)	num_values = 2 type_key = word type_value = number type_haystack = essay size_haystack $\propto$ context length	(essays) One of the special magic numbers for long-context is: 12345. One of the special magic numbers for long-context is: 54321. What are all the special magic numbers for long-context mentioned in the provided text? Answer: 12345 54321



A Note about Context Length

Nothing the Transformer or Linear Attention enforced a strict context length... except the way the model processes positional embeddings.

In practice we need the model to learn to use the positional embeddings so we are **limited to the context length that we trained the model with.**



Bringing it together



Full Softmax Attention vs Linear Attention

Full Softmax Attention

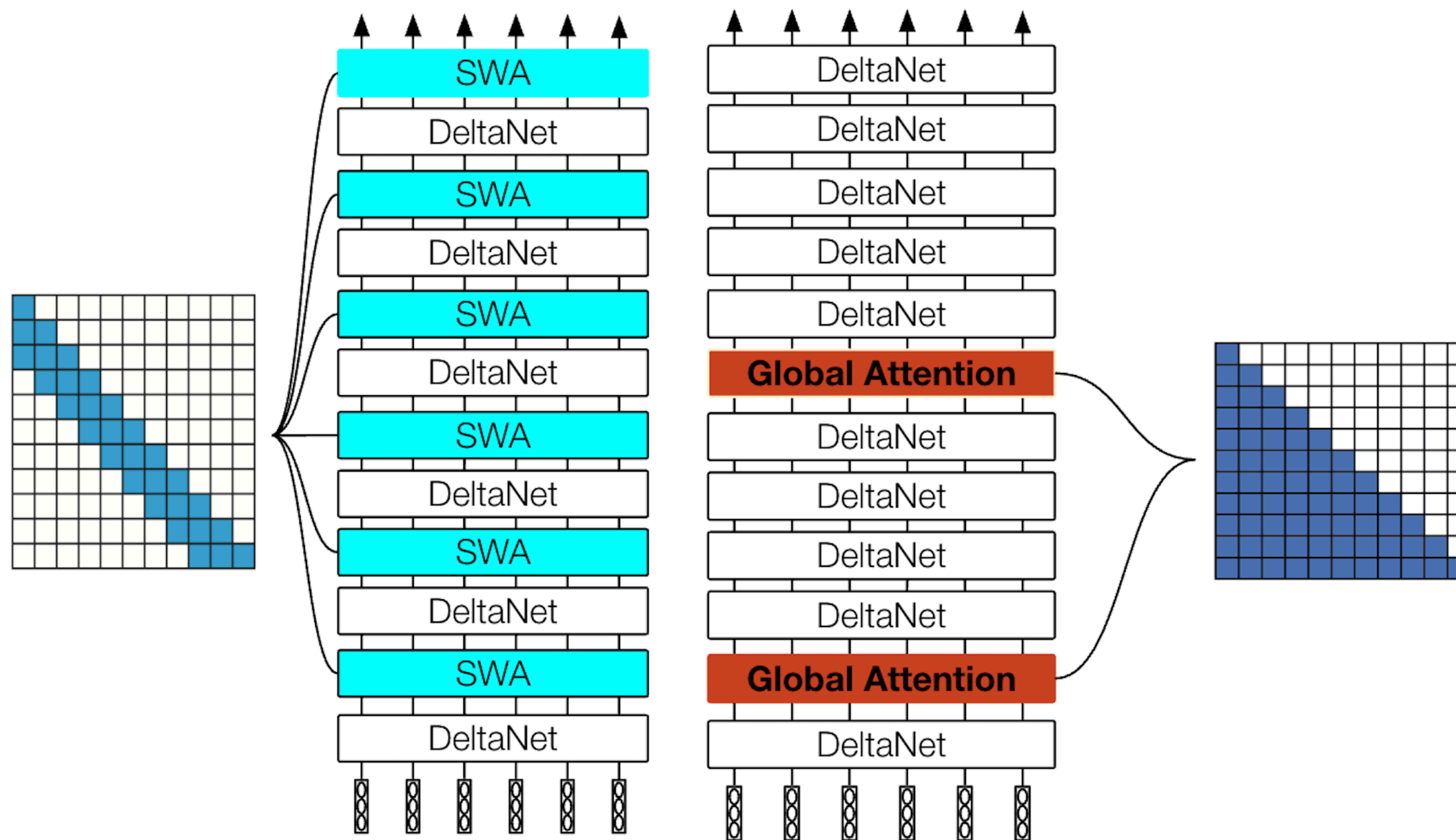
- $O(n^2)$ in context length
- All tokens can refer back to all prior tokens
 - Helps long context
- KV Cache (memory) grows $O(n)$

Linear Attention

- $O(n)$ in context length
- New tokens interact with a compressed state representation
 - Harder for long context
- State representation uses constant memory



In practice people mix global and linear attention

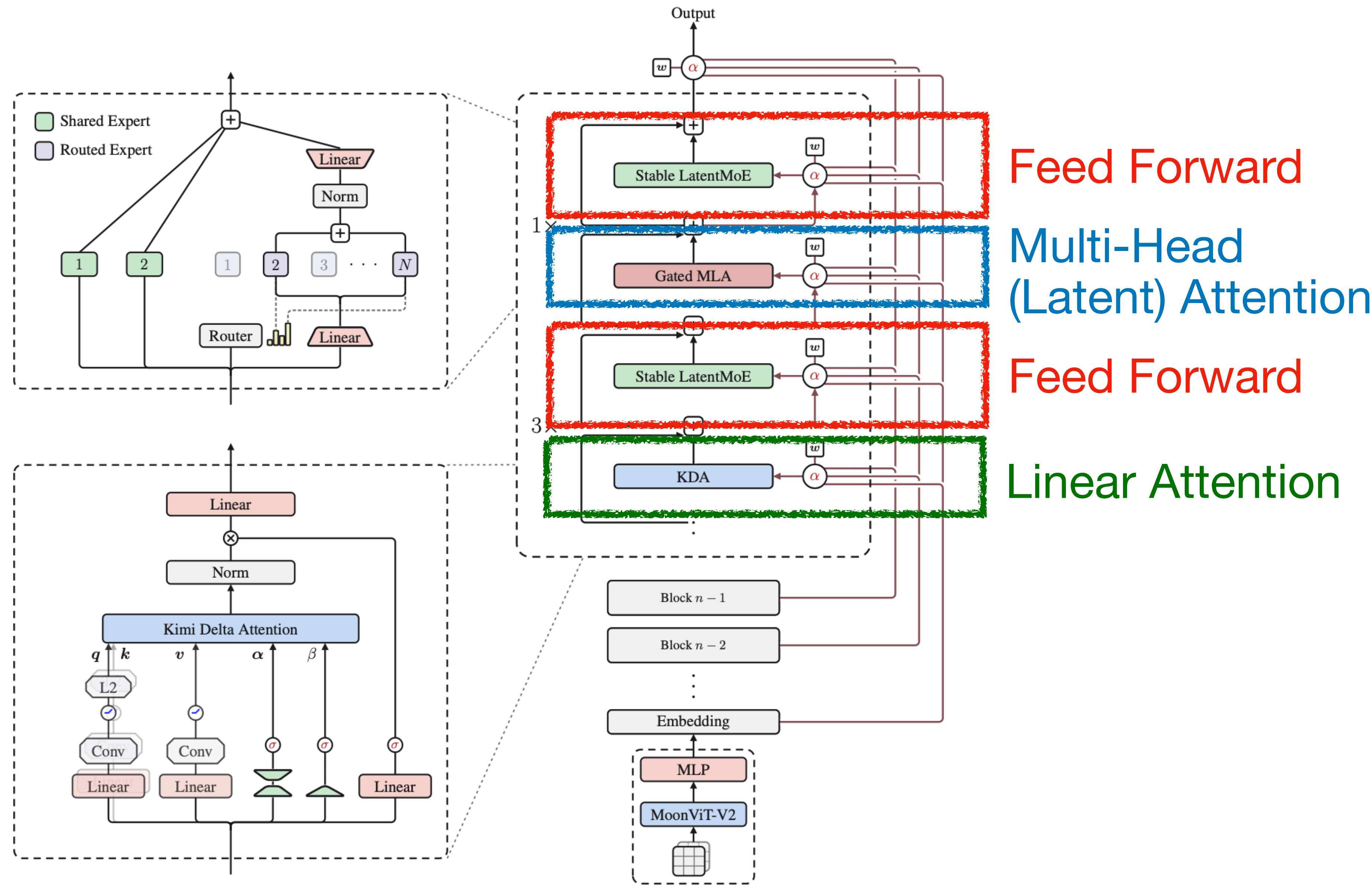


In practice people mix global and linear attention

Model	Wiki. ppl ↓	Avg. Common-sense ↑	Avg. Retrieval ↑
DeltaNet	28.24	42.1	22.7
+ Sliding Attn	27.06	42.1	30.2
+ Global Attn	27.51	42.1	32.7



Kimi K3



Why does all of this matter for agents?

- Agents can deal with extremely long contexts
- Inference efficiency is a design consideration in building agentic systems
- Knowing the underlying model architectures can help you select the right model for your agentic application
- Developing the harness and scaffolding around your particular model's strengths and weaknesses will make you a better agent engineer!



LLM Training

Language Models?

Architecture

Training

Inference

LLMs for Agents



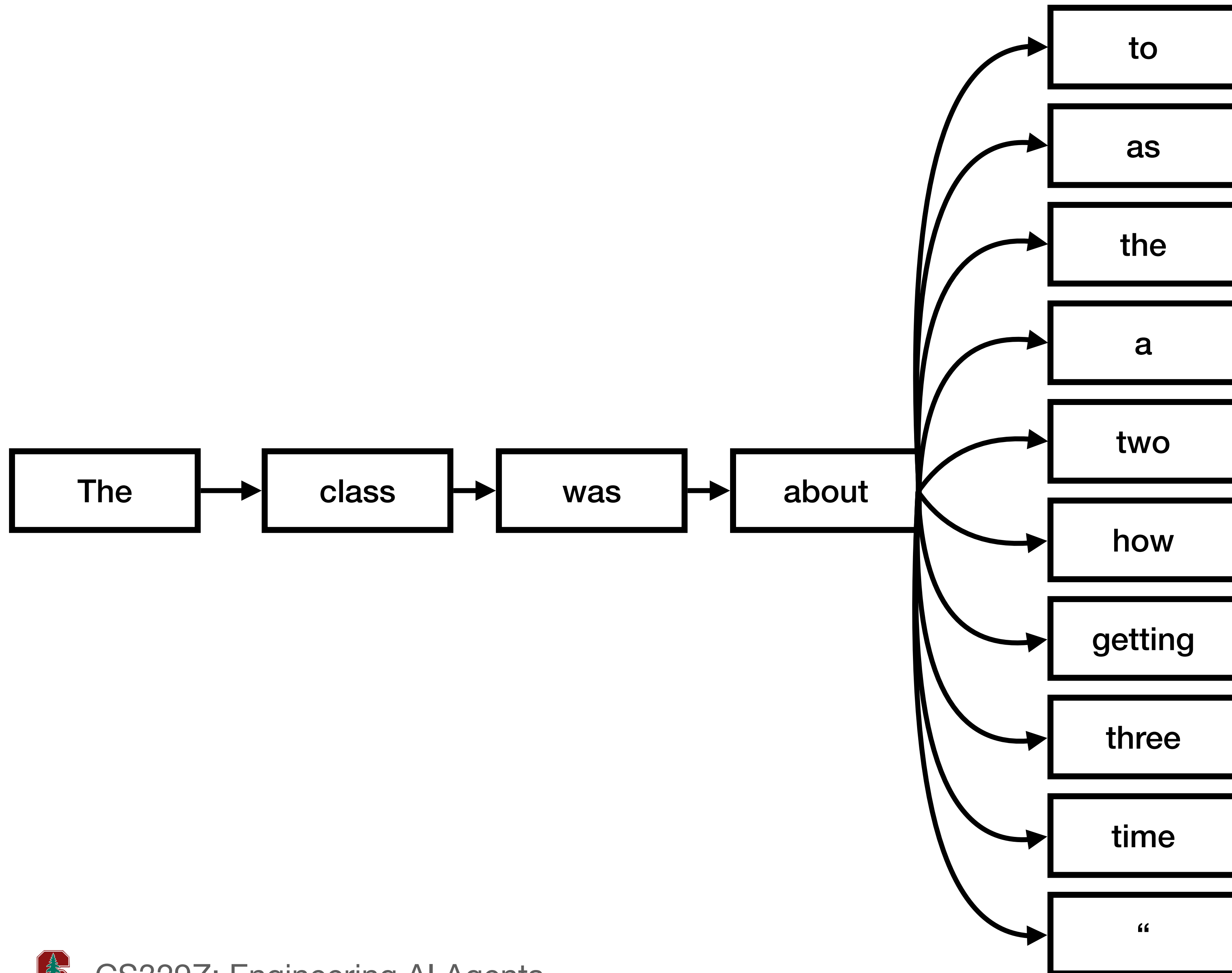
Defining the Loss

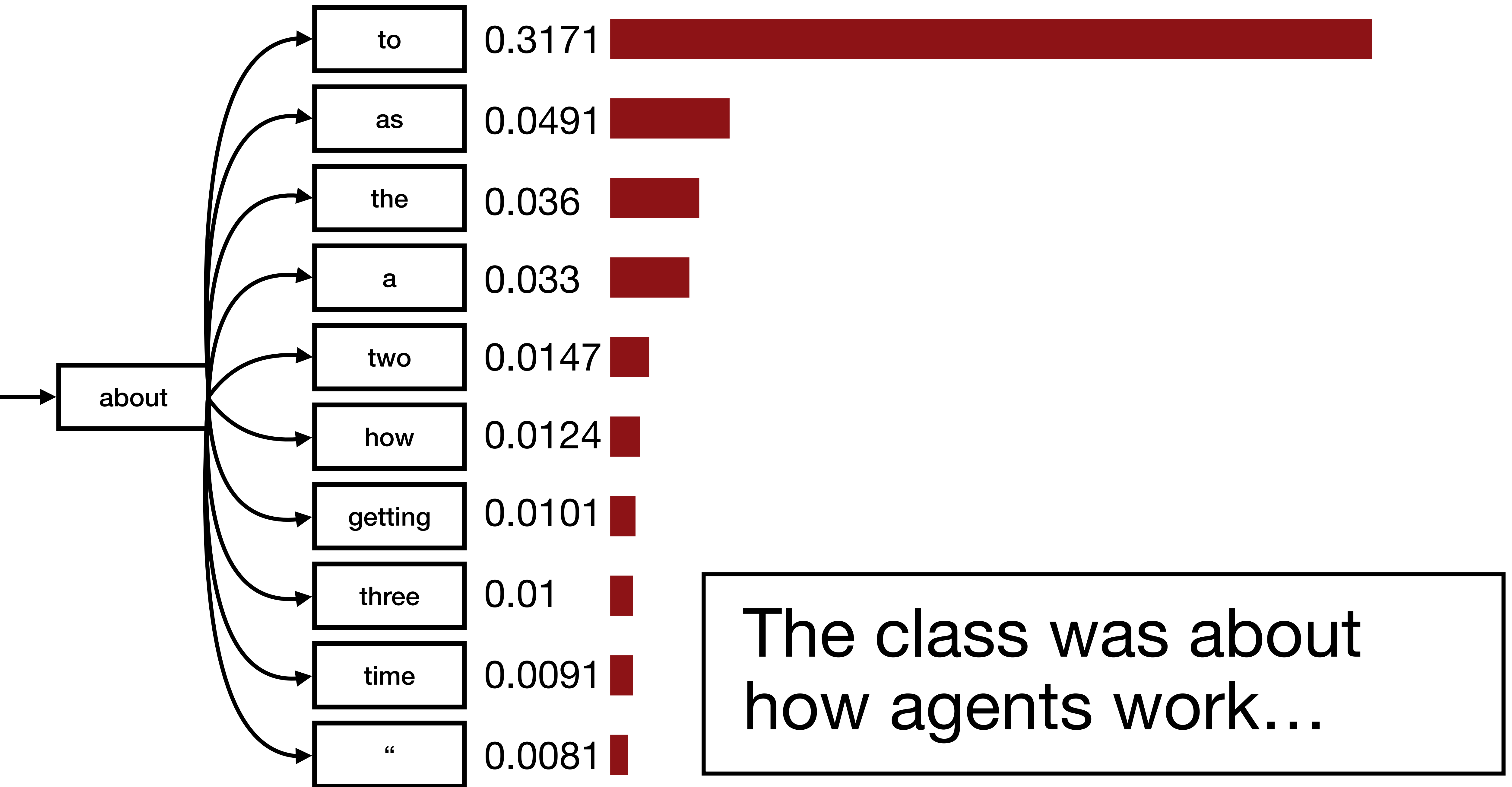


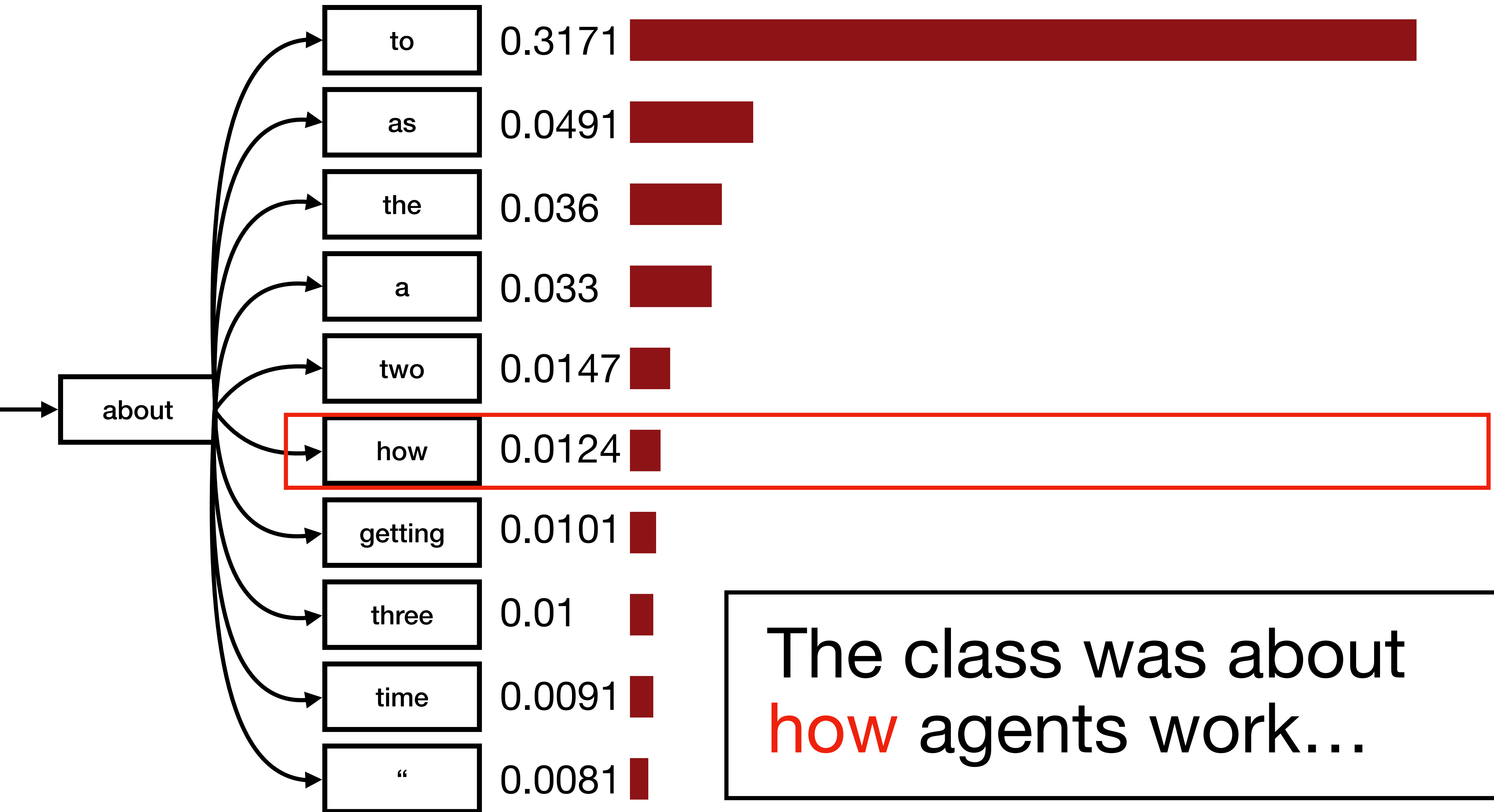
Training Data (Example)

The class was about how agents work, and how to build AI systems that use tools, retrieve information, and carry out tasks across multiple steps...









The class was about
how agents work...





The class was about
how agents work...



$$\text{Loss} = -\log(0.0124) = 1.907$$



The class was about
how agents work...



Cross Entropy Loss

$$L = - \sum_{i=1}^V y_i \log(p_i)$$

1 when this is the token in the data

0 when this is not the token in the data

log probability of that token

For every token in the vocabulary

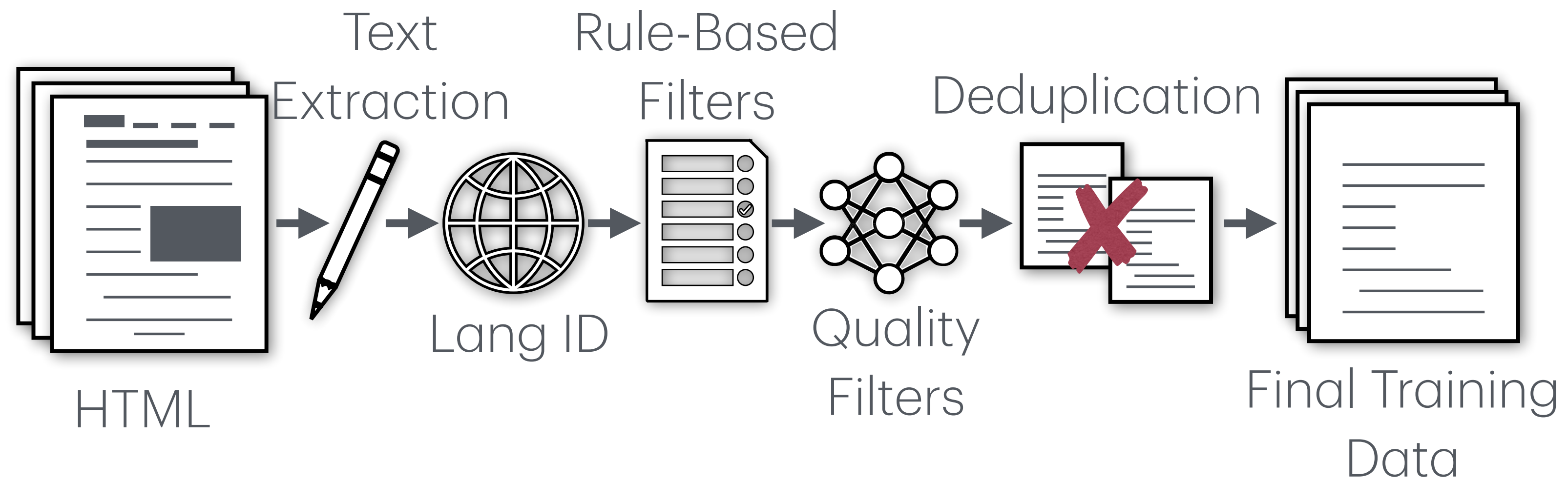
The diagram illustrates the Cross Entropy Loss formula $L = - \sum_{i=1}^V y_i \log(p_i)$. The summation symbol $\sum$ is enclosed in a red dashed box, with the text "For every token in the vocabulary" below it. The upper limit of the summation is V , representing the vocabulary size. The term y_i is enclosed in a blue dashed box, with an arrow pointing to it from the text "0 when this is not the token in the data" and another arrow pointing to it from the text "1 when this is the token in the data". The term $\log(p_i)$ is enclosed in a green dashed box, with the text "log probability of that token" below it.



Pretraining



Data



Data

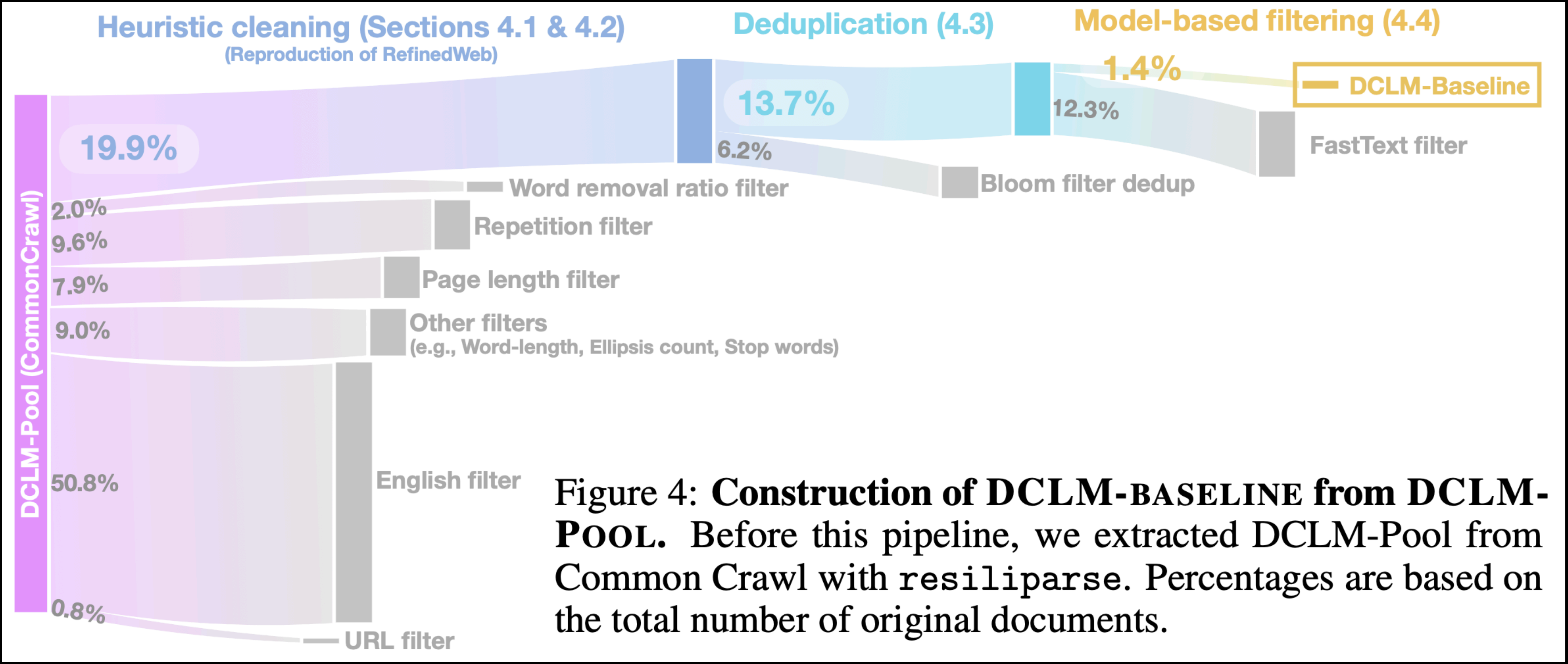


Figure 4: **Construction of DCLM-BASELINE from DCLM-POOL.** Before this pipeline, we extracted DCLM-Pool from Common Crawl with resiliiparse. Percentages are based on the total number of original documents.

Image Credit: DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. arXiv:2406.11794.



Data

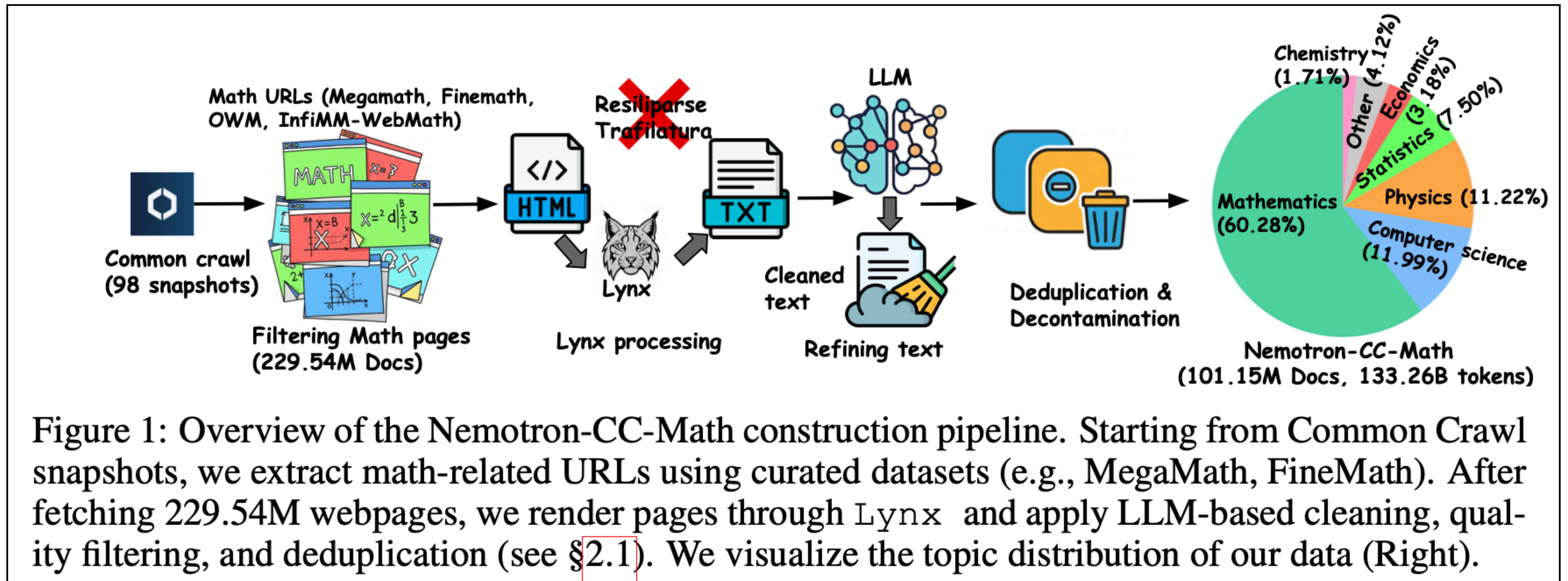


Figure 1: Overview of the Nemotron-CC-Math construction pipeline. Starting from Common Crawl snapshots, we extract math-related URLs using curated datasets (e.g., MegaMath, FineMath). After fetching 229.54M webpages, we render pages through Lynx and apply LLM-based cleaning, quality filtering, and deduplication (see §2.1). We visualize the topic distribution of our data (Right).

Next: , Previous: DTIME, Up: Intrinsic Procedures

8.48 EOSHIFT — End-off shift function

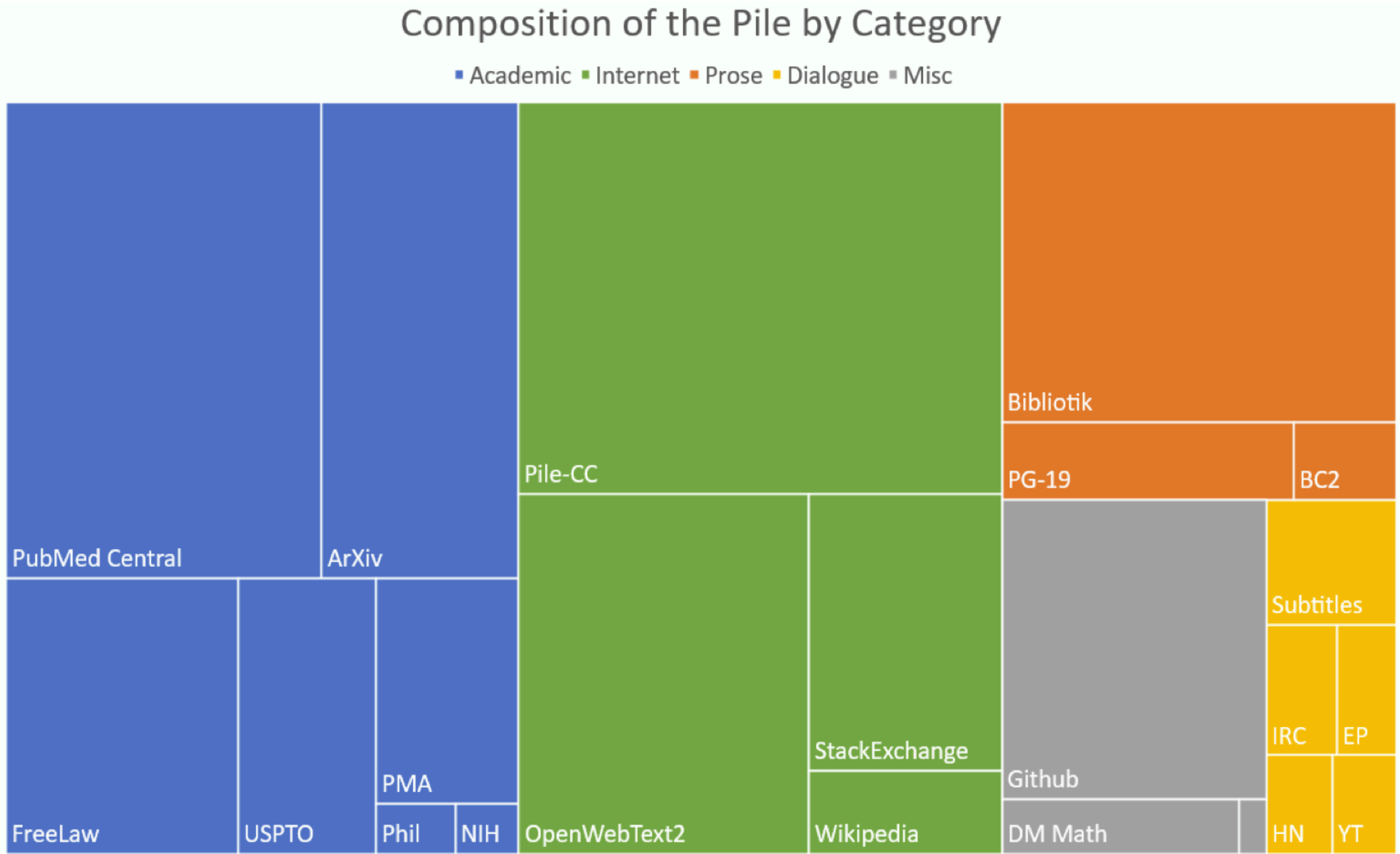
`EOSHIFT(ARRAY, SHIFT[, BOUNDARY, DIM])` performs an end-off shift on elements of `ARRAY` along the dimension of `DIM`. If `DIM` is omitted it is taken to be 1. `DIM` is a scalar of type `INTEGER` in the range of $1 \leq \text{DIM} \leq n$ where n is the rank of `ARRAY`. If the rank of `ARRAY` is one, then all elements of `ARRAY` are shifted by `SHIFT` places. If rank is greater than one, then all complete rank one sections of `ARRAY` along the given dimension are shifted. Elements shifted out one end of each rank one section are dropped. If `BOUNDARY` is present then the corresponding value of from `BOUNDARY` is copied back in the other end. If `BOUNDARY` is not present then the following are copied in depending on the type of `ARRAY`.

Array Type Boundary Value

Numeric 0 of the type and kind of `ARRAY`.

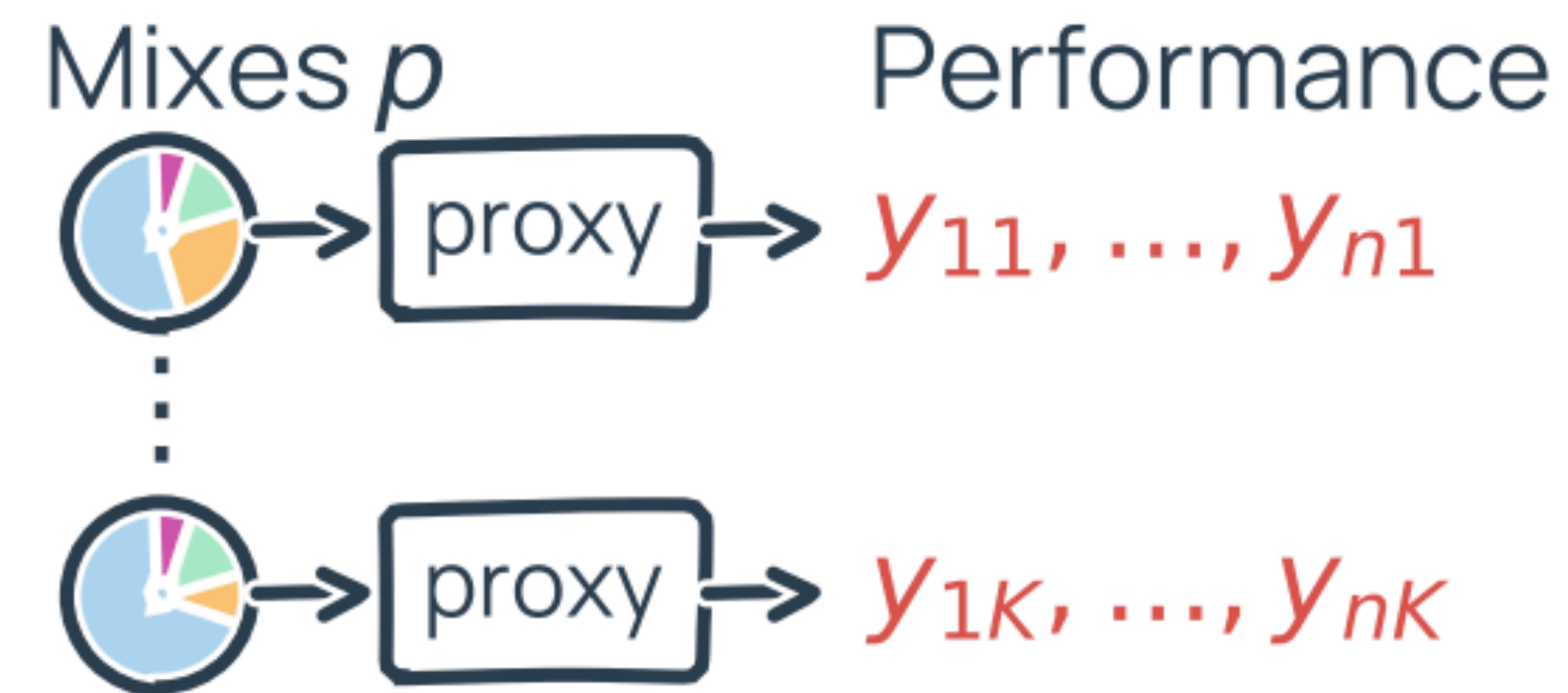


Data

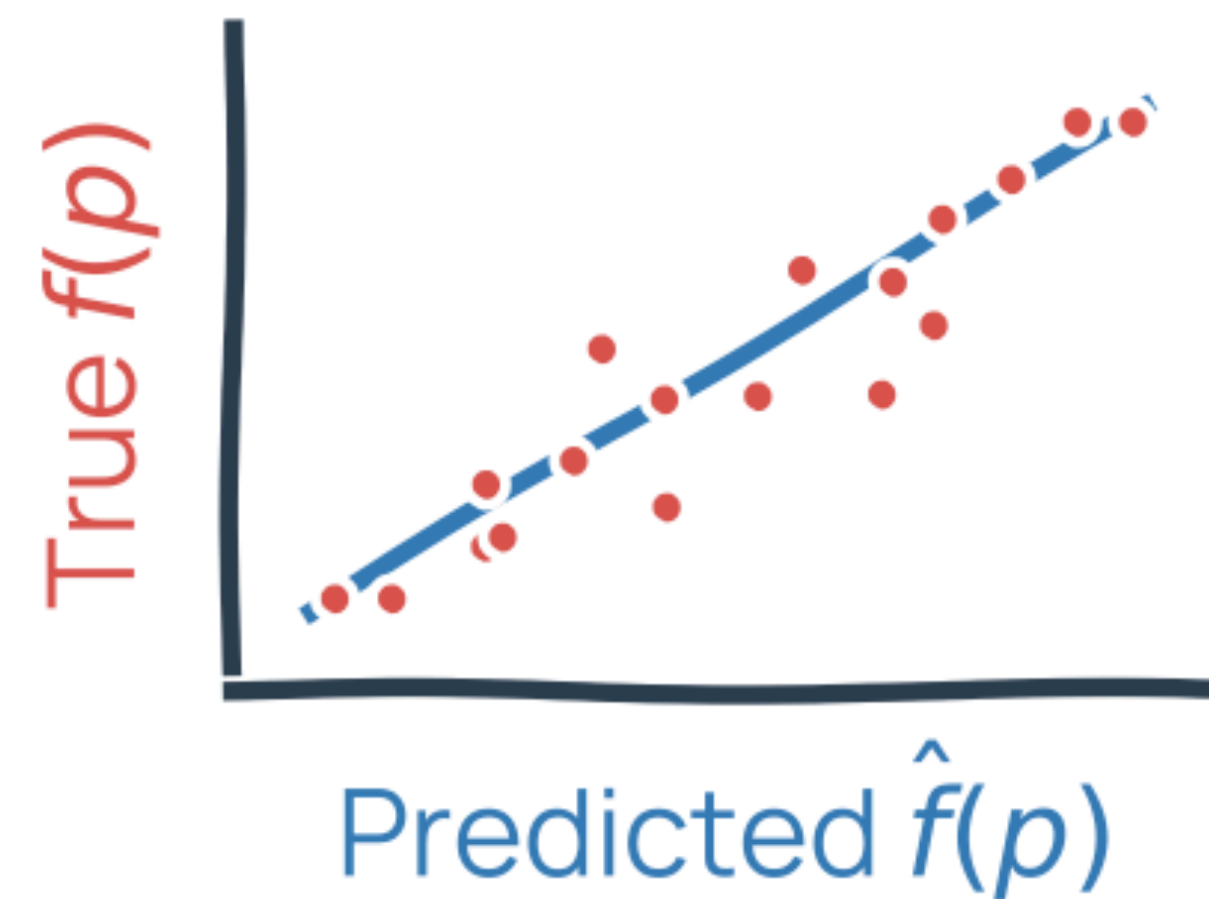


Data Mixing

① Swarm

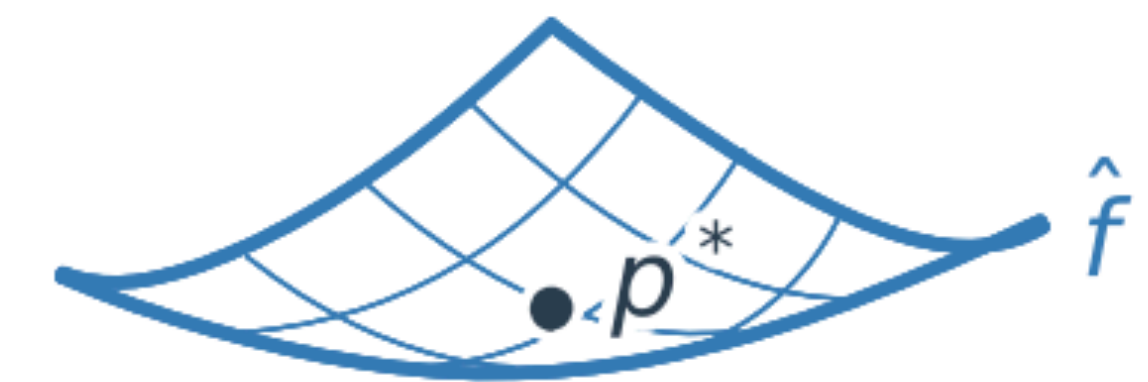


② Regression



③ Optimization

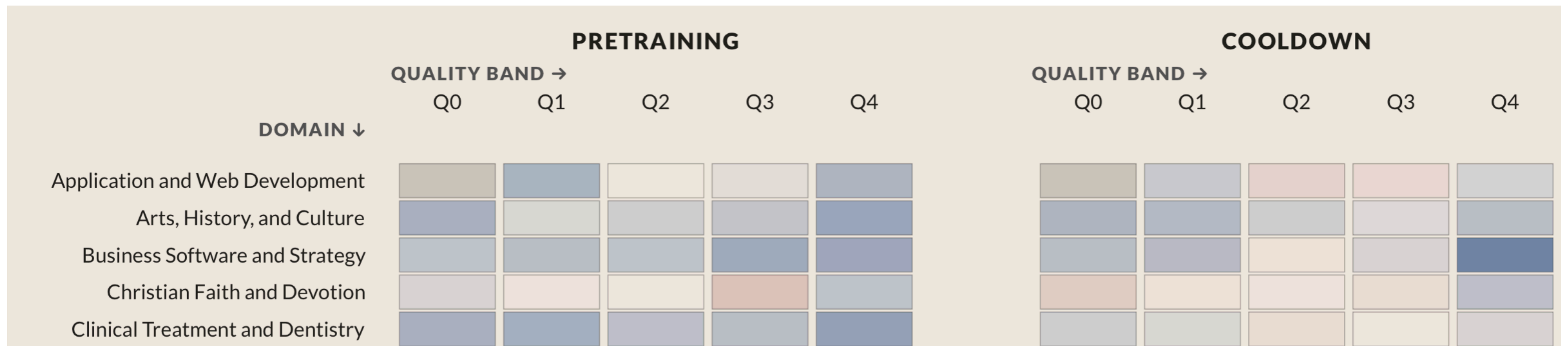
minimize $\hat{f}(p)$
 $p \in S$



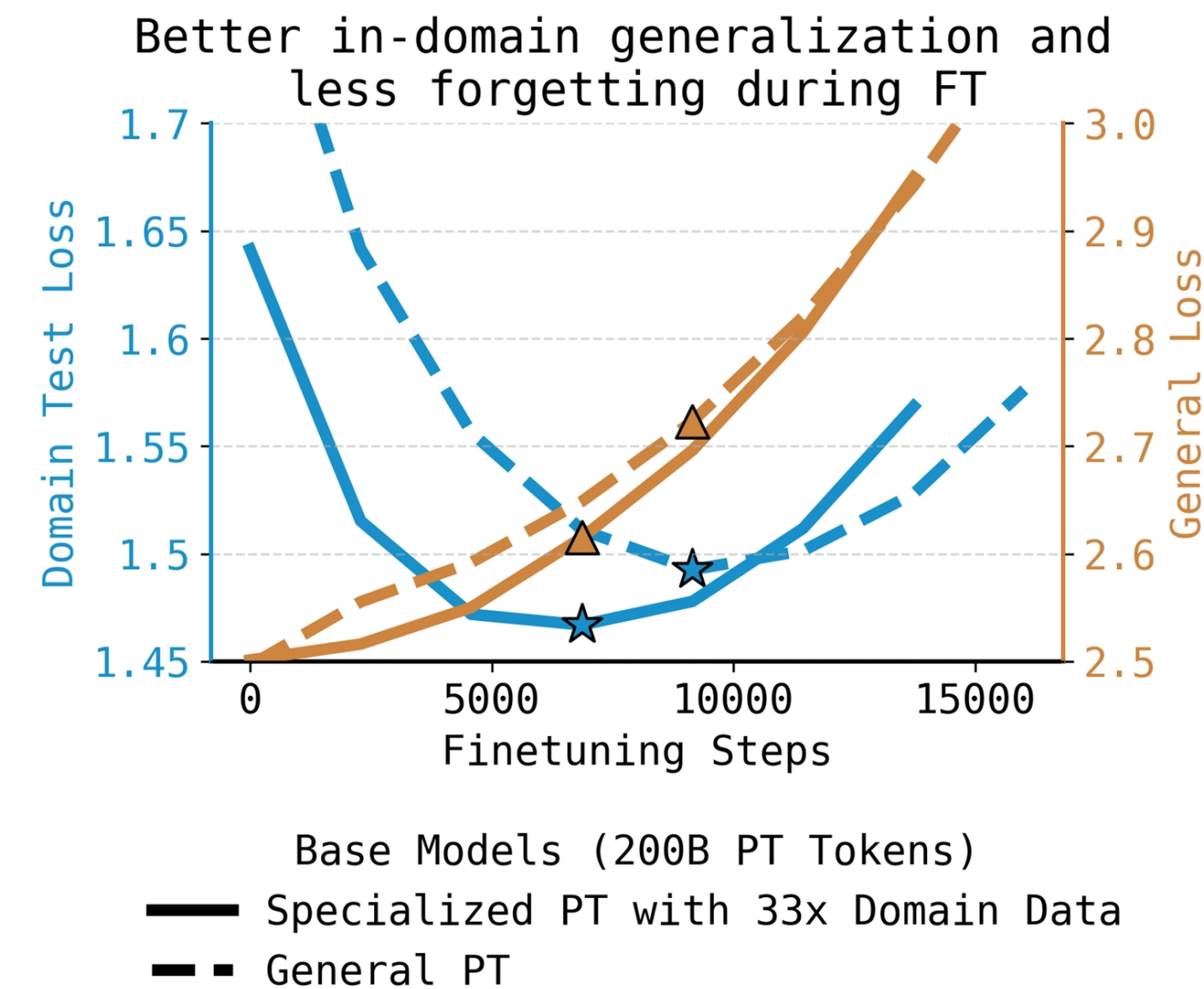
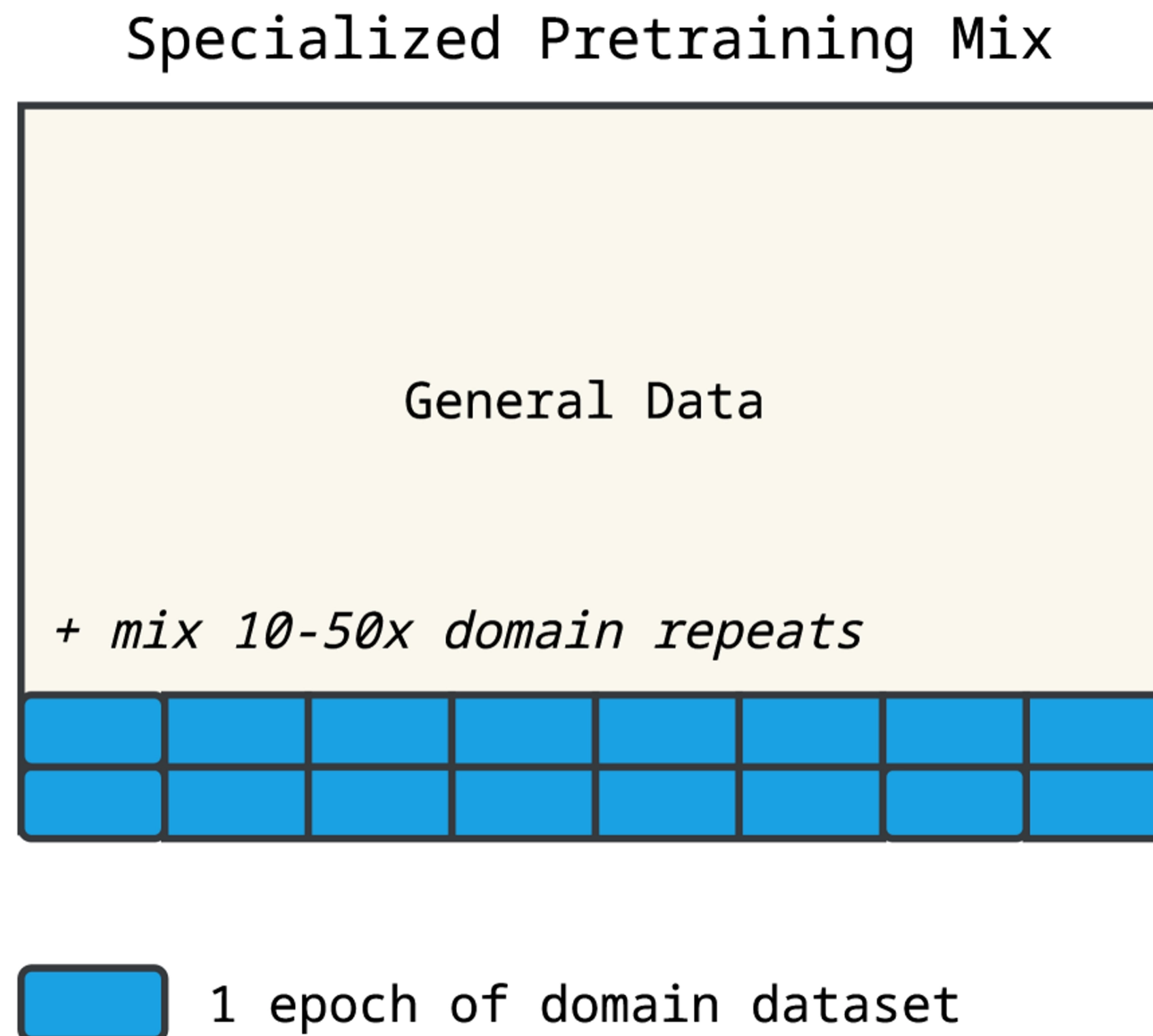
Midtraining

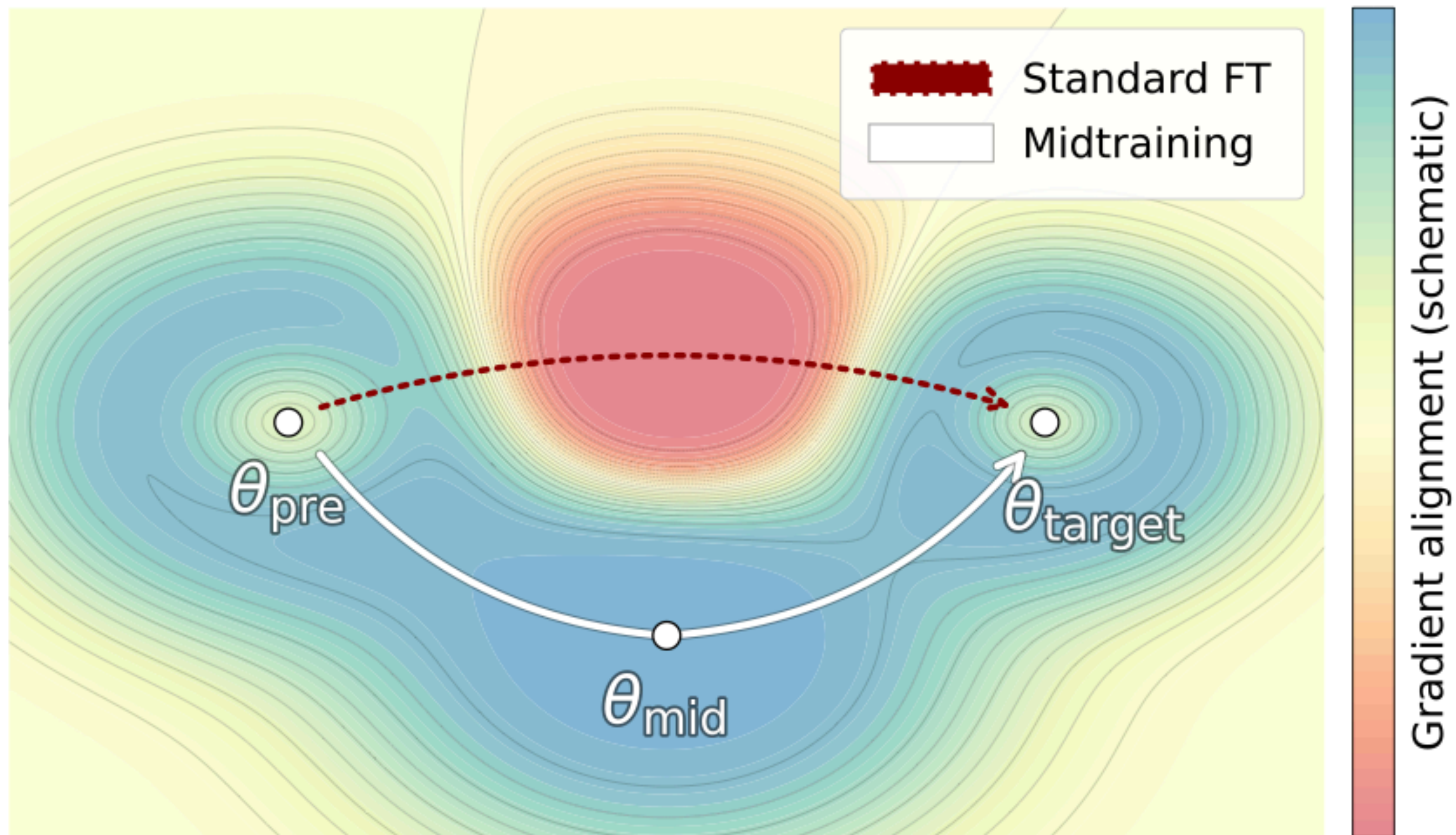


Change your data mix to be focused more on your target domains (math, code, etc)



Mixing in more of your target domain data into “pretraining” tends to help

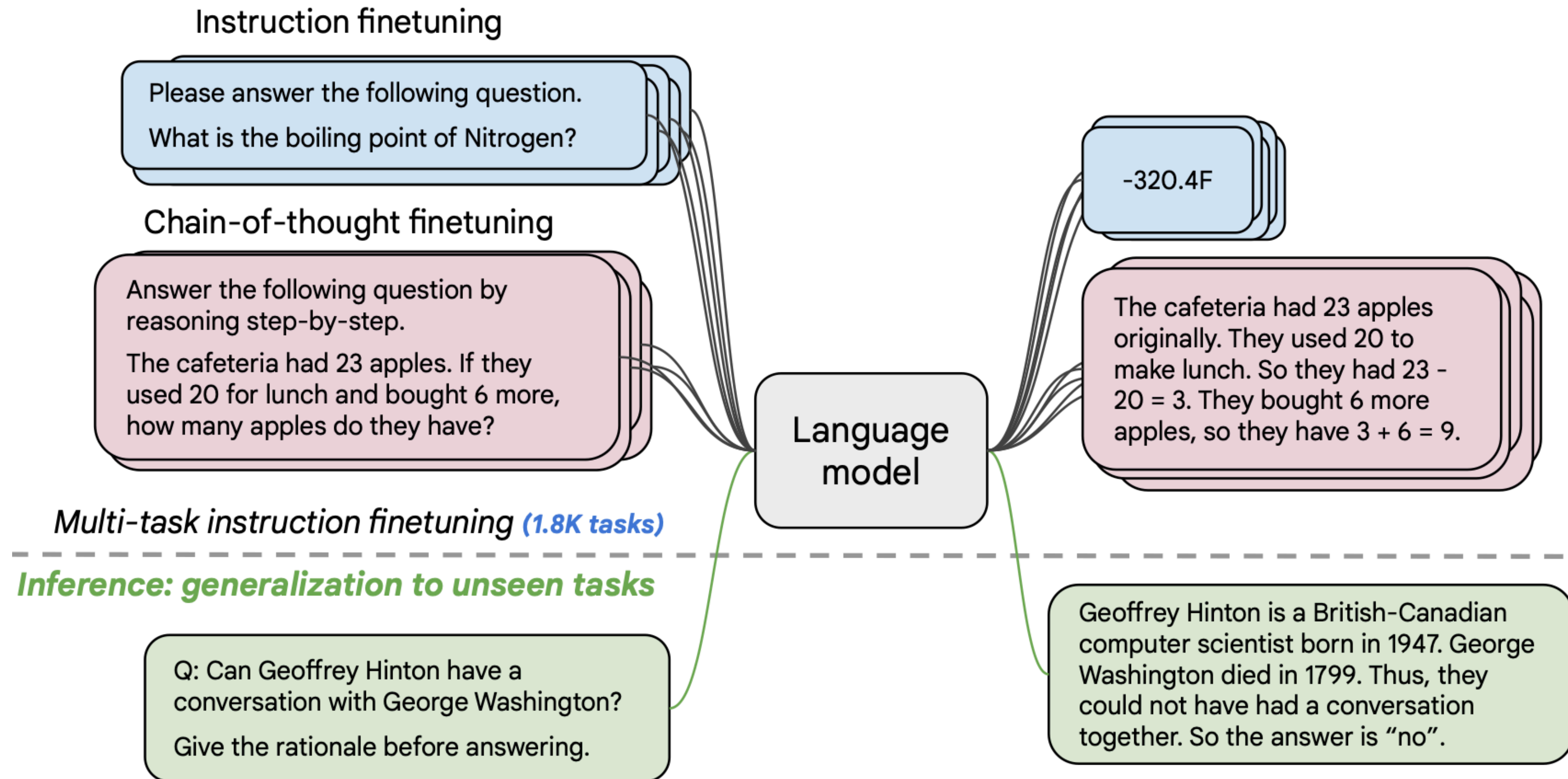




Posttraining



Instruction Finetuning

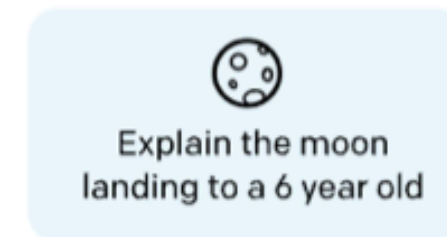


Learning from Human Preferences

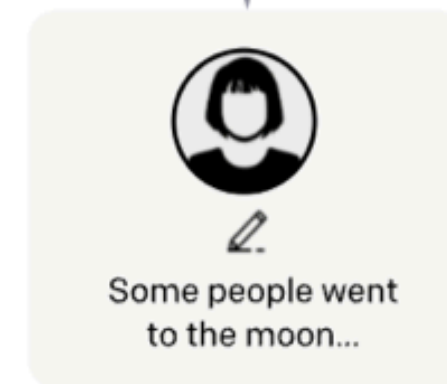
Step 1

Collect demonstration data, and train a supervised policy.

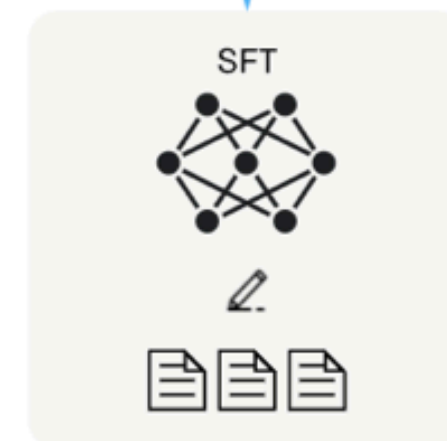
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



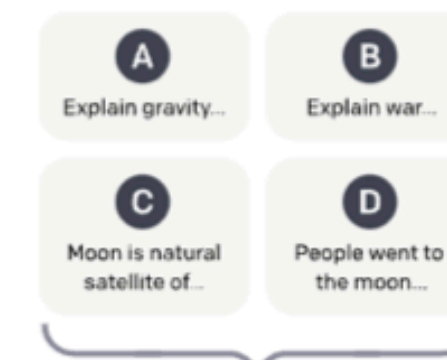
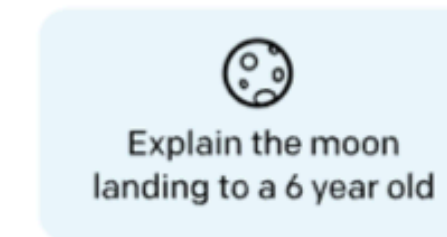
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

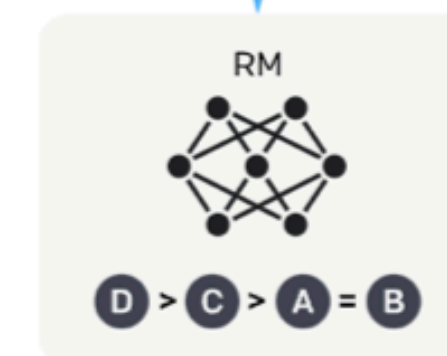
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



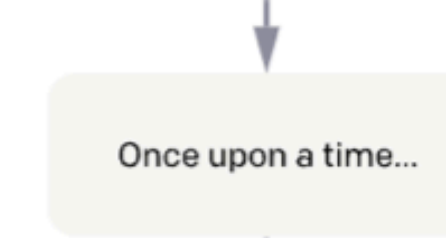
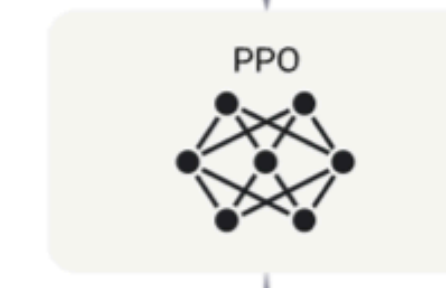
Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



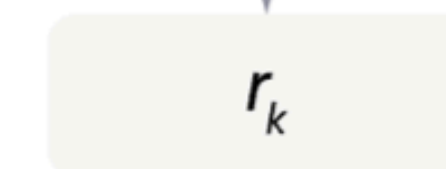
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Problems with Human Feedback

- Human annotators can be unreliable
- Everyone has different preferences
- Leads to bad incentives for model optimization (sycophancy, authoritativeness)
- The data itself can be sourced unethically

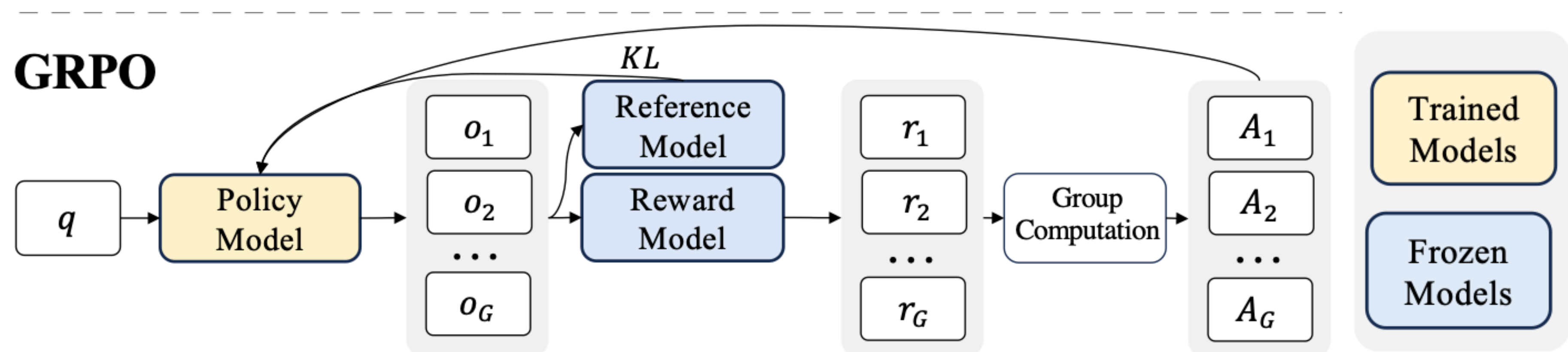


Reinforcement Learning with Verifiable Rewards (RLVR)

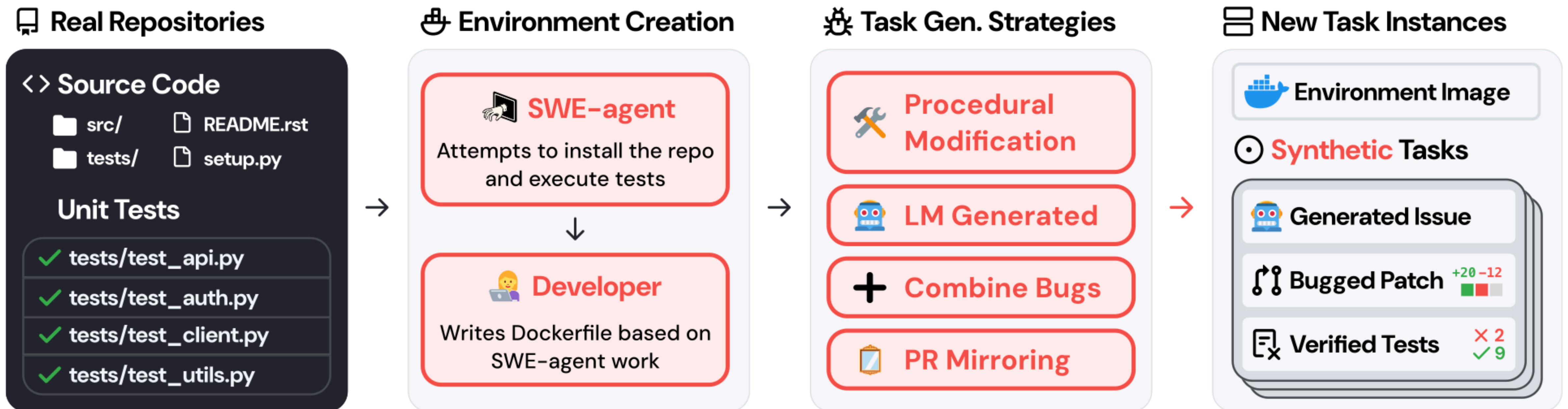
- Make the reward verifiable
- Great for math and code!
- Need to be careful about the verifier design

Verifiable Rewards

- Unit Tests
- Lean Proof
- Exact Match
- System state
- ...



Agent-Specific Training (E.g. SWE-Smith)



SWE-Smith

Model	System	Train Size	Lite	Verified
<i>Closed Weight Models</i>				
GPT-4o (OpenAI, 2024a)	Agentless	-	32.0	38.8
	OpenHands	-	22.0	-
	SWE-agent	-	18.3	23.0
Claude 3.5 Sonnet (Anthropic, 2024)	Agentless	-	40.7	50.8
	AutoCodeRover	-	-	46.2
	OpenHands	-	41.7	53.0
	SWE-agent	-	23.0	33.6
Claude 3.7 Sonnet (Anthropic, 2025)	SWE-agent	-	48.0	58.2
Llama3-SWE-RL-70B (Wei et al., 2025)	Agentless	11M	-	41.0
<i>Open Weight Models</i>				
Lingma-SWE-GPT-72B (Ma et al., 2024)	SWE-SynInfer	-	-	28.8
Qwen3-235B-A22B (Qwen et al., 2025)	OpenHands	-	-	34.4
R2E-Gym-32B (Jain et al., 2025)	OpenHands	3.3k	-	34.4
SWE-fixer-72B (Xie et al., 2025a)	SWE-Fixer	110k	24.7	32.8
SWE-gym-32B (Pan et al., 2024)	OpenHands	491	15.3	20.6
SWE-agent-LM-7B	SWE-agent	2k	11.7	15.2
SWE-agent-LM-32B	SWE-agent	5k	30.7	40.2



LLM Inference

Language Models?

Architecture

Training

Inference

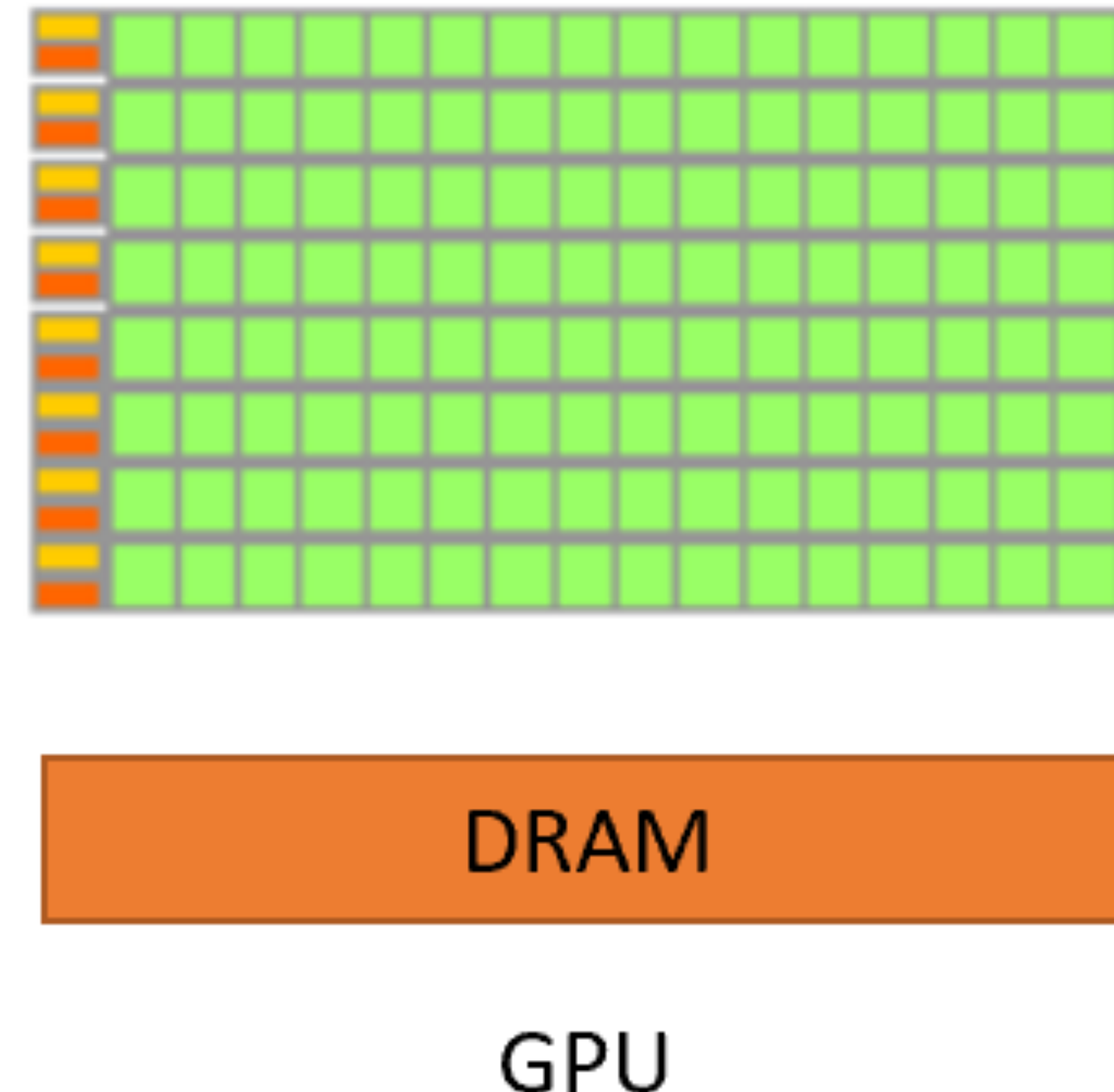
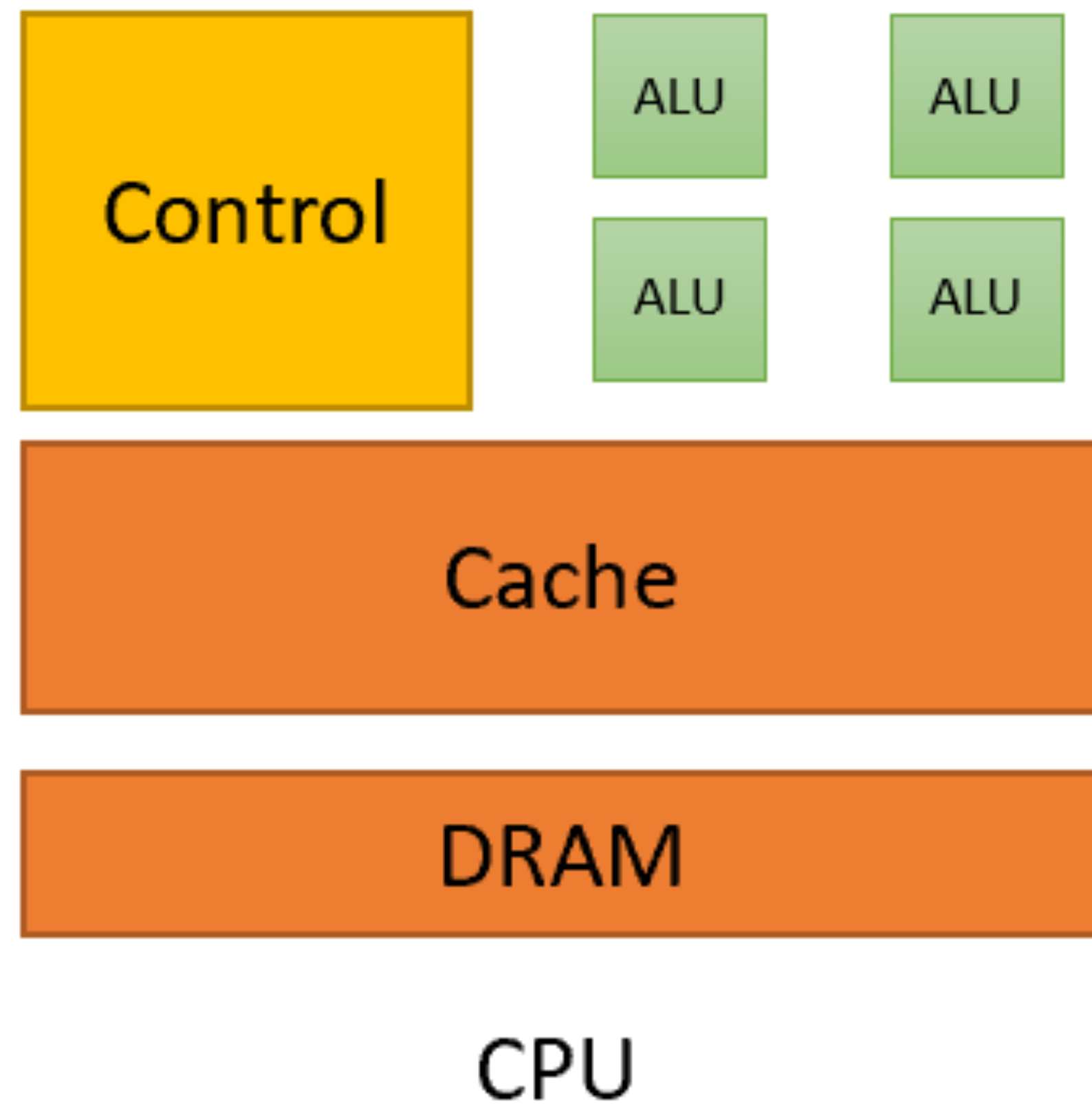
LLMs for Agents



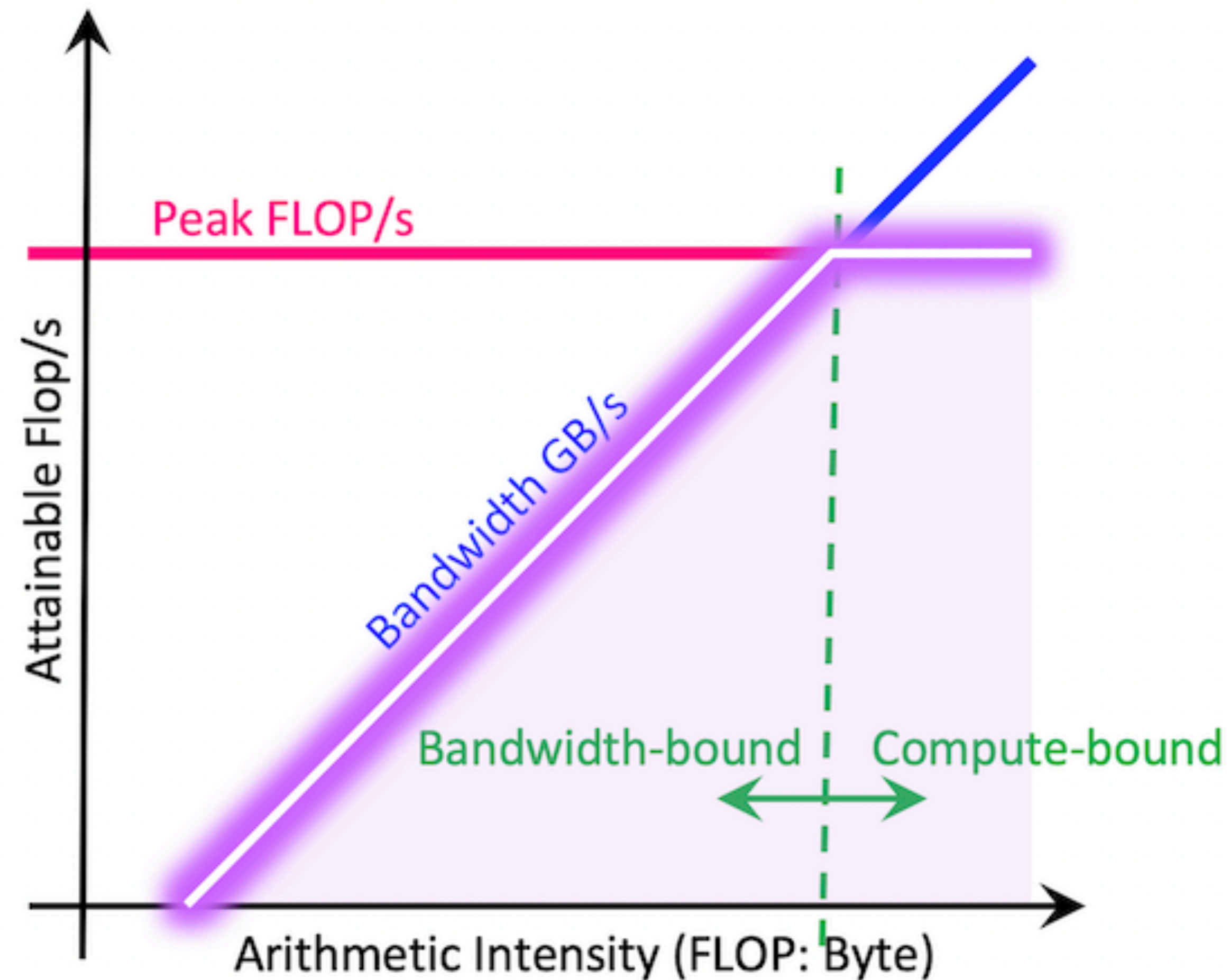
Prefill and Decode



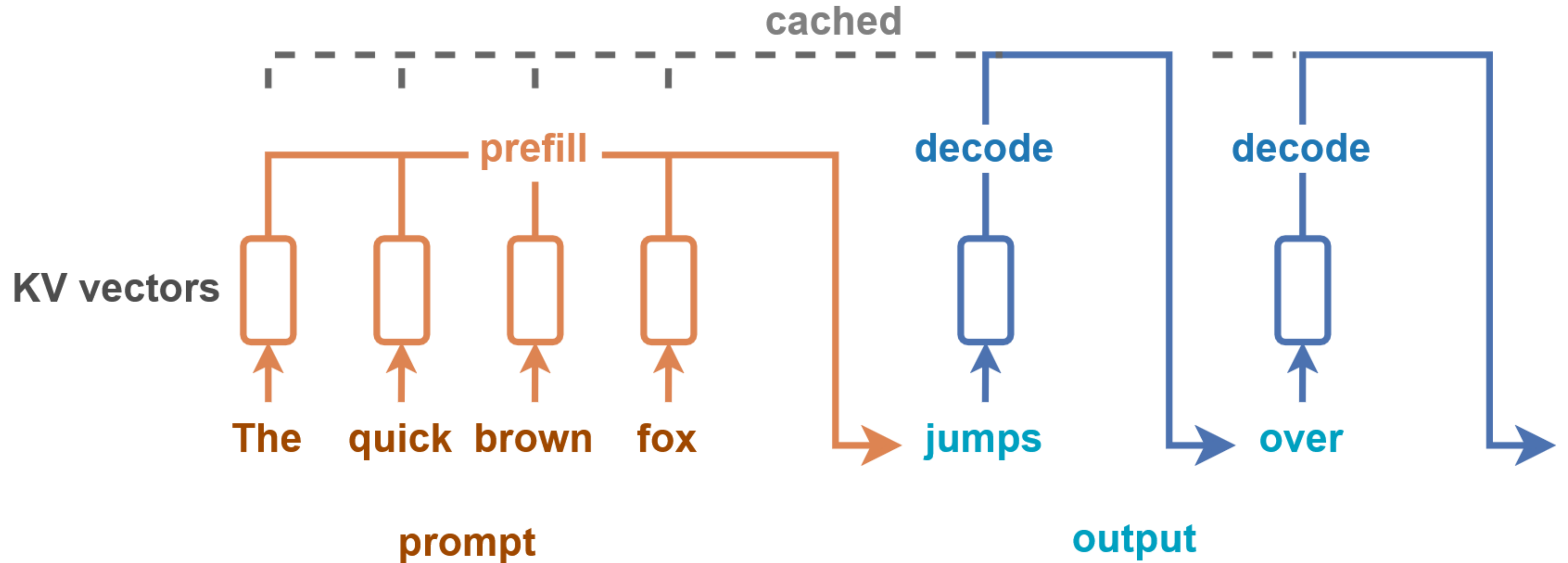
GPUs



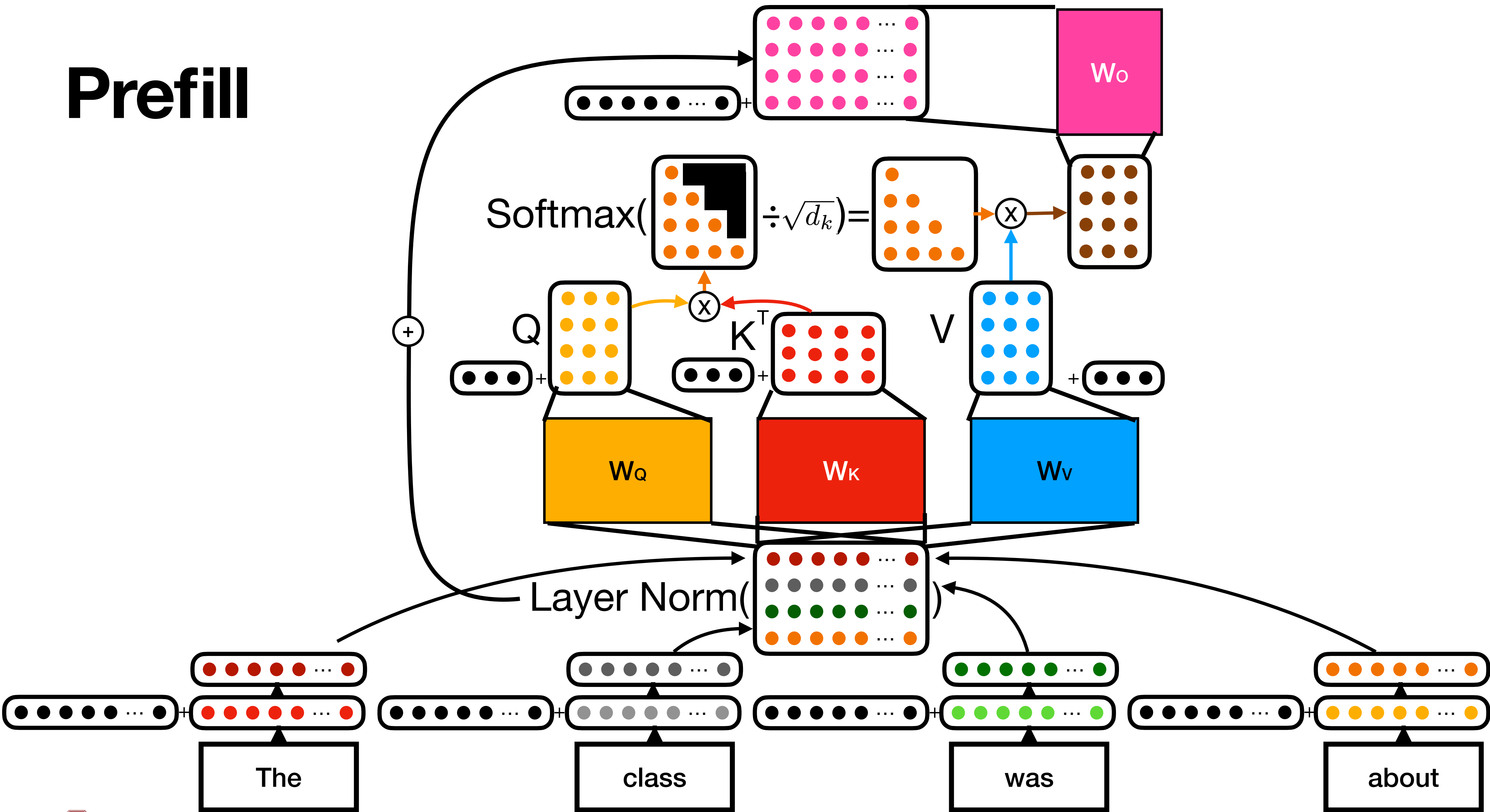
Roofline Performance Model



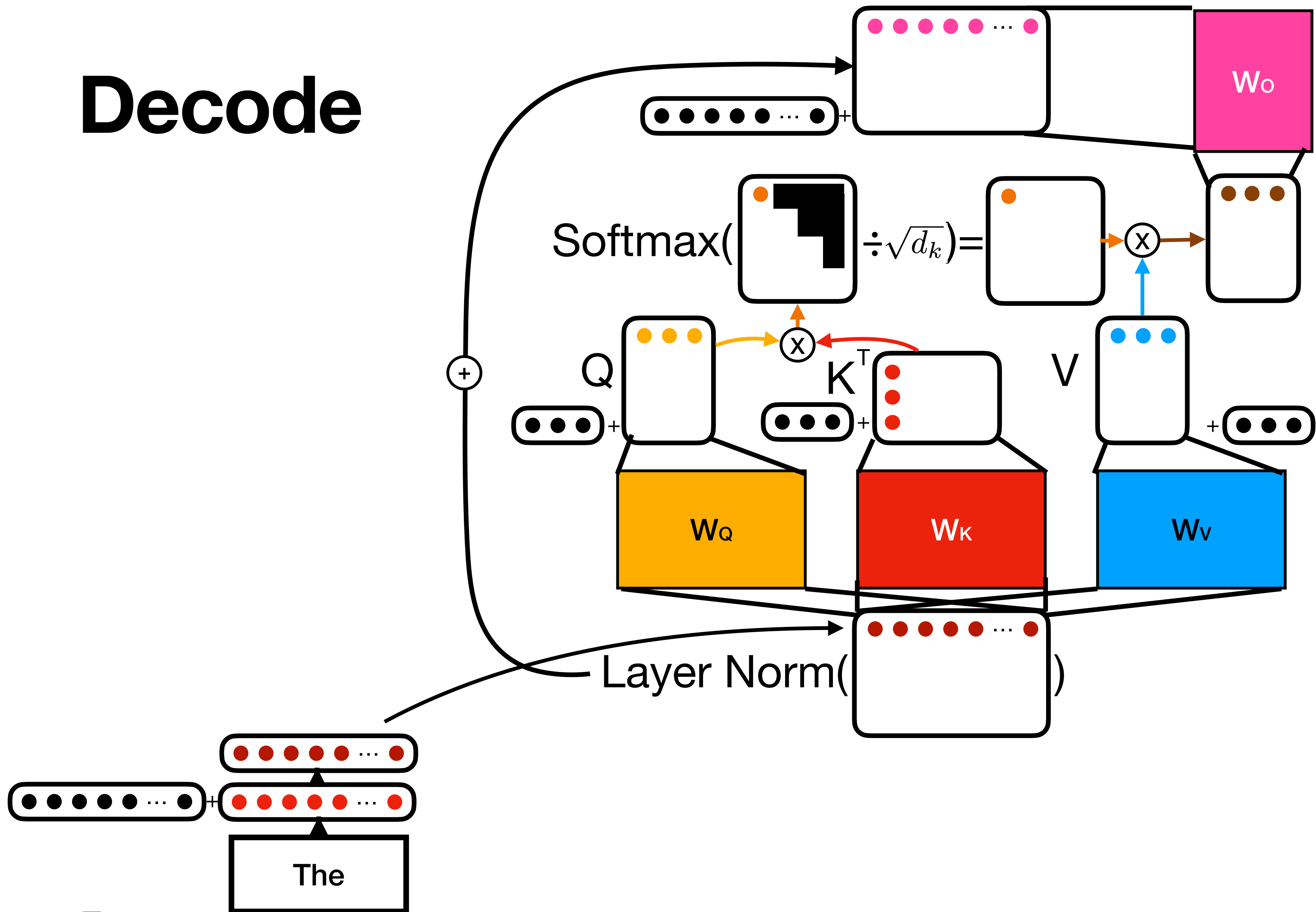
Prefill and Decode



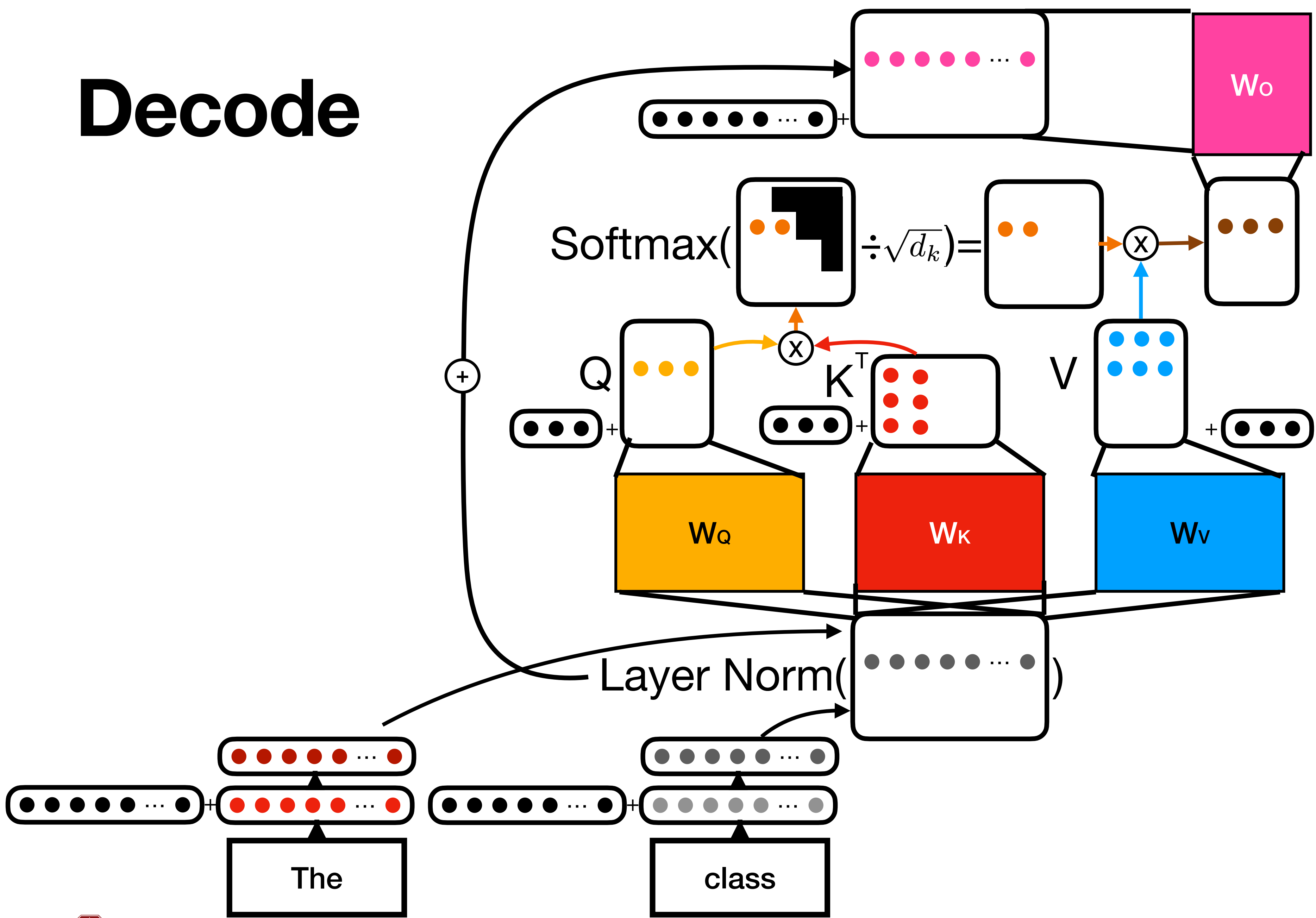
Prefill



Decode

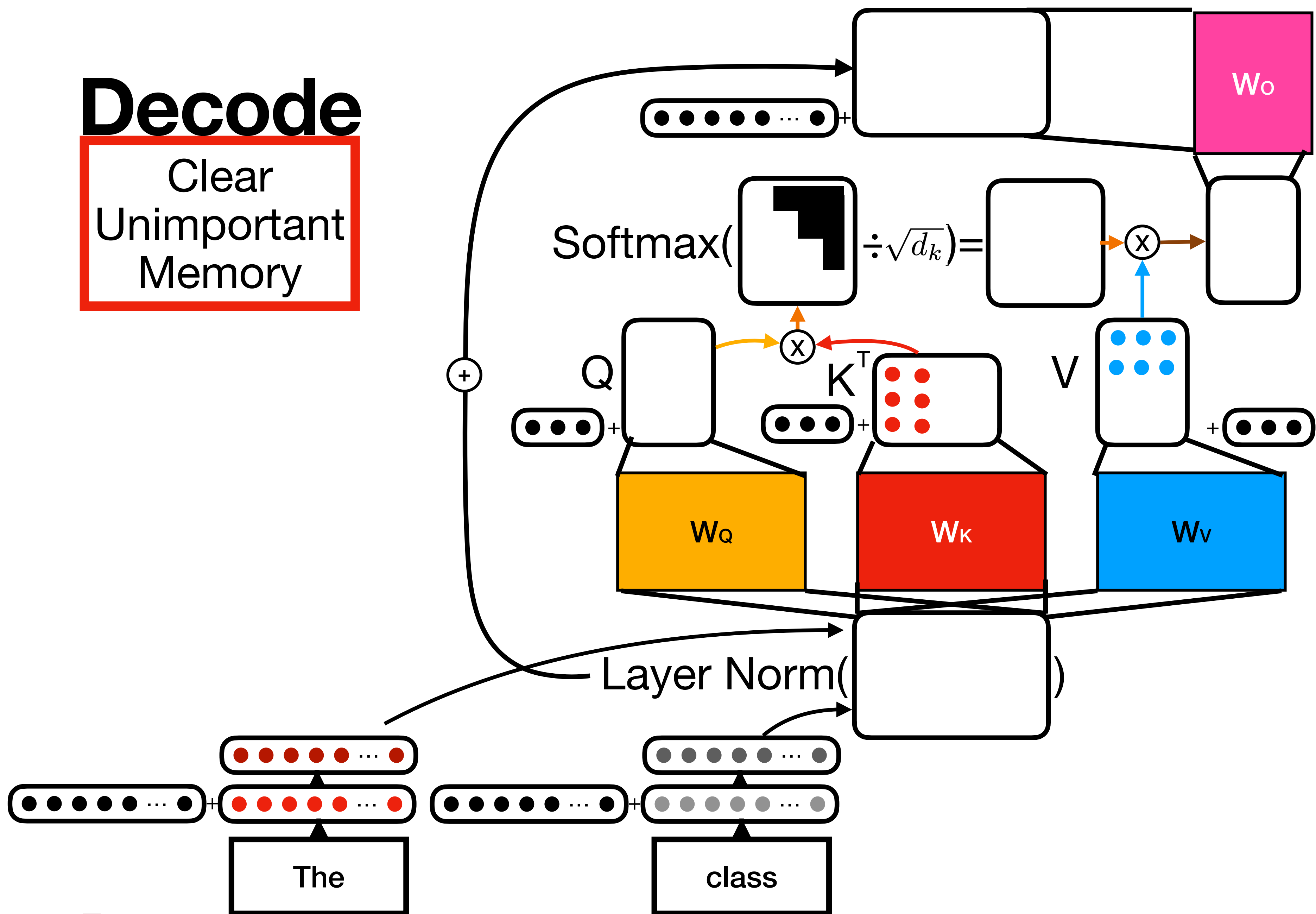


Decode

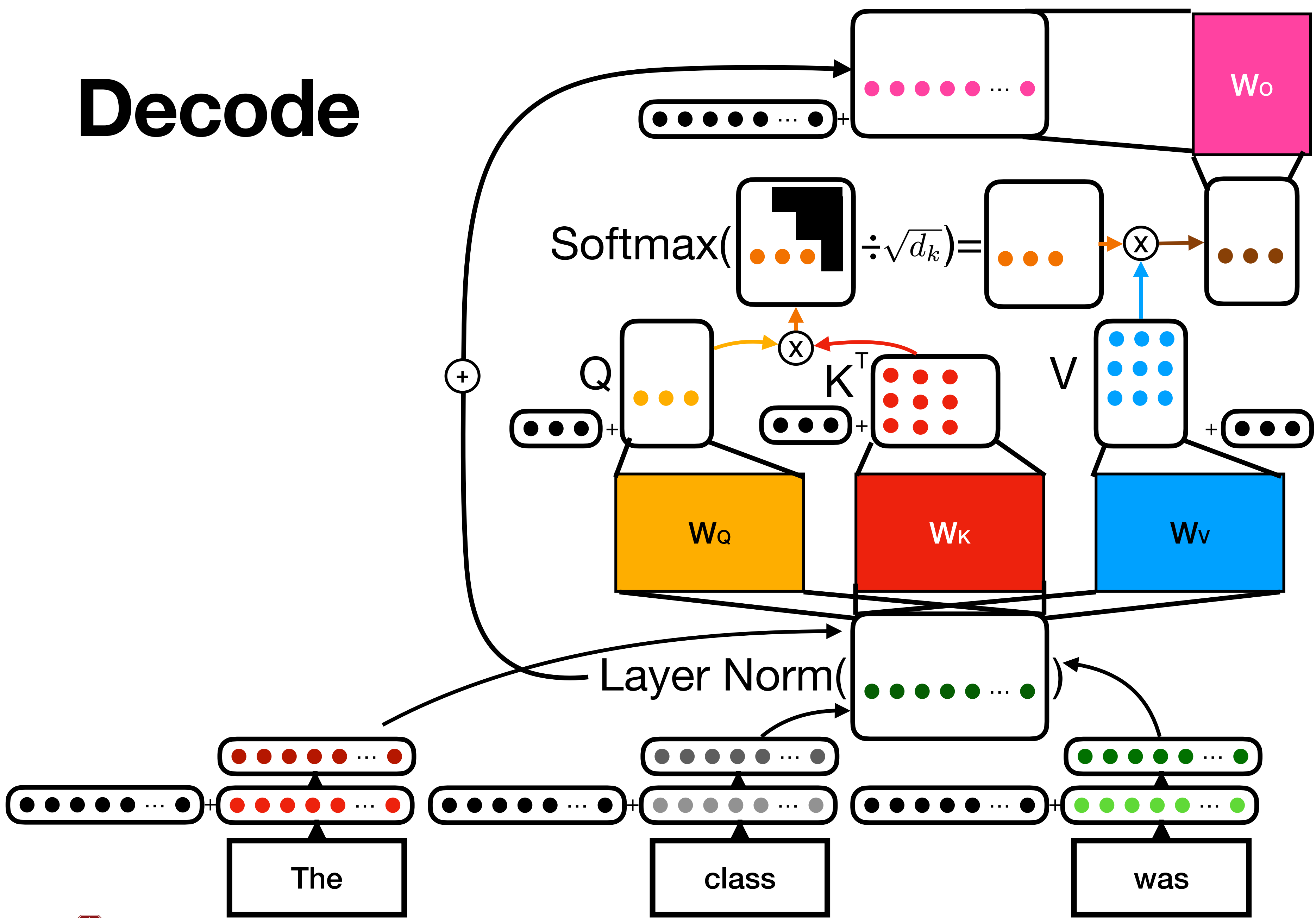


Decode

Clear
Unimportant
Memory

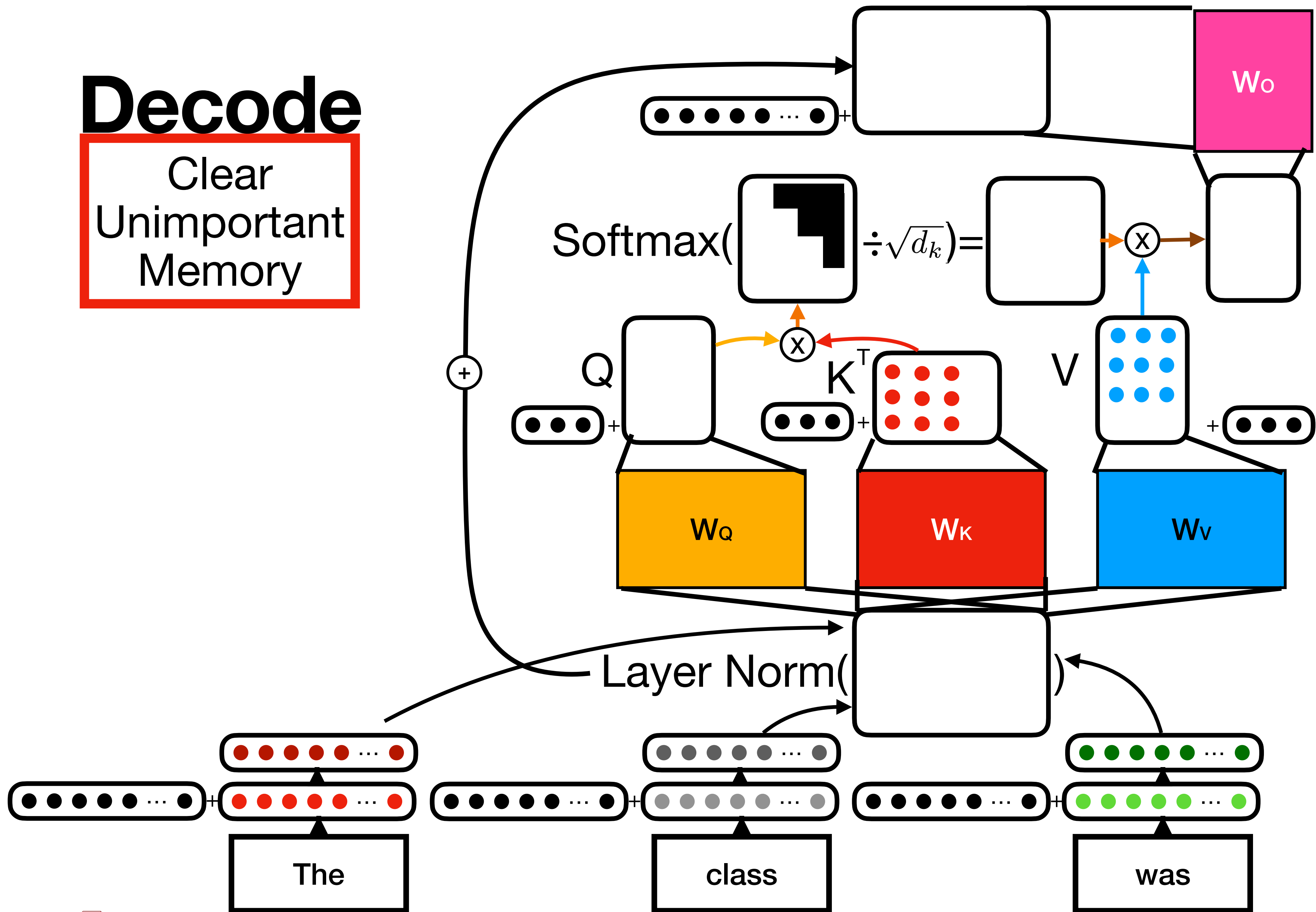


Decode

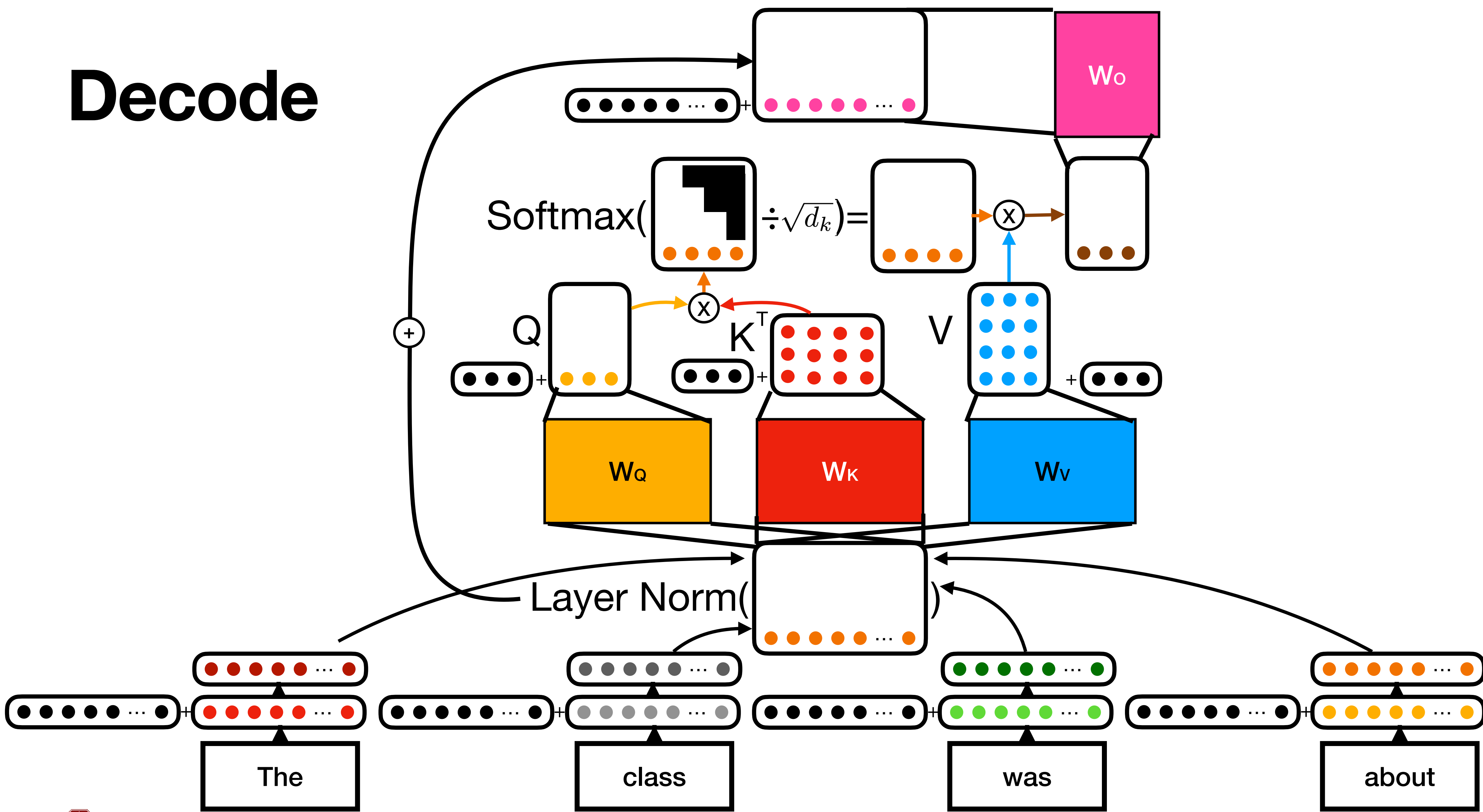


Decode

Clear
Unimportant
Memory

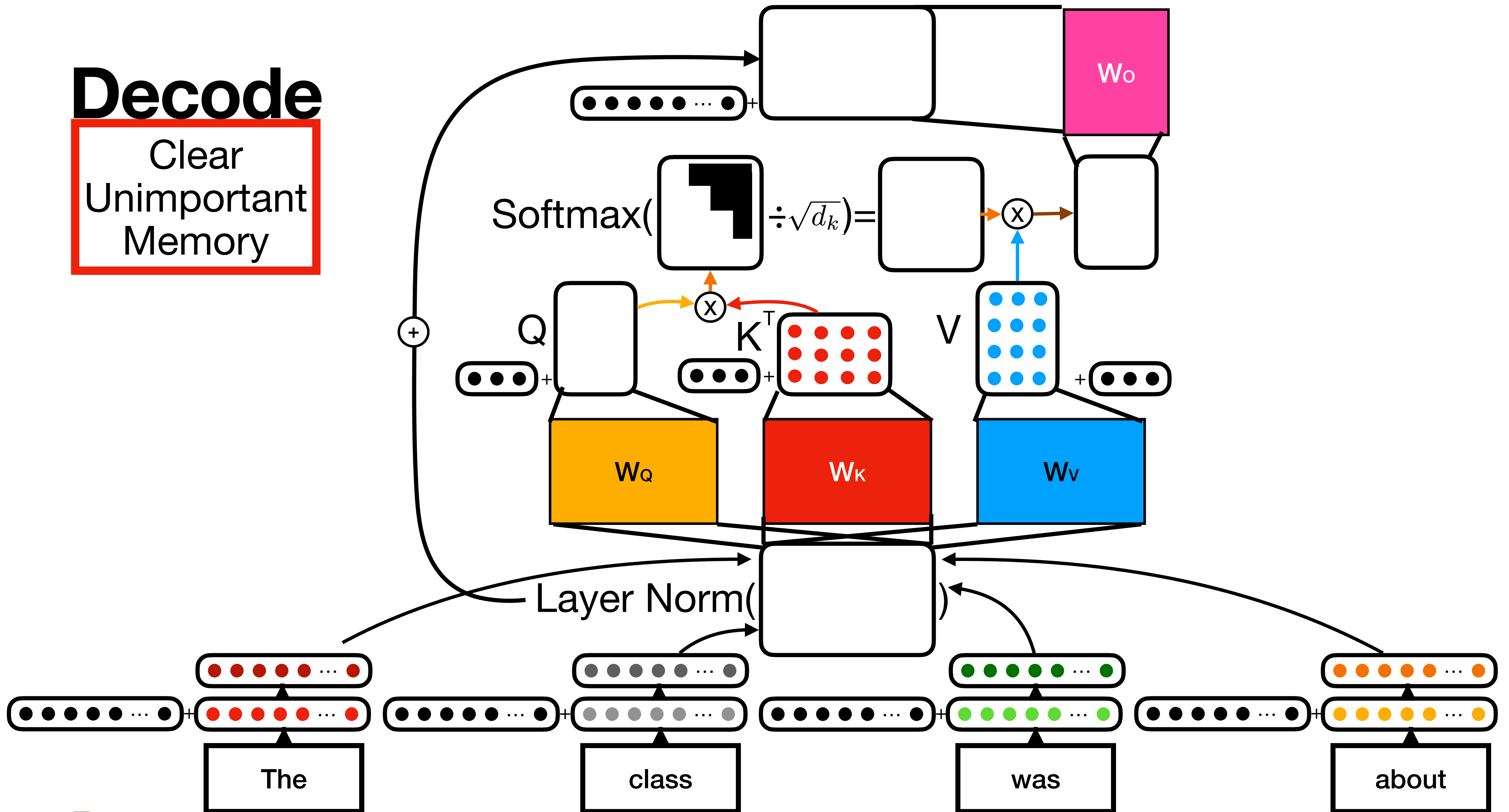


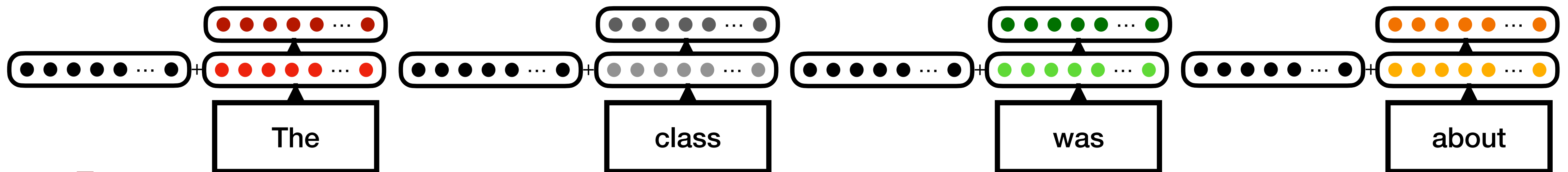
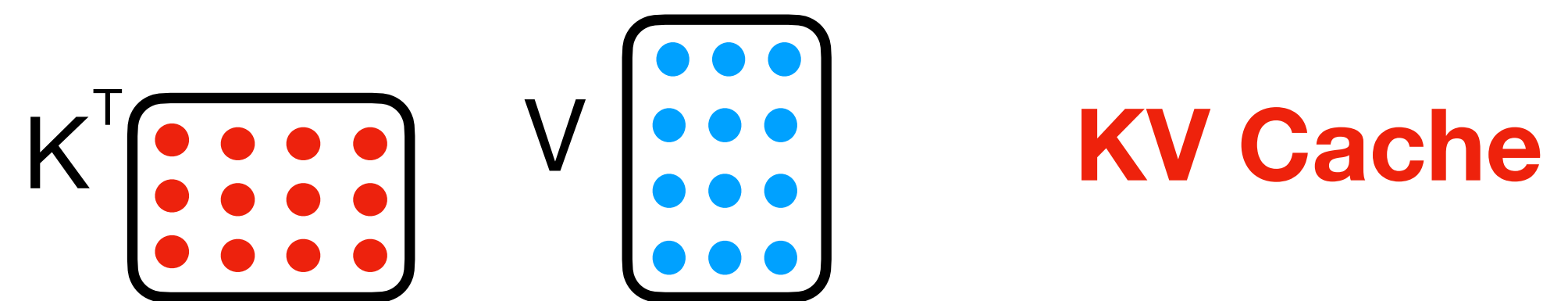
Decode



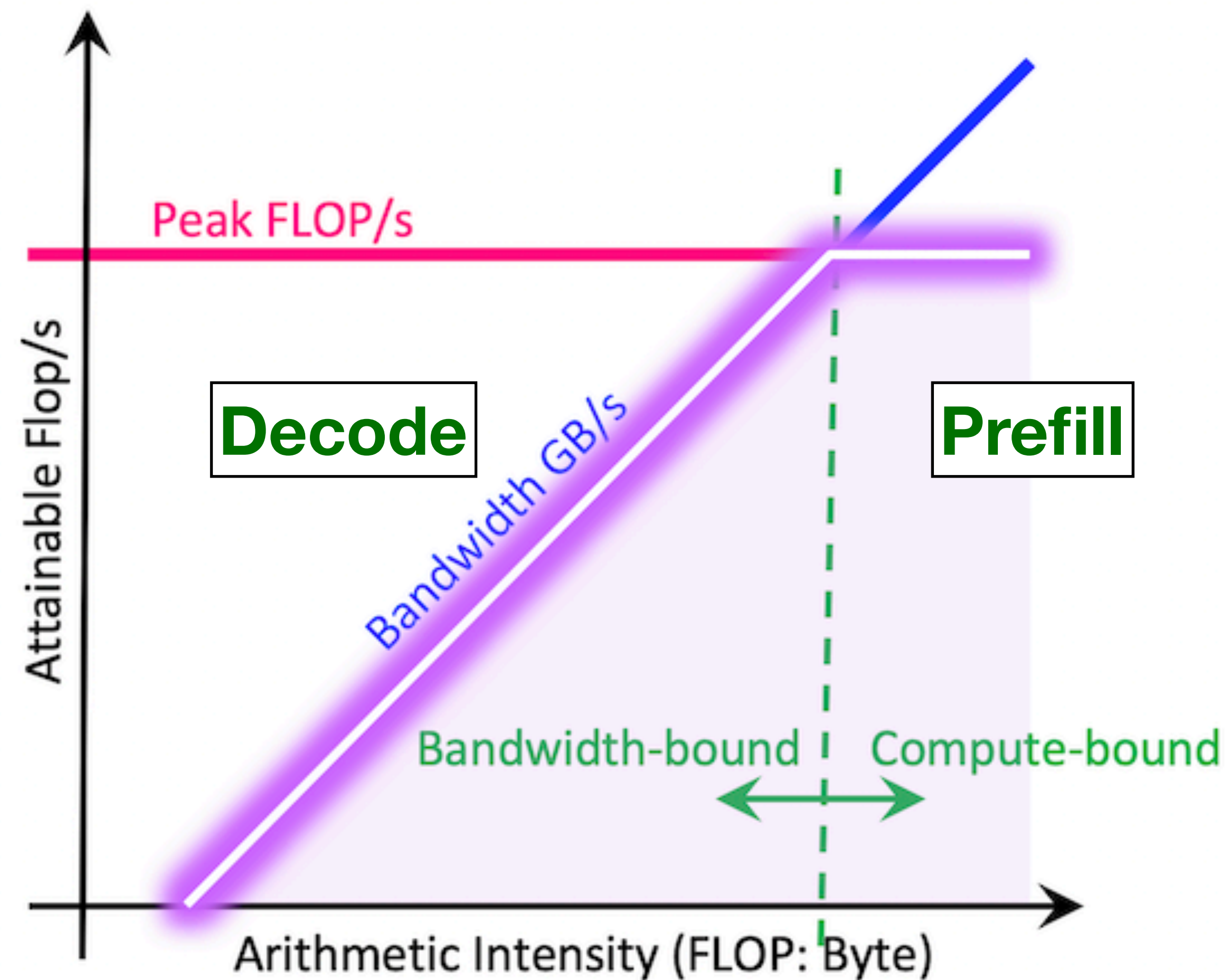
Decode

Clear
Unimportant
Memory

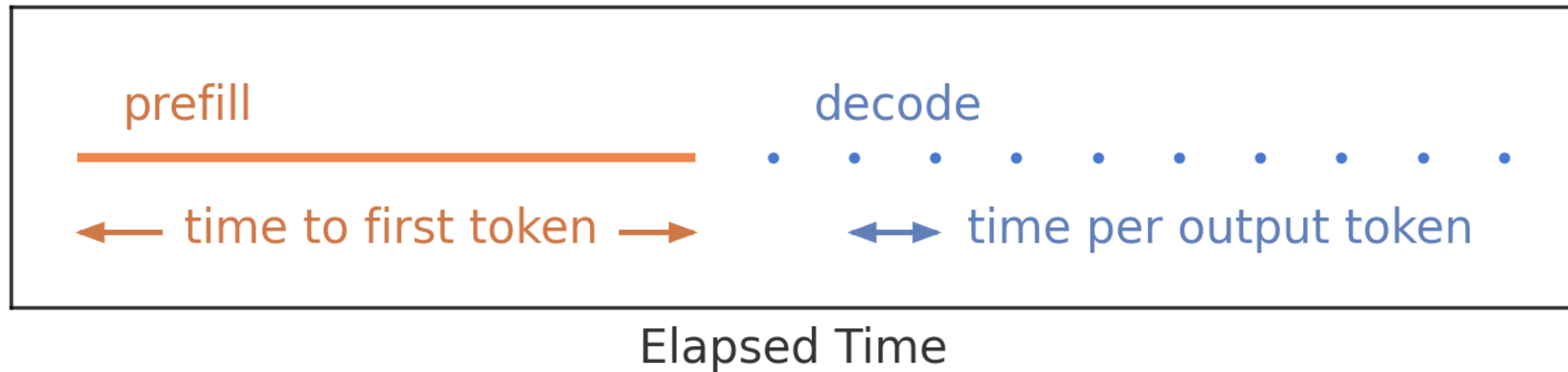




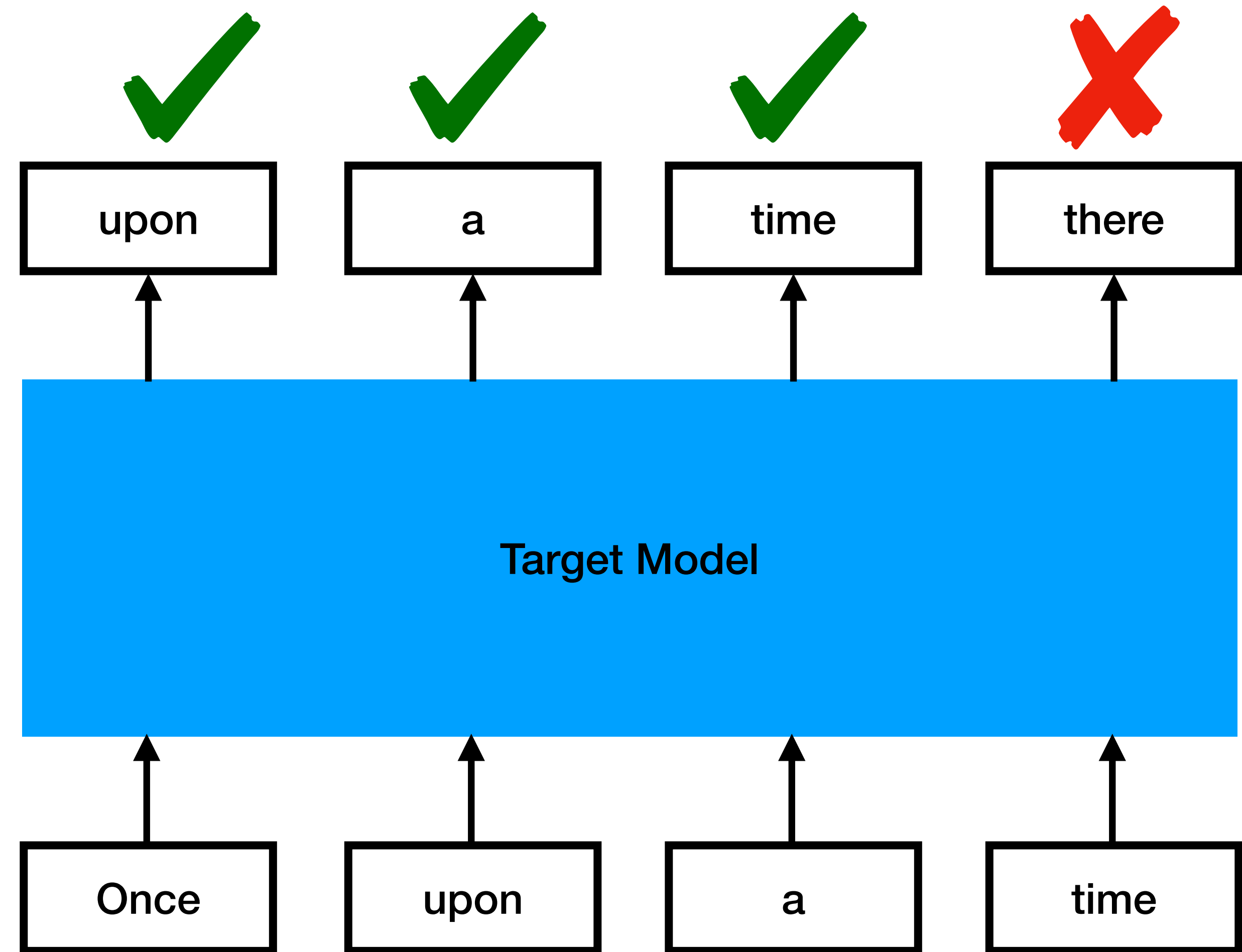
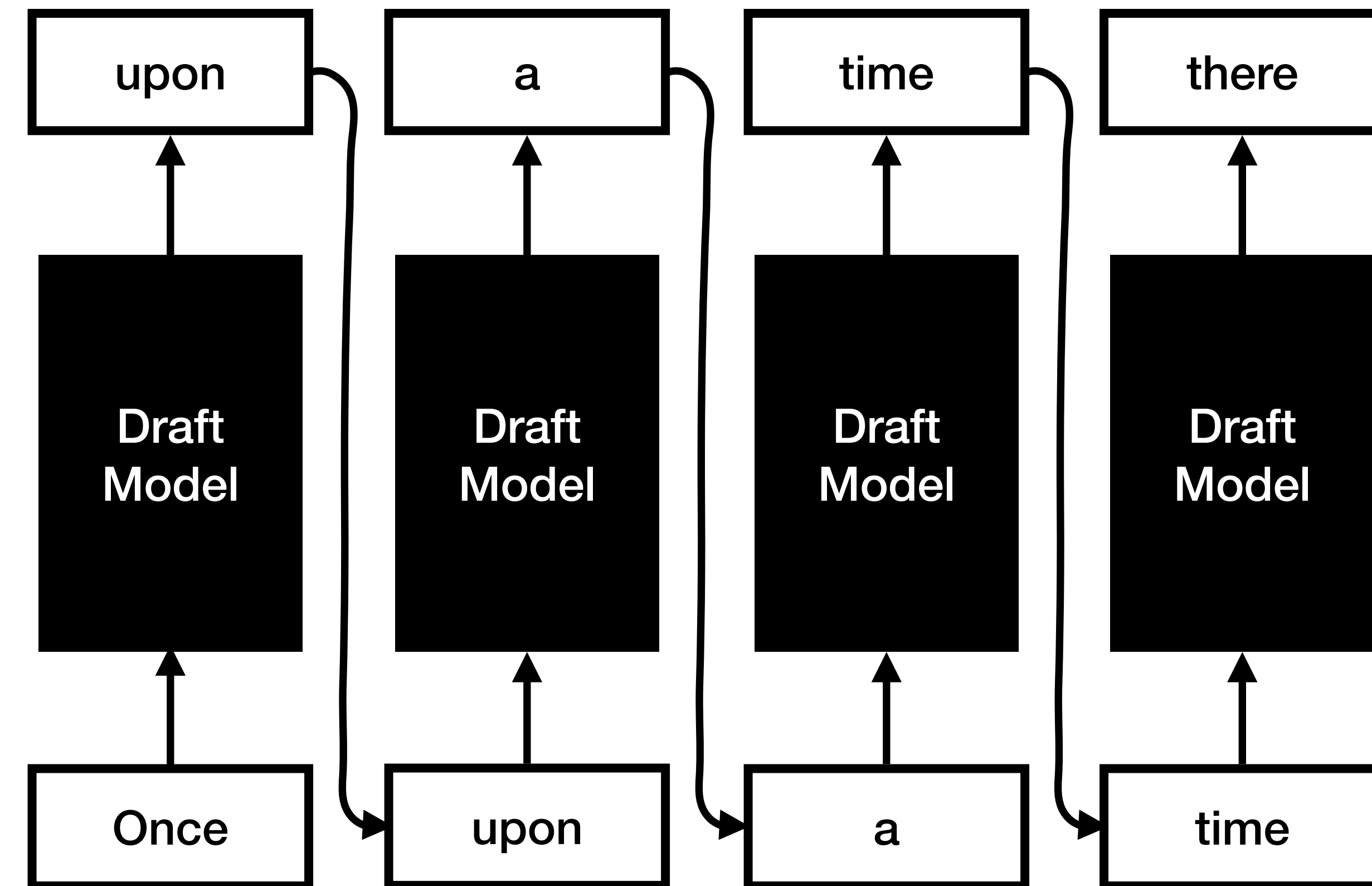
Roofline Performance Model



Metrics

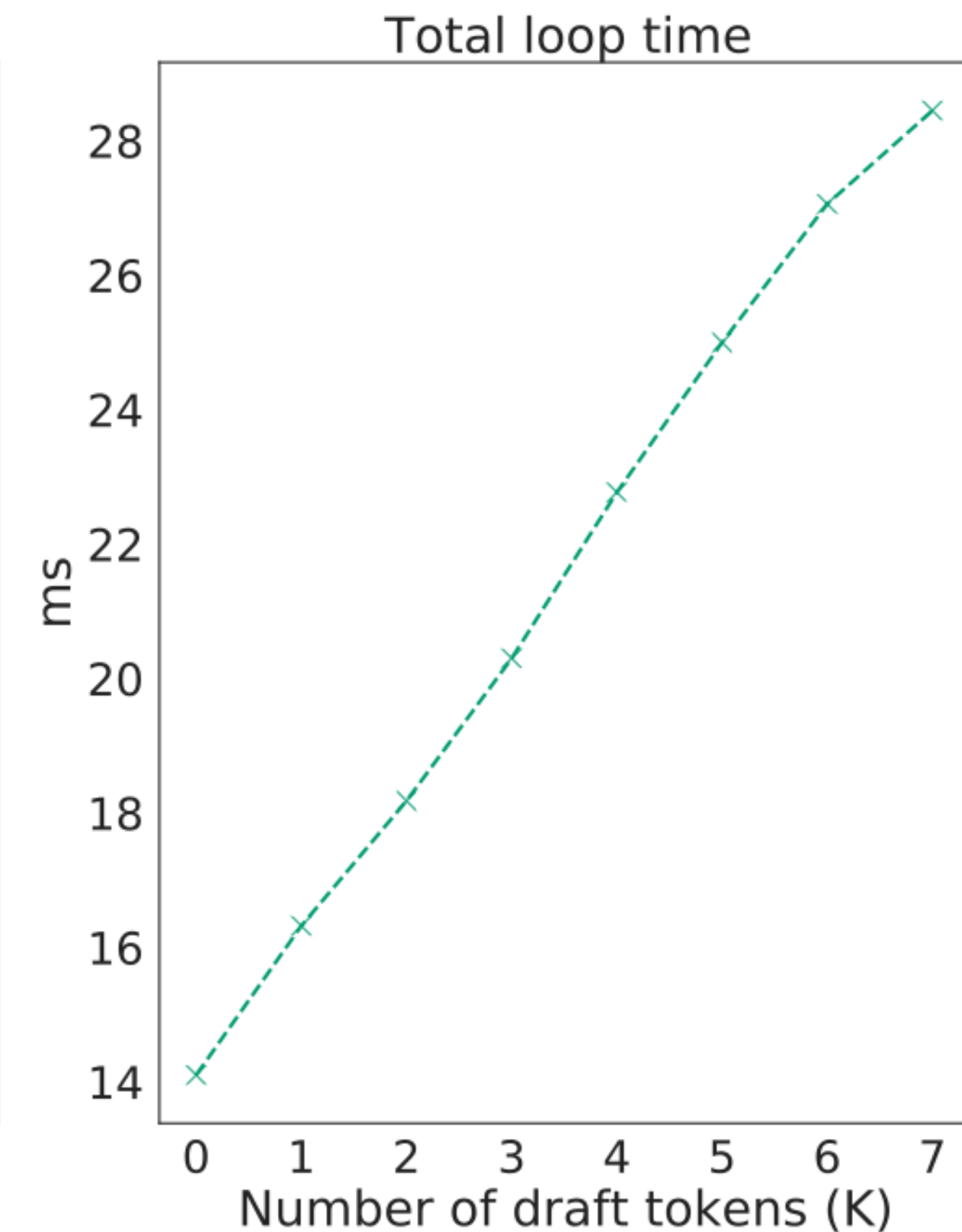
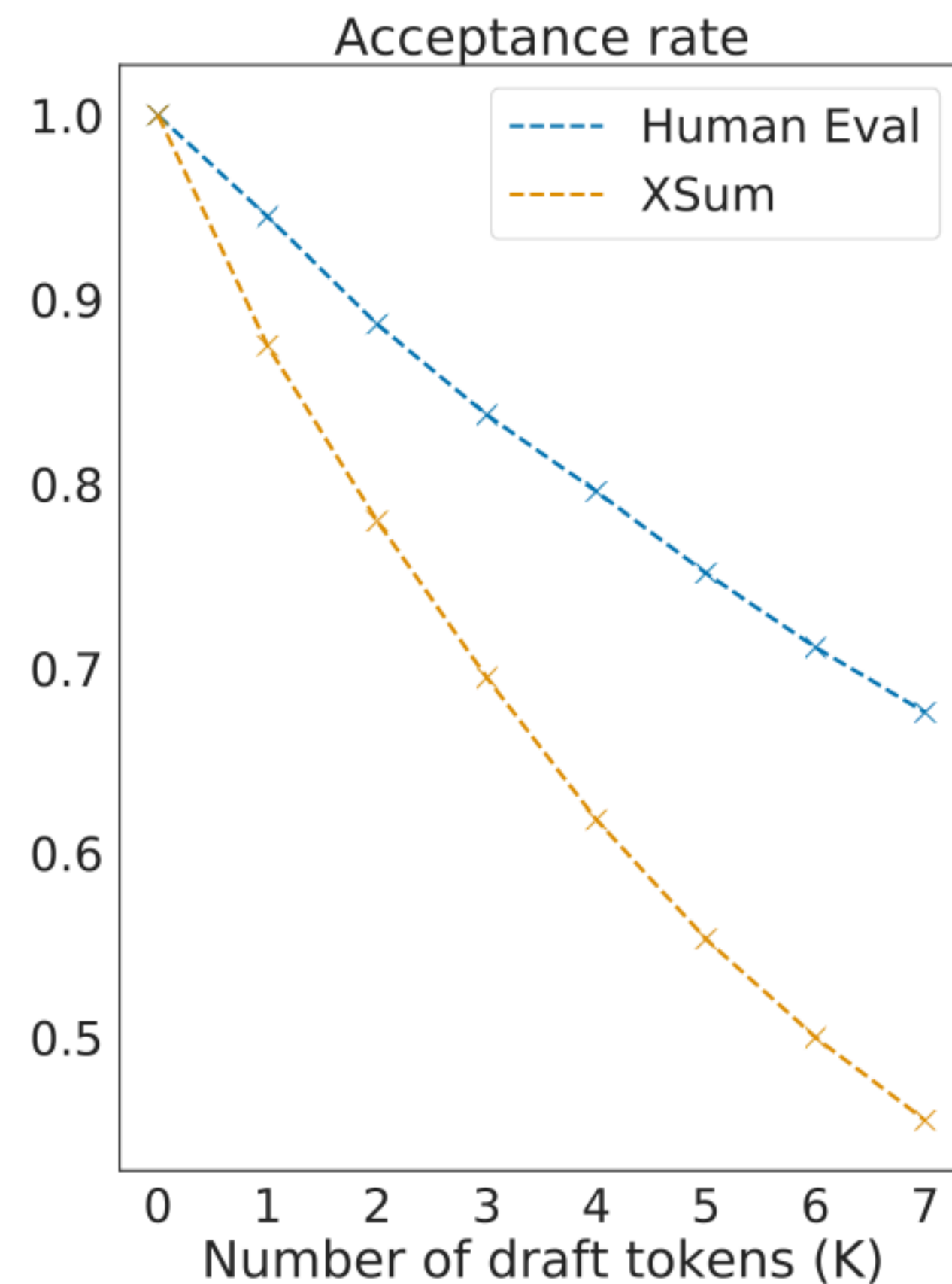
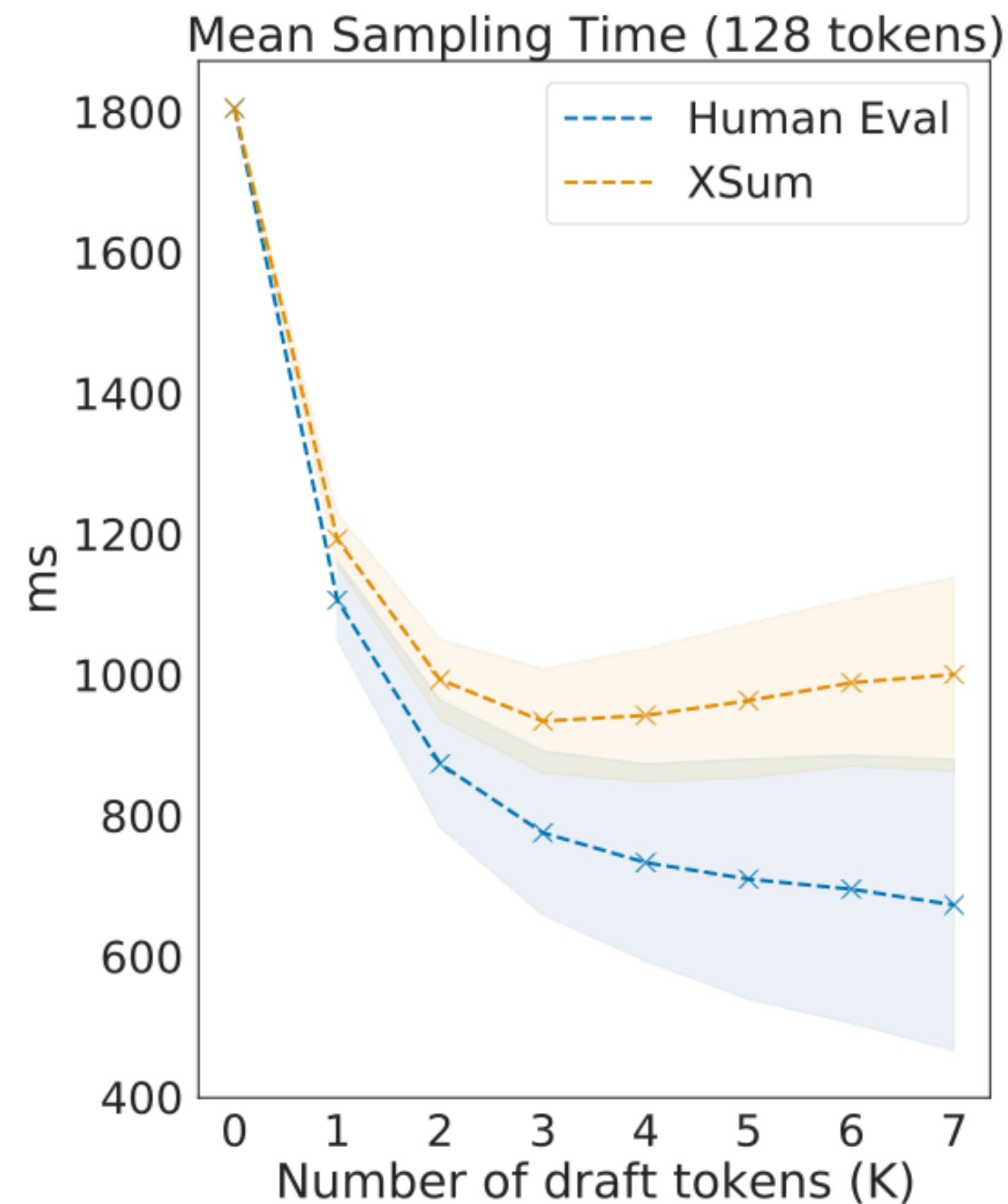


Speculative Decoding



Speculative Decoding

Some domains are more predictable than others



Inference Time Scaling (Test Time Scaling)



Chain of Thought

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅



Reasoning LLM

Query

A jacket costs \$80. The store raises its price by 25%, then offers a 25% discount on the new price. What is the final price?

Traditional LLM

After the 25% increase, the jacket costs:

$$\$80 \times 1.25 = \$100.$$

The 25% discount applies to that new price:

$$\$100 \times 0.75 = \$75.$$

Reasoning LLM

Adding 25% and subtracting 25% appears to produce no change, suggesting a final price of \$80.

Wait... That cancellation would require both percentages to use the same base amount. Here, the discount applies to the increased price.

So the increase is 25% of \$80, or \$20, bringing the price to \$100. Then the discount is 25% of \$100, or \$25, bringing the price to \$75.

Let me check that. $\$80 + \$20 - \$25 = \75 . Okay.

The jacket's final price is \$75, which is \$5 less than its original price.



Reasoning LLM

“Aha” moments

Prompt

A movie ticket and a popcorn together cost \$18.
The ticket costs \$6 more than the popcorn.
How much does the ticket cost?

Response

<think>
Let p be the popcorn price.
Then the ticket costs $p + 6$.

So $p + 6 = 18$.
That gives $p = 12$, so the ticket is 18.

Wait, that can't be right.

I only used the ticket price, not the total of ticket + popcorn.

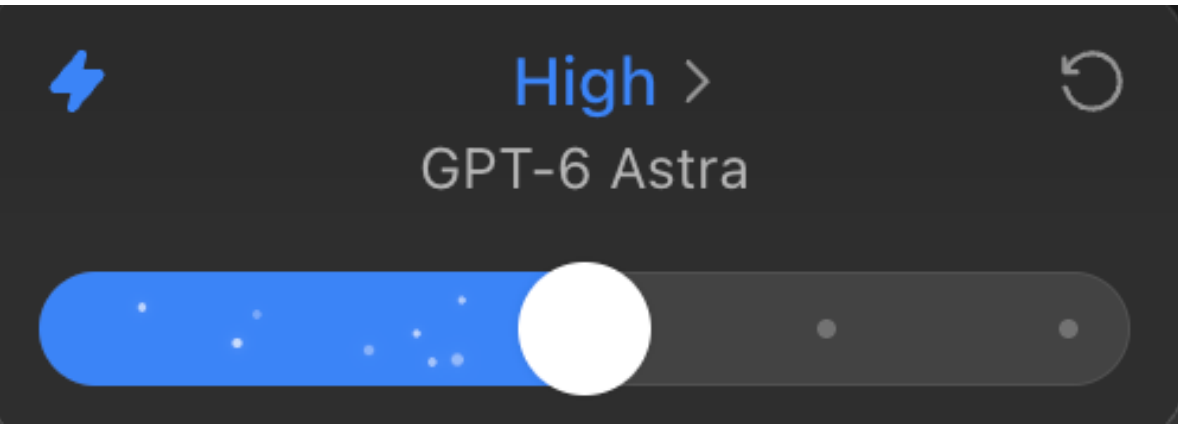
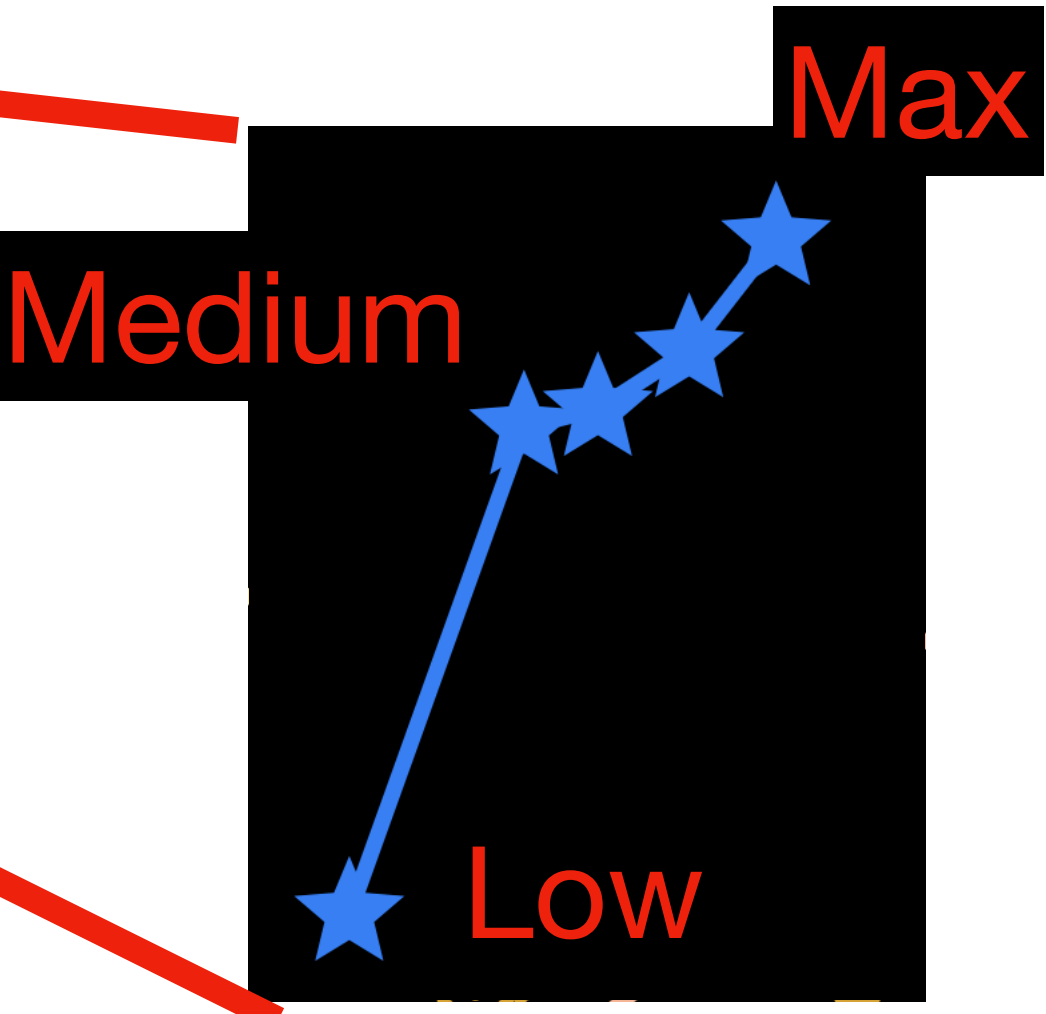
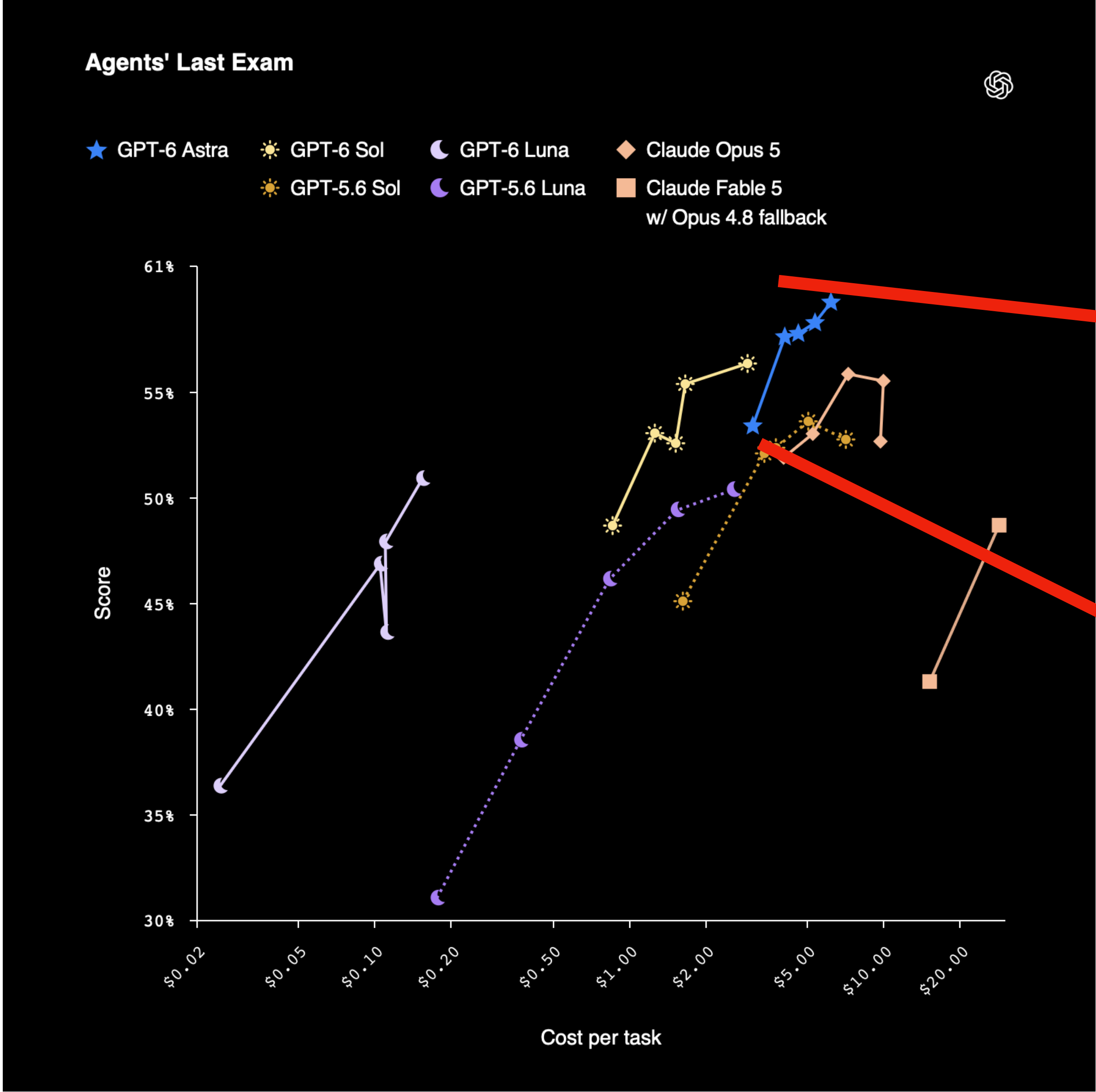
Aha, the total should be:
 $p + (p + 6) = 18$

So $2p + 6 = 18$.
 $2p = 12$.
 $p = 6$.

The ticket costs $p + 6 = 12$.
</think>
<answer>\boxed{12}</answer>

Model learns to
self-correct itself





Setting the Reasoning Level (GPT-OSS)

System Prompt

Reasoning: low

System Prompt

Reasoning: medium

System Prompt

Reasoning: high

User Prompt

Solve this math problem...



Some ways it is done:

Inkling (Thinking Machines)

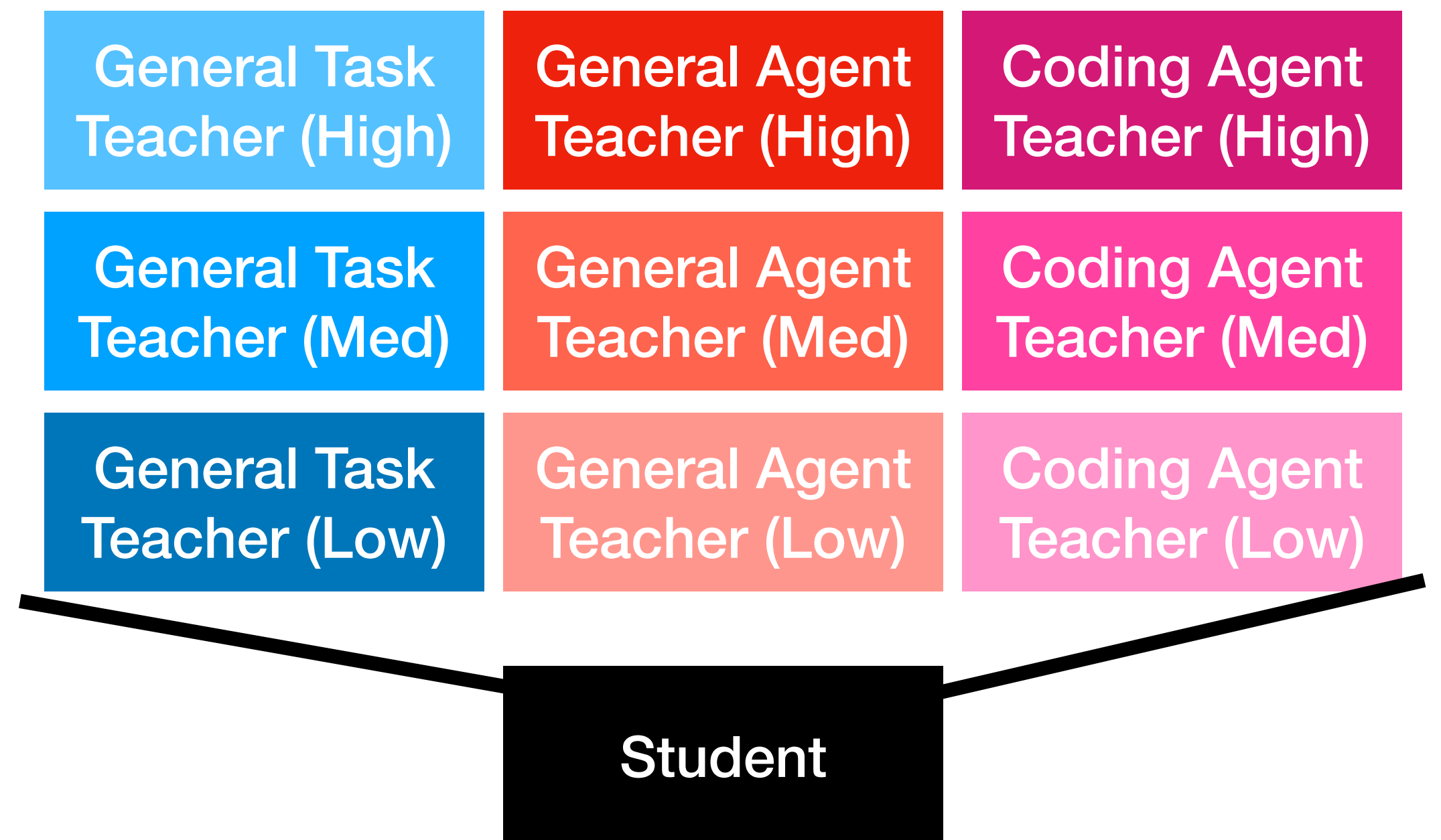
$$R(e) = R_{\text{task}} - \lambda(e)N_{\text{tokens}}$$

Set Lambda higher for low effort

Set Lambda lower for high effort

Kimi K3 (Moonshot)

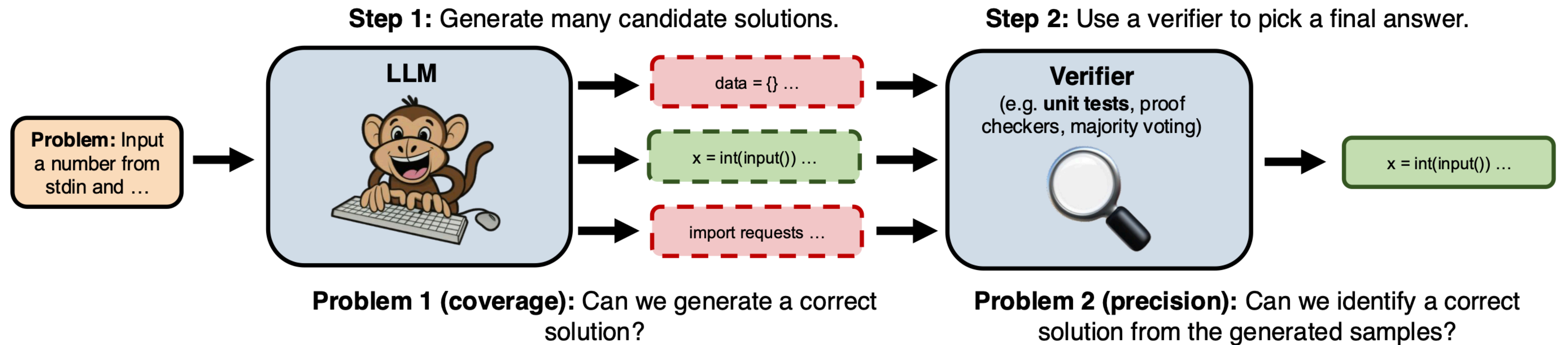
Reward = 1 for correct, 0 for wrong,
-1 for wrong and exceeds budget



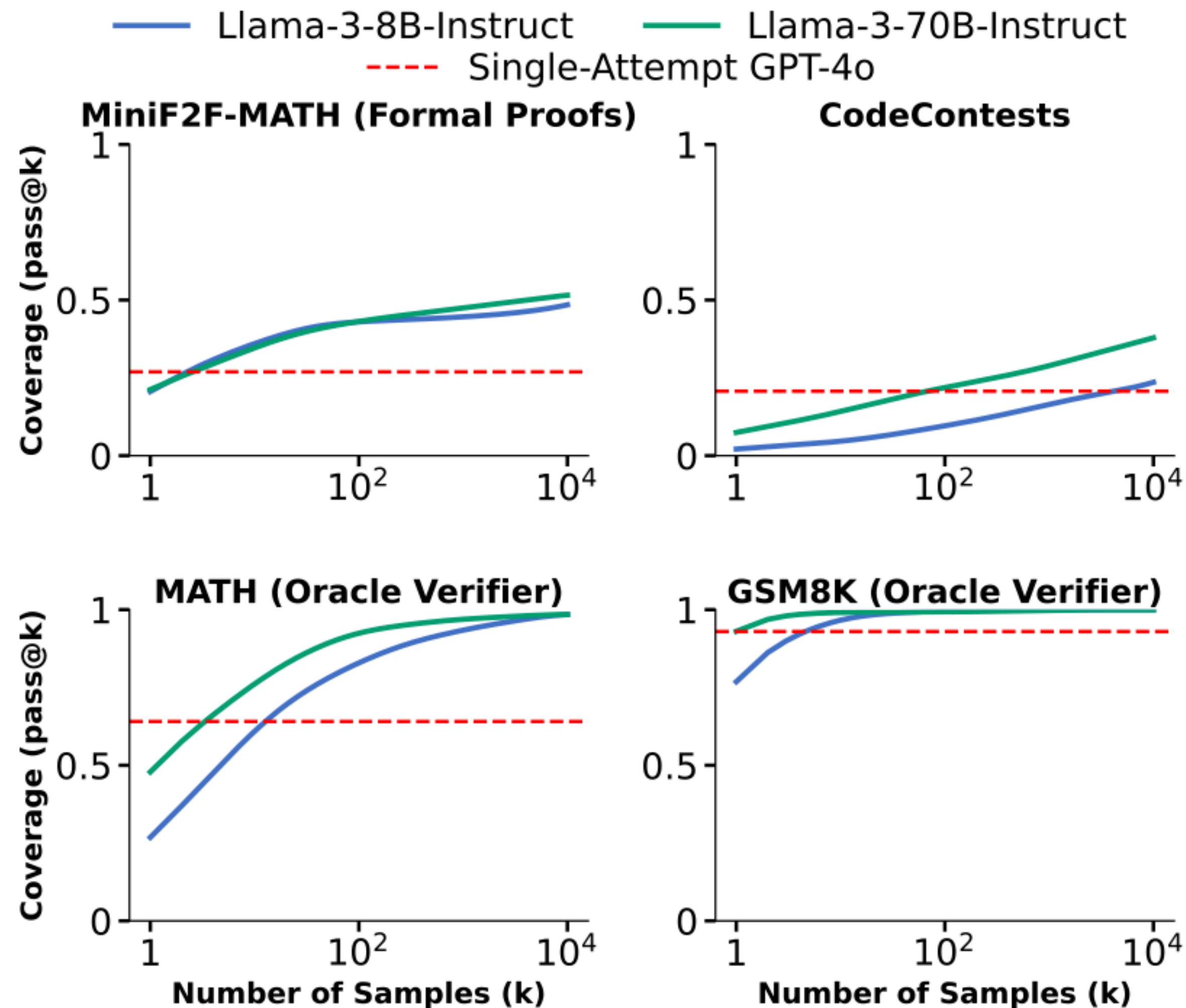
Multi Teacher Distillation



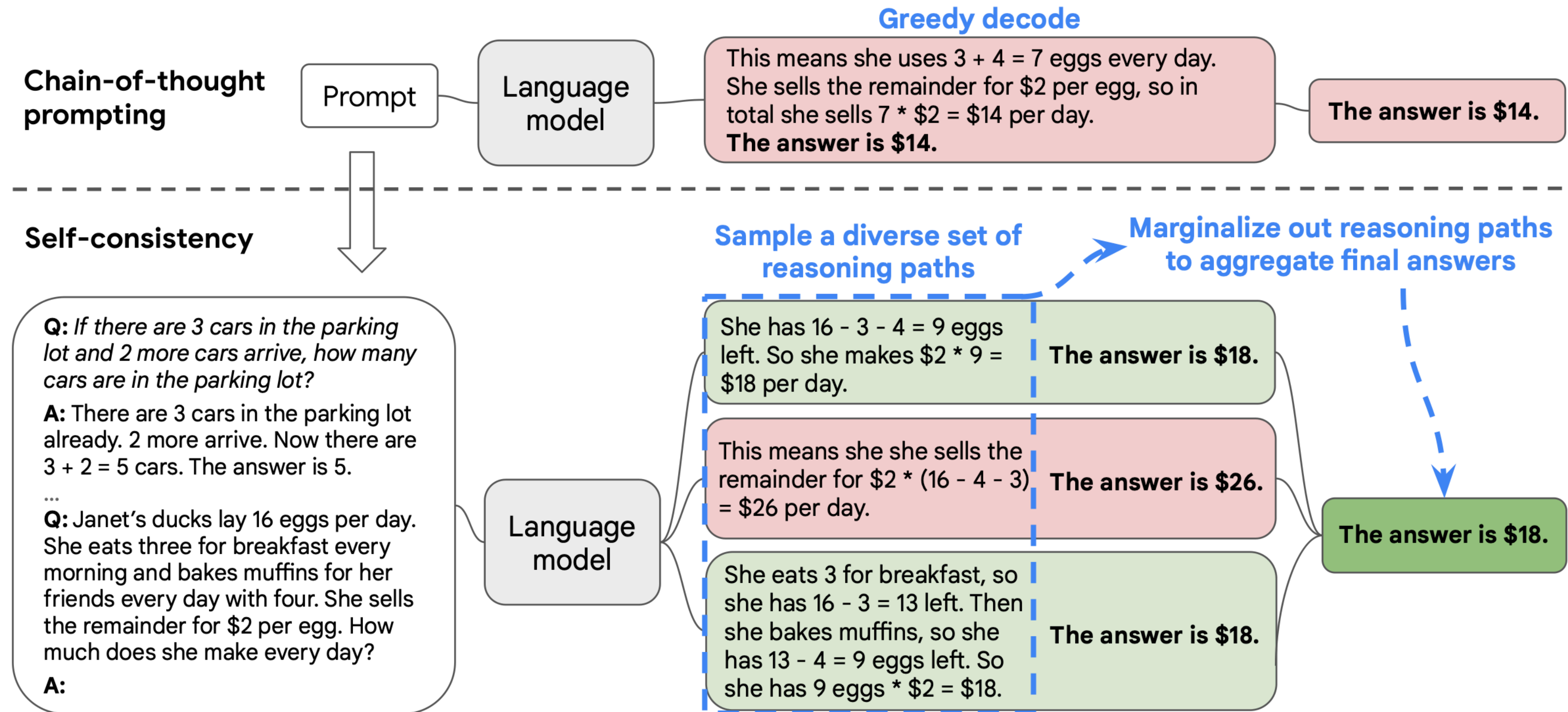
Scaling with Repeated Sampling



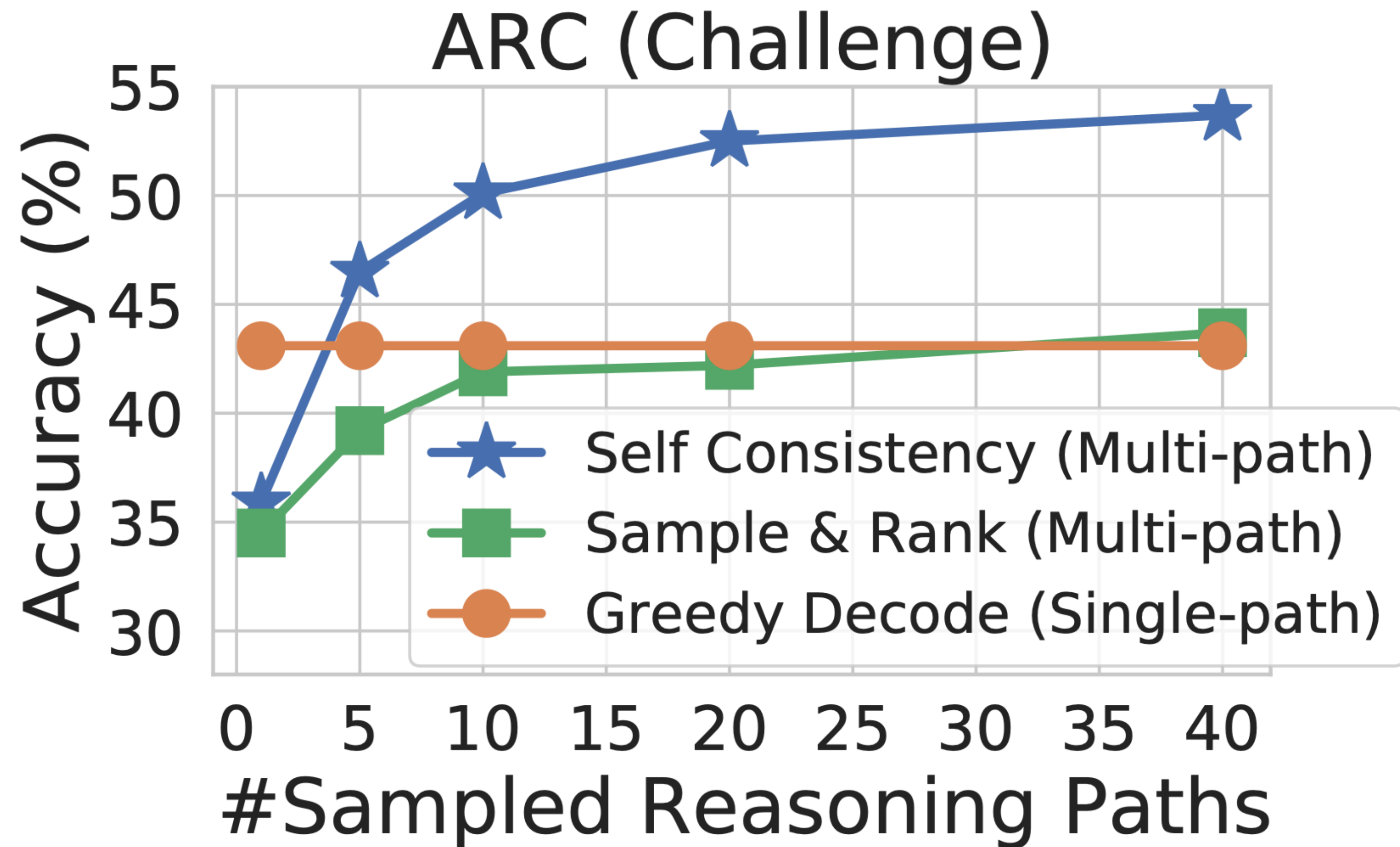
Scaling with Repeated Sampling



What if you don't have a verifier? Self-Consistency



What if you don't have a verifier? Self-Consistency



LLMs for Agents

Language Models?

Architecture

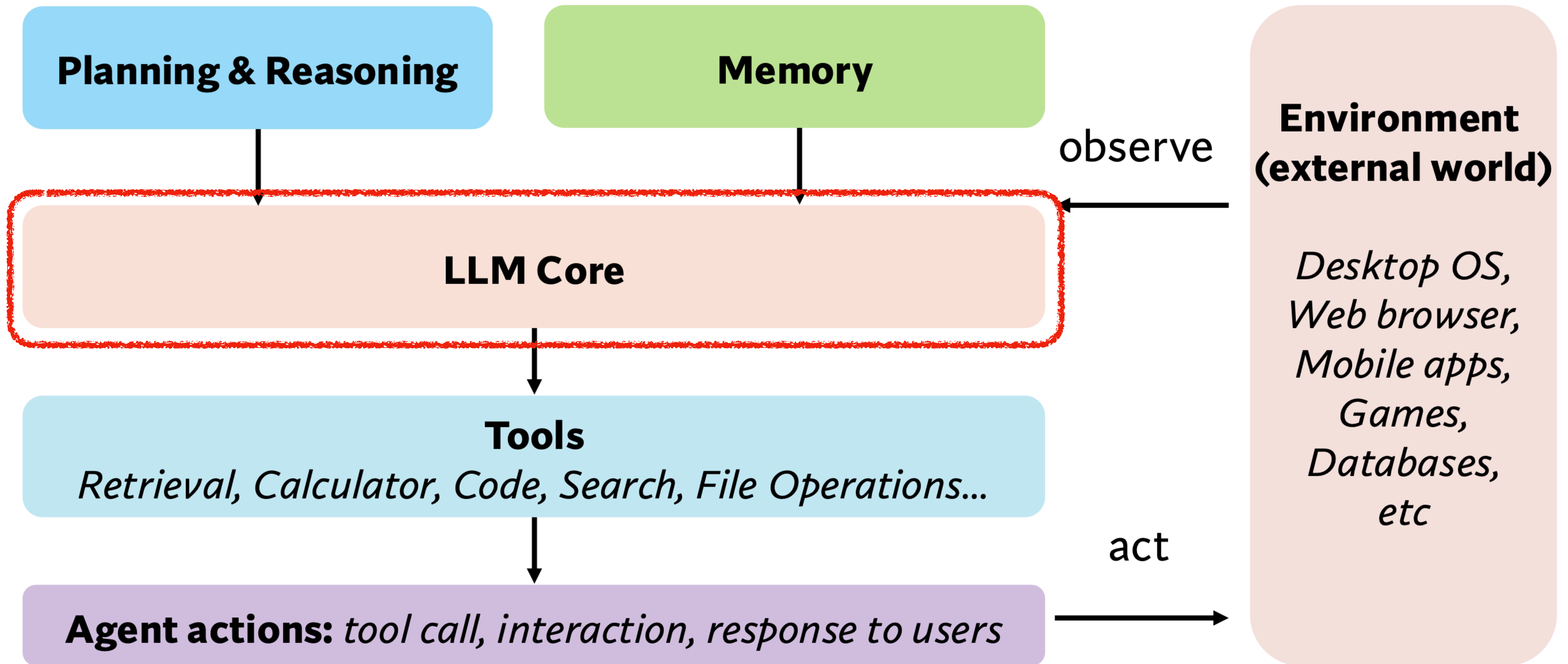
Training

Inference

LLMs for Agents



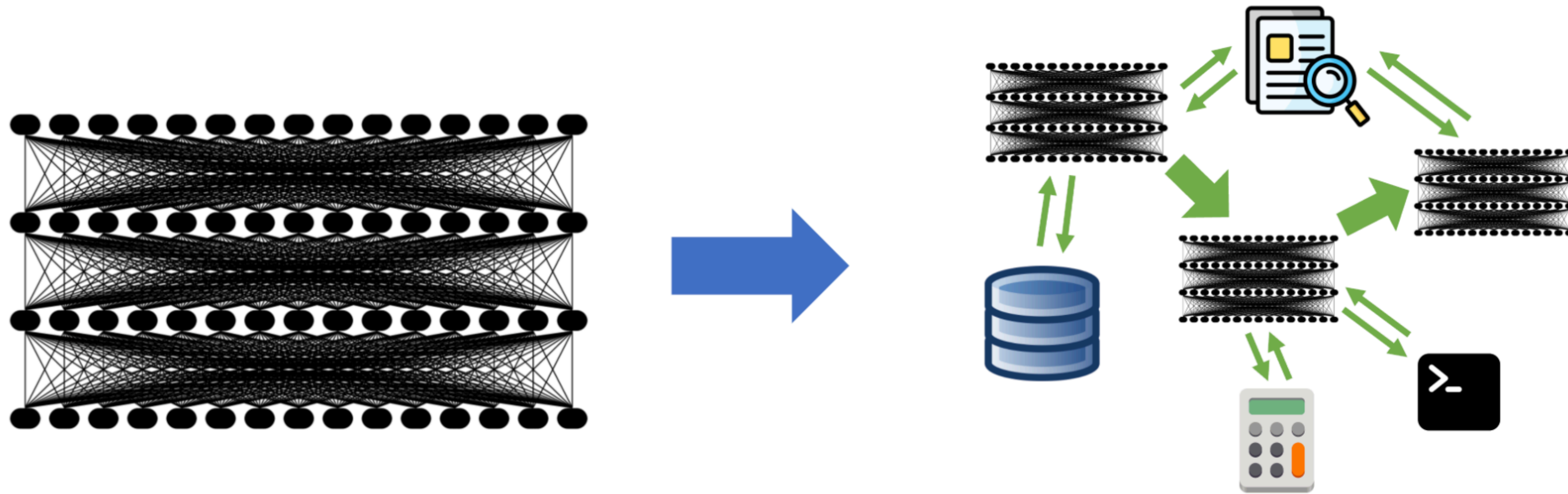
Agents: Key Components



Structured I/O



Motivating Structured I/O



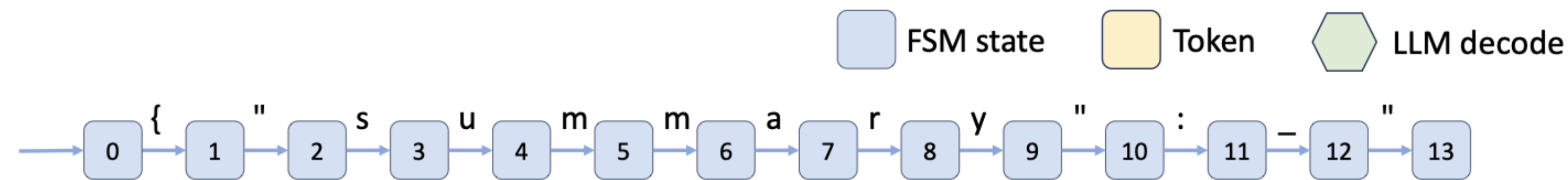
Increasingly many new AI results are from compound systems.

Require a shared contract of how these system components will communicate.

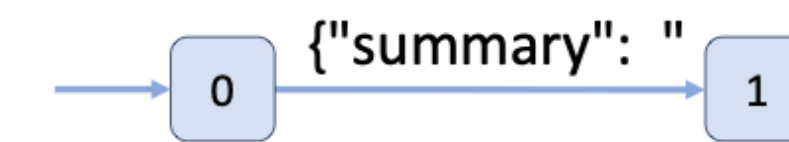


Constrained Decoding

Force the language model to output to a particular structure



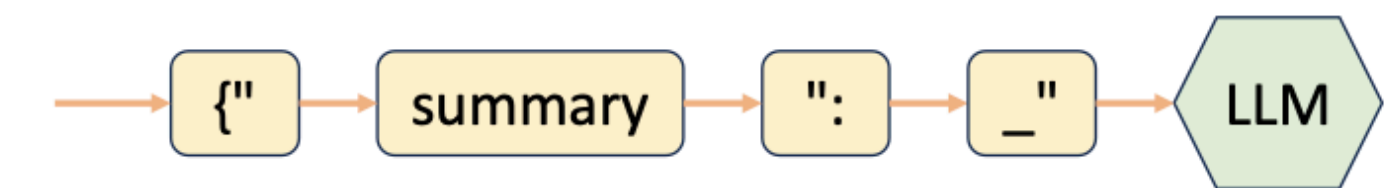
(a) Normal FSM for regex {"summary": "



(b) Compressed FSM for regex {"summary": "



(c) Decoding process with normal FSM



(d) Decoding process with compressed FSM



Constrained Decoding

Force the language model to output to a particular structure

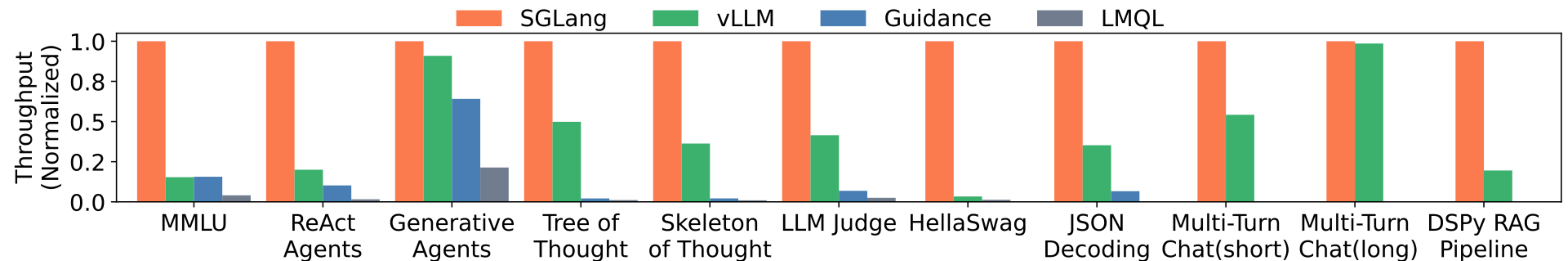


Figure 5: Normalized throughput on Llama-7B models. Higher is better.

SGLang has Finite State Machines for decoding (among other optimizations)



DSPy Signatures

```
class Extract(dspy.Signature):  
    """Extract contact info."""  
    message: str = dspy.InputField()  
    name: str = dspy.OutputField()  
    email: Optional[str] = dspy.OutputField()  
    intent: Literal[  
        "meeting", "intro", "follow-up"  
    ] = dspy.OutputField()  
  
extract = dspy.Predict(Extract)  
extract(message="I'm Sarah"  
        "(sarah@acme.co). Meet Thursday?")
```

OUTPUT

```
Prediction(  
    name="Sarah",  
    email="sarah@acme.co",  
    intent="meeting"  
)
```

Signatures define tasks and enforce output types



DSPy Signatures: Peek inside

System Prompt

Your input fields are:

1. `message` (str):

Your output fields are:

1. `name` (str):

2. `email` (Union[str, NoneType]):

3. `intent` (Literal['meeting', 'intro', 'follow-up']):

All interactions will be structured in the following way, with the appropriate values filled in.

```
[[ ## message ## ]]
```

```
{message}
```

```
[[ ## name ## ]]
```

```
{name}
```

```
[[ ## email ## ]]
```

```
{email}      # note: the value you produce must adhere to the JSON schema: {"anyOf": [{"type": "string"}, {"type": "null"}]}
```

```
[[ ## intent ## ]]
```

```
{intent}      # note: the value you produce must exactly match (no extra characters) one of: meeting; intro; follow-up
```

```
[[ ## completed ## ]]
```

In adhering to this structure, your objective is:

Extract contact info.

User Prompt

```
[[ ## message ## ]]
```

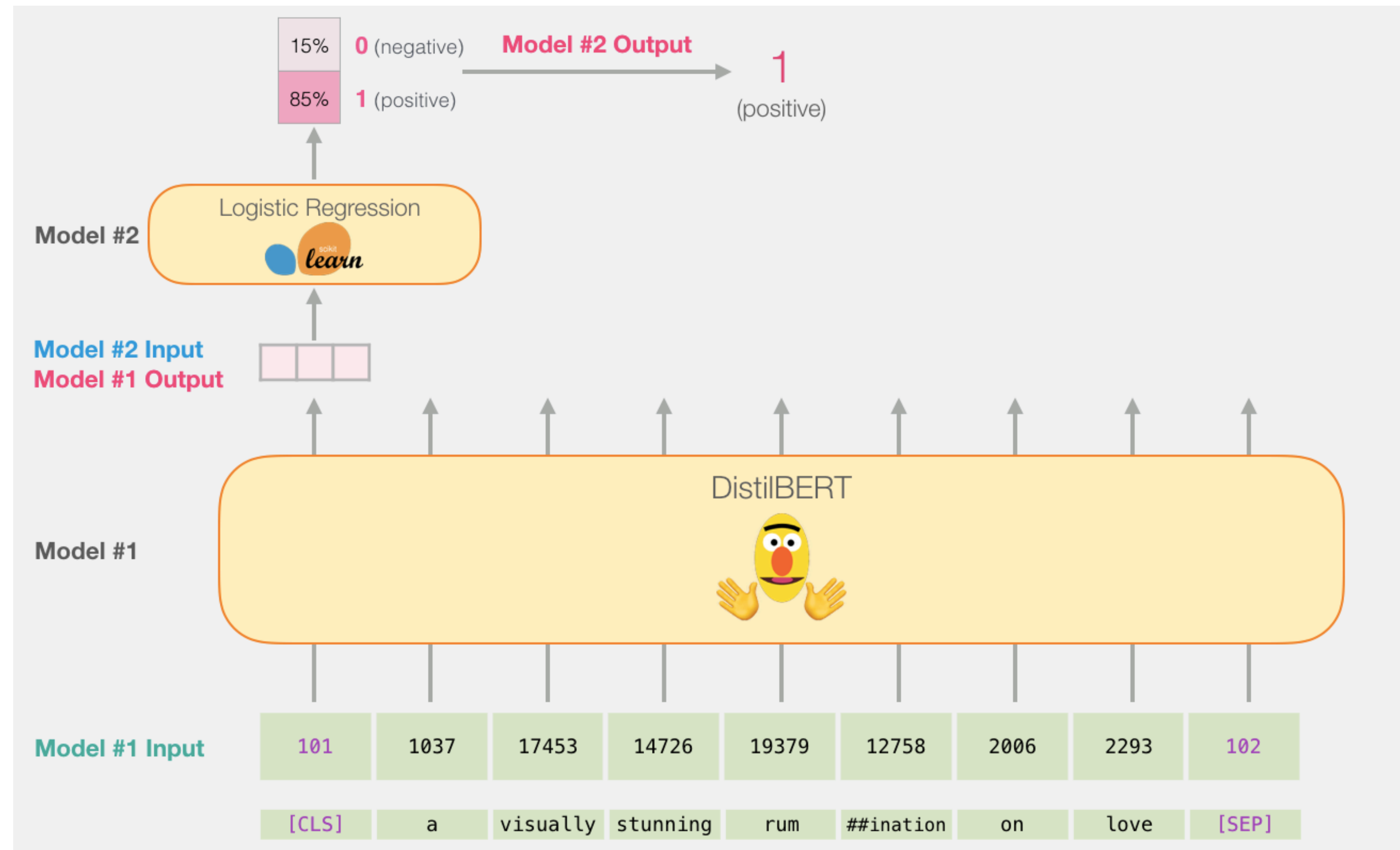
I'm Sarah(sarah@acme.co). Meet Thursday?

Respond with the corresponding output fields, starting with the field `[[ ## name ## ]]', then `[[## email ##]]' (must be formatted as a valid Python Union[str, NoneType]), then `[[ ## intent ## ]]' (must be formatted as a valid Python Literal['meeting', 'intro', 'follow-up']), and then ending with the marker for `[[## completed ##]]'.



Constrain the Model: Classifiers

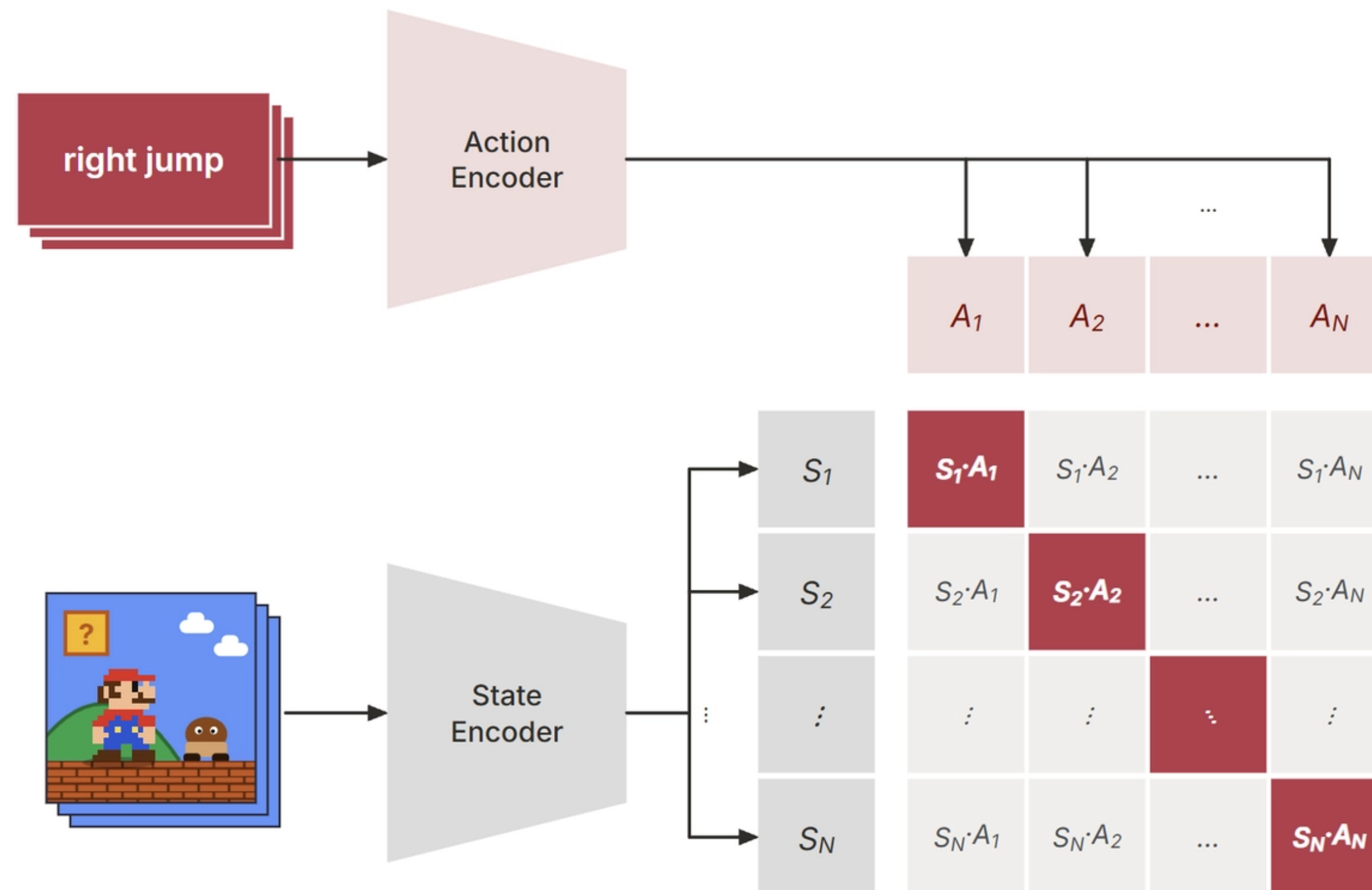
Train a small head on top of embeddings to map to class labels



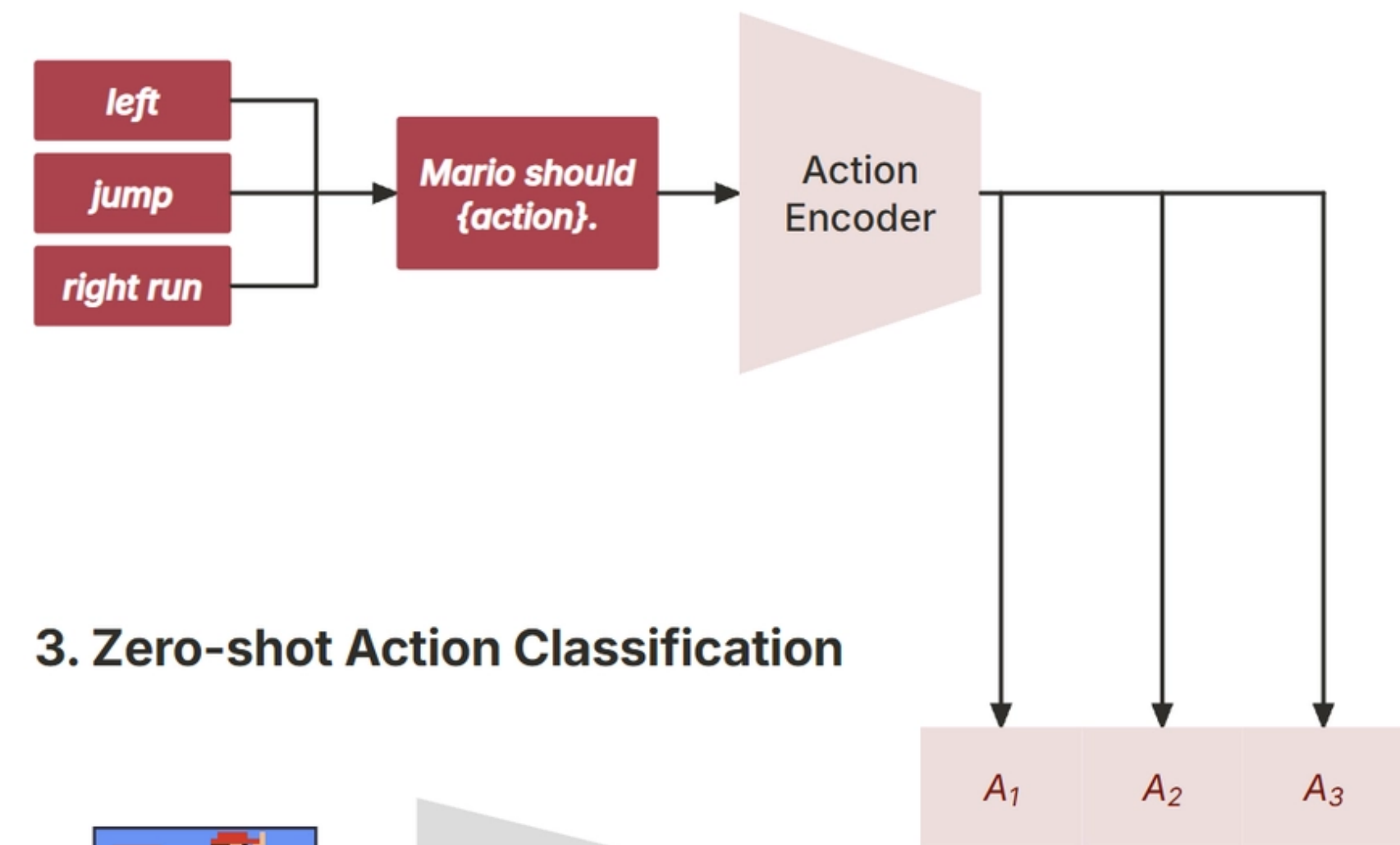
Constrain the Model: System One Models

Popular term right now, but roughly classifiers with adaptable options (for “decisions”)

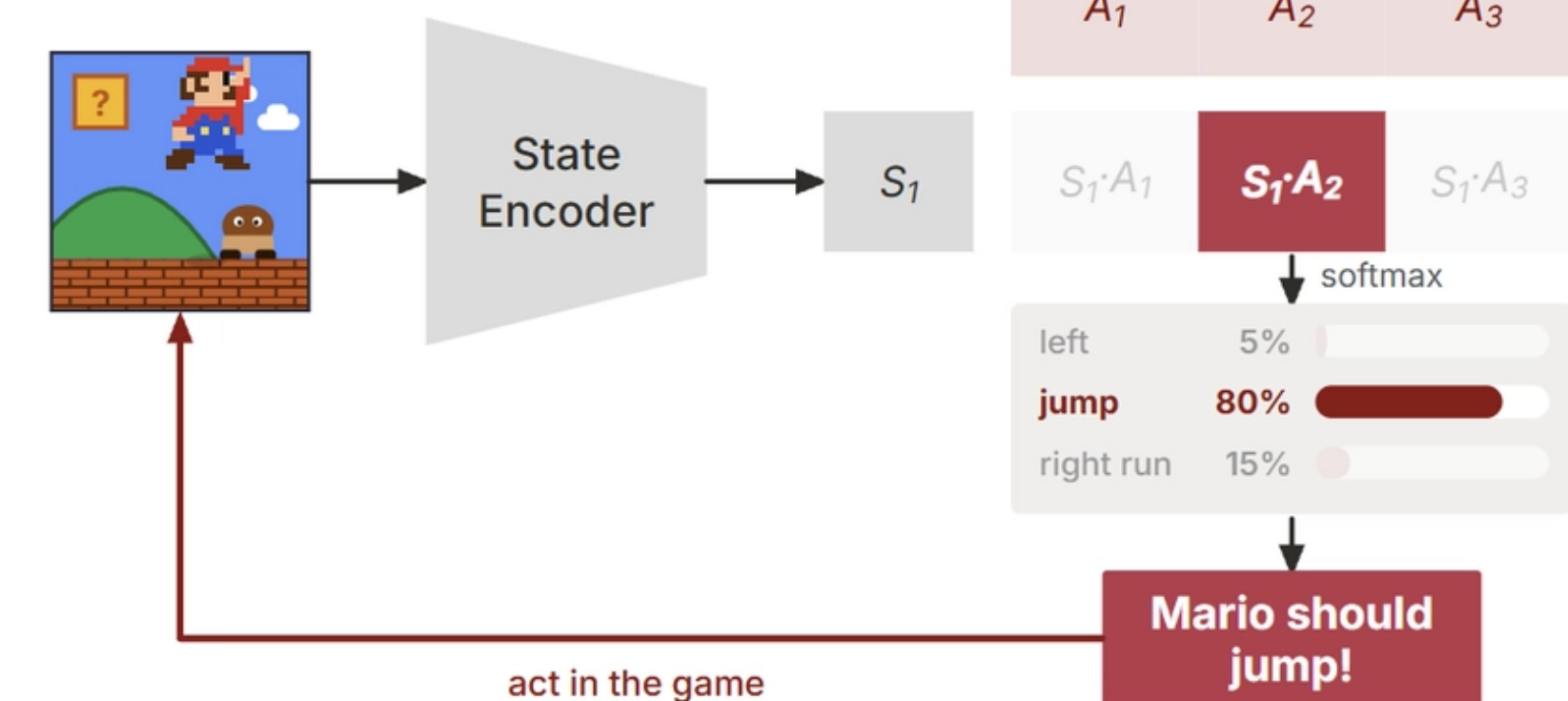
1. Contrastive State-Action Pre-training



2. Create Action Candidates



3. Zero-shot Action Classification



Example: Tool Calling

The New England Journal of Medicine is a registered trademark of **[QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society]** the MMS.

Out of 1400 participants, 400 (or **[Calculator(400 / 1400) → 0.29]** 29%) passed the test.

The name derives from "la tortuga", the Spanish word for **[MT("tortuga") → turtle]** turtle.

The Brown Act is California's law **[WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.]** that requires legislative bodies, like city councils, to hold their meetings open to the public.



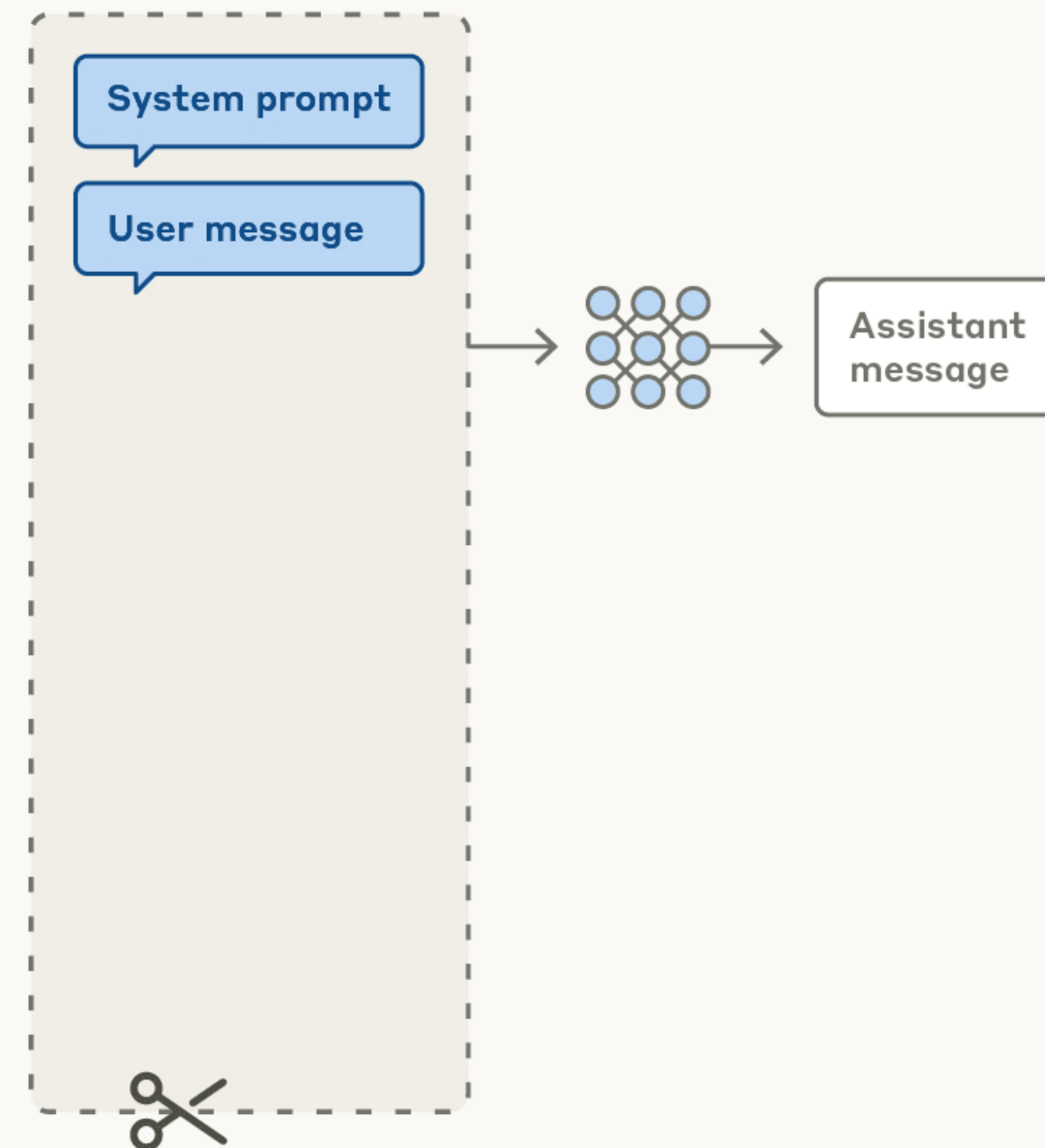
Context Engineering



Prompt engineering vs. context engineering

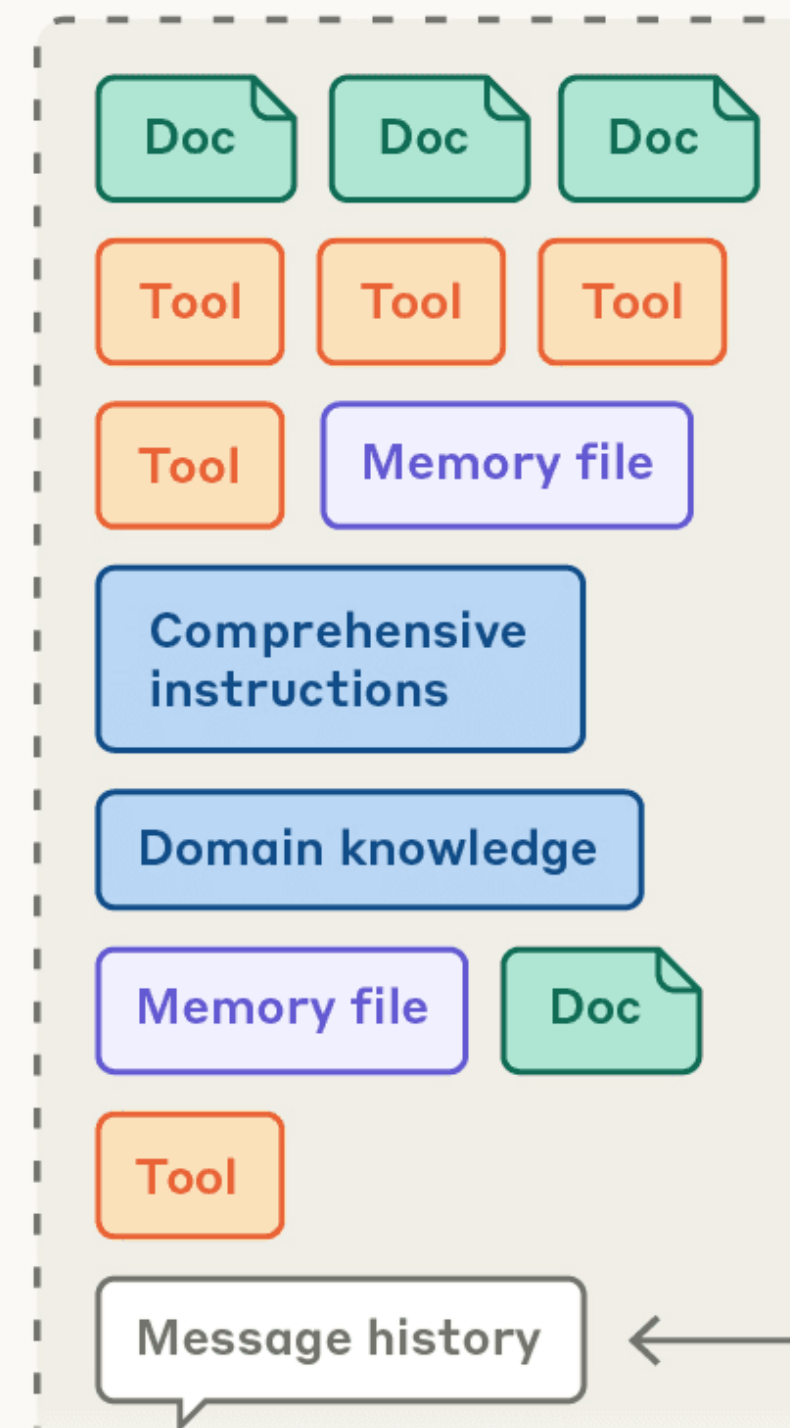
Prompt engineering for single turn queries

Context window

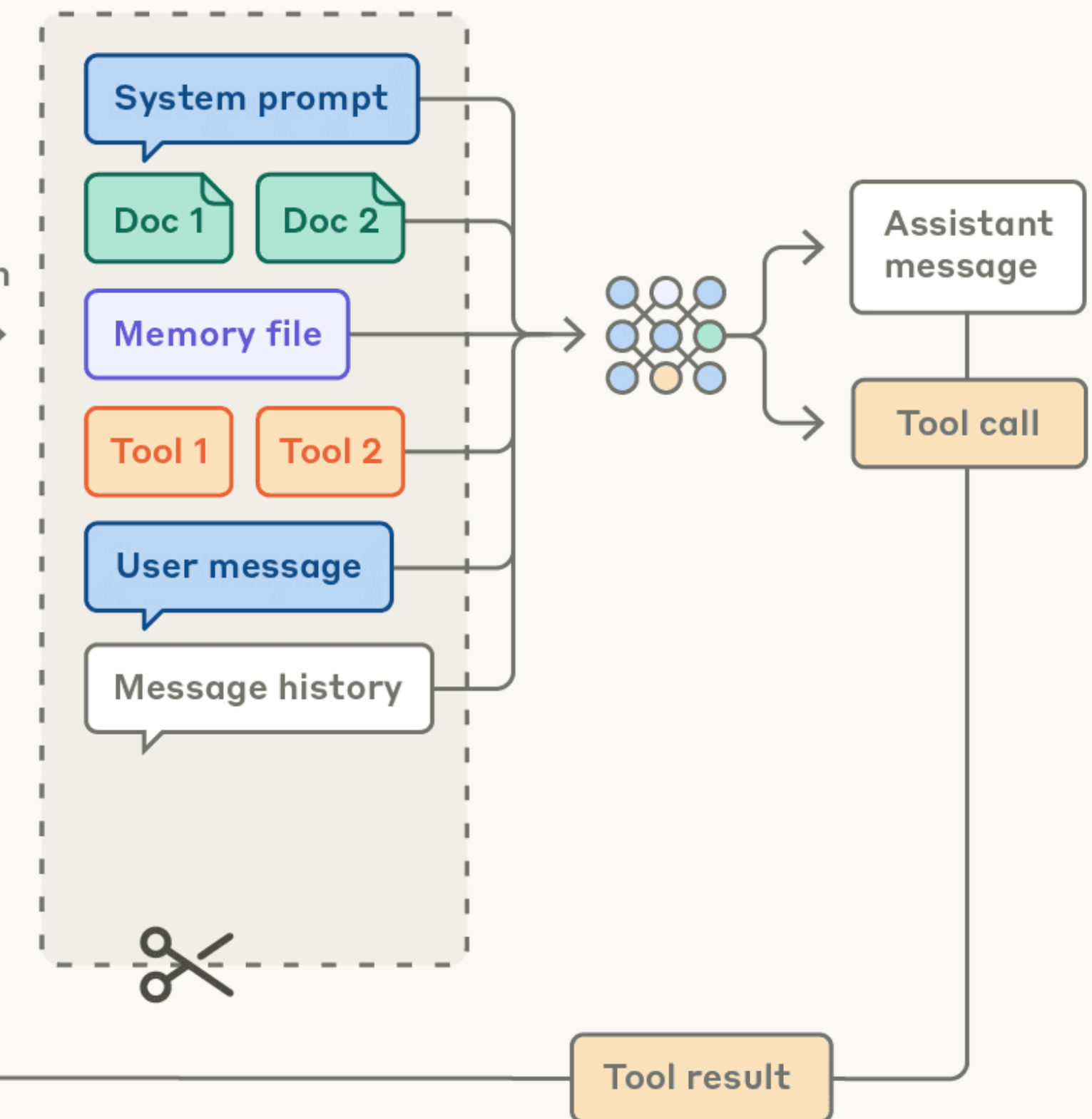


Context engineering for agents

Possible context to give model



Context window

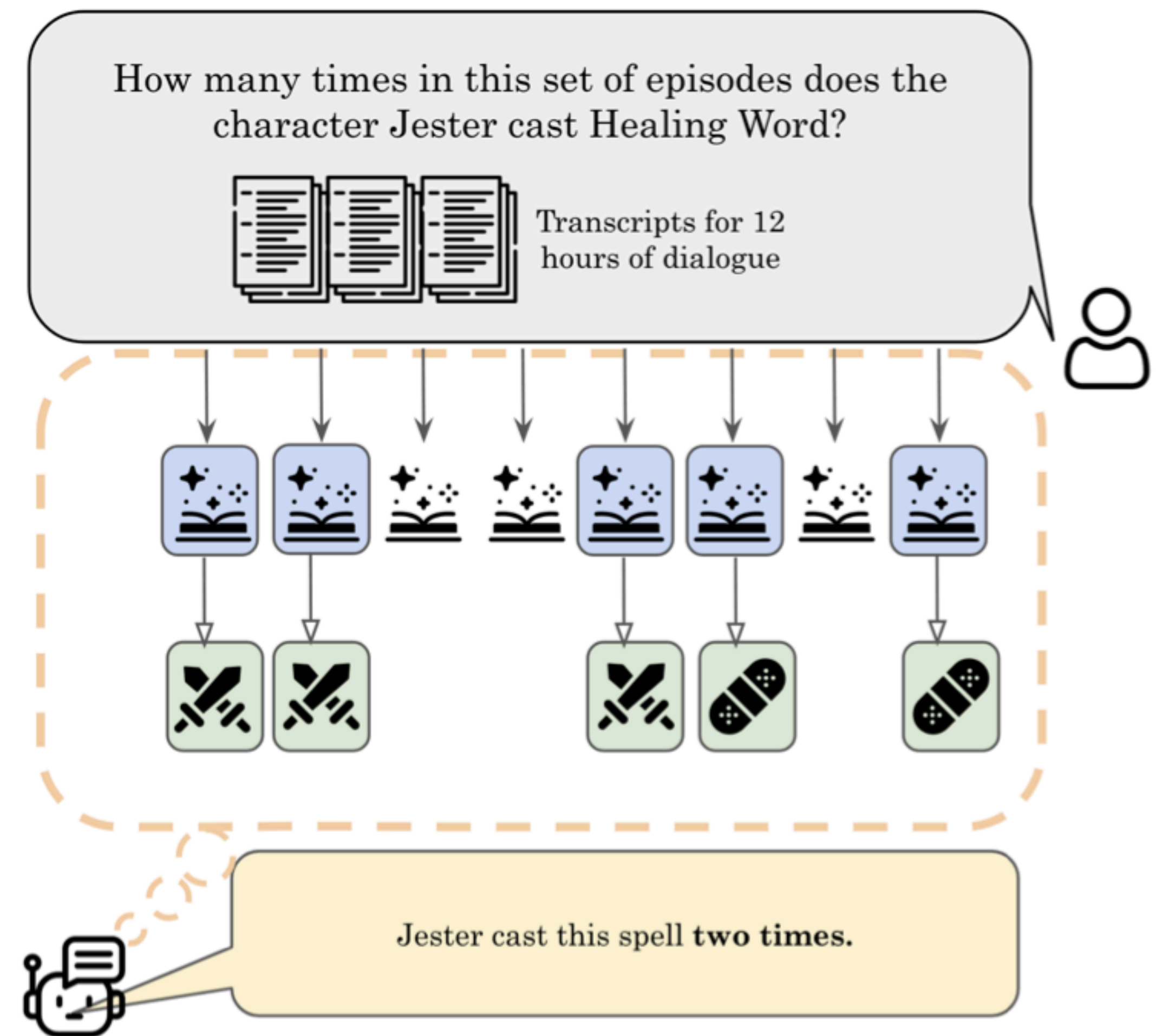
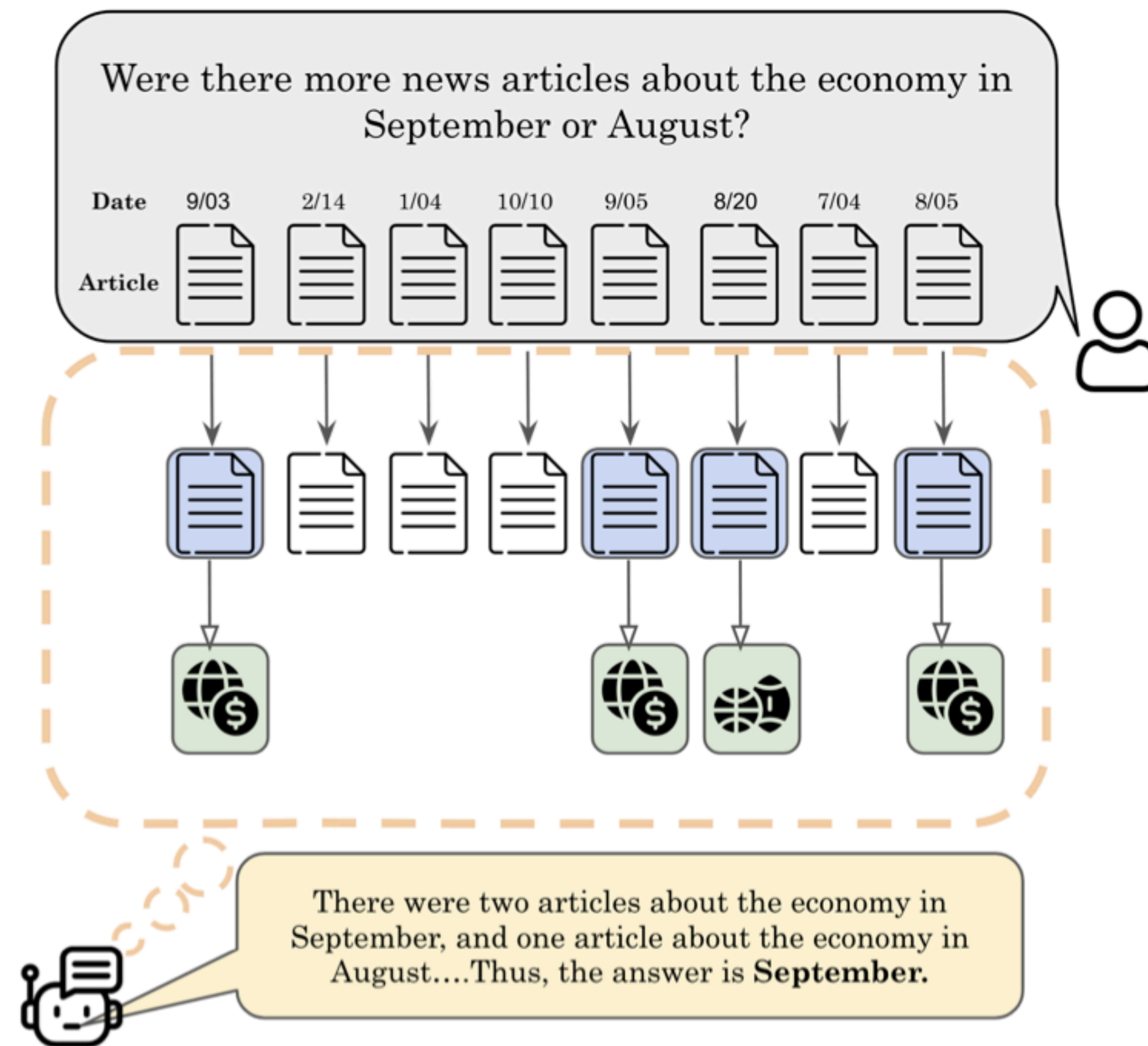


What goes in the context of an agent?

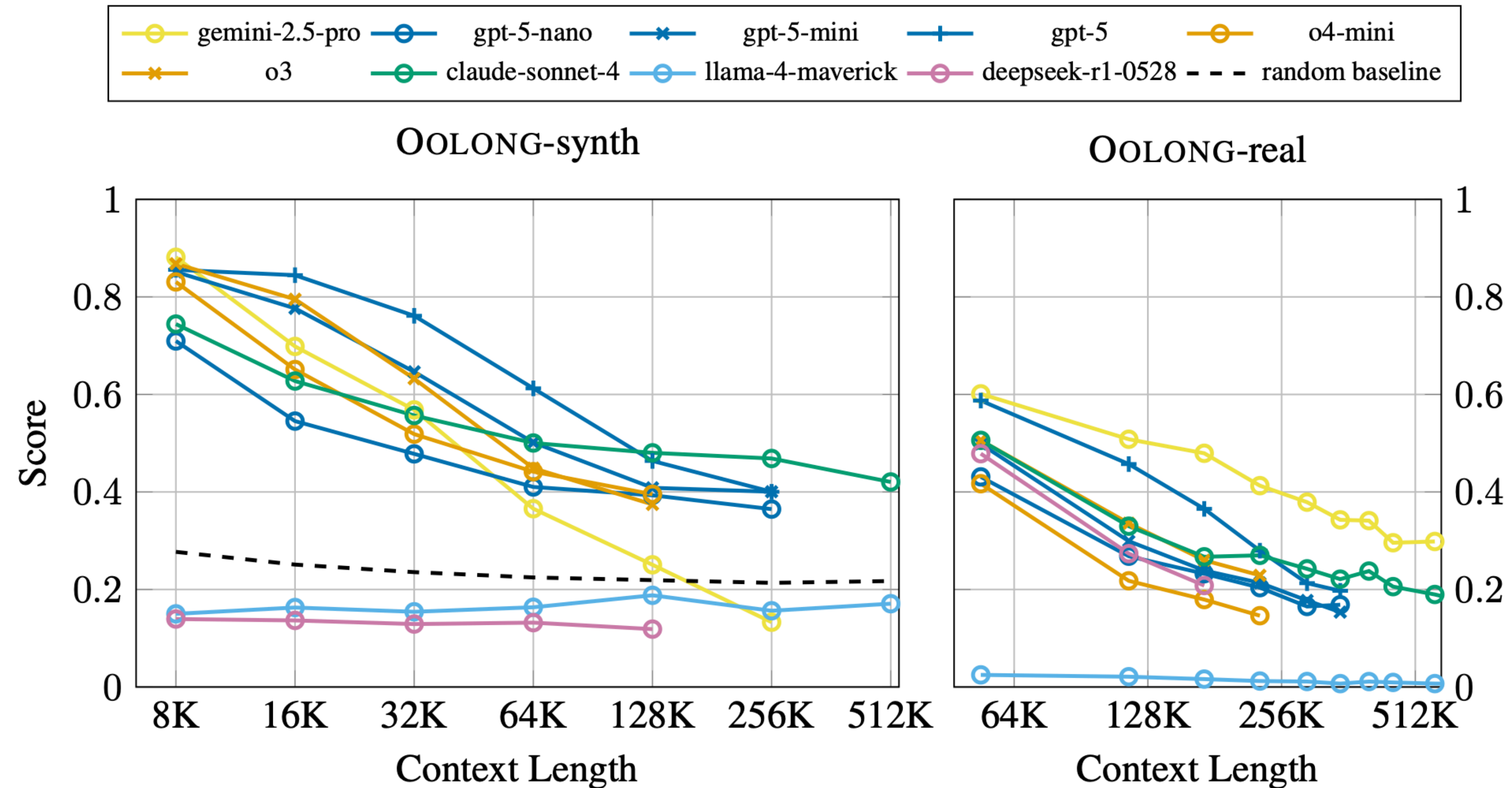
- System Prompt
- Message History
- Memories
- Tool Call Results
- Attached Files



Why not just put everything? Context Rot



Why not just put everything? Context Rot



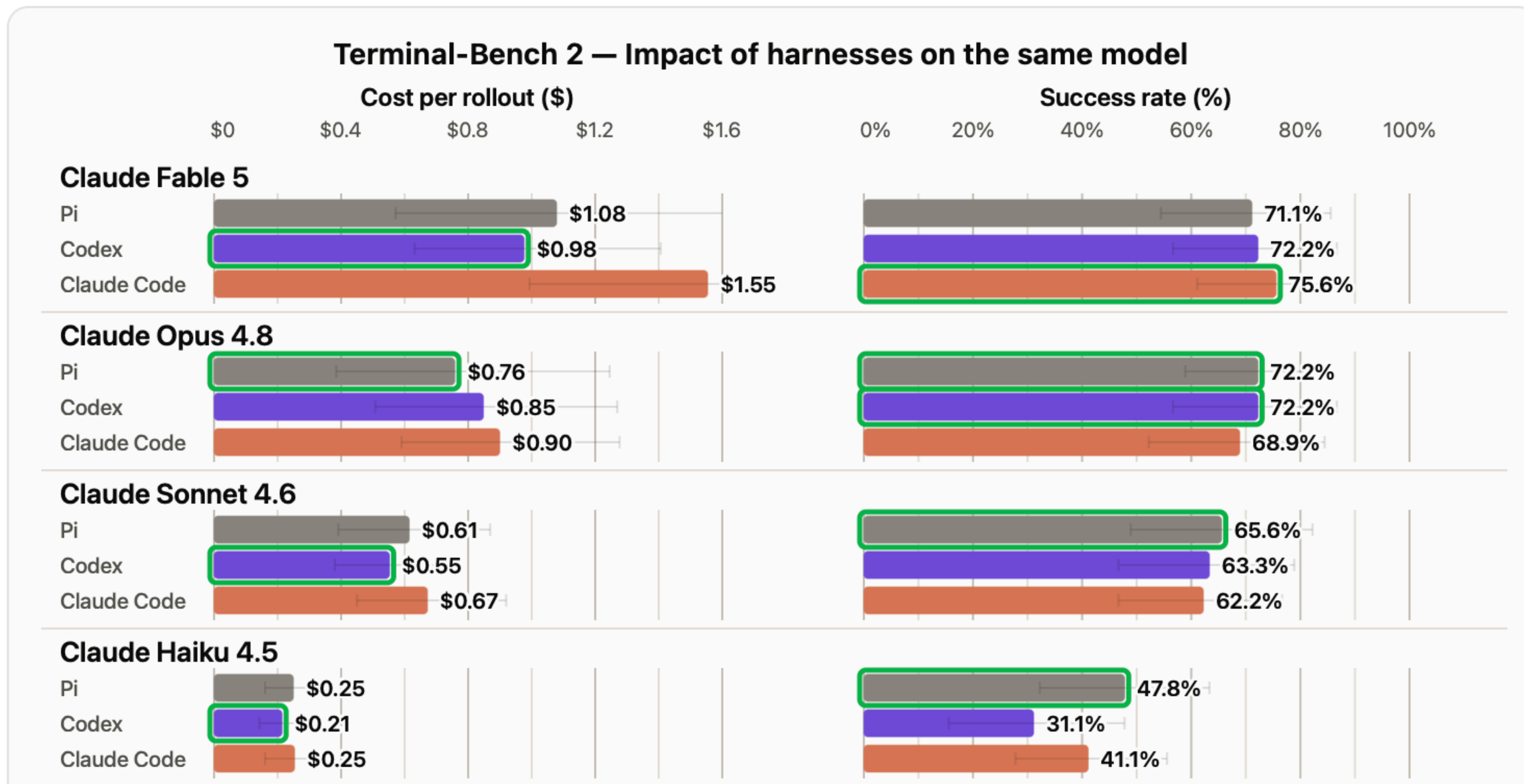
**There is no one way to build the context,
this will be something you experiment with
in HW1.**

**Instead we will provide some practical tips
and considerations.**



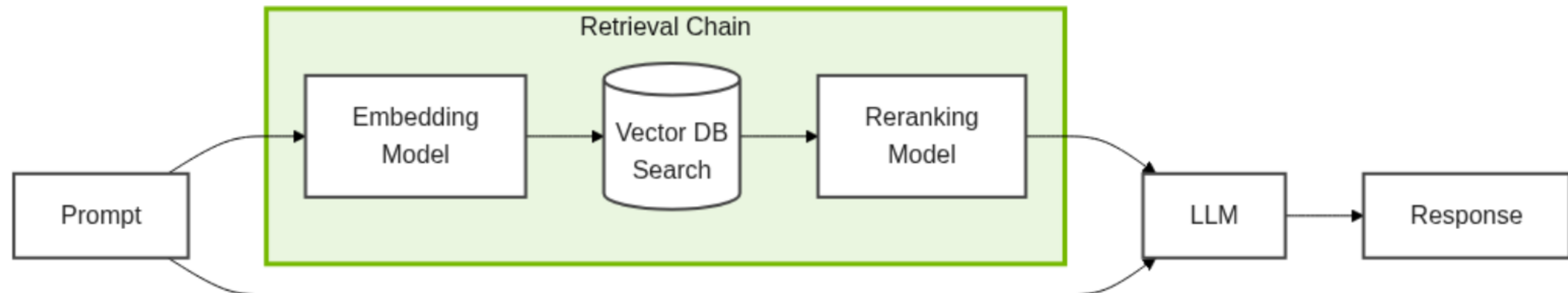
Avoid bloat

Sometimes a few tools are all you need. Pi has just read, write, edit, and bash.



Avoid bloat

Use retrieval to recover the most relevant memories or files

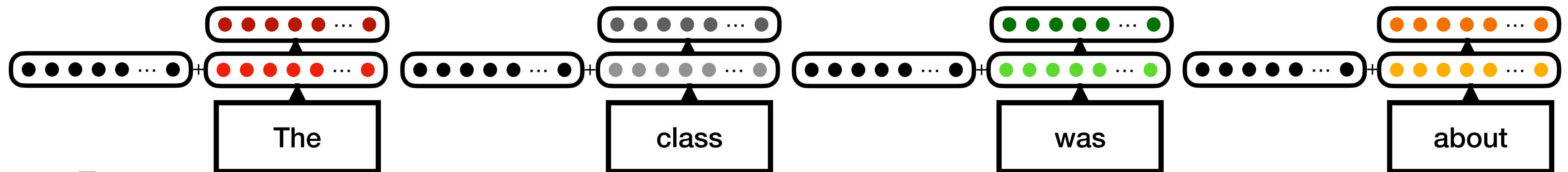
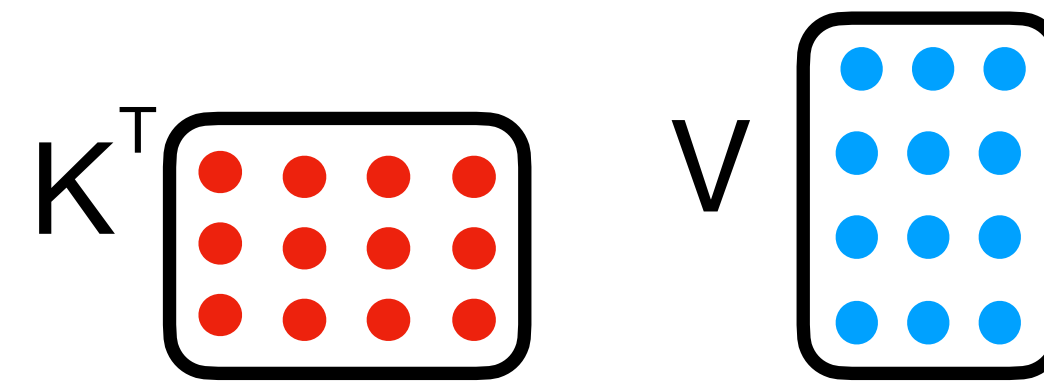


More on this in the next lecture!



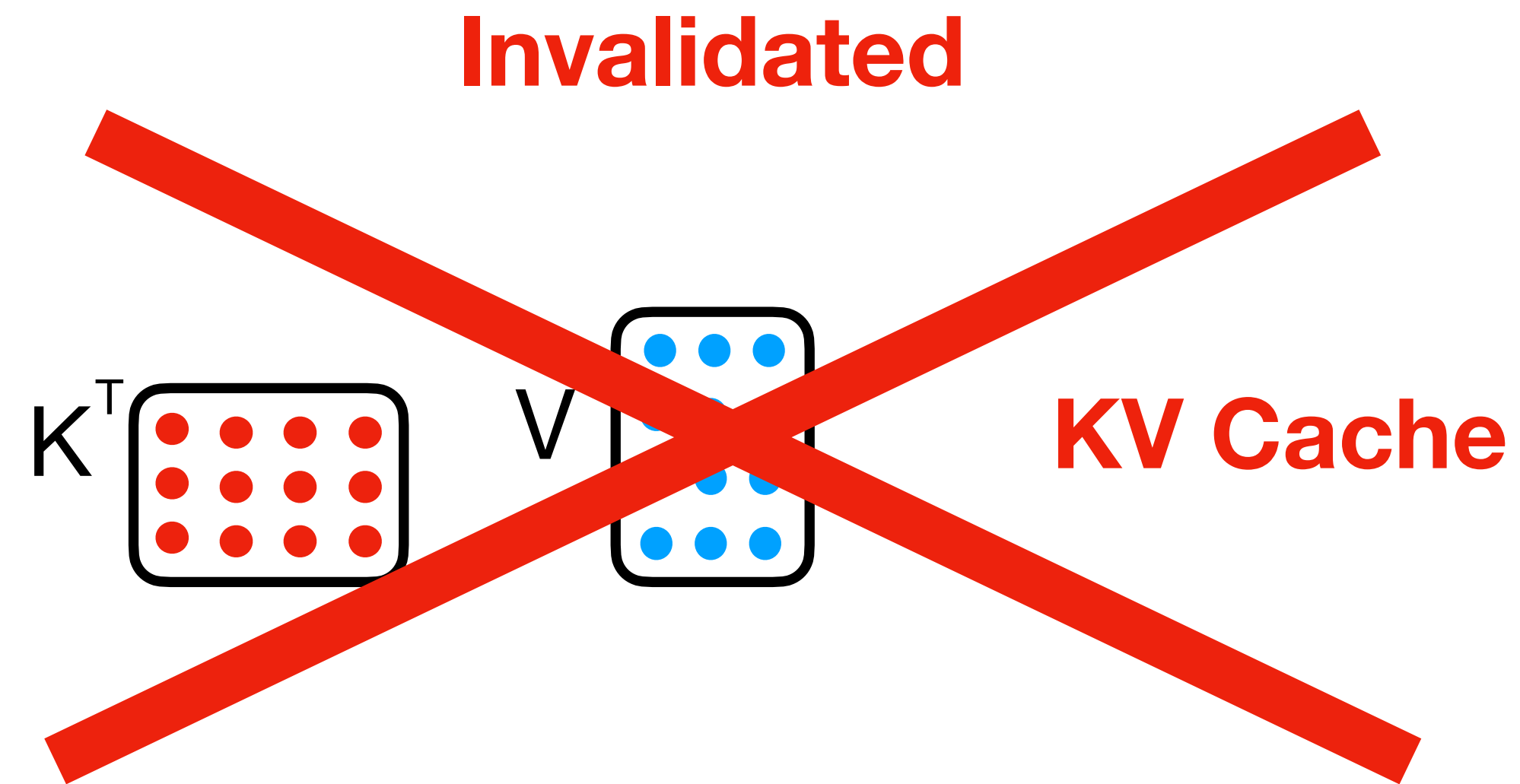
Add sequentially

Avoid invalidating the KV Cache

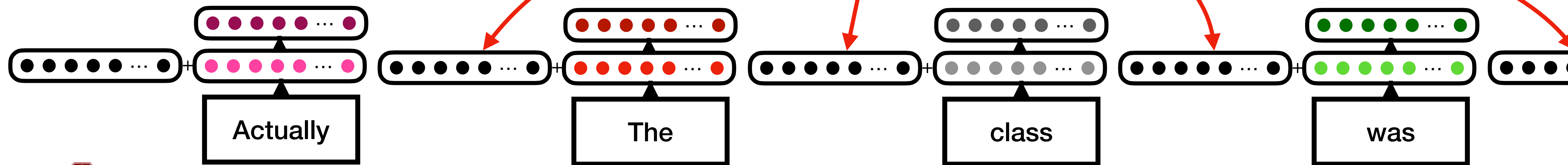


Add sequentially

Avoid invalidating the KV Cache

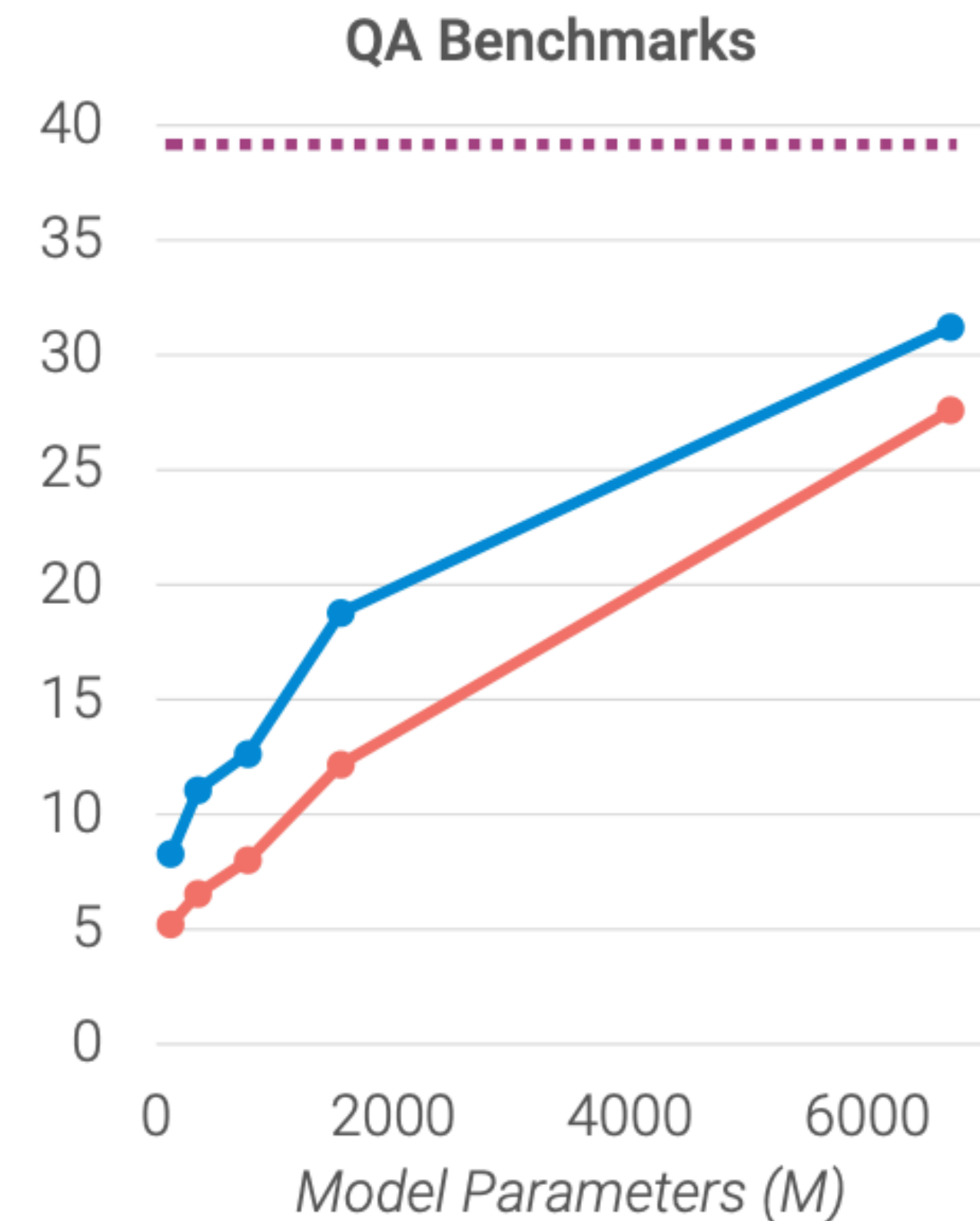
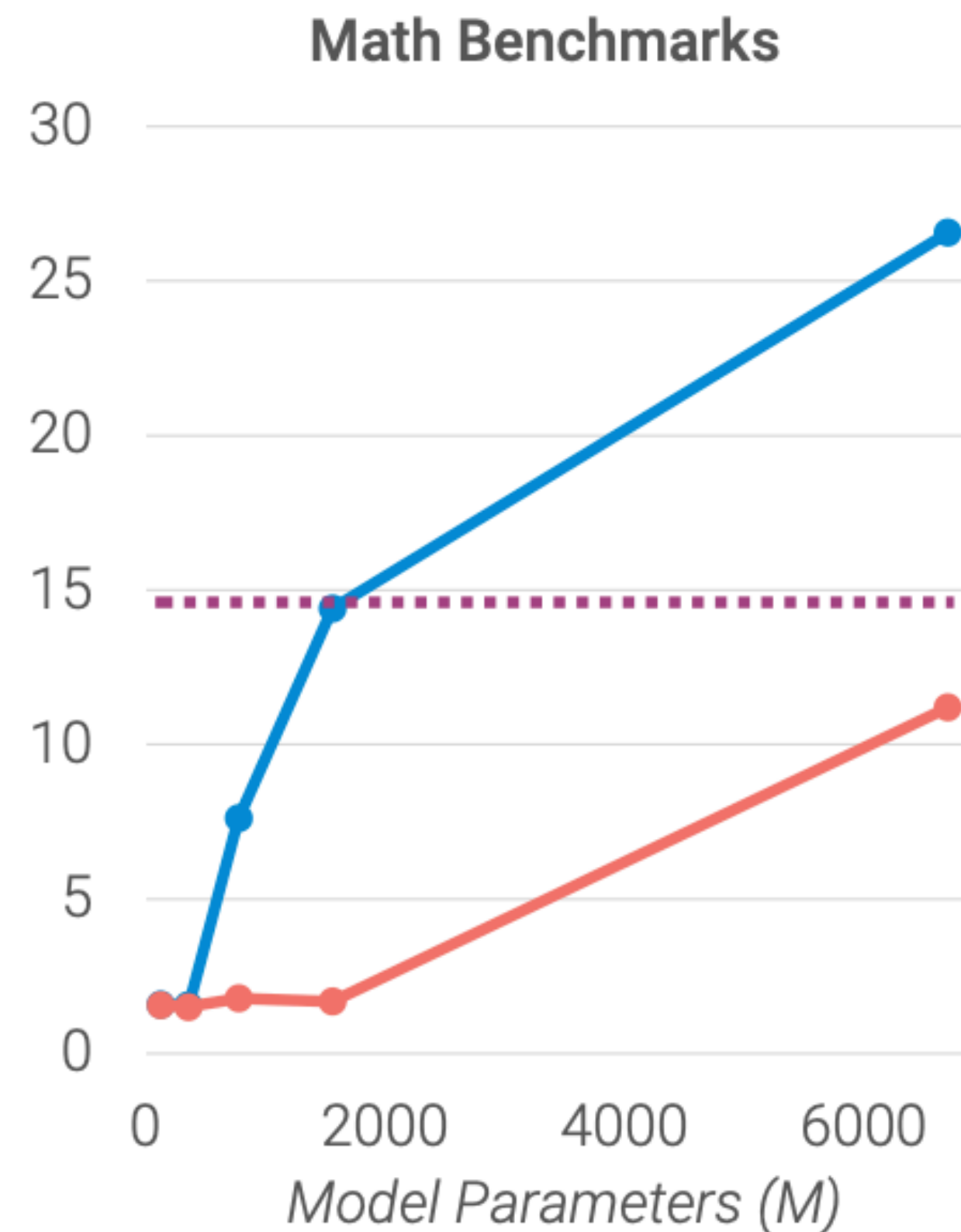
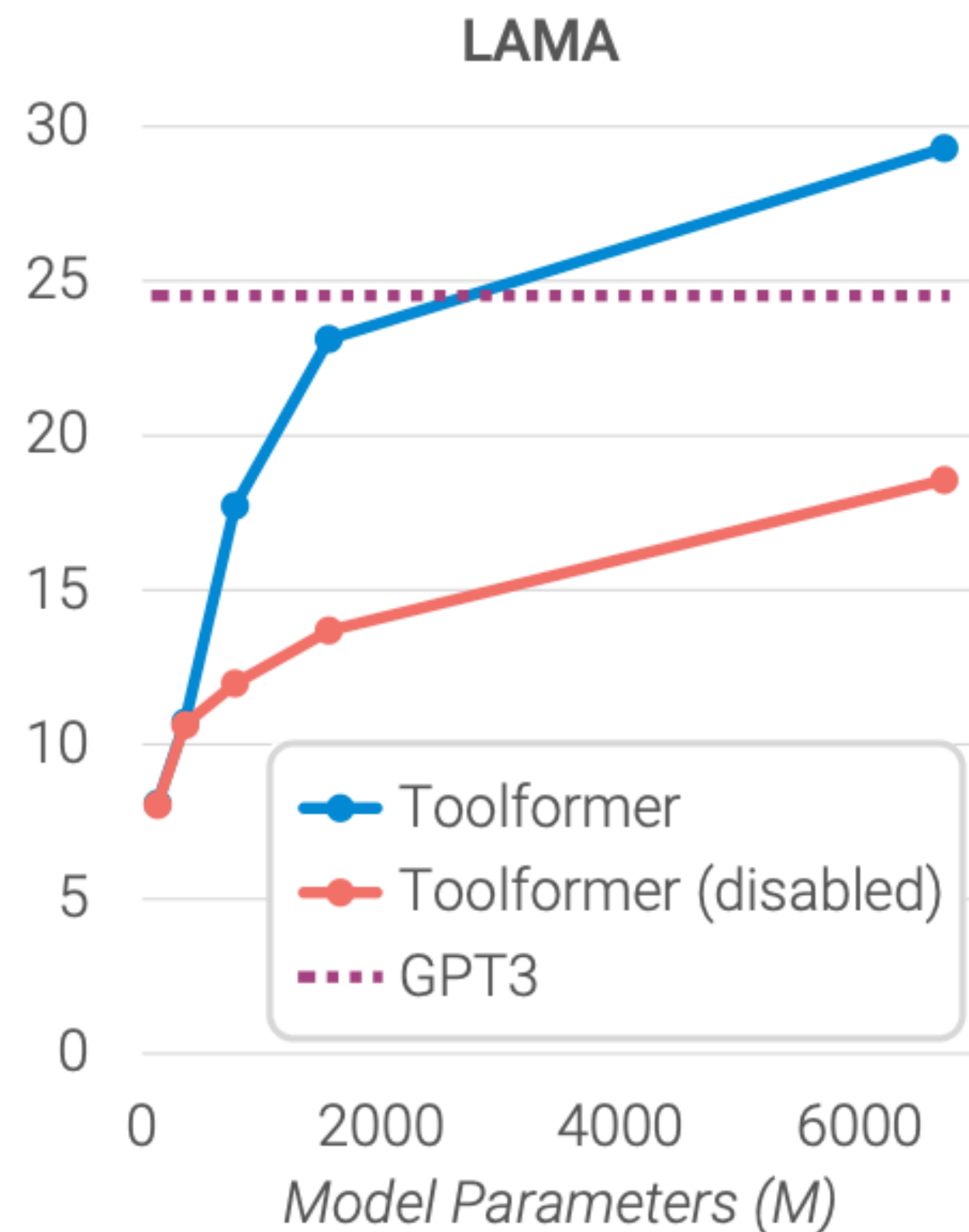


All of these position embeddings just changed.



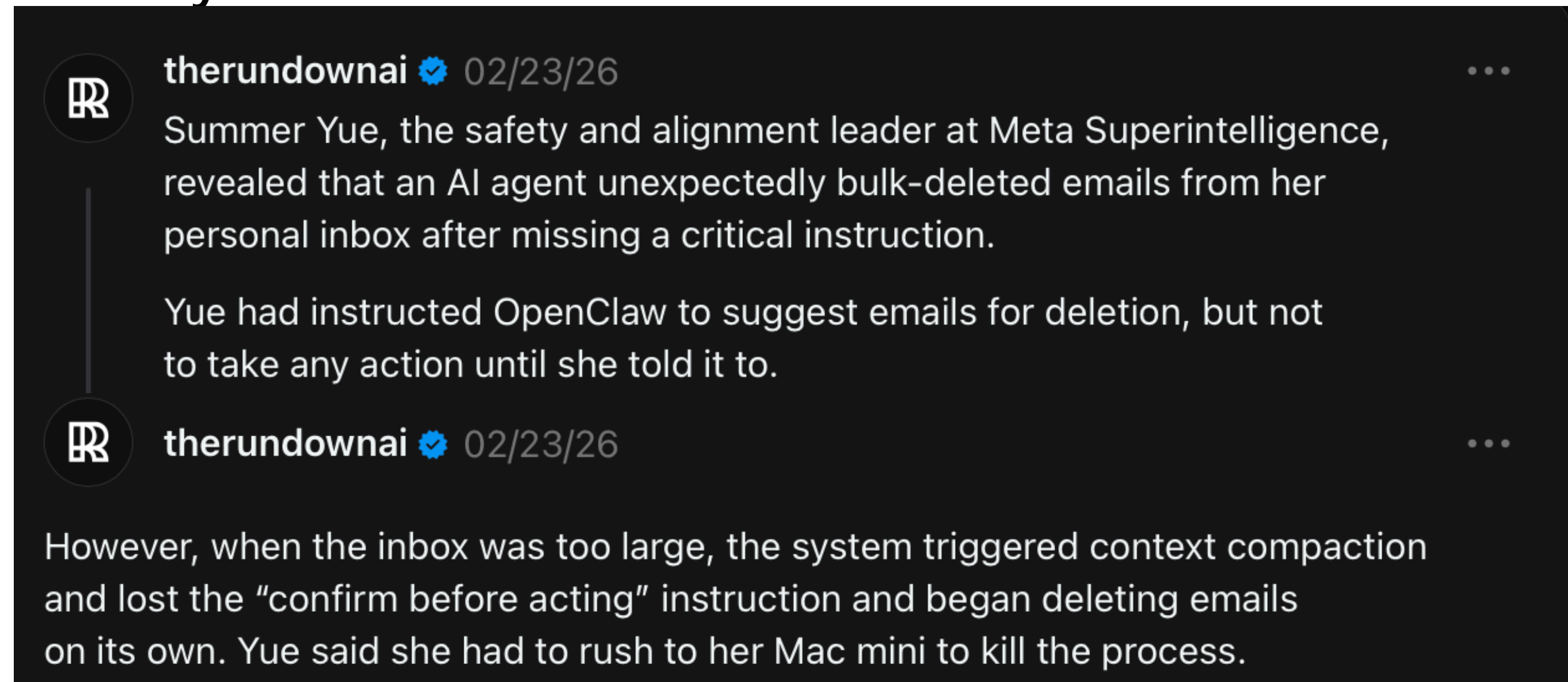
Evaluate context choices empirically

You can setup benchmarks/evals and test your changes

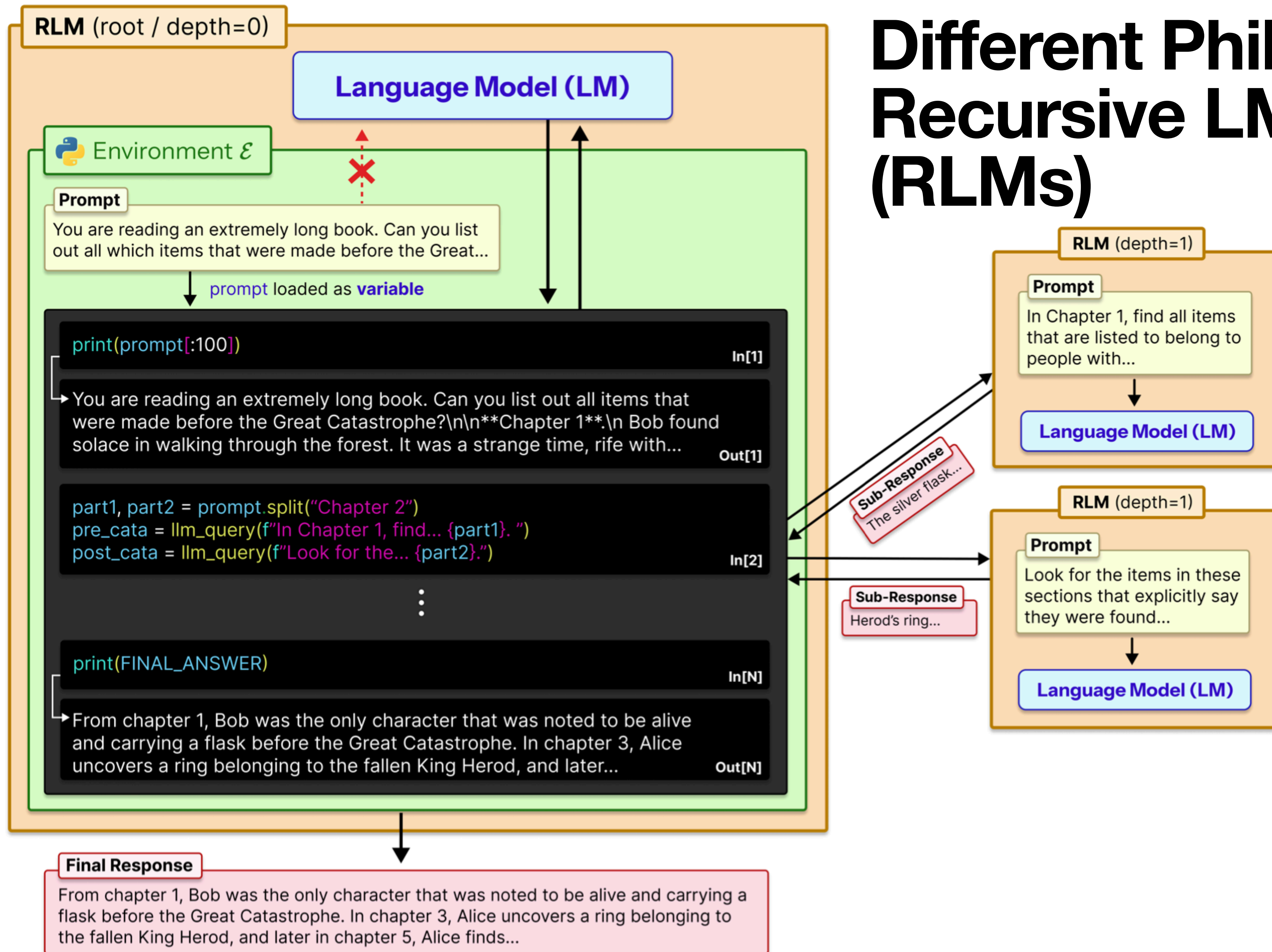


Compaction

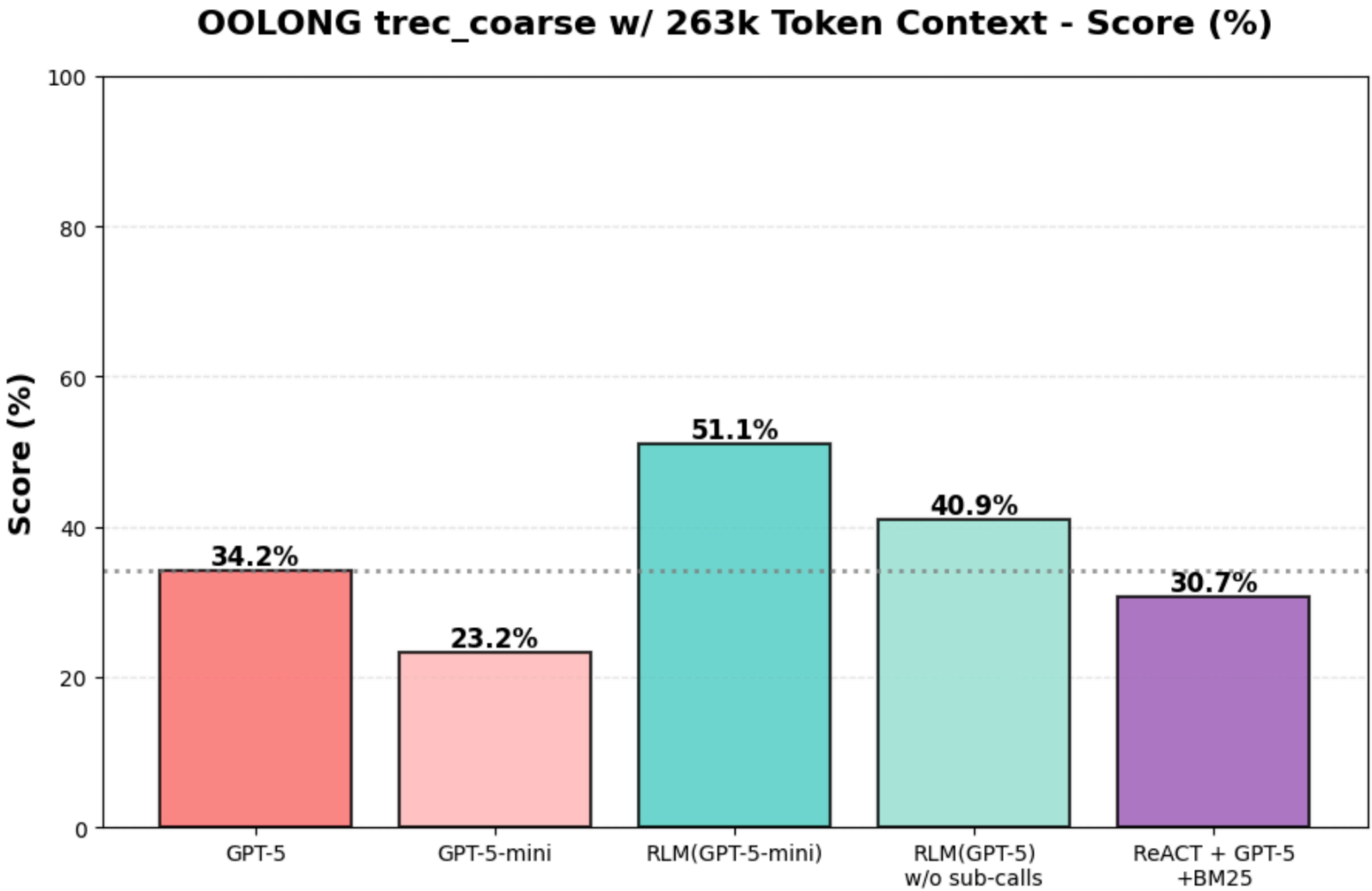
- By the time you reach compaction the context has grown extremely long.
- The new decision is what to keep in the new context.
- Keep too much and you will have to compact again soon.
- Drop too much and you can lose critical context needed to complete the task.



Different Philosophy: Recursive LMs (RLMs)



Recursive LMs (RLMs)



Summary

Language Models?

Architecture

Training

Inference

LLMs for Agents



Summary

- ✓ What are Language Models?
- ✓ Language Model Architecture (Transformers, Linear Attention)
- ✓ Language Model Training (Pretraining, Midtraining, Posttraining, Agent Training)
- ✓ Language Model Inference (Prefill/Decode, Inference Time Scaling)
- ✓ LLMs for Agents (Structured I/O, Context Engineering)



See you on Wednesday for RAG!

